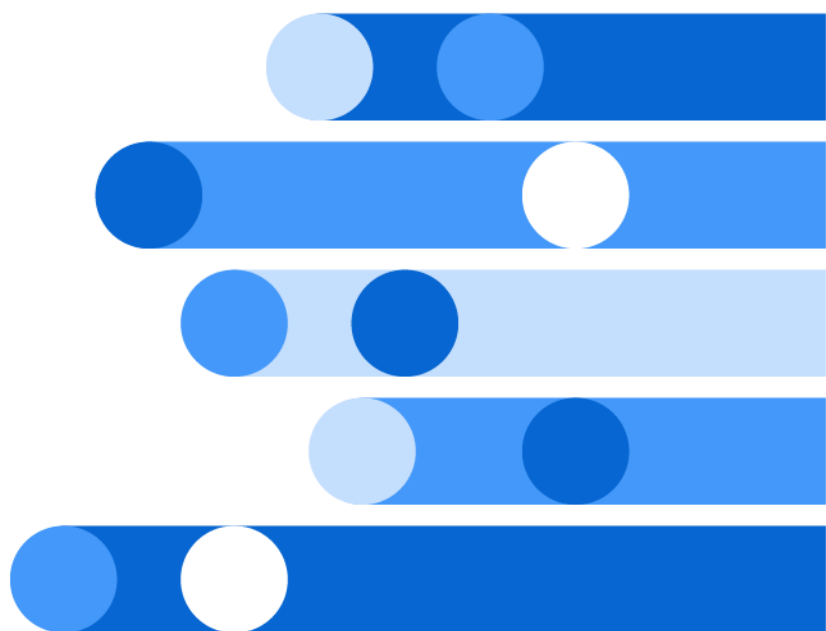




SAS[®] 9.4 DS2 Programmer's Guide



The correct bibliographic citation for this manual is as follows: SAS Institute Inc. 2017. *SAS® 9.4 DS2 Programmer's Guide*. Cary, NC: SAS Institute Inc.

SAS® 9.4 DS2 Programmer's Guide

Copyright © 2017, SAS Institute Inc., Cary, NC, USA

All Rights Reserved. Produced in the United States of America.

For a hard copy book: No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, or otherwise, without the prior written permission of the publisher, SAS Institute Inc.

For a web download or e-book: Your use of this publication shall be governed by the terms established by the vendor at the time you acquire this publication.

The scanning, uploading, and distribution of this book via the Internet or any other means without the permission of the publisher is illegal and punishable by law. Please purchase only authorized electronic editions and do not participate in or encourage electronic piracy of copyrighted materials. Your support of others' rights is appreciated.

U.S. Government License Rights; Restricted Rights: The Software and its documentation is commercial computer software developed at private expense and is provided with RESTRICTED RIGHTS to the United States Government. Use, duplication, or disclosure of the Software by the United States Government is subject to the license terms of this Agreement pursuant to, as applicable, FAR 12.212, DFAR 227.7202-1(a), DFAR 227.7202-3(a), and DFAR 227.7202-4, and, to the extent required under U.S. federal law, the minimum restricted rights as set out in FAR 52.227-19 (DEC 2007). If FAR 52.227-19 is applicable, this provision serves as notice under clause (c) thereof and no other notice is required to be affixed to the Software or documentation. The Government's rights in Software and documentation shall be only those set forth in this Agreement.

SAS Institute Inc., SAS Campus Drive, Cary, NC 27513-2414

November 2025

SAS® and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.

Other brand and product names are trademarks of their respective companies.

9.4-P11:ds2pg

Contents

<i>What's New in SAS DS2 Programmer's Guide</i>	<i>vii</i>
---	------------

PART 1 Introduction 1

Chapter 1 / Introduction to the DS2 Language	3
Introduction to the DS2 Language	3
Running DS2 Programs	4
Supported Data Sources	5
Intended Audience	6
When to Use DS2	7
Converting DATA Step Programs to DS2 Programs	7
Syntax Conventions for the DS2 Language	8

PART 2 DS2 Concepts

Chapter 2 / DS2 Programming Semantics	13
Basic DS2 Program Syntax	13
Basic DS2 Program Semantics	14
Scope of DS2 Identifiers	17
Chapter 3 / DS2 Variables	23
Introduction to DS2 Variables	23
Variable Declaration	24
Variable Lists	28
Variable Scope	35
Predefined DS2 Variables	36
Chapter 4 / DS2 Constants	39
Definition of a Constant	39
Numeric Constants	39
Character Constants	41
Binary Constants	44
Date and Time Constants	45
Constant List	46
Chapter 5 / DS2 Data Types	47
What Are the Data Types?	47
Data Type Characteristics	51

Define Data Types for a Column	55
Error Messages That Use the DOUBLE and REAL Data Types	56
Chapter 6 / DS2 Identifiers	57
Overview of Identifiers	57
Regular Identifiers	58
Delimited Identifiers	58
Support for Non-Latin Characters	60
Chapter 7 / How DS2 Processes Nulls and SAS Missing Values	61
DS2 Modes for Nonexistent Data	61
Differences between Processing Null Values and SAS Missing Values	63
Reading and Writing Nonexistent Data in ANSI Mode	65
Reading and Writing Nonexistent Data in SAS Mode	66
Testing for Null Values	67
Testing for Missing Values	68
Chapter 8 / DS2 Type Conversions	69
Type Conversion Definitions	69
Overview of Type Conversions	70
Type Conversion for Unary Expressions	71
Type Conversion for Logical Expressions	71
Type Conversion for Arithmetic Expressions	72
Type Conversion for Relational Expressions	73
Type Conversion for Concatenation Expressions	74
Type Conversions and Ambiguous Method Calls	74
Chapter 9 / DS2 Expressions	77
What Is an Expression?	77
Types of Expressions	78
Operators in Expressions	95
Chapter 10 / Dates and Times in DS2	101
DS2 Dates, Times, and Timestamps	101
SAS Date, Time, and Datetime Values	103
Converting SAS Date, Time, and Datetime Values to a DS2 Date, Time, or Timestamp Value	104
Converting DS2 Date, Time, and Timestamp Values to SAS Date, Time, or Datetime Values	105
Date, Time, and Datetime Functions	106
Date, Time, and Datetime Formats	107
Chapter 11 / DS2 Arrays	113
Overview of DS2 Arrays	113
Temporary Arrays	114
Variable Arrays	115
Declaring Arrays with a HAVING Clause	120
Array Assignment	122
Array Arguments	125
How to Query Array Dimensions	128
How to Write Array Content	129

Chapter 12 / DS2 Packages	131
Introduction to DS2 Packages	131
Packages, Scope, and Lifetime	134
Dot Operator in Packages	139
Package Constructors and Destructors	140
User-Defined Packages	140
Predefined DS2 Packages	143
Chapter 13 / Threaded Processing	195
Overview of Threaded Processing	195
Threading and DS2 Programs	197
Automatic Variables That Are Useful in DS2 Threading	199
Threaded Processing and the SAS In-Database Code Accelerator	199
Chapter 14 / SAS In-Database Code Accelerator	201
Overview of the SAS In-Database Code Accelerator	202
Requirements for Using the SAS In-Database Code Accelerator	202
SAS In-Database Code Accelerator for Greenplum	203
SAS In-Database Code Accelerator for Hadoop	204
SAS In-Database Code Accelerator for Teradata	209
Using the DS2ACCEL Option to Control In-Database Processing	209
BY-Group Processing When Running Thread Programs inside the Database	210
Considerations and Limitations	210
SAS In-Database Code Accelerator Examples	215
Chapter 15 / Using DS2 and FedSQL	223
Dynamically Executing FedSQL Statements from DS2	223
Chapter 16 / DS2 Input and Output	225
Overview of DS2 Input and Output	225
Reading Data Using the SET Statement	226
Reading Data Using the Hash Package	230
Reading and Writing Data Using the SQLSTMT Package and the SQLEXEC Function	231
Writing Data Using the OUTPUT Statement	231
Column Order in Output Tables When Using Data Sources Outside SAS	232
NLS Transcoding Failures	232
Chapter 17 / Combining Tables	235
Definitions for Combining Data	235
Combining Tables: Basic Concepts	236
Combining DS2 Tables: Methods	246
Chapter 18 / Reserved Words	267
Reserved Words in the DS2 Language	267

PART 3 DS2 and CAS

Chapter 19 / DS2 in CAS: Concepts	273
How DS2 Runs in CAS	273

BY-Group Processing in CAS	281
Integer Type Conversion in the CAS Server	282
DS2 Logging in the CAS Server	283
DS2 Session Encoding in the CAS Server	283
Chapter 20 / Running DS2 Programs in CAS	285
How to Run DS2 Programs in CAS	285
DS2 Program Walk-Through	287
Chapter 21 / DS2 and DATA Step Model Publishing and Scoring in CAS	291
Introduction to DS2 and DATA Step Model Publishing and Scoring in CAS	291
Using PROC SCOREACCEL to Publish, Run, and Delete Models in CAS	294
Using DS2 Actions to Publish, Run, and Delete Models in CAS	295
Using DS2 Actions to Copy to, Publish from, Run, and Delete Models in an External Data Source	297
 PART 4 Appendixes	
Appendix 1 / DS2 Type Conversions for Expression Operands	305
DS2 Automatic Type Conversions for Expression Operands	305
Appendix 2 / DS2 Loggers	307
Overview of DS2 Loggers	307
Configuration Loggers	308
Run-Time Loggers	308
HTTP Package Logger	310
JSON Package Logger	310
Example: Logging All SQL Operations	312
Example: Logging Trace Variables	313
Appendix 3 / Resolving DS2 Errors	317
Types of DS2 Errors	317
How to Resolve DS2 Errors	317
DS2 Loggers	318

What's New in SAS DS2 Programmer's Guide

Overview

DS2 is a SAS proprietary programming language that is appropriate for advanced data manipulation. DS2 is included with Base SAS and SAS Viya and intersects with the SAS DATA step. It also includes additional data types, ANSI SQL types, programming structure elements, and user-defined methods and packages.

Changes and enhancements for SAS 9.4 and SAS Viya releases are detailed in the following sections.

DS2 Loggers

Beginning in February 2021, a new DS2 runtime logger is available on [SAS Viya 3.5](#). The logger, `App.TableServices.DS2.RunTime.TraceVariables`, enables you to trace variable activity in DATA, PACKAGE, and THREAD program blocks. The logger is used with the TRACEVARIABLES option, which identifies the program blocks that are to be examined. For more information, see [“Run-Time Loggers” on page 308](#) and [“Example: Logging Trace Variables” on page 313](#).

SCOREACCEL Procedure

In [SAS 9.4M6](#), a new procedure, SCOREACCEL, provides an interface to the CAS server for DATA step and DS2 model publishing and scoring. Models can be published and run in CAS or in Hadoop or Teradata.

SAS Viya 3.4 has the following changes and enhancements:

- Arguments to delete models from an external data source.
- An option in the RunModel statement to run your scoring model with either MapReduce or Spark.
- Arguments to provide an analytic store without a DS2 model program as input to the publishModel action. The DS2 model program is then generated in CAS by the publishModel action.

In SAS Viya 3.3, a new procedure, SCOREACCEL, provides an interface to the CAS server for DATA step and DS2 model publishing and scoring. Models can be published and run in CAS or in Hadoop or Teradata.

DSTODS2 Procedure

In SAS 9.4M5, a new procedure, DSTODS2, enables you to translate a subset of your SAS DATA step code into DS2 code. Then, if necessary, you can revise your program to take advantage of DS2 features and submit your program using PROC DS2.

Predefined DS2 Packages

The SAS Viya 3.5 release has the following new features and enhancements:

- The documentation about the scope of package instances has been updated. The THIS and *package-instance* arguments of the _NEW_ operator have been deprecated and removed from the _NEW_ operator for packages. The lifetime of a package instance now depends on whether a DS2 package variable references it. The lifetime of a package instance can extend beyond the scope in which the instance itself was created.

In SAS 9.4M5, two new predefined packages are available, PCRXFIND and PCRXREPLACE, for regular expression matching and substitution. These packages are based on the PCRE 2 open-source regular expression library. The DS2 PRX functions have been deprecated because PCRXFIND and PCRXREPLACE provide superior performance to the existing PRX functions.

General Enhancements

In the [SAS 9.4M9](#) release, SAS In-Database Code Accelerator always runs as a Spark application, for UNIX and Windows. You do not have to specify `HADOOPPLATFORM=SPARK`, because it is the default. MapReduce is no longer supported. Changes might be required in configuration and usage. For more information, see [“Overview of the SAS In-Database Code Accelerator” on page 202](#).

In the [SAS 9.4M7](#) release, `GROUP`, `INDSNUM`, `OF`, `ORDER`, and `PARTITION` have been added to the list of DS2 reserved words.

The [SAS Viya 3.5](#) release has the following new features and enhancements:

- Because some DS2 error messages that are written to the SAS log now contain an internal error code, an appendix that provides more information about the codes has been added to the documentation. If you contact SAS Technical Support, these internal error codes can help SAS more quickly determine the source of the problem.
- A DS2 program that runs in CAS can create a column with a VARBINARY data type, read a VARBINARY column from a CAS table, and write a VARBINARY column to a CAS table. In addition, a DS2 program that runs in CAS can create a column with a BINARY data type. However, it cannot read a BINARY data column from a CAS table or write a BINARY column to a CAS table.
- If you use in-database processing with the SAS Embedded Process on Teradata, note that the `SASTransform` table function has been deprecated. The `SASTblOp` table operator should be used with Teradata 14.10 or later.
- The ability to read and write to MongoDB and Salesforce nonrelational databases with DS2 is added when appropriate SAS/ACCESS software is installed. The functionality is available when accessing the databases through a SAS library and a CAS library. For information about data type support in a SAS library, see “Data Types for MongoDB” and also “Data Types for Reading from Salesforce” and “Data Types for Writing to Salesforce” in [“Data Type Reference” in SAS DS2 Language Reference](#). For information about data type support in a caslib, see [“Data Types for CAS” in SAS DS2 Language Reference](#). For general information about working with the databases and details about connecting with a SAS library, see information about the databases in [SAS/ACCESS for Nonrelational Databases: Reference](#). For information about assigning caslibs, see [“Data Connectors” in SAS Cloud Analytic Services: User’s Guide](#).
- Support for the VARBINARY data type when reading and writing to CAS tables.
- There are improvements to data type support for various third-party relational databases when appropriate SAS/ACCESS software is installed. These features are available when the databases are accessed through a SAS library. This functionality is available in SAS Viya and in SAS 9.4M6.

- ❑ DS2 now creates VARCHAR columns containing more than 65,535 characters as type STRING in Hive.
- ❑ DS2 now reads and writes Teradata NUMBER columns.
- ❑ DS2 now reads JSON columns in MySQL.

In the [April 2019](#) release of SAS 9.4M6, the MongoDB and Salesforce non-relational databases are supported as data sources. Access is Read-only, and must be made through a SAS library. Appropriate SAS/ACCESS software must be installed.

[SAS 9.4M6](#) has the following new features and enhancements:

- The SAS In-Database Code Accelerator can be executed as either a MapReduce job or as a Spark application. A new system option, HADOOPPLATFORM, determines which execution platform is used. However, in SAS 9.4M6, the HADOOPPLATFORM=SPARK option is not supported on the Windows operating system for the SAS In-Database Code Accelerator.
- User-written DS2 methods that return a DOUBLE value now return the specified missing value. Previously, the regular SAS missing value (a period) was returned.
- CEDA processing of SPD Engine input files is supported by the SAS In-Database Code Accelerator for Hadoop.
- SQL queries using a WHERE IN clause are now supported by the SAS In-Database Code Accelerator for Hadoop.
- New methods are available for the HTTP package that enable you to specify a URL or proxy URL and a user name and password for those URLs. You can either specify an Open Authorization (OAuth) token or search for one in the SAS environment.
- A RETAIN option has been added to the MERGE statement that produces a Cartesian product on a many-to-many match merge that is similar to a DATA step merge.
- The DS2 language supports these new data sources: Spark as well as databases (such as PostgreSQL) that are compliant with JDBC.
- The SAS In-Database Code Accelerator for Hadoop supports the SCRATCH_DB option for a Hive database that is used when a temporary table is created.

[SAS Viya 3.4](#) has the following new features and enhancements:

- New methods are available for the HTTP package that enable you to specify a URL or proxy URL and a user name and password for those URLs. You can either specify an Open Authorization (OAuth) token or search for one in the SAS environment.
- A RETAIN option has been added to the MERGE statement that produces a Cartesian product on a many-to-many match merge that is similar to a DATA step merge.
- A DS2 action enables you to delete models from an external data source.
- An option in the runModelExternal action enables you to run your scoring model with either MapReduce or Spark.

- DS2 action options can provide an analytic store without a DS2 model program as input to the publishModel action. The DS2 model program is generated in CAS by the publishModel action.
- The DS2 language supports these new data sources: Spark as well as databases (such as PostgreSQL) that are compliant with JDBC.

SAS Viya 3.3 has the following changes and enhancements:

- You can use new DS2 actions to publish and run DATA step and DS2 models in CAS or in Hadoop or Teradata. Alternatively, you can use the new SCOREACCEL procedure from the SAS client.
- The SAS In-Database Code Accelerator for Hadoop supports the SCRATCH_DB option for a Hive database that is used when a temporary table is created.
- DS2 supports BIGINT (INT64) and INTEGER (INT32) as well as CHAR, DOUBLE, and VARCHAR data types in the CAS server. Columns that are defined as SMALLINT and TINYINT data types in CAS are now created as INTEGER instead of DOUBLE.

SAS 9.4M5 has the following changes and enhancements:

- When a variable is used but not declared, a warning is sent to the SAS log. The warning now indicates the data type, length, and, in some cases, precision, that is assigned to the undeclared variable.
- In logical operations, a missing value in any expression, such as an IF expression, evaluates to False in SAS mode. A null value evaluates to neither true nor false in ANSI mode.
- Methods with in_out parameters can have return values.
- Methods in thread programs now allow packages to be passed in as parameters.
- When the length of the target field of the \$QUOTE format is not large enough to contain the string and its quotation marks, SAS returns as much of the string that fits into the field starting from the right most character taking into account two characters for the double quotation marks instead of blanks.

Documentation Enhancements

For SAS 9.4M8, the documentation is updated to clarify DS2 handling of hexadecimal values. For more information, see [“Binary Constants” on page 44](#).

In August 2021, the documentation was enhanced to describe how DS2 loggers might be used to resolve problems. For more information, see [Appendix 3, “Resolving DS2 Errors,” on page 317](#).

SAS 9.4M6 has the following changes and enhancements:

- Documentation was added for using brackets ([[]]) for long string constants (NCHAR, NVARCHAR, and VARCHAR data types) during assignment. This is similar to the way that the LUA programming language delimits strings.
- Documentation for IBM BigInsights and Pivotal has been removed. IBM BigInsights and Pivotal are no longer supported as Hadoop vendors. IBM and Pivotal asked their customers to deploy Hortonworks.

In SAS 9.4M5, the DS2 language concepts have been moved from the *SAS DS2 Language Reference* into this document. In addition, to provide a more comprehensive user experience, information for using DS2 with the CAS server has been incorporated into this document.

PART 1

Introduction

Chapter 1

<i>Introduction to the DS2 Language</i>	3
---	----------

Introduction to the DS2 Language

<i>Introduction to the DS2 Language</i>	3
<i>Running DS2 Programs</i>	4
<i>Supported Data Sources</i>	5
<i>Intended Audience</i>	6
<i>When to Use DS2</i>	7
<i>Converting DATA Step Programs to DS2 Programs</i>	7
<i>Syntax Conventions for the DS2 Language</i>	8
Typographic Conventions	8
Syntax Conventions	8

Introduction to the DS2 Language

DS2 is a new SAS proprietary programming language that is appropriate for advanced data manipulation. DS2 is included with Base SAS and SAS Viya and intersects with the SAS DATA step. It also includes additional data types, ANSI SQL types, programming structure elements, and user-defined methods and packages.

Several DS2 language elements accept embedded FedSQL syntax, and the runtime-generated queries can exchange data interactively between DS2 and any supported database. This allows SQL preprocessing of input tables, which effectively combines the power of the two languages.

In addition, DATA step logic can be transformed to run in environments where DS2 is supported and the DATA step is not. These environments include the following:

- SAS Federation Server
- SAS LASR Analytic Server
- SAS Embedded Process
- SAS Enterprise Miner

- SAS Decision Services

The DS2 procedure enables you to submit DS2 language statements from the SAS windowing environment or SAS Studio. For more information about PROC DS2, see [Base SAS Procedures Guide](#).

Note: Because the DS2 language can be used with many data sources, the terms row, column, and table are used to describe the data elements. Comparing this to SAS DATA step terminology, a row corresponds to an observation, a column corresponds to a variable, and a table corresponds to a data set.

Running DS2 Programs

You can submit DS2 programs in one of the following ways.

- Through the SAS windowing environment or SAS Studio using the DS2 procedure. The DS2 procedure can be used to run DS2 code in Base SAS, SAS Viya, or on the CAS server. A single PROC DS2 step can contain several DS2 programs.

For more information, see “[DS2 Procedure](#)” in [Base SAS Procedures Guide](#).

- In SAS Studio using the runDS2 action on the CAS server. The runDS2 action is used in conjunction with the CAS procedure.

Note: Unless you are using Python or Lua, it is recommended that you use PROC DS2 to submit DS2 code to the CAS server.

For more information, see “[DS2 Action Set](#)” in [SAS Viya: System Programming Guide](#) and [Chapter 19, “DS2 in CAS: Concepts,”](#) on page 273.

- Directly to a data source using the SAS In-Database Code Accelerator.

For more information about using the SAS In-Database Code Accelerator, see [SAS In-Database Products: User's Guide](#).

- Directly to the SAS Federation Server using the SAS LIBNAME engine for SAS Federation Server.

Note: Some of these execution methods might require additional software licenses. For example, accessing any relational database management system (RDBMS) from Base SAS requires the appropriate SAS/ACCESS software license.

Supported Data Sources

DS2 can access the following data sources:

- Amazon Redshift¹
- Aster
- CAS tables¹
- DB2 for UNIX* and Windows operating environments
- Greenplum
- Hadoop (Hive¹ and HDMD¹)
- Google BigQuery on Linux x64¹
- Impala¹
- databases¹ that are compliant with JDBC (such as PostgreSQL)
- Memory Data Store (MDS)
- Microsoft SQL Server¹
- MongoDB on Linux for x64^{1 2}
- MySQL (Beginning with SAS Viya 3.5, includes MySQL 8 Server)
- Netezza
- databases¹ that are compliant with ODBC (such as Microsoft SQL Server)
- Oracle¹
- PostgreSQL¹
- Salesforce on Linux for x64¹
- SAP (Read-only)
- SAP HANA¹
- SAP IQ
- SAS data sets¹
- SAS Scalable Performance Data Engine (SPD Engine) data set¹
- SAS Scalable Performance Data Server (SPD Server) tables in UNIX and Windows operating environments
- Snowflake on Linux x64¹
- Spark
- Teradata for UNIX¹ and Windows operating environments
- Vertica

- Yellowbrick

Note: The following data sources are not supported:

- Informix
 - OLEDB
 - SAP ASE
-

¹These data sources are supported on the CAS Server.

²The MongoDB data source has special requirements for creating tables. Depending on your data, you might want to create these tables: a root table, a parent table, and a child table. You can create only root tables with DS2. To create parent and child tables, you must use FedSQL. See the information about MongoDB in [SAS/ACCESS for Relational Databases: Reference](#).

The DS2 procedure and SAS Federation Server support different data sources. See *Base SAS Procedures Guide* and *SAS Federation Server: Administrator's Guide* for information about the data sources that each one supports.

For information about accessing the data sources, see [SAS/ACCESS for Relational Databases: Reference](#) and [SAS/ACCESS for Nonrelational Databases: Reference](#), as appropriate.

Intended Audience

The information in this document is intended for the following users who perform in these roles:

- **Application developers** who write the client applications. They write applications that create tables, bulk load tables, manipulate tables, and query data.
- **Database administrators** who design and implement the client/server environment. They administer the data by designing the databases and setting up the data source metadata. That is, database administrators build the data model.
- **SAS programmers** who want or need to take advantage of the advanced features of the DS2 language such as increased numeric precision, parallel computation for CPU-bound processes, using explicit pass-through SQL queries as direct input to a DATA step process, or in-database processing in big data environments.
- **Data analysts** who want to push SAS processing into big-data domains using the scoring and code accelerators.

When to Use DS2

Typically, DS2 programs are written for applications that carry out the following actions:

- require the precision that results from using the new supported data types
- benefit from using the new expressions or write methods or packages available in the DS2 syntax
- need to execute SAS FedSQL from within the DS2 program
- execute outside a SAS session, for example, in-database processing on Hadoop or Teradata, in SAS Viya, or the SAS Federation Server
- take advantage of threaded processing in products such as the SAS In-Database Code Accelerator and SAS Enterprise Miner

Converting DATA Step Programs to DS2 Programs

In [SAS 9.4M5](#), you can use the DSTODS2 procedure to translate the DATA step code into DS2. Not all DATA step code is supported for translation, and some manual translation might be required. Code lines that cannot be translated are placed in comments. For more information, see the [“DSTODS2 Procedure” in Base SAS Procedures Guide](#).

After you have converted the DATA step code to DS2 and your program is syntactically complete, you can use the DS2 procedure to run your program from within SAS or SAS Viya, or you can use the HPDS2 procedure to run your program on the High-Performance Analytic Server distributed computing environment.

Syntax Conventions for the DS2 Language

Typographic Conventions

Type styles have special meanings when used in the documentation of the DS2 language syntax.

UPPERCASE BOLD

identifies DS2 keywords such the names of statements and functions (for example, PUT).

UPPERCASE ROMAN

identifies arguments and values that are literals (for example, FROM).

italic

identifies arguments or values that you supply. Items in italic can represent user-supplied values that are either one of the following.

- nonliteral values assigned to an argument (for example, ALTER=*alter-password*).
- nonliteral arguments (for example, KEEP=(*column-list*).

If more than one of an item in italics can be used, the items are expressed as *item* [, ...*item*].

`monospace`

identifies examples of SAS code.

Syntax Conventions

This documentation uses the Backus-Naur Form (BNF), specifically the same syntax notation used by Jim Melton in *SQL:1999 Understanding Relational Language Components*.

The main difference between traditional SAS syntax and the syntax that is used in the DS2 language reference documentation is in how optional syntax arguments are displayed. In traditional SAS syntax, angle brackets (< >) are used to denote optional syntax. In DS2 language syntax, square brackets ([]) are used to denote optional syntax and angle brackets are used to denote non-terminal components.

The following symbols are used in the DS2 language syntax.

::=

This symbol can be interpreted as “consists of” or “is defined as”.

<>

Angle brackets identify a non-terminal component (that is, a syntax component that can be further resolved into lower level syntax grammar).

[]

Square brackets identify optional arguments. Any argument that is not enclosed in square brackets is a required argument. Do not enter square brackets unless they are preceded by a backward slash (\), which denotes that they are literal.

{ }

Braces provide a method to distinguish required multi-word arguments. Do not enter braces unless they are preceded by a backward slash (\), which denotes that they are literal.

|

A vertical bar indicates that you can choose one value from a group. Values that are separated by bars are mutually exclusive.

...

An ellipsis indicates that the argument or group of arguments that follow the ellipsis can be repeated any number of times. If the ellipsis and the following arguments are enclosed in square brackets, they are optional.

\

A backward slash indicates that the next character is a literal.

The following examples illustrate the syntax conventions that are described in this section. These examples contain selected syntax elements, not the complete syntax.

```
1 SET 2 <table-reference> [... [<table-reference>] [INDSNAME=variable];
    [ 3 BY [DESCENDING] 4 column [ 5 ... [DESCENDING] column];
    6 <table-reference>::=
    {table (table-options)} 7 | 8 \{sql-text 8 \}
```

- 1 SET is in uppercase bold because it is the name of the statement.
- 2 <table-reference> is in angle brackets because it is a non-terminal argument that is further resolved into lower level syntax grammar. You must supply at least one <table-reference>.
- 3 BY and DESCENDING are in uppercase roman because they are literal arguments. DESCENDING is in square brackets because it is an optional argument.
- 4 *column* is in italics because it is an argument that you can supply.
- 5 The square brackets and ellipsis around the second instance of *column* indicate that you can repeat this argument any number of times as long as the arguments are separated by commas.
- 6 The <table-reference>::= non-terminal argument syntax is read as follows: A <table-reference> consists of a table name and table options or embedded SQL text.
- 7 The vertical bar (|) indicates you can supply either *table* [table-options] or *sql-text*, but not both.

- 8 The backslash (\) before the braces around *sql-text* indicate that those braces are literals and must be entered.

PART 2

DS2 Concepts

Chapter 2	
DS2 Programming Semantics	13
Chapter 3	
DS2 Variables	23
Chapter 4	
DS2 Constants	39
Chapter 5	
DS2 Data Types	47
Chapter 6	
DS2 Identifiers	57
Chapter 7	
How DS2 Processes Nulls and SAS Missing Values	61
Chapter 8	
DS2 Type Conversions	69
Chapter 9	
DS2 Expressions	77
Chapter 10	
Dates and Times in DS2	101
Chapter 11	
DS2 Arrays	113
Chapter 12	
DS2 Packages	131
Chapter 13	
Threaded Processing	195
Chapter 14	
SAS In-Database Code Accelerator	201

Chapter 15	
<i>Using DS2 and FedSQL</i>	223
Chapter 16	
<i>DS2 Input and Output</i>	225
Chapter 17	
<i>Combining Tables</i>	235
Chapter 18	
<i>Reserved Words</i>	267

DS2 Programming Semantics

Basic DS2 Program Syntax	13
Basic DS2 Program Semantics	14
Variable Declaration Statements	14
Methods	15
Scope of DS2 Identifiers	17
Programming Blocks	17
Variable Lookup	18
Definition of Scope	19
Variable Lifetime	20

Basic DS2 Program Syntax

A DS2 program consists of a list of declarations followed by a list of method statements. Here is an example of a simple declare list:

```
declare int x;
dcl double d;
```

Here is an example of a simple method statement list:

```
method init()
    put 'in init';
end;

method run();
    put 'in run';
end;

method term();
    put 'in term';
end;
```

Combining the two lists creates a simple DS2 program.

```
declare int x;
declare double d;
```

```

method init();
end;

method run();
end;

method term();
end;

```

Although a DS2 program is typically more complex, this simple program contains several syntactic elements:

Keywords:

- DECLARE/DCL
- DOUBLE
- METHOD
- END

Identifiers:

- x
- d
- INIT
- RUN
- TERM

Lexical Separators:

- (
-)
- ;

The program illustrates how to declare an identifier, either in a DECLARE statement or in a METHOD statement. It also illustrates how the high-level structure of a DS2 program consists of a sequence of variable declarations followed by a sequence of METHOD statements. The next section explains what these terms mean in DS2.

Basic DS2 Program Semantics

Variable Declaration Statements

A variable declaration allocates memory space and identifies that memory with an identifier, called the variable name. The declaration, either explicitly or implicitly, allocates memory for the variable and designates the type of data that can be

saved at that memory location. In DS2, you declare variables by using the DECLARE statement. A DECLARE statement performs the following actions:

- assigns an identifier to a memory location. That identifier becomes the variable name.
- designates the type of data that the variable can hold.
- allocates a specified amount of memory to the variable.

More than one variable can be declared in one DECLARE statement. For more information, see the [“DECLARE Statement” in SAS DS2 Language Reference](#).

Methods

Methods are basic program execution units. In DS2, the method structure is used to group related program statements in one syntactically identifiable location. The group of statements in the method can then be easily invoked, or executed, multiple times.

DS2 methods are similar to functions, in languages such as C, and methods in Java. In traditional DATA step programming, the LINK and RETURN statements were used similarly to method definitions in DS2 DATA programs. However, because LINK and RETURN cannot create variables with local scope, they are much less useful for delivering reusable code modules. The FCMP procedure enables you to create reusable functions with local scoping, and it is the equivalent of a DS2 method in traditional Base SAS programming.

A method defines a scoping block. Therefore, any parameters and any variable declarations in the method body are local to the method.

TIP DS2 methods can support up to 1000 arguments. A DS2 method that has more than 1000 arguments can generate a compilation error.

A **type signature**, or simply **signature**, is defined to be the ordered list of the method's parameter types. If any two method definitions have the same name, but different type signatures, the method is **overloaded**. An error occurs if two method definitions have the same name and same type signature.

Note: You cannot overload a method based on CHAR and NCHAR data types alone if session encoding requires multiple bytes. If a session encoding requires multiple bytes per character (for example, UTF-8), then CHAR and NCHAR are identical types and both use NCHAR. Consequently, the two method definitions would be seen as the same.

There are two types of methods in DS2: *system* methods, and *user-defined* methods.

A DS2 program must contain at least one of the following three system methods:

```
method init();
```

```

    end;
method run();
    end;
method term();
    end;

```

These methods are meant to provide a more structured framework than the SAS DATA Step implicit loop concept. In Base SAS, the entire DATA Step program was included in the implicit loop. In DS2, the implicit loop is represented by the RUN method, with the INIT and TERM methods providing initialization and finalization code, respectively.

When a DS2 program executes, here are the results:

- 1 The INIT method runs. Any initializations take place.
- 2 Variables in the program data vector that have not been retained are set to the appropriate missing values. For more information, see the [“RETAIN Statement” in SAS DS2 Language Reference](#).
- 3 The RUN method executes.
- 4 Execution control then depends on the status of any input statement in the RUN method. Currently, the only input statement in DS2 is the SET statement. If the RUN method meets one of these conditions, then processing proceeds to Step 5. Otherwise, processing proceeds to Step 2 so that the RUN method can execute again:
 - No input statements
 - An input statement that has completed execution
- 5 The TERM method executes, and any final statements execute.

The INIT, RUN, and TERM methods must be defined without any parameters and without a return value. If you specify a parameter for the INIT, RUN, or TERM methods, an error occurs.

If you do not specify an OUTPUT statement in the DS2 program, the DS2 compiler provides one with no parameters that executes at the end of the RUN method. If you have not written a RUN method, one is created implicitly by the DS2 compiler. Therefore, the minimum DS2 program form is a single declaration within a DATA program as follows:

```

data;
dcl double x;
enddata;

```

This code is functionally equivalent to the following two sets of code:

```

data;
dcl double x;
method run();
    end;
enddata;

```

```

data;
dcl double x;
method run();
    output;

```

```
end;
enddata;
```

Each set of code produces a single, uninitialized value of x in the output window.

If you attempt to call the INIT, RUN, or TERM method directly from a DS2 program, an error occurs.

User-defined methods can be created by enclosing statements that you would like executed one or more times within METHOD and END statements. For more information about user-defined methods, see the [“METHOD Statement” in SAS DS2 Language Reference](#).

Note: When using PROC DS2, DS2 programs are delimited by RUN statements. If additional DS2 code is found after a RUN statement, this code composes a new, distinct DS2 program from the DS2 program before the previous RUN statement.

Scope of DS2 Identifiers

Programming Blocks

A programming block is a section of code that begins and ends with an ordered pair of keywords. The following keywords create programming blocks:

- DATA...ENDDATA
- PACKAGE...ENDPACKAGE
- THREAD...ENDTHREAD
- DO...END
- METHOD...END

In this documentation, these terms are used for programming blocks.

- A data programming block or **data program** refers to code bounded by DATA...ENDDATA statements.
- A package programming block or **package** refers to the stored library of variables and methods bounded by PACKAGE...ENDPACKAGE statements. The variables and methods of a package can be used by DS2 programs, threads, or other packages.
- A thread programming block, or **thread program**, refers to a stored program that is bounded by the THREAD...ENDTHREAD statements. The thread program can be called by the SET FROM statement in a DS2 program or package.
- A DO programming block, or **DO loop**, refers to a subblock of programming statements that are bounded by the DO and END statements.

- A method programming block or **method block** refers to a subblock of programming statements that are bounded by the METHOD and END statements.

Some blocks can be nested. In this example, there is one data program, defined by the DATA and ENDDATA statements, and three nested method blocks, defined by the three method statements.

```
data _null_;
  declare int x;

  method init();
    declare double d;
  end;

  method run();
  end;

  method term();
  end;
enddata;
```

A variable declared in the outermost programming block is called a **global** variable, or a variable having **global scope**. A variable declared in any nested block is called a **local** variable, or a variable having scope that is local to that block. DS2 also assigns global scope to undeclared variables. In the preceding example, X is a global variable, and D is a variable that is local to the nested INIT method.

Note: A DS2 program can have multiple subprograms followed by an optional data program. The following restrictions apply:

- There can be only one data program and the data program must be the last subprogram.
 - The ENDPACKAGE, ENDTHREAD, or ENDDATA statements are optional for the last subprogram of the DS2 program but are recommended for good programming form. These statements are required for all other subprograms.
-

Variable Lookup

When a variable is referenced, DS2 always searches for the variable's declaration beginning in the block of the reference. Then, if it is not found there, it searches successively in any outer containing blocks or program. In this example, any reference to X in the INIT method refers to the global declaration of X.

```
declare int x;

method init();
end;
```

Because methods are blocks, they can contain declarations themselves. In this example, any reference to X in the INIT method refers to the local declaration of X, but any reference to X in the RUN method refers to the global declaration of X.

```
declare int x;

method init();
    declare int x;
end;

method run();
end;
```

Definition of Scope

Scope can be considered an attribute of identifiers. Identifiers can refer to a number of program entities: method names, functions, data names, labels, or program variables. This section uses program variables as examples, but any identifier is subject to scoping rules.

Scope describes where in a program a variable can be accessed. Global variables have global scope and are accessible from anywhere in the program. Local variables have local scope and are accessible only from within the program or block in which the variable was declared.

In DS2, a variable is accessible only as long as program execution is taking place within the scope of the variable. That is, the values of variables are accessible only when a statement in the scope of the variable is actively executing.

In the following example, the variable X is in scope only while the INIT method is executing. Neither the RUN or the TERM methods can refer to it.

```
data;
    declare int x;    /* global x in global scope */

    method init();
        x = 5;        /* global x assigned 5 */
    end;

    method run();
    end;

    method term();
    end;

enddata;
```

Each variable in any given scope must have a unique name, but variables in different scopes can have the same name. When scopes are nested, if a variable in an outer scope has the same name as a variable in an inner scope, the variable within the outer scope is hidden by the variable within the inner scope. For example, in the following program two different variables share the same name, X. Global variable X has global scope, and local variable X has local scope. Within the

local scope of method INIT, local variable X hides global variable X. Therefore, the assignment statement assigns 5 to local variable X.

```
data;
  declare int x;    /* global x in global scope */

  method init();
    declare int x;  /* local x in local scope */

    x = 5;          /* local x assigned 5 */
  end;

  method run();
  end;

  method term();
  end;

enddata;
```

Variable Lifetime

The lifetime of a variable is the time during which the variable exists. Global variables exist for the duration of the program. Local variables exist for the duration of the block in which the variable was declared. The value of a global variable is set to a missing or null value before entry into the RUN method unless that global variable appears within a RETAIN statement in the current program block.

In the following example, the variable X exists only while the INIT method is executing and the variable Y exists for the duration of the data program.

```
data;
  declare double y;

  method init();
    declare double x;
  end;

enddata;
```

During a variable's lifetime, it can be overshadowed by a locally declared variable of the same name, as in this example:

```
declare int x;

method init();
  declare int x;
end;

method run();
end;
```


Although the global variable X has lifetime for the entire program, it is not directly accessible from the INIT method in this example because of the local declaration of X in the INIT method.

DS2 Variables

Introduction to DS2 Variables	23
Variable Declaration	24
How to Declare a Variable	24
Controlling How DS2 Handles an Undeclared Variable	27
Variable Lists	28
Overview of DS2 Variable Lists	28
Types of Variable Lists	29
The OF Operator with Variable Lists	31
Expansion of Variable Lists	32
Creating Named Variable Lists	33
Unnamed Variable Lists	34
Passing Variable List Arguments	34
Variable Scope	35
Global Variables	35
Local Variables	35
Global and Local Variables in DS2 Output	35
Example of Global and Local Variables	36
Predefined DS2 Variables	36
Predefined Method Variables	36
DS2 Thread Variables	38

Introduction to DS2 Variables

The properties of DS2 program variables are that they have a name, a scope, and a data type.

A name, or identifier, is one or more tokens, or symbols, that is given to a variable. Names are discussed in [Chapter 6, “DS2 Identifiers,” on page 57](#).

Variables can have either global or local scope depending on where the variable is declared. For more information, see [“Variable Scope” on page 35](#).

Variable data types are assigned either implicitly or explicitly depending on how they are declared. For more information, see [“Variable Declaration” on page 24](#).

Note: The term “data type” includes any data type attributes such as precision, character set encoding, and length. For complete information about data types, see [Chapter 5, “DS2 Data Types,” on page 47](#).

Variable Declaration

How to Declare a Variable

There are three ways to declare a variable and its data type:

- Explicit declaration by using the DECLARE statement

The DECLARE statement associates a data type with each variable in a variable list or an array. If the DECLARE statement is used outside a method, a global variable is created. If the DECLARE statement is used within a method, a local variable is created. Within a method, DECLARE statements must precede method statements. Otherwise, an error occurs.

Variables that are explicitly declared are assigned a **Variable** attribute.

For more information, see [“Variable Scope” on page 35](#) and [“DECLARE Statement” in SAS DS2 Language Reference](#).

- Implicit declaration by using a SET or MERGE statement

The SET or MERGE statement reads the column information for each specified table. For each column in each table, the SET or MERGE statement creates a global variable in the DS2 program with the same data types as those of the column.

Variables that are implicitly declared are assigned a **Label** attribute.

Here is an example.

```
proc ds2;
data input1 (overwrite=yes);
    dcl double x y z;
    method init();
        x=1; y=2; z=3;
    output;
end;
enddata;
run;
quit;

proc ds2;
```

```

data results (overwrite=yes);
    method run();
        set input1;
    end;
enddata;
run;
quit;

proc compare data=input1 compare=results error note criterion=1e-4;
run;
quit;

```

The following output from PROC COMPARE shows that the columns of table input1 have a variable attribute and the columns of table results have a label attribute.

Figure 3.1 PROC COMPARE OUTPUT

Listing of Common Variables with Differing Attributes					
Variable	Dataset	Type	Length	Label	
x	WORK.INPUT1	Num	8		
	WORK.RESULTS	Num	8	x	
y	WORK.INPUT1	Num	8		
	WORK.RESULTS	Num	8	y	
z	WORK.INPUT1	Num	8		
	WORK.RESULTS	Num	8	z	

For more information, see [“Reading Data Using the SET Statement” on page 226](#), [“MERGE Statement” in SAS DS2 Language Reference](#), and [“SET Statement” in SAS DS2 Language Reference](#).

- Implicit declaration by using an undeclared variable in a programming block

Note: It is strongly recommended that you explicitly declare every variable. By doing so, you can avoid data type mismatches among data sources.

If you use a variable without declaring it, DS2 assigns the variable a data type:

- The data type for an undeclared variable on the left side of an assignment statement is determined by the data type of the value on the right side of the assignment statement. If the data type of the value on the right side of the assignment statement is numeric or NULL, then type DOUBLE is assigned to the left side variable. Otherwise, the data type of the value on the right side is assigned to the left side variable.

Note: If the right side of the assignment statement is an expression, the length of the left side variable is the resultant length of the right side expression. In this example, the length of variable b is 9, the result of adding the string lengths of x and y.

```

x='abc';
y='rstuvw';

```

```
b=x || y;
```

Note: Implicitly declared character types are assigned the session encoding for the variables character set.

- The data type for an undeclared variable on the right side of an assignment statement is DOUBLE.

By default, a warning is sent to the SAS log. The warning indicates the data type, length, and, where appropriate, precision, that is assigned to the undeclared variable. However, you can use the DS2SCOND system option or the PROC DS2 SCOND option to issue an error or note or ignore the undeclared variable. For more information, see [“Controlling How DS2 Handles an Undeclared Variable” on page 27](#).

The following example shows how to define variables explicitly and implicitly, or by assignment. FIRSTNAME and LASTNAME are declared explicitly as variables with the data type of CHAR(20). PNUM is declared implicitly by assignment. As you can see by the warning written to the log, DS2 assigns PNUM a data type based on the value on the right side of the assignment, which is CHAR(11):

```
data phonenums;
  dcl char(20) firstName lastname;
  method init();
    firstName='Sam';
    lastName='Alesski';
    pnum = '19192223454';
    ... more DS2 statements ...
  end;
enddata;
```

```
WARNING: Line 19: No DECLARE for assigned-to variable pnum; creating it as a
global variable of type char(11).
```

You need to be aware of type conversions and missing values when assignments and expressions are used. In this example, the equality operand is evaluated with a DOUBLE operand and a VARCHAR operand. The DOUBLE operand takes precedence over the VARCHAR operand and consequently the VARCHAR operand is converted to a DOUBLE. Because the string, 'on1', is not a valid numeric value, the conversion results in the numeric SAS missing value for the right-hand side operand. Because the variable `in2` was not initialized by the program, `in2` is also set to a SAS missing value. In SAS mode, missing = missing for DOUBLE data types evaluates to true.

```
data _null_;
  dcl varchar(100) "results";
  dcl varchar(100) "in1";

  method run();
    "in1"='onlystring2';

    /* Mistype in2 instead of in1 */
    if ("in2"='on1') then do;
      "results" = 'Rule is fired AGAIN';
    end;
```

```

        put 'here are the results';
        put "results";
    end;
enddata;
run;

```

The following lines are written to the SAS log.

```

here are the results
Rule is fired AGAIN
WARNING: Line 65: No DECLARE for referenced variable in2; creating it as a global
variable of type double.

```

Controlling How DS2 Handles an Undeclared Variable

To control how DS2 handles an undeclared variable, you can use the DS2SCOND system option or the SCOND option on the DS2 procedure:

Table 3.1 Settings to Control Variable Declaration

DS2SCOND/ SCOND Setting	Effect on Variable Declaration
WARNING	Declaration by assignment occurs. Warning messages are written to the SAS log. This is the default behavior.
NONE	Declaration by assignment occurs. No messages are written to the SAS log.
NOTE	Declaration by assignment occurs. A note is written to the SAS log.
ERROR	Declaration by assignment does not occur. An error message is written to the SAS log. This is also known as variable declaration strict mode.

For more information, see the “[DS2SCOND= System Option](#)” in *SAS DS2 Language Reference* and “[DS2 Procedure](#)” in *Base SAS Procedures Guide*.

Variable Lists

Overview of DS2 Variable Lists

A DS2 variable list is a collection of DS2 variables. Many DS2 statements and table options use variable lists for specification of sets of variables. For example, the KEEP, DROP, and VARARRAY statements can use variable lists for the specification of the set of variables to keep, drop, or reference. Here are some examples:

```
keep name address city state zip phone;
drop rep1-rep5;
vararray int grades[*] assignment: quiz: exam;;
```

A variable list can also provide a convenient mechanism for specifying variables of interest to a DS2 method or a package.

DS2 supports the following forms of variable lists.

- name variable list
- numbered range variable list
- name range variable list
- name prefix variable list
- type variable list
- special name variable list

For more information, see [“Types of Variable Lists” on page 29](#).

The different forms of variable lists can be mixed within a single variable list. For example, the following is a valid variable list.

```
u x1-x3 u:
```

Assuming the program data vector illustrated below, the above variable list would expand to u x1 x2 x3 u u1 u2. Note that a single variable can be referenced multiple times in a variable list expansion.

PDV	u	v	w	x1	x2	x3	x4	u1	u2
-----	---	---	---	----	----	----	----	----	----

Types of Variable Lists

Name Variable Lists

A name variable list is simply a list of variable names.

```
location date pressure temperature
```

Numbered Range Variable Lists

A numbered range variable list expands to reference global variables with a specified prefix and a numeric suffix ranging between two specified numbers. The numbered range variable list has the following syntax.

*prefix**n1-prefixn2*

n1 and *n2* represent the beginning and ending of the range, inclusive. Variable names are constructed during the numbered range list expansion by concatenating the prefix with each number in the numbered range.

For example, the numbered range variable list *x1-x5* expands to *x1 x2 x3 x4 x5*. The numbered range variable list *score10-score5* expands to *score10 score9 score8 score7 score6 score5*.

Name Range Variable Lists

A name range variable list expands to reference all global variables whose variable definition occurred between the definition of two specified variables, inclusive. The name range variable list has the following syntax.

var1--var2

sales_jan--sales_mar is an example.

DS2 maintains a seen list of defined variables. As variables are defined, the variables are added to the seen list in their order of definition. For example, consider the following statements.

```
declare double reg1_id reg2_id reg1_rev reg2_rev;
declare double reg1_exp reg2_exp total_rev total_exp;
keep reg1_rev--reg2_exp;
```

The seen list after executing the DECLARE statements would be *reg1_id reg2_id reg1_rev reg2_rev reg1_exp reg2_exp total_rev total_exp*. Therefore, the variable list *reg1_rev--reg2_exp* in the KEEP statement expands to *reg1_rev reg2_rev reg1_exp reg2_exp*.

Name range variable lists rely on the order of variable definition, that is, `x--a` includes all variables in order of variable definition from `x` to `a` inclusive.

Name Prefix Variable Lists

A name prefix variable list expands to reference all global variables that begin with a specified prefix. The name prefix variable list has the following syntax.

prefix:

An example of a name prefix variable list is `sales:`, which expands to all variables whose names begin with “sales”, such as `sales_jan`, `sales_feb`, and `sales_mar`.

Type Variable Lists

A type variable list expands to reference all global variables of a specified type. The type variable list has the following syntax.

data-type:

An example of a type variable list is `smallint int`, which expands to all variables of type SMALLINT or INT. The types that are supported by type variable lists are these.

- TINYINT
- SMALLINT
- INTEGER
- BIGINT
- REAL
- FLOAT (matches DOUBLE and FLOAT)
- DOUBLE (matches DOUBLE and FLOAT)
- BINARY (matches BINARY and VARBINARY)
- CHAR (matches CHAR, VARCHAR, and CHARACTER)
- NCHAR (matches NCHAR and NVARCHAR)
- CHARACTER (matches CHAR, VARCHAR, and CHARACTER)
- DATE
- TIME
- TIMESTAMP

Special Name Variable Lists

A special name variable list expands to reference a specific group of Read-only, global variables. The `_ALL_` variable is supported. `_ALL_` references all global variables in the DS2 program.

For more information, see [“Predefined DS2 Variables” on page 36](#).

The OF Operator with Variable Lists

The OF operator precedes the arguments of DS2 functions that can take variable lists and arrays as arguments. Here is the general syntax for DS2 functions that use the OF operator:

```
function-name(OF variable-list) | (OF array-name[*])
```

The OF syntax for a function can include general expressions for arguments where each `zi` is a regular expression or an OF list operator. In other words, you can mix regular expressions, several OF lists, and so on.

```
y = f(z1, z2, ..., zn);
```

For more information, see [“Using the OF Operator with Arrays” in SAS DS2 Language Reference](#).

Note: There are some cases where the DS2 OF operator gives different results from the DATA step OF operator when used with some functions. This is because of how the variable list is processed in each language. The DATA step takes only the variables up to the point of the function call. DS2 takes all the variables that match across the whole program. Here is an example where the SUM function in the DATA step program uses only the variables A1 and A4. The SUM function in the DS2 program uses the variables A1, A4, and A2.

```
/* DATA step program */
/* sums A1 and A4 with a result of 4 */

data _null_;
  A1 = 1.0;
  A4 = 3.0;
  y = 1;
  lab:
  if (y = 2) then
    X = SUM(OF A: );
    A2 = 2.0;
  if (y = 1) then do;
    y = 2;
    goto lab;
  end;
  put X=;
;
```

```

run;

/* DS2 program */
/* sums A1, A4, and A2 with a result of 6*/

proc ds2;
data X (overwrite=yes);
  dcl double x A1 A2 A4 y;
  method run();
    A1 = 1.0;
    A4 = 3.0;
    y = 1;
  lab:
    if (y = 2) then
      X = SUM(OF A: );
      A2 = 2.0;
    if (y = 1) then do;
      y = 2;
    goto lab;
    end;
  put X=;
end;
enddata;
run;
quit;

```

Expansion of Variable Lists

Variable lists expand to reference global scalar variables that match the variable list type. Local variables are never included in a variable list expansion.

In the following example, variable **x** is a global variable and variable **y** is local to method **INIT**. The **x:** name prefix variable list in the **KEEP** statement expands to reference only variable **x1**. Local variable **x2** is not included in the variable list expansion. Therefore, variable **x2** is not written to the table **example**.

```

data example;
  declare double x1;
  keep x:;

  method init();
    declare double x2;
  end;
enddata;
run;

```

Variable list expansion considers all global variables regardless of where the variable is defined in the program. In the following example, **_ALL_** in the **KEEP** statement expands to reference global variables **x1**, **x2**, and **x3**. Therefore, variables **x1**, **x2**, and **x3** are written to table **example**.

```

data example;
  declare double x1;
  keep [_all_];
  declare double x2;

  method init();
    x3=17;
  end;
enddata;
run;

```

Creating Named Variable Lists

The VARLIST statement creates a named variable list.

Here is an example where a VARLIST is created. The INIT method is used to initialize the variable list. In the RUN method, four variables are dropped (nc1, nc2, cty1, and cty2).

```

data april;
  dcl double nc1 nc2 nc3;
  dcl char(8) cty1 cty2 cty3;
  varlist cost [nc1-nc3 cty1--cty3];

  method init();
    nc1=-377.90;
    nc2=-2922.78;
    nc3=98333.56;
    cty1='Raleigh';
    cty2='Durham';
    cty3='Roxboro';
  end;
enddata;
run;

data aprilloss (overwrite=yes drop=(nc1-nc2 cty1-cty2));
  method run();
    set april;
  end;
enddata; run;

proc print data=aprilloss;
run;

```

Note: The VARLIST statement is limited to the global scope of the DS2 package or program. The VARLIST statement cannot be used to create a local variable list.

For more information, see the “[VARLIST Statement](#)” in *SAS DS2 Language Reference* and “[Passing Variable List Arguments](#)” on page 34.

Unnamed Variable Lists

DS2 provides a mechanism for specification of an anonymous, or unnamed, variable list as part of an expression of another statement. The unnamed variable list has the following syntax.

[variable-list]

An example of an unnamed variable list is `compute ([x y z]) ;`.

In this method expression, an unnamed variable list referencing variables **x**, **y**, and **z** is created and passed as the single argument to the `compute` method.

If unnamed variable list syntax is used in an expression, then an unnamed variable list is created in the global scope referencing the variables in *variable-list*. An unnamed variable list is inaccessible from statements other than the statement in which the unnamed variable list was specified. Therefore, it cannot be reused in other expressions.

Passing Variable List Arguments

A DS2 variable list can be passed as an argument to DS2 methods or DS2 functions that accept a variable list argument. A variable list can also be used in the `VARARRAY` statement. The DS2 V* variable functions (`VNAME`, for example) are functions that accept a variable list argument. Several of the `SQLSTMT` package methods accept a variable list argument.

DS2 variable lists are always passed by reference and cannot be passed by value. A DS2 variable list argument can either be a named variable list that is created with a `VARLIST` statement, or it can be an anonymous variable list. The values of DS2 variables in a variable list cannot be extracted by the DS2 code. Only the metadata describing the variable can be extracted.

Use the following syntax to specify a variable list parameter for a DS2 method.

VARLIST *parameter-name*

Here is an example.

```
data _null_;
  vararray double x[5];
  dcl bigint xyz;
  dcl date xanadu;

  method printNames(varlist v);
    dcl varchar(100) name type;
    dcl bigint i;
    do i = 1 to dim(v);
      name = vname(v[i]);
      type = vtype(v[i]);
      put name= type=;
```

```
        end;  
    end;  
  
    method init();  
        printNames([x:]);  
    end;  
enddata;  
run;
```

Variable Scope

Global Variables

A variable with global scope, a global variable, is declared in one of three ways: in the outermost programming block of a DS2 program, using a DECLARE statement, implicitly declared inside a programming block using a SET statement, or implicitly declared inside a programming block by using an undeclared variable. Variables with global scope can be accessed from anywhere in the program and exist for the duration of the program. Global variables can be used in a THIS expression in any program block. For information about declaring DS2 variables, see [“Variable Declaration” on page 24](#). For information about using the THIS expression, see [“THIS Expression” on page 88](#).

Local Variables

A variable with local scope, a local variable, is declared within an inner programming block, such as a method or package, by using a DECLARE statement. Variables with local scope are known only within the inner programming block where they are declared. For more information, see [“Programming Blocks” on page 17](#).

Global and Local Variables in DS2 Output

Only global variables, by default, are included in the output. Local variables that are used for program loops and indexes do not need to be explicitly dropped from the output. Local variables are always created at the start of a method invocation, such as an iteration of the implicit loop of the RUN method, and are destroyed at the end of each invocation. Therefore, it is not recommended to use local variables as accumulator variables in the RUN method.

All global variables are named in the program data vector (PDV). The PDV is the set of values that are written to an output table when DS2 writes a row. For more information about the PDV, see [“Processing a DATA Step: A Walk-through” in SAS Language Reference: Concepts](#).

Example of Global and Local Variables

The following program shows both global (A, B, and TOTAL) and local variables (C):

```
data;
  dcl int a; 1 /* A is a global variable */
  method init();
    dcl int c; 2 /* C is a local variable */
    a = 1; 3
    b = 2; 4 /* B is undeclared so it is global */
    c = a + b;
    this.total = a + b + c; 5
  end;
enddata;
run;
```

- 1 A is a global variable because it is declared in the outermost DS2 program.
- 2 C is a local variable because it is declared inside the method block, METHOD INIT ().
- 3 Because A is a global variable, it can be referenced within the method block, METHOD INIT().
- 4 Because B is not declared in METHOD INIT(), it defaults to being a global variable. DS2 assigns B a data type of DOUBLE. B appears in the PDV and the output table.
- 5 THIS.TOTAL simultaneously declares the variable TOTAL as a global variable with the data type of DOUBLE and assigns a value to it based on the values of A, B, and C.

Predefined DS2 Variables

Predefined Method Variables

Predefined variables are variables that are automatically declared within a method block and discarded when the method is complete. The values of predefined variables are retained from one iteration of a RUN method to the next. These variables are added to the program data vector but are not written to the table

being created. Predefined variables are temporary and are not saved with your data. The predefined variable is available in the method block, and you can use them just like any variable that you declare yourself.

Two predefined variables are created:

_N

is initially set to 0 by the INIT method. Each time the RUN method executes, the value increments by 1. The data type for _N is BIGINT. This is a Read-only variable; you cannot assign a value to _N.

N

is initially set to 1 by the INIT method. Each time the RUN method executes, the value increments by 1. That is, the first time the RUN method executes, the value of _N_ is 1. The value indicates the number of times that the program has looped through the method. The variable can be used to count rows, but it is a good row counter only when one record is read per iteration. The data type for _N_ is BIGINT. You can assign a numeric value to _N_, but in the next iteration of the RUN method, the value reverts to the value assigned by the program

Although the predefined variables are not output variables, you can use the PUT statement with the _ALL_ argument to print the values of predefined variables to the SAS log.

The following simple DS2 program illustrates using the PUT _ALL_ statement to print the values of the _N and _N_ variables to the SAS log.

```
data inp /overwrite=yes;
  dcl double a;
  method init();
    a = 1; output;
  end;
enddata; run;

data;
  method init();
    put 'init' _n=; put _ALL_;
  end;
  method run();
    set inp;
    put 'run' _n=; put _ALL_;
  end;
  method term();
    put 'term' _n=; put _ALL_;
  end;
enddata; run;
```

Example Code 3.1 SAS Log with `_N` and `_N_` Variable Values

```
.  
.   
.   
NOTE: Execution succeeded. One row affected.  
init _n=0  
a=. _n_=1  
run _n=1  
a=1 _n_=1  
term _n=2  
a=1 _n_=2
```

DS2 Thread Variables

The following automatic variables are used for subsetting a problem across DS2 threads. These automatic variables are also useful for providing context when you are debugging with PUT statements.

- `_HOSTNAME_`
- `_NTHREADS_`
- `_THREADID_`

For more information, see [“Automatic Variables That Are Useful in DS2 Threading” on page 199](#).

DS2 Constants

Definition of a Constant	39
Numeric Constants	39
Character Constants	41
Overview of Character Constants	41
Long Character Constants	41
Character Encoding of DS2 Constants	43
Referencing a Macro Variable in a Character Constant	44
Binary Constants	44
Date and Time Constants	45
Constant List	46

Definition of a Constant

A constant is a number, character string, binary number, date, time, or timestamp that indicates a fixed value. Here are some examples of DS2 constants:

```
107
'Trends in Business'
date '2008-01-01'
b'10011001'
x'FFE3546F'
```

Numeric Constants

A numeric constant is a negative or positive numeric value that is either an integer constant or a fractional constant.

integer constant

is a numeric value without a fractional component, that is, a whole number. An integer constant value is stored as an exact numeric. An integer constant without the N or n suffix is a BIGINT, INTEGER, SMALLINT, or TINYINT data type value. An integer constant with the N or n suffix is a DECIMAL or NUMERIC data type value. An integer constant has the following forms where *integer* is a sequence of one or more digits, 0 through 9:

integer

*integer***N|n**

The following values are valid integer constants:

```
0
124
124N
+124n
-124n
```

fractional constant

is a numeric value that has a fractional or decimal component. A fractional constant value is stored as either an exact numeric or an approximate numeric. A fractional constant without the N or n suffix is a REAL or DOUBLE data type value that is stored as an approximate numeric. A fractional constant with the N or n suffix is a DECIMAL or NUMERIC data type value that is stored as an exact numeric up to the maximum precision of the data type. Approximate fractional constants support standard notation or scientific (E) notation. Exact fractional constants support only standard notation. A fractional constant has the following forms where *integer*, *fraction*, and *exponent* is a sequence of one or more digits, 0 through 9:

integer.

integer.**N|n**

integer.fraction

.fraction

*integer.fraction***E|e**[+|-]*exponent*

*integer***E|e**[+|-]*exponent*

*.fraction***E|e**[+|-]*exponent*

*integer.fraction***N|n**

*.fraction***N|n**

The following values are valid fractional constants:

```
0.
124.0
+124.0
-57.33
.5
1E100
99.99e4
-.33e-2
-57.33n
```

```
123456789012345678901234567890.12N
```

Character Constants

Overview of Character Constants

A character constant is a sequence of characters enclosed in single quotation marks and can be written using the following formats:

```
'character-string'
```

```
n'character-string'
```

For character strings that contain national characters, use the NCHAR constant, `n'character-string'`. National characters can take multiple bytes of storage.

Character and NCHAR constants can include a newline for character constants that span multiple lines.

Note: A sequence of characters enclosed in double quotation marks is not a character constant, it is an identifier. For more information, see [“Delimited Identifiers” on page 58](#).

The following character constants are valid:

```
'PHONE'
n'STÄDTE'
'Phone
Number' /* constant that spans more than one line */
```

Note: To have a single quotation mark (') or apostrophe (') in a string, use an additional quotation mark. Here is an example.

```
'Isn''t life beautiful'
```

Long Character Constants

Long string constants (NCHAR, NVARCHAR, and VARCHAR data types) can be enclosed in double brackets ([[]]) during assignment. This is similar to the way in which the LUA programming language delimits strings.

A long character constant begins with [[, [=], [==] (that is, two left brackets with zero or more equal signs between them). A long character constant ends with]],]=],]==] (that is, two right brackets with the same number of equal signs between them). The end token must match the pattern of the beginning token.

IMPORTANT Comments within double brackets are discarded.

CAUTION

Do not put “run;” or “quit;” inside a long character constant that is enclosed in brackets. Putting “run;” or “quit;” inside the long character constant that is enclosed in brackets causes a compilation error, and your program fails.

Here is an example:

```
proc ds2;
data;
  dcl nvarchar(1000) cc;
  method init();
    cc = [====[
      proc ds2;
      data;
      dcl nchar(42) cc;
      method init();
      cc = [= [
        this is a test
      ]=];
      put '|' cc= '|';
      end;
      enddata;
    ]====];
    put '|' cc= '|';
  end;
enddata;
run;
quit;
```

Output 4.1 Output from Constant Brackets Example

cc

```
proc ds2; data; dcl nchar(42) cc; method init(); cc = [= [ this is a test ]=]; put '|' cc= '|'; end; enddata;
```

Note: A long character constant might include newlines, but newlines are not required. This means that n'xyzy' can be replaced with [= [xyzy]=] without affecting the behavior of the program.

CAUTION

When you parse long character constants, if you specify a single bracket (]) next to an end token (]) with or without an equal sign (=) between the brackets and you fail to add a whitespace character, a parse error failure

message is generated. Adding a whitespace character results in successful code execution. However, the content changes.

For example, the statement `str=[==[xyz]]==;` results in a parse error failure because the ending double brackets are consecutive with no whitespace character between the brackets. To avoid this parse error, add a whitespace character, which forces the parser to disregard the initial double-bracket end token, `str=[==[xyz] ]==;`. Adding a whitespace character changes the content of the string. The statement must end with a single bracket that is preceded by an equal sign.

Here are successful examples:

- `[==[foo]]==;` ; The end tokens of `]]` and `]==]` match the initial token, so the first tokens are added to the literal.
- `[==[foo]=]]==;` ; The end tokens of `] =]` and `] ==]` match the initial token, so the first tokens are added to the literal.
- `[==[foo] ]==;` ; The end tokens of `]]` and `] ==]` match the initial token, so the first tokens are added to the literal.

Character Encoding of DS2 Constants

All DS2 character data has an associated character encoding. The same is true of character constants. DS2 character constant values are either encoded with the DS2 session encoding or the DS2 national encoding. National character variables are always assigned UTF-8 as the character encoding regardless of the DS2 session encoding.

To specify a character constant to be encoded using the DS2 session encoding, do not specify a prefix for the constant in the DS2 program text.

```
'Hello World'
'你好世界'
```

To specify a character constant to be encoded using the DS2 national character encoding, prefix the constant with an `n` in the DS2 program text.

```
n'Hello World'
n'你好世界'
```

The encoding of the DS2 program text might differ from the encoding assigned to a character constant during processing of the DS2 program. For example, if the character encoding of the DS2 program text is UTF-8 and the DS2 session encoding is latin-1, the character constant in the following `PUT` statement fails to transcode during processing of the program:

```
put '你好世界';
```

Since the character constant is not specified as a national character constant, the character constant is assigned the DS2 session encoding, latin-1. The latin-1 character set does not support the Chinese characters '你好世界'. Thus, the transcode from UTF-8 (DS2 program text encoding) to latin-1 (DS2 session

encoding) fails. The transcode failure can be eliminated by changing the DS2 session encoding to an encoding that supports Chinese characters (for example, EUC-CN or UTF-8) or by specifying the character constant to be encoded using the DS2 national character encoding by prefixing an n in the DS2 program text.

Referencing a Macro Variable in a Character Constant

To reference a macro variable in a character constant, use the SAS macro function %TSLIT, which overrides the need for single quotation marks around the constant value. For example, the following statement uses the %TSLIT function to specify the macro variable %profit:

```
put %tslit(PROFIT: &profit);
```

The %TSLIT macro function is stored in the default autocall macro library. The syntax is as follows:

%TSLIT(<*literal-text* >);

The %TSLIT macro function references any macro variables in the literal text argument and puts single quotation marks around the resulting text value. This creates a DS2 character constant value.

Binary Constants

Binary constants can be written as either a bit string or as a hexadecimal string using the following formats:

b'*bit-string*

x'*hexadecimal-string*

bit-string is a sequence of binary digits, 0 and 1.

hexadecimal-string is a string of hexadecimal characters, 0 - 9 and A - F.

Both *bit-string* and *hexadecimal-string* must be enclosed in single quotation marks.

DS2 converts both the bit constant and the hexadecimal constant to their binary equivalents for use in the DS2 program. Binary and hexadecimal constants are padded with zeros to a byte boundary.

The following binary constants are valid:

```
b'11100011'
x'FF143E99'
```

When assigning a hexadecimal constant to a character variable, the hexadecimal constant must be in the same encoding as the session encoding for the assignment

to be valid. A text value is encoded into a data value for a column using a character encoding. The bytes used to store any given text value differ depending on the encoding. For example, consider some hexadecimal values of the character string *âgê* in the UTF-8 and Latin-1 character encodings.

Table 4.1 *Latin-1 and UTF-8 Encodings for Character String *âgê**

Character Encoding	Text Data in Hexadecimal
Latin-1	E1 67 EA
UTF-8	C3 A1 67 C3 AA

Specifying the hexadecimal value for the UTF-8 encoding in a Latin-1 session can result in an error or simply an incorrect result. Here is the log output of example DS2 code that submits the appropriate hexadecimal value for a Latin-1 session encoding.

```

1  proc ds2;
2      data _null_;
3          method init();
4              dcl varchar(100) s1;
5              s1 = x'E167EA';
6              put s1=;
7          end;
8      enddata;
9      run;
s1=âgê
NOTE: Execution succeeded. No rows affected.
10  quit;
```

Note: Blank spaces must be removed from the input value.

DS2 does not have a hexadecimal notation for numeric constants.

Date and Time Constants

Date and time constants can be written as date, time, or timestamp strings using the following formats:

date 'yyyy-mm-dd'

time 'hh:mm:ss'

time 'hh:mm:ss.fraction'

timestamp 'yyyy-mm-dd hh:mm:ss'

timestamp 'yyyy-mm-dd hh:mm:ss.fraction'

yyyy is a four-digit year.

mm is a two-digit month.

dd is a two-digit day.

hh is a two-digit hour.

mm is a two-digit minute.

ss is a two-digit second.

fraction is a sequence of numbers that represents a fraction of a second.

All date and time constants must be enclosed in single quotation marks. Year, month, and day must be separated by a hyphen. Hours, minutes, and seconds must be separated by colons. Seconds and fraction of a second must be separated by a period. In the timestamp constant, a space separates the date from the time.

The length of a date constant must be ten characters, which includes the hyphens that separate the year, month, and day.

The length of the time constant without the fraction must be eight characters, which includes the colons that separate the hours, minutes, and seconds. The length of the time constant with the fraction can vary with the fraction of a second.

The length of the timestamp constant without the fraction must be 19 characters, which includes the hyphens and colons to separate the date and time components. The length of the timestamp with the fraction can vary with the fraction of a second.

Here are some examples of date and time constants:

```
date '2008-01-01'
time '11:59:59'
timestamp '2007-09-30 02:33:31.59'
```

Constant List

A constant list is a series of constant values that are enclosed in parentheses, separated by commas or blanks, that can be used in simple array assignments and in IN expressions. A constant list can also contain nested constant lists. An example of a constant list is (2, '33', -5, (1, '3')). In this example, (1, '3') is a nested constant list.

Each constant value or nested list can be prefixed with an iterator. An iterator is a number followed by the asterisk (*). The iterator indicates the number of times to repeat the constant or constant list that follows the asterisk. For example, in the constant list (2, '33', 3*-5, 2*(1, '3')), the list item 3*-5 results in -5, -5, -5, and the list item 2*(1, '3') results in 1, '3', 1, '3'. There is no difference between using the iterator "5*" and repeating the list element five times.

Note: Iterators are unlikely to be useful in an IN expression, as duplicate values in an IN expression could slow processing.

DS2 Data Types

<i>What Are the Data Types?</i>	47
<i>Data Type Characteristics</i>	51
Numeric Data Types	51
Character Data Types	52
Date and Time Data Types	55
Binary Data Types in CAS	55
<i>Define Data Types for a Column</i>	55
<i>Error Messages That Use the DOUBLE and REAL Data Types</i>	56

What Are the Data Types?

A data type is an attribute of every column that specifies what type of data the column stores. The data type is the characteristic that identifies a piece of data as a character string, an integer, a floating-point number, or a date or time. The data type also determines how much memory to allocate for the column's value.

The following table lists the set of data types that are supported by DS2. Note that not all data types are available for table storage on each data source.

Table 5.1 DS2 Data Types

Data Type	Description
BIGINT	stores a large signed, exact whole number, with a precision of 19 digits. The range of integers is -9,223,372,036,854,775,807 to 9,223,372,036,854,775,807. Integer data types do not store decimal values; fractional portions are discarded.
BINARY(<i>n</i>)	stores fixed-length binary data, where <i>n</i> is the maximum number of bytes to store. The maximum number of bytes

Data Type	Description
	is required to store each value regardless of the actual size of the value.
CHAR(<i>n</i>)	stores a fixed-length character string, where <i>n</i> is the maximum number of characters to store. The maximum number of characters is required to store each value regardless of the actual size of the value. If <code>char(10)</code> is specified and the character string is only five characters long, the value is right padded with spaces.
DATE	stores a calendar date. A date literal is specified in the format <code>yyyy-mm-dd</code>: a four-digit year (0001 to 9999), a two-digit month (01 to 12), and a two-digit day (01 to 31). For example, the date September 24, 1975 is specified as <code>1975-09-24</code>. DS2 complies with ANSI SQL:1999 standards regarding dates. However, not all data sources support the full range of dates. For example, dates between 0001-01-01 and 1582-12-31 are not valid dates for a SAS data set or an SPD data set.
DECIMAL NUMERIC(<i>p,s</i>)	stores a signed, exact, fixed-point decimal number, with user-specified precision and scale. The precision and scale determines the position of the decimal point. The precision is the maximum number of digits that can be stored to the left and right of the decimal point, with a range of 1 to 52. The scale is the maximum number of digits that can be stored following the decimal point. Scale must be less than or equal to the precision. For example, <code>decimal(9,2)</code> stores decimal numbers up to nine digits, with a two-digit fixed-point fractional portion, such as 1234567.89.
DOUBLE	stores a signed, approximate, double-precision, floating-point number. Allows numbers of large magnitude and permits computations that require many digits of precision to the right of the decimal point.
FLOAT	stores a signed, approximate, double-precision, floating-point number. Data defined as FLOAT is treated the same as DOUBLE.
INTEGER	stores a regular size signed, exact whole number, with a precision of ten digits. The range of integers is -2,147,483,647 to 2,147,483,647. Integer data types do not store decimal values; fractional portions are discarded.

Data Type	Description
	Note: Integer division by zero does not produce the same result on all operating systems. It is recommended that you avoid integer division by zero.
NCHAR(<i>n</i>)	stores a fixed-length character string like CHAR but uses a Unicode national character set, where <i>n</i> is the maximum number of multibyte characters to store. Depending on the platform, Unicode characters use either two or four bytes per character and support all international characters.
NVARCHAR(<i>n</i>)	stores a varying-length character string like VARCHAR but uses a Unicode national character set, where <i>n</i> is the maximum number of multibyte characters to store. Depending on the platform, Unicode characters use either two or four bytes per character and can support all international characters.
REAL	stores a signed, approximate, single-precision, floating-point number.
SMALLINT	stores a small signed, exact whole number, with a precision of five digits. The range of integers is -32,767 to 32,767. Integer data types do not store decimal values; fractional portions are discarded.
TIME(<i>p</i>)	stores a time value. A time literal is specified in the format <i>hh:mm:ss[.nnnnnnnnnn]</i> ; a two-digit hour 00 to 23, a two-digit minute 00 to 59, and a two-digit second, with an optional fraction value. For example, the time 6:30 a.m. is specified as 06:30:00. When supported by a data source, the <i>p</i> parameter specifies the seconds precision. <i>p</i> is an optional fraction value that is up to nine digits long.
TIMESTAMP(<i>p</i>)	stores both date and time values. A timestamp literal is specified in the format <i>yyyy-mm-dd hh:mm:ss[.nnnnnnnnnn]</i> ; a four-digit year 0001 to 9999, a two-digit month 01 to 12, a two-digit day 01 to 31, a two-digit hour 00 to 23, a two-digit minute 00 to 59, and a two-digit second, with an optional fraction value. For example, the date and time September 24, 1975 6:30 a.m. is specified as 1975-09-24 06:30:00. When supported by a data source, the <i>p</i> parameter specifies the seconds precision. , <i>p</i> is an optional fraction value that is up to nine digits long.
TINYINT	stores a very small signed, exact whole number, with a precision of three digits. The range of integers is -127 to

Data Type	Description
	127. Integer data types do not store decimal values; fractional portions are discarded.
VARBINARY(<i>n</i>)	stores varying-length binary data, where <i>n</i> is the maximum number of bytes to store. The maximum number of bytes is not required to store each value. If varbinary (10) is specified and the binary string uses only five bytes, only five bytes are stored in the column.
VARCHAR(<i>n</i>)	stores a varying-length character string, where <i>n</i> is the maximum number of characters to store. The maximum number of characters is not required to store each value. If varchar (10) is specified and the character string is only five characters long, only five characters are stored in the column.

The following program is an example of declaration of many of the data types available in DS2. Note that you must be connected to a data source that supports these data types for the program to run successfully.

```
data ds2data (overwrite=yes);
  dcl int          v01;
  dcl bigint       v02;
  dcl smallint     v03;
  dcl tinyint      v04;
  dcl double       v05;
  dcl char(8)      v06;
  dcl varchar(8)   v07;
  dcl nchar(8)     v08;
  dcl nvarchar(8)  v09;
  dcl date         v10;
  dcl time         v11;
  dcl timestamp    v12;
  dcl binary(8)    v13;
  dcl varbinary(32) v14;
  dcl decimal(5,2) v15;

method init();
  v01 = 1;
  v02 = 1;
  v03 = 1;
  v04 = 1;
  v05 = 1.0;
  v06 = 'aa';
  v07 = 'aa';
  v08 = 'aa';
  v09 = 'aa';
  v10 = date'2001-01-01';
  v11 = time'01:01:01';
  v12 = timestamp'2001-01-01 01:01:01';
  v13 = x'0123456789ABCDEF';
```

```

v14 = x'0123456789ABCDEF';
v15 = 32.23n;
output;
end;
enddata;
run;

```

You can use a PROC FEDSQL DESCRIBE TABLE statement to produce a summary of the contents of the ds2data table.

```

CREATE TABLE DS2DATA (
    "v01"    INTEGER,
    "v02"    BIGINT,
    "v03"    INTEGER,
    "v04"    INTEGER,
    "v05"    DOUBLE PRECISION,
    "v06"    WCHAR(8),
    "v07"    WVARCHAR(8),
    "v08"    WCHAR(8),
    "v09"    WVARCHAR(8),
    "v10"    DATE,
    "v11"    TIME(0),
    "v12"    TIMESTAMP(4),
    "v13"    BINARY(8),
    "v14"    VARBINARY(32),
    "v15"    NUMERIC
);

```

Data Type Characteristics

Numeric Data Types

Numeric data types store numbers (for example, quantities and currency values). The choice of a numeric data type depends on the type of number that is being stored and on how the number is used.

Characteristics to consider when choosing a numeric data type include the following:

- whether you want to use exact numeric data types or approximate numeric data types. Exact numeric data types such as BIGINT, DECIMAL, INTEGER, NUMERIC, and TINYINT, represent a value exactly. Approximate numeric data types, such as REAL, DOUBLE, and FLOAT, do not store the exact values that are specified for many numbers. For many applications, the tiny difference between the specified value and the stored approximation is not noticeable unless exact numeric behavior is required.
- whether you want to store whole numbers or decimal numbers. Note that for integer data types, if a decimal value is inserted, the fractional portion is discarded. For example, if 2.7 is inserted, the stored value is 2.

- how to store numbers with decimal points. Floating-point format is a number in which the decimal point is not fixed. A floating-point data type is used for longer decimal values and for quickly calculating a large range of numbers. For the floating-point data types, if you enter an integer that is too large, an error occurs. Enter the value using floating-point notation. Fixed point is a method for storing a number in which the decimal points line up. The fixed-point data type DECIMAL is used for exact precision of fractional components, such as money.
- a data type's precision. The precision for a data type specifies the total number of digits that a number can contain. The more digits, the higher the precision for the value. If a value is out of range, an error occurs. For example, for the BIGINT data type, an error occurs for a value that is larger than 9,223,372,036,854,775,807.

To avoid errors or incorrect results, you must consider the results when performing operations on numeric values, particularly for the integer data types. The INTEGER data type has a precision of 10 digits and a range from – 2,147,483,647 to 2,147,483,647. If you multiply two large integers (for example, 33432*79879) and the result is larger than 2,147,483,647, an overflow error occurs. If you expect a result that is larger than the data type's precision or range, you can assign the result as a larger data type such as DOUBLE or BIGINT, or you can enter the expression as a double constant (for example, 33432.0*79879.0), not an integer, in order to force the expression to be evaluated as a double precision, floating-point number.

A specific example of when you must consider results is when performing division in DS2. In the following example, 80/100 performs integer division and the result is 0. Floating-point division of 80.0/100 results in .8.

```
data;
  dcl double x y;
  method run();
    /* this gives an integer value result of 0 */
    x=80/100;
    put x=;

    /* this gives the result of .8 */
    y=80.0/100;
    put y=;
  end;
enddata;
run;
```

Character Data Types

Overview of Character Data Types

DS2 provides several character data types that store character string (text) data. Character data types can contain alphabetic characters, numeric digits 0 through 9, and other special characters.

Note: If a character string includes a number, DS2 automatically converts it to a numeric type and uses that number in any calculation.

Each character data type provides a parameter for specifying the maximum number of characters.

For fixed-length data types (CHAR and NCHAR) the character length is the exact number of characters of text data. For the DATA step or in the CAS server, the character length is the exact number of bytes of text data.

For varying-length data types (NVARCHAR and VARCHAR), the character length is the maximum allowed number of characters of text data for DS2, the DATA step, or in the CAS server.

For example, assume a fixed-length character column with a length of 10 and UTF-8 encoding. A Base SAS or CAS table can store up to 10 bytes of text data. DS2 interprets the length of 10 as 10 characters. The DS2 column can store any ten UTF-8 characters including ten 4-byte characters, which requires 40 bytes of storage. If the ten 4-byte characters are written to a SAS data set or a CAS table, a truncation error occurs because the 10 characters exceed the 10 bytes of storage allotted to the SAS or CAS table for the data.

Character Data Types and Encoding

All DS2 character data has an associated character encoding. Although DS2 supports processing data in multiple character encodings in the same program, each character variable has one associated character encoding that is fixed for the duration of the DS2 program. Data that is assigned to a character variable either through a SET statement or an Assignment statement is transcoded to the variable's character encoding if that data is in a different character encoding.

The character encoding for a character variable can be specified with the CHARACTER SET option of a DECLARE statement. Consider the following statements:

```
declare varchar(1024) character set utf8 s1;
declare varchar(1024) character set latin1 s2;
declare varchar(1024) s3;
declare nvarchar(1024) s4;
```

Character variable s1 is declared with character encoding UTF-8. Character variable s2 is declared to with character encoding latin-1. Character variable s3 is declared without an explicit character encoding. Therefore, DS2 assigns the DS2 session encoding as the character encoding for variable s3. When executed by using PROC DS2, the DS2 session encoding is the SAS session encoding. If the DS2 session encoding is EUC-CN, character variable s3 is assigned character encoding EUC-CN. National character variables are a special case that do not support the CHARACTER SET option with a DECLARE statement. National character variables are always assigned UTF-8 as the character encoding regardless of the DS2 session encoding. Therefore, national character variable s4 is assigned UTF-8 as its character encoding.

An NCHAR[*n*] declaration is equivalent to CHARACTER[*n*] CHARACTER SET UTF-8, and an NVARCHAR[*n*] declaration is equivalent to CHARACTER VARYING[*n*] CHARACTER SET UTF-8.

Character variables that are implicitly declared by a SET or MERGE statement are assigned the encoding of the corresponding column in the data source. Character variables that are implicitly declared by a statement other than the SET or MERGE statement are assigned the DS2 session encoding for the variable's character encoding.

Note: If the output data source does not support multiple encodings such as a SAS data set, then the different encodings are distilled to a common encoding for the table.

Default encoding depends on your operating system and locale. For a complete list of character set encoding values, see “Character Sets for Encoding in NLS” in *SAS National Language Support (NLS): Reference Guide*.

Limitations and Considerations of Character Data Types

Be aware of the following limitations and considerations with character variables:

- Columns with a data type of CHAR, VARCHAR, NCHAR, and NVARCHAR have a length restriction of 10485760 characters for any column that is read in or generated. The characters can be multibyte characters (for example, UTF-8).
- Trailing blanks are ignored in comparison operations for both CHAR and VARCHAR data types. Trailing blanks are not ignored during assignment, concatenation, output, as arguments passed to methods, and so on, for both CHAR and VARCHAR. Trailing blanks are not ignored when copying. If a VARCHAR(10) value 'abc' is copied to a VARCHAR(20), the value is copied including trailing blanks.
- The ENCODING= system option is not supported when you are reading or updating SAS data sets. DS2 converts a given ENCODING= value to the least common denominator encoding when multiple encoding values are specified with the CHARACTER SET syntax. The ENCODING= value that is used can differ from what really is created in the data set. This conflict can cause potential transcoding errors.
- DS2 follows ANSI SQL standards for data casting. For example, if you assign a variable with a CHAR data type to a new variable with a VARCHAR data type, the contents of the CHAR variable are transferred to the VARCHAR variable if it fits. Here is an example.

```
dcl char(8) u;
dcl varchar(6) v;
method run();
u='eighteen';
v=u;
```

After the assignment, the resulting value of `v` is 'eighte' and has length of six. If the contents of the CHAR variable do not fit, it is truncated regardless of whether it is a whitespace character. If you do not want to copy trailing whitespace characters from a CHAR value stored in variable during assignment of the value to a VARCHAR variable, use the TRIM function to eliminate the trailing whitespace characters from the value before the assignment.

Date and Time Data Types

DS2 supports several data types for the specific purpose of storing dates and times. The date and time ranges are data-source specific.

Binary Data Types in CAS

Beginning with SAS Viya 3.5, a DS2 program that runs in CAS can create a column with a VARBINARY data type, read a VARBINARY column from a CAS table, and write a VARBINARY column to a CAS table. Before SAS Viya 3.5, DS2 could not create a VARBINARY column in a CAS table. In addition, no error was reported when a VARBINARY column was read. However, the data that was read was incorrectly treated as character data.

A DS2 program that runs in CAS can create a column with a BINARY data type. However, it cannot read a BINARY data column from a CAS table or write a BINARY column to a CAS table. If a DS2 program reads a BINARY column from a CAS table or writes a BINARY column to a CAS table, an error occurs. To prevent this error, drop the BINARY column before reading from or writing to the CAS table. Before SAS Viya 3.5, no error was reported when reading a BINARY column. However, the data that was read was incorrectly treated as character data.

Define Data Types for a Column

When defining a data type, use the data type keywords for the data types supported by DS2.

Keep in mind that for data to be stored, the data type must be available for data storage in that data source. Although DS2 supports several data types, the data types that can be defined for a particular table depend on the data source, because each data source does not necessarily support all of the DS2 data types. In addition, data sources support variations of the standard SQL data types. That is, a specific data type that you specify might map to a different data type and might also have different attributes in the underlying data source. This occurs when a data source does not natively support a specific data type, but data values of a

similar data type can be converted without data loss. For example, to support the INTEGER data type, a SAS data set maps the data type definition to SAS numeric, which is a DOUBLE.

For details about data source implementation for each data type, see [“Data Type Reference” in SAS DS2 Language Reference](#).

In addition, the CT_PRESERVE= connection argument, which controls how data types are mapped, can affect whether a data type can be defined. The values FORCE (default) and FORCE_COL_SIZE do not affect whether a data type can be defined. The values STRICT and SAFE can result in an error if the requested data type is not native to the data source or the specified precision or scale is not within the data source range. For information about the CT_PRESERVE= connection argument, see *SAS Federation Server: Administrator's Guide*.

Error Messages That Use the DOUBLE and REAL Data Types

When error messages contain numeric values for DOUBLE and REAL data types, these values are written to the log using the BESTw. format. The BESTw. format is the default format for writing numeric values. When you use BESTw., SAS chooses the format that provides the most information about the value according to the available field width.

Because SAS uses the BESTw. format, the DOUBLE and REAL data type values might not be exactly the same as the values that you use in your programs. BESTw. rounds the value, and if SAS can display at least one significant digit in the decimal portion, within the width specified, BESTw. produces the result as a decimal. Otherwise, it produces the result in scientific notation. SAS always stores the complete value regardless of the format that you use to represent it.

DS2 Identifiers

<i>Overview of Identifiers</i>	57
<i>Regular Identifiers</i>	58
<i>Delimited Identifiers</i>	58
<i>Support for Non-Latin Characters</i>	60

Overview of Identifiers

An identifier is one or more tokens, or symbols, that name programming language entities, such as variables, method names, package names, and arrays, as well as data source objects, such as table names and column names.

The DS2 language supports ANSI SQL:1999 standards for both regular and delimited identifiers.

Regular identifiers are the type of identifiers that you see in most programming languages. They are **not case-sensitive** so that the identifier `Name` is the same as `NAME` and `name`. Only certain characters are allowed in regular identifiers.

Delimited identifiers are case-sensitive only for identifiers that require double quotation marks, that is, table and schema names. Other delimited identifiers are not case-sensitive. Delimited identifiers allow any character and must be enclosed in double quotation marks. Variable names `"Name"`, `"NAME"`, `Name`, `name`, and `NaMe` all represent the same variable.

By supporting ANSI SQL:1999 identifiers, the DS2 language is compatible with data sources that also support the ANSI SQL:1999 identifiers.

Note: Identifiers for SAS and SPD data sets are limited to 32 characters.

Regular Identifiers

When you name regular identifiers, use these rules:

- The length of a regular identifier can be 1 to 255 characters.
- The first character of a regular identifier must be a letter or underscore. Subsequent characters can be letters, digits, or underscores. For example, a forward slash or parenthesis is not allowed in an identifier unless that identifier is quoted. Note that you cannot begin a regular identifier with two underscores.
- Regular identifiers are case-insensitive.

The following regular identifiers are valid:

```
firstName  
lastName  
_phonenum  
phone_num1
```

Letters in regular identifiers are stored internally as uppercase letters, which allows letters to be written in any case. For example, an unquoted, input column name of `phone_num1`, `Phone_Num1`, or `PHONE_NUM1` is displayed in the output as uppercase, `PHONE_NUM1`. Identifiers that do not contain special characters or spaces and are enclosed in double quotation marks are treated as regular identifiers as if the quotation marks were not present.

Note: Each data source has its own naming conventions, all of which are supported by the DS2 language. When your program contains identifiers specific for a particular data source, you must follow the naming conventions for that data source. For more information, see the topic on naming conventions for your data source in *SAS/ACCESS for Relational Databases: Reference*.

Delimited Identifiers

When you name delimited identifiers, follow these rules:

- Begin and end delimited identifiers with double quotation marks.
- The length of a delimited identifier can be 1 to 253 characters. (Two of the 255 maximum characters are reserved for the quotation marks.)
- Delimited identifiers consist of any sequence of characters, including spaces and special characters, between the beginning and ending double quotation marks.

- The only delimited identifiers that are case-sensitive are table and schema names. Other delimited identifiers are not case-sensitive.
- Special characters in delimited identifiers must be enclosed in matching double quotation marks in addition to the beginning and ending double quotation marks that are required for the identifier.
- Do not use a forward slash (/) in a delimited identifier. This can cause errors in some operating environments. For example, "/" means subdirectory in UNIX systems.

A string of characters enclosed in double quotation marks is interpreted as an identifier and not as a character constant. Character constants can be enclosed only in single quotation marks.

The following delimited identifiers are valid:

```
"x&y&z"
"Ü1"
"(area)phone_num"
"a**B"
```

You can use delimited identifiers for terms that might otherwise be a reserved word. For example, to use the term "date" other than for a date declaration, you would use it as the delimited identifier "date". Here is an example.

```
/* In DATA step, there are no reserved words for variables. */
/* So, this doesn't cause an error. However, how do you */
/* use such a variable in DS2? */
data a;
    date = mdy(8,14,2012);
run;

/* This program gives an error because the reserved word DATE */
/* is used when a variable is expected. */
proc ds2;
data b(overwrite=yes);
    method run();
        set a;
        if month(date) = 8 then
            put 'August';
        else
            put 'Not August';
        end;
    enddata;
run; quit;

/* One solution is to quote the reserved word */
proc ds2;
data b(overwrite=yes);
    method run();
        set a;
        if month("date") = 8 then
            put 'August';
        else
            put 'Not August';
        end;
    enddata;
```

```

run; quit;

/* Another solution is to rename the variable on input and output */
/* to avoid the reserved word in DS2 code. */
proc ds2;
data b(overwrite=yes rename=(sas_date="date"));
  method run();
    set a(rename=("date"=sas_date));
    if month(sas_date) = 8 then
      put 'August';
    else
      put 'Not August';
  end;
enddata;
run; quit;

```

A warning is issued for tables that are created with delimited column names that are then referenced in DS2 programs that are submitted to data sources that are not case-sensitive. Data sources that are not case-sensitive remove the quotation marks and treat the column name as not delimited.

A delimited identifier is similar to – but not the same as – name literals in the SAS DATA step language.

Note: Each data source has its own naming conventions, all of which are supported by the DS2 language. When your program contains identifiers specific for a particular data source, you must follow the naming conventions for that data source. For more information, see the topic on naming conventions for your data source in *SAS/ACCESS for Relational Databases: Reference*.

CAUTION

Using the `PRESERVE_TAB_NAMES=NO` option in your `LIBNAME` statement can cause unexpected results.

Support for Non-Latin Characters

The DS2 language supports only Latin characters for regular identifiers. To use non-Latin characters, the identifier must be delimited using double quotation marks.

How DS2 Processes Nulls and SAS Missing Values

<i>DS2 Modes for Nonexistent Data</i>	61
<i>Differences between Processing Null Values and SAS Missing Values</i>	63
<i>Reading and Writing Nonexistent Data in ANSI Mode</i>	65
<i>Reading and Writing Nonexistent Data in SAS Mode</i>	66
<i>Testing for Null Values</i>	67
<i>Testing for Missing Values</i>	68

DS2 Modes for Nonexistent Data

Many relational databases such as Oracle and DB2 implement ANSI SQL null values. Therefore, the concept of null values using DS2 is the same as using the SQL language for databases that support ANSI SQL. It is important to understand how DS2 processes SAS missing values because data can be lost.

Note: Only DOUBLE or CHAR(*n*) data types can have missing values. Missing values that are assigned to variables of any data types other than DOUBLE or CHAR(*n*) are automatically converted to null values.

Note: SAS Cloud Analytic Services (CAS) currently supports CHAR, VARCHAR, and DOUBLE data types for table storage on each data source.

Because there are significant differences in processing null values and SAS missing values, DS2 has two modes for processing nonexistent data: ANSI SQL null mode (ANSI mode) and SAS missing value mode (SAS mode).

The behavior of nonexistent data depends on how you connect to your data source:

- DS2 code that is submitted to PROC DS2 or PROC HPDS2 processes the data using SAS mode. PROC DS2 provides the ANSIMODE option that enables you to process data using ANSI mode.
- DS2 code that is submitted to the SAS In-Database Code Accelerator or the SAS Scoring Accelerator processes the data using SAS mode.
- DS2 code that is submitted using the DS2 runDS2 action processes the data using SAS mode. The DS2 runDS2 action provides the “nullBehavior”:“ANSI” parameter that enables you to process data using ANSI mode.
- DS2 code that is submitted to the SAS Federation Server processes data in ANSI mode.

In most instances, no mode change is necessary to process nonexistent data. You can be in any mode and still operate on null and missing values together.

The following are instances of when you might want to change the mode:

- when a client application processes SAS data sets and the mode for nonexistent data is in ANSI mode
- when the processing of SAS data sets is complete and the client application is ready to return to ANSI mode

CAUTION

If the mode is not set for the desired results, data is lost. In ANSI mode, when DS2 reads a SAS numeric missing value from a SAS data set, it converts the SAS missing value to an ANSI null value. If the ANSI null value is then written to a SAS data set, DS2 converts the ANSI null value to the SAS numeric missing value (.). If the SAS numeric missing value from the input data set is a special numeric missing value, such as .A, the .A is lost during the conversion to and from ANSI null, and the SAS numeric missing value (.) is written to the output data set. In the following example, column *x* is of data type DOUBLE. The example illustrates how a SAS special numeric missing value (.A in the fourth row) is transformed to the SAS numeric missing value (.) by a DS2 program run in ANSI mode.

```
/* assume input data set indata contains */
x
100
.
.A

/* Run the following program using ANSI mode */
proc ds2 ansimode;
data outdata;
  method run();
  set indata;
end;
enddata;
run;
quit;

/* the output dataset outdata contains */
x
100
.
.
```

In SAS mode, when DS2 reads an ANSI null value of data type CHAR(*n*) from an ANSI data source, it converts the ANSI null value to a SAS character missing value (character string consisting entirely of blank spaces). When DS2 writes a nonexistent value to a CHAR(*n*) column to an ANSI data source, the value is written as an ANSI null value. However, during processing, DS2 treats the nonexistent value as a SAS character missing value. In the following example, column *x* is of data type CHAR(5). The example illustrates that the ANSI null value (in the second row) is transformed to a string of blank spaces (' ') by a DS2 program that is run in SAS mode.

```
/* assume input data set indata contains */
x
'abc '
null

/* run the following program using SAS mode */
proc ds2;
data db.outdata;
    method run();
    set db.indata;
end;
enddata;
run;
quit;

/* the output dataset outdata contains */
x
'abc '
' ' '
```

Differences between Processing Null Values and SAS Missing Values

Processing SAS missing values is different from processing ANSI null values and has significant implications in these situations:

- when filtering data (for example, in a WHERE clause, a HAVING clause, a subsetting IF statement, or an outer join ON clause). SAS mode interprets each null value as a SAS missing value (.), which is a known value. ANSI mode interprets a null value as an unknown value.
- when submitting outer joins in ANSI mode, internal processing might generate nulls for intermediate result tables. DS2 might generate SAS missing values in SAS mode for intermediate result tables. Therefore, for intermediate result tables, nulls are interpreted as unknown values in ANSI mode and in SAS mode, missing values are interpreted as known values.
- when comparing a character value that consists entirely of blank spaces, SAS mode interprets the value as MISSING. In ANSI mode, a character value that

consists entirely of blank spaces is treated as a string made up of space characters (it has no special meaning).

- DS2 interprets a null as a SAS missing value in these cases:
 - in SAS mode, a null is used in a computation or assignment involving floating-point numbers or fixed-length character values. Missing values in varying-length character strings are treated like ANSI null values.
 - a null is passed to a SAS format or function that expects a DOUBLE or CHAR data type.

Note: If you are using SAS Federation Server, ANSI null values are translated to SAS missing values in FedSQL CALL invocations when the DS2_SASMISSING environment variable is set to TRUE.

The following are attribute and behavior differences between null values and SAS missing values:

Table 7.1 *Attribute and Behavior Differences between ANSI SQL Null Values and SAS Missing Values*

Attribute or Behavior	ANSI SQL Null value	SAS Missing Values
internal representation	metadata	floating-point or character
evaluation by logical operators	is an unknown value that is compared by using three-valued logic, whose resolved values are True, False, and null. For example, <code>WHERE col1 = null</code> returns null. Note: In ANSI mode, <code>null=null</code> is null.	is a known value that when compared, resolves to a Boolean result. For example, <code>WHERE col1 = .A</code> returns True when the same missing value is found. Missing values have a value of False (zero) when you use them with logical operators. Note: Only missing values of DOUBLE and CHAR(<i>n</i>) data types compare equally within the data type.
collating sequence order	appears as the smallest value	appears as the smallest value

Reading and Writing Nonexistent Data in ANSI Mode

Many relational databases such as Oracle and DB2 implement ANSI SQL null values. Therefore, the concept of null values using DS2 is the same as using the SQL language for databases that support ANSI SQL. It is important to understand how DS2 processes SAS missing values because data can be lost.

SAS missing value data types can be only DOUBLE or CHAR. Therefore, only the conversion for these data types is shown. The following table shows the value returned to the client application when DS2 reads a null value or a SAS missing value from a data source in ANSI mode:

Table 7.2 *Reading Nonexistent Data Values in ANSI Mode*

Column Data Type	Nonexistent Data Value	Value Returned to the Application
DOUBLE	., _ or .A-.Z ¹	null
DOUBLE	null	null
CHAR	'_ ' ²	'_ ' ²
CHAR	null	null

- ¹ The value ._ (period followed by an underscore) or .A-.Z represent a special numeric missing value. When SAS prints a special missing value, it prints only the letter or underscore.
- ² The value '_ ' is a blank space between single quotation marks, which in ANSI mode, is a blank space, not nonexistent data.

This next table shows the value stored when nonexistent data values are written to data sources in ANSI mode:

Table 7.3 *Storing Nonexistent Data in ANSI Mode*

Column Data Type	Nonexistent Data Value	Value Stored in the SAS Data Set	Value Stored in the ANSI SQL Null Supported Data Source
DOUBLE	null	.	null

Column Data Type	Nonexistent Data Value	Value Stored in the SAS Data Set	Value Stored in the ANSI SQL Null Supported Data Source
CHAR	null	' '	null

¹ The value ' ' is a blank space between single quotation marks, which in ANSI mode, is a blank space, not nonexistent data.

In ANSI mode, a blank space is always interpreted as a blank space and not as nonexistent data. Also, in ANSI mode, SAS missing values are converted to ANSI null values at input or assignment.

Reading and Writing Nonexistent Data in SAS Mode

When the client application uses SAS mode, nonexistent data values are treated like SAS missing values in the Base SAS environment.

The following table shows how nonexistent data values are read in SAS mode:

Table 7.4 Reading Nonexistent Data Values in SAS Mode

Column Data Type	Nonexistent Data Value	Value Returned to the Application
DOUBLE	., _ or .A-Z ¹	., _ or .A-Z ¹
DOUBLE	null	.
CHAR	' ' ²	' ' ²
CHAR	null	' ' ^{2, 3}

¹ The value . (period followed by an underscore) or .A-Z represent a special numeric missing value. When SAS prints a special missing value, it prints only the letter or underscore.

² The value ' ' is a blank space between single quotation marks, which in SAS mode, is nonexistent data.

³ When the SET statement encounters a null for a fixed-width character string, the string contains blank characters for the length of the string.

The next table shows how missing data values are written to a data destination in SAS mode:

Table 7.5 Writing Nonexistent Data Values in SAS Mode

Column Data Type	Nonexistent Data Value	Value Stored in the SAS Data Set	Value Stored in ANSI SQL Null Supported Data Source
DOUBLE ²	., _ or .A	., _ or .A-Z	null
CHAR ²	' _ '1	' _ '1	null

¹ The value ' _ ' is a blank space between single quotation marks, which in SAS mode, is nonexistent data.

² If the output is written to a SAS data set, other data types are forced to a DOUBLE or CHAR(*n*) data type. Null values become missing values (.) during that type conversion.

In SAS mode, ANSI null values for type DOUBLE or CHAR(*n*) are converted to SAS missing values at input or assignment. For other data types, FLOAT and VARCHAR for example, null values are preserved at input or assignment.

Note: Operations on special numeric missing values produce a regular SAS missing value—that is, a single period. For example, if $Y=.A$, $X=Y+1$ produces $X=.$ However, unlike other operations, a direct assignment preserves a special numeric missing value. For example, if $Y=.A$, the following output is produced, depending on the assignment.

```
X=Y+Y produces X=.
X=Y   produces X=A.
X=Y+1 produces X=.
```

Testing for Null Values

DS2 provides the NULL function to test for a null value. The NULL function has one argument, which can be an expression. If the expression is null, the function returns 1. If the expression is not null, the function returns 0.

This example shows a test for a null value:

```
if null(numCopies) then put 'Number of copies is unknown.'
  else put 'Number of copies is' numCopies;
```

The NMISS function returns the number of null and SAS missing numeric values. The NMISS function requires numeric values and works with multiple numeric values, whereas NULL works with only one value that can be either numeric or character.

For more information, see the “NULL Function” in *SAS DS2 Language Reference* and the “NMISS Function” in *SAS DS2 Language Reference*.

Testing for Missing Values

DS2 provides the MISSING function to test for a null value. The MISSING function has one argument, which can be a numeric or character expression. If the expression is null, the function returns 1. If the expression is not null, the function returns 0.

The NMISS function returns the number of null and SAS missing numeric values. The NMISS function requires numeric values and works with multiple numeric values, whereas MISSING works with only one value that can be either numeric or character.

For more information, see the [“MISSING Function” in SAS DS2 Language Reference](#) and the [“NMISS Function” in SAS DS2 Language Reference](#).

DS2 Type Conversions

<i>Type Conversion Definitions</i>	69
<i>Overview of Type Conversions</i>	70
<i>Type Conversion for Unary Expressions</i>	71
<i>Type Conversion for Logical Expressions</i>	71
<i>Type Conversion for Arithmetic Expressions</i>	72
<i>Type Conversion for Relational Expressions</i>	73
<i>Type Conversion for Concatenation Expressions</i>	74
<i>Type Conversions and Ambiguous Method Calls</i>	74

Type Conversion Definitions

binary data type

refers to the BINARY and VARBINARY data type

character data type

refers to the CHAR, VARCHAR, NCHAR, and NVARCHAR data types

coercible data type

a data type that can be converted to multiple data types, not just a character data type

date/time data type

refers to the DATE, TIME, and TIMESTAMP data types

non-coercible data type

a data type that only can be converted to a character data type

numeric data type

refers to the DECIMAL, DOUBLE (or FLOAT), REAL, BIGINT, INT, NUMERIC, SMALLINT, and TINYINT data types

standard character conversion

if an expression is not one of the character data types, it is converted to a CHAR data type.

standard numeric conversion

if an expression has a coercible, non-numeric data type, it is converted to a DOUBLE data type.

Overview of Type Conversions

Operands in an expression must be of the same general data type, numeric, character, binary, or date/time, in order for DS2 to resolve the expression. When it is necessary, DS2 converts an operand's data type to another data type, depending on the operands and operators in the expression. This process is called **type conversion**. For example, the concatenation operator (||) operates on character data types. In a concatenation of the character string 'First' and the numeric integer 1, the INTEGER data type for the operand 1 is converted to a CHAR data type before the concatenation takes place.

When an operand data type is converted within the same general data type, the operand data type is promoted. Operands with a data type of SMALLINT and TINYINT are promoted to INTEGER, and operands of type REAL are promoted to DOUBLE. Type promotion is performed for all operations on SMALLINT, TINYINT, and REAL, including arguments for method and function expressions.

Numeric and character data types are coercible. The BINARY, VARBINARY and the date/time data types DATE, TIME, and TIMESTAMP, are non-coercible and only can be converted to one of the character data types.

When DS2 evaluates an expression, if the data types of the operands match exactly, no type conversion or promotion is necessary and the expression is resolved. Otherwise, each operand must go through a standard numeric conversion or a standard character conversion, depending on the operator.

The results of a numeric or character expression are based on a data type precedence. If both operands have different types within the same general data type, the data type of the expression is that of the operand with the higher precedence, where 1 is the highest precedence. For example, for numeric data types, a data type of DOUBLE has the highest precedence. If an expression has an operand of type INTEGER and an operand of type DOUBLE, the data type of the expression is DOUBLE. A list of precedences can be found in the topics that follow, if applicable, for the different types of expressions.

For a table showing all type conversions, see [Appendix 1, "DS2 Type Conversions for Expression Operands,"](#) on page 305.

Type Conversion for Unary Expressions

In unary expressions, such as +1 or -444, the standard numeric conversion is applied to the operand. The following table shows the data type for unary expressions:

Table 8.1 Data Type Conversion for Unary Expressions

Expression Type	Expression Data Type
Unary plus	same as the operand or DOUBLE for converted operands
Unary minus	same as the operand or DOUBLE for converted operands
Unary not	INTEGER

Type Conversion for Logical Expressions

In logical expressions, such as `a & b` or `(a ~= start) | (f = finish)`, the standard numeric conversion is applied to each operand. The following table shows the precedence used to determine the data type of the expression, where 1 is the highest precedence and 3 is the lowest. The data type of the expression is the data type of the operand that has the higher precedence.

Table 8.2 Data Type Conversion for Logical Expressions

Precedence	Data Type of Either Operand	Expression Data Type
1	DOUBLE	DOUBLE
2	BIGINT	BIGINT
3	all other numeric data types	INTEGER

Type Conversion for Arithmetic Expressions

In arithmetic expressions, such as $a <> b$ or $a + (b * c)$, the standard numeric conversion is applied to each operand.

The following table shows the precedence used to determine the data type of arithmetic expressions for the addition, subtraction, multiplication, and division operators, where 1 is the highest precedence and 3 is the lowest. The data type of the expression is the data type of the operand that has the higher precedence.

Table 8.3 *Type Conversion for Addition, Subtraction, Multiplication, and Division Expressions*

Precedence	Data Type of Either Operand	Expression Data Type
1	DOUBLE	DOUBLE
2	DECIMAL, NUMERIC	DECIMAL, NUMERIC
3	BIGINT	BIGINT
4	all other numeric data types	INTEGER

The following table shows the data type for arithmetic expressions that use the min, max, and power operators:

Table 8.4 *Data Type Conversion for the Min, Max, and Power Operator Expressions*

Operator	Operator Data Type	Expression Data Type
min or max	DOUBLE	DOUBLE
min or max	DECIMAL, NUMERIC	DECIMAL, NUMERIC
min or max	BIGINT	BIGINT
**	all numeric data types	DOUBLE

Type Conversion for Relational Expressions

In relational expressions, such as $x \leq y$ or $i > 4$, the standard conversion that is applied depends on the operand data types. The data type of the expression is always INTEGER, as shown in the following tables.

Table 8.5 Data Type Conversion for Relational Expressions except IN Expressions

Order of Data Type Resolution	Data Type of Either Operand	Standard Conversion	Expression Data Type
1	any numeric data type	numeric	INTEGER
2	CHAR, NCHAR	character	INTEGER
3	DATE, TIME, TIMESTAMP	none, data types must match	INTEGER
4	NVARCHAR, VARCHAR	none, error returned	not applicable

Table 8.6 Data Type Conversion for IN Expressions

Precedence	Data Type of Either Operand	Expression Data Type
1	non-numeric	DOUBLE
2	DOUBLE	DOUBLE
3	DECIMAL, NUMERIC	DECIMAL
4	all other types that are not BIGINT	BIGINT

Type Conversion for Concatenation Expressions

In concatenation expressions, such as `a || b` or `x || y`, the standard character conversion is applied to each operand. The following table shows the precedence used to determine the data type of the expression, where 1 is the highest precedence and 2 is the lowest. The data type of the expression is the data type of the operand that has the higher precedence.

Table 8.7 Data Type Conversion for Concatenation Expressions

Precedence	Data Type of Either Operand	Expression Data Type
1	if either is type NCHAR	NCHAR
2	CHAR	CHAR

Type Conversions and Ambiguous Method Calls

If DS2 finds multiple methods that have the same name of the invoked method, DS2 tries to resolve the method overloading to determine which method to invoke. However, there are certain situations that DS2 cannot resolve.

In this example, two **ambig** methods are created, one with a BIGINT parameter and one with a CHAR parameter. When the method is invoked in the INIT method, the program passes an INTEGER argument. This generates an ambiguous method call error because DS2 cannot determine whether to call `ambig(bigint a)` or `ambig(char(1) a)`.

```
data _null_;
  dcl integer inp;
  dcl char(100) res;
  method ambig(bigint a) returns char(10);
    return 'bigint';
  end;
  method ambig(char(1) a) returns char(10);
    return 'char(1)';
```

```

end;

method init();
    inp = 2;
    res = ambig(inp);
    put inp= res=;
end;
enddata;
run;

```

To avoid the error, add a method **ambig** that has an INTEGER parameter.

```

data _null_;
    dcl integer inp;
    dcl char(100) res;
    method ambig(bigint a) returns char(10);
        return 'bigint';
    end;
    method ambig(char(1) a) returns char(10);
        return 'char(1)';
    end;

    /* Define ambig with a int parameter */
    /* fixes the problem by routing "integer" to "bigint" */

    method ambig(int a) returns char(10);
        dcl bigint l;
        l = a;
        return ambig(l);
    end;

    method init();
        inp = 2;
        /* call ambig(int) is no longer ambiguous */
        res = ambig(inp);
        put inp= res=;
    end;
enddata;
run;

```

Note that if you attempt to call the method **ambig** with a DOUBLE argument, you get an ambiguous method call error because DOUBLE does not exactly match one of the existing method declarations. Instead, you need to create a new method definition—this one taking a DOUBLE parameter—to work around the ambiguity.

DS2 Expressions

<i>What Is an Expression?</i>	77
<i>Types of Expressions</i>	78
Overview of Expressions	78
Primary Expression	79
Complex Expression	80
Array Expression	81
Function Expression	82
IN Expression	82
LIKE Expression	83
Method Expression	85
Package Method Expression	87
SYSTEM Expression	88
THIS Expression	88
IF Expression	89
SELECT Expression	92
<i>Operators in Expressions</i>	95
Operator Precedence	95
Expression Values by Operator	96
Short-Circuit Evaluation in SAS	99

What Is an Expression?

An expression is made up of operands, and optional operators, that form a set of instructions and that resolves to a value.

An operand can be a single constant or variable, or it can be an expression. Operators are the symbols that represent either a calculation, comparison, or concatenation of operators.

Here are some examples of DS2 expressions:

```
a = b * c
"coll"
s || 1 || z
a >= b**(c - 8)
```

```
system.put(a*5,hex.)
```

Types of Expressions

Overview of Expressions

The basic type of expression is a primary expression. Complex expressions combine expressions and operators. Other expressions invoke a DS2 construct, such as a method expression or a function expression. The `system` expression and the `THIS` expression refer to expressions that are global in scope. In SAS mode, the `IN` expression returns a Boolean result based on whether the result of an expression is contained in a list.

Expression kinds commonly refer to a segment of code. For example, an `AND` expression refers to the `AND` operator and the operands that it processes. A binary expression refers to a binary constant or a hexadecimal constant such as `x'ff00effc'`.

Whether an expression is simple or complex, invokes a construct, or is global in scope, expressions of all kinds have a value and a data type.

Note: In logical operations, a missing value in any expression, such as an `IF` expression, evaluates to `False` in SAS mode. A null value evaluates to neither true nor false in ANSI mode. If you run this example in SAS mode, `False` is written to the SAS log. If you run this example in ANSI mode, `Null` is written to the SAS log.

```
proc ds2;
data _null_;
  declare double d;
  method init();
    if d then put 'True';
    else if not d then put 'False';
    else put 'Null';
  end;
enddata;
run;
quit;
```

Primary Expression

In their simplest form, primary expressions are numbers, character strings, binary and hexadecimal constants, literal values, date and time values, identifiers, and null values, as in these primary expressions:

```
a
"var_a"
5.33
'Company'
date '2007-08-24'
```

The following table shows basic primary expressions, their data type(s), and an example:

Table 9.1 Data Types and Examples of Primary Expressions

Type of Expression	Short Description	Data Type	Example
Array	Groups same type data	Same type as individual items in the array	<code>a[5, b+c]</code>
Binary	Binary and hexadecimal constants	VARBINARY	<code>x'FE'</code> <code>b'01000011'</code>
Character	Character string	CHAR, VARCHAR	<code>'New Report'</code> <code>a='Stock';</code> <code>b='Report';</code> <code>c=a b;</code> <code>1</code>
National Character	National Character string	NCHAR, NVARCHAR	<code>n'New Report'</code> <code>a=n'Stock';</code> <code>b=n'Report';</code> <code>1</code>
Dot	System, THIS, and package method expressions	The resolved type of the expression	<code>system.put(x,5.)</code> <code>this.s</code> <code>p.calc(2,6,9)</code>
Date / time	Date and time values	DATE, TIME, TIMESTAMP, or DOUBLE	<code>date '2007-01-01'</code>

Type of Expression	Short Description	Data Type	Example
Identifier	Provides a name for various language elements or is a keyword	The declared or default type. The default is DOUBLE.	a "part1" IN
Integer	Integer numbers	INTEGER, BIGINT	123
New	Instantiates a package method	The resolved type of the expression	a= <code>new_package_name()</code> ; 1
Numeric	Integer, real, and floating point numbers, or a missing or null value	DOUBLE, FLOAT, REAL	5 4.3
Null	Null expression	none	NULL
Parentheses	Operator and operands enclosed in parentheses for higher evaluation precedence	The resolved type of the expression enclosed in parentheses.	<code>(a + b)</code> - c 1

1 For the purpose of the example, the primary expression is contained in an expression or an assignment statement. The example expression is highlighted.

Complex Expression

A complex expression combines expressions and operators to create a more expansive expression, as in these expressions:

```
a + b * -c - 5
a = b = 5
x | y & a < c * d
x**y**z - 9 >= f
z >< c + e <> u**y * 10
x || c + 7
a in (1,2,3)
```

Evaluation of complex expressions is based on the operator order of precedence, as shown in [Table 9.5 on page 95](#), and the data types of the primary expressions. Before any calculations can be done, operand data types must be the same general data type: numeric, character, binary, or date/time. If the data types are the same, processing can proceed. If they are not the same, the operand data types are converted based on the operator and the data type of the operands.

In the expression `a + b / -c - 5`, assume that `a` is 1.35 with a type of DOUBLE, `b` is 2 with a data type of INTEGER, and `c` is 3 with a data type of INTEGER. `b/-c` or

2/-3 evaluates to INTEGER 0. The INTEGER 0 is converted to a DOUBLE 0 before being added to a. Then $a + 0.0$ evaluates to DOUBLE 1.35. The INTEGER 5 is converted to a DOUBLE 5 before the addition of the DOUBLE 1.35. The final result of the expression is DOUBLE -3.65.

For information about data type conversion, see [Chapter 8, “DS2 Type Conversions,” on page 69](#).

In the expression $a = b = 5$, if b is a value other than 5, then $b = 5$ is evaluated to 0. Therefore, a is assigned a value of 0. The first equal sign ($=$) is an assignment operator and the second equal sign is a logical equality operator. For more information, see [“Using an Expression with Multiple Equal Signs” in SAS DS2 Language Reference](#).

Note: DS2 supports using `eq` as well as the equal sign.

Array Expression

An array expression is a primary expression that represents a grouping of data items of the same data type. Although an array can have multiple dimensions, individual data item values are scalar values. Data items are accessed by specifying an index into the array.

The array expression consists of an array identifier followed by an array index expression for each dimension in the array, as in this syntax:

array-identifier `\[ index-expression < , ...index-expression > \]`

Note: Brackets in the syntax convention indicate optional syntax. The escape character (`\`) before a bracket indicates that the bracket is required for the syntax. Indexes in an array expression must be contained by brackets (`[ ]`).

The array identifier can be either a declared array variable or a variable used in a THIS expression. The index expression is a primary expression that resolves to an integer.

Here are some examples of array expressions:

```
a[i]
s[j * 2, k-3]
this.c[2, vwind, a[i]]
```

When an array is declared, the index values specify the boundaries for the array. If an index expression is beyond the boundaries of the array, DS2 issues an error.

The value of an array expression is the value of the indexed value in the array. For example, if the array values are $a[1] = 12$, $a[2] = 15$, and $a[3] = 20$, the value of the array expression $a[2]$ is 15.

Note: Arrays are 1-based. The array index starts at 1.

Function Expression

A function expression invokes a function within a DS2 program. To invoke a function, use this syntax:

```
function-name ( < argument < , ...argument > > )
```

Functions might require arguments. If the function expression contains arguments, the argument data types are converted, if necessary, to the data types of the function **signature**, which is the argument order and data type for a function. Parentheses in the function call are required, whether the function takes arguments or no arguments are required. For example, the TIME function does not require arguments:

```
t = time();
```

A function expression resolves to the value returned by the function. In the function expression above, `time()` resolves to the current time of day.

Methods and functions are similar. Functions have global scope. Methods are programming blocks and have local scope.

If the name of a function is identical to a method name, DS2 invokes the method. Functions with the same name as a method can be invoked only by using a SYSTEM expression. For more information, see [“SYSTEM Expression” on page 88](#).

For a list of DS2 functions, see [“DS2 Functions” in SAS DS2 Language Reference](#).

IN Expression

An IN expression determines whether an expression is contained in a constant list. See [“Constant List” on page 46](#).

Here is the syntax of an IN expression:

```
expression < not-operator > IN constant-list
```

Note: Any valid data type for your data source can be used in *constant-list*. If any argument is non-numeric, the argument is converted to DOUBLE. If any argument is DOUBLE or REAL, all arguments are converted to DOUBLE (if not so already). If any argument is DECIMAL, all arguments are converted to DECIMAL (if not so already). Otherwise, all arguments are converted to a BIGINT. The result is always INTEGER, either 0 or 1.

Any of the NOT operators (~, ^, or NOT) are valid before the IN operator, which results in the logical negation of the expression value.

Consider these differences, according to mode:

- In SAS mode, an IN expression evaluates to TRUE (a nonzero value) or FALSE (0).
- In ANSI mode, an IN expression evaluates to TRUE (a nonzero value), FALSE (0), or NULL.

This is the same behavior as DS2's logical (AND, OR, NOT) and relational ($\leq$, $<$, and so on) operations.

If the IN expression and the constants in the constant list do not contain a NULL or SAS missing value, the result of the IN operation is TRUE or FALSE regardless of the mode.

If the IN expression or a constant in the constant list contains a SAS missing value or NULL, consider these differences, according to mode:

- In SAS mode, if the IN expression or a constant in the constant list contains a SAS missing value, the result of the IN operation is still TRUE or FALSE.
- In ANSI mode, if the IN expression or a constant in the constant list contains a NULL value, the result of the IN operation is NULL.

For more information about how DS2 processes nulls and SAS missing values, see [“DS2 Modes for Nonexistent Data” on page 61](#).

The following table shows the results of some IN expressions:

Table 9.2 Examples of IN Expressions

Input Values	IN Expression	Results
a = 2	a in (5,34,2,67)	1
b = 3	b not in (3,22,43,65)	0

LIKE Expression

Overview of the LIKE Expression

A LIKE expression determines whether a character string matches a pattern-matching specification.

Here is the syntax of a LIKE expression:

expression < NOT > **LIKE** *pattern-matching-expression* <ESCAPE *character-expression*>

The expressions can be any character string or binary string data type.

If *expression* matches the pattern specified by *pattern-matching-expression*, a value of 1 (true) is returned. Otherwise, a value of 0 (false) is returned.

NOT LIKE returns the inverse value of LIKE. For example, if `x like y` is true, then `x not like y` is false.

The ESCAPE argument is used to search for literal instances of the percent (%) and underscore (_) characters, which are usually used for pattern matching.

Patterns for Searching

Patterns consist of three classes of characters.

Table 9.3 *Pattern-matching Characters*

Character	Description
underscore (_)	matches any single character
percent sign (%)	matches any sequence of zero or more characters Note: Be aware of the effect of trailing blanks. To match values, you might have to use the TRIM function to remove trailing blanks.
any other character	matches that character

Searching for Literal % and _

Because the % and _ characters have special meaning in the context of the LIKE expression, you must use the ESCAPE argument to search for these character literals in the input character string.

These examples use the values `app`, `a_`, `a__`, `bbaa1`, and `ba_1`.

- The condition `like 'a_ %'` matches `app`, `a_`, and `a__`, because the underscore (_) in the search pattern matches any single character (including the underscore), and the percent (%) in the search pattern matches zero or more characters, including '%' and '_'.
- The condition `like 'a_ ^%' escape '^'` matches only `a_`, because the escape character (^) specifies that the pattern search for a literal '%'.
- The condition `like 'a_ %' escape '_'` matches none of the values, because the escape character (underscore) specifies that the pattern search for an 'a' followed by a literal '%', which does not apply to any of these values.

Searching for Mixed-case Strings

The DS2 LIKE expression is case sensitive. To search for mixed-case strings, use the UPCASE function as the following example shows:

```
upcase(str) like 'SM%'
```

LIKE Expression Examples

The following table shows examples of the matches that would result when searching these strings: Smith, Smooth, Smothers, Smart, Smuggle.

Table 9.4 Examples of LIKE Expressions

LIKE Expression Example	Matching Results
str like 'Sm%'	Smith, Smooth, Smothers, Smart, Smuggle
str like '%th'	Smith, Smooth
str LIKE 'S__gg%'	Smuggle
str like 'S_o'	(no matches)
str like 'S_o%'	Smooth, Smothers
str like 'S%th'	Smith, Smooth
str not like 'Z'	Smith, Smooth, Smothers, Smart, Smuggle

Method Expression

A method expression invokes a method that has been defined by the METHOD statement.

To invoke a method, use this syntax:

```
method-name ( < expression < , ... expression > > )
```

Methods are invoked based on the method name and signature. DS2 first identifies the method name. If a method name is identical to a function name, DS2 invokes the method. A function with the same name as a method can be invoked by using a SYSTEM expression. For more information, see [“SYSTEM Expression” on page 88](#).

Because DS2 allows overloaded methods, DS2 invokes the method whose arguments best match the number of arguments and the argument data types in the method signature, which is the argument order and data type for the method. The best match is the one for which the number of method parameters is equal to the number of arguments, and such that no other method signature has as many exact parameter type matches for the given argument list. If a best match is not found, an error occurs.

TIP DS2 methods can support up to 1000 arguments. A DS2 method that has more than 1000 arguments can generate a compilation error.

Once the method to execute is identified, DS2 converts argument data types to the data type of the corresponding method parameter, if necessary. The method then executes.

A method expression resolves to the value returned by the method.

In the following example, methods CONCAT and ADD are defined and then invoked in the INIT() method. The highlighted expressions in the INIT() method are method expressions:

```
method concat(char(100) x, char(100) y) returns char(200);
    return trim(x) || y;
end;

method add(double x, double y) returns double;
    return x + y;
end;

method init();
    dcl char(200) r;
    r = concat('abc', 'def');
    d = add(100,101);
end;
```

In this next example, the D method is an overloaded method. DS2 must compare the method expression arguments to the method signatures to find the best match:

```
method d(double x, double y) returns double;
    return x + y;
end;

method d(int x, int y) returns int;
    return x + y;
end;

method init();
    dcl double r;
    dcl int i;
    r = d(1.2345, 5.6789);
    i = d(99, 100);
end;
```

The first method calls the D method whose signature requires values with DOUBLE data types. The second method calls the D method whose signature requires values with INTEGER data types.

This final example shows that DS2 cannot determine whether the values in the method expression have a data type of INTEGER or DOUBLE. Because it is ambiguous, DS2 issues an error:

```
method d(int x, double y) returns double;
    return x + y;
end;

method d(double x, int y) returns double;
    return x + y;
end;

method run();
    d = d(100, 102);
end;
```

For more information, see the [“METHOD Statement” in SAS DS2 Language Reference](#).

Package Method Expression

A package method expression instantiates a method that is defined in a package. To invoke a package method expression, use this syntax:

package-name.method-name (< method-argument < , ... method-argument > >)

Package method expressions execute in a manner similar to method expressions. That is, once DS2 has determined that the package and the method exist, the best match of method signatures is determined, argument data types are converted if necessary, and the method executes.

TIP DS2 methods can support up to 1000 arguments. A DS2 method that has more than 1000 arguments can generate a compilation error.

In the following example, the highlighted expressions are package method expressions:

```
declare package myadd a1() a2();
a1.sale(3,4);
a1.add(1,2);
a2.bonus(5,12);
a2.add(10,20);
```

The first two package method expressions invoke the SALE and ADD methods in the A1 package, which was instantiated from the MYADD package. The last two package method expressions invoke the BONUS and ADD methods in the A2 package.

Note: You can invoke a DS2 package method expression as a function in a FedSQL SELECT statement. For more information, see [“Using DS2 Packages in Expressions” in SAS FedSQL Language Reference](#).

For information about packages, see the “[PACKAGE Statement](#)” in *SAS DS2 Language Reference* and Chapter 12, “DS2 Packages,” on page 131.

SYSTEM Expression

When a method and a function have identical names, the method call takes precedence over the function call. The function can then be invoked only by using a SYSTEM expression.

To invoke a SYSTEM expression, use this syntax:

SYSTEM.*function-expression*

A SYSTEM expression prepends a function expression with the dot notation, `system..` For example, if SUM is the name of a method as well as the name of a function, the SUM function only can be invoked by using the SYSTEM expression: `system.sum(a,b,c)`.

THIS Expression

A THIS expression provides an alternate method to simultaneously declare and use a global scalar variable from anywhere within a DS2 program. A THIS expression is used to circumvent the standard variable lookup. In a THIS expression, DS2 searches for a scalar variable declaration of the identifier in global scope. If there is no such declaration, DS2 declares the identifier in global scope with DOUBLE type. Global variables can be referenced by all programming blocks in a DS2 program.

To invoke a THIS expression, use this syntax:

THIS.*variable-name*

A THIS expression prepends a variable with the dot notation, `THIS..`

Note: DS2 stores `THIS.variable-name` only as *variable-name*. If you have a local variable with the same name as the global scalar variable and DS2 issues a diagnostic message about the variable, you will not be able to distinguish which variable is a problem. For example, if DS2 issues a warning message that `x` is not declared, you would not know whether the message refers to the global variable, `THIS.x`, or the local variable, `x`.

In the following example, the variable `s` becomes a global variable by using a THIS expression:

```
method init();
  this.s = sum(a,b,c,d,e);
end;
method run();
  t = put(this.s 5.4);
end;
```

The THIS expression provides a method to access a global variable that is hidden by a local variable with the same name. Here is an example.

```
data;
  declare double x;    /* declare global x */

  method run();
    declare double x;  /* declare local x */

    /* Two variables exist with same name, "x". */
    /* Identifier "x" refers to local x in      */
    /* scope of run method. Global x is hidden */
    /* by local x.                             */

    this.x = 1.0;      /* assign 1.0 to global x */
    x = 0.0;           /* assign 0.0 to local x  */

    output;            /* global x with 1.0 output */
  end;
enddata;
run;
```

IF Expression

Overview of the IF Expression

The conditional IF expression is used to select between two values based on whether a conditional expression evaluates to true (a nonzero value) or false (zero).

To invoke an IF expression, use this syntax:

IF *expression-1* **THEN** *expression-2* **ELSE** *expression-3*

If *expression-1* is a nonzero value, the result of the IF expression is the value of *expression-2*. Otherwise, the result of the IF expression is the value of *expression-3*. Here is an example.

```
m=(if missing(u) then 0 else u);
```

The IF expression can be used wherever any other expression can be used. The precedence of an IF expression is lower than the arithmetic and logic operators. Therefore, parentheses are necessary in mixed expressions like this one.

```
r = 25.5 + (if sum < 15 then -a else b*2);
```

Without the parentheses, the plus (+) operator would be evaluated first resulting in a parse error from the subexpression 25.5 + if.

Nested IF Expressions

IF expressions can be nested to select between many values for a multi-way decision.

```
IF condition-expression-1 THEN result-expression-1
    ELSE IF condition-expression-2 THEN result-expression-2
    ...
    ELSE IF condition-expression-n THEN result-expression-n
    ELSE result-expression-default
```

The condition expressions are evaluated in order. The result of the nested IF expression chain is the associated *result-expression* of the first *condition-expression* that evaluates to true (a nonzero value). If all the *condition-expressions* evaluate to false (zero), the result of the IF expression is the *result-expression-default*. Here is an example.

```
grade = if score >= 90 then 'A'
        else if score >= 80 then 'B'
        else if score >= 70 then 'C'
        else if score >= 60 then 'D'
        else if score >= 0 then 'F'
        else NULL;
```

Note: Use a SELECT statement rather than a series of IF-THEN/ELSE statements when you have a long series of mutually exclusive conditions. A large number of nested IF-THEN/ELSE statements could cause an internal error. By contrast, the SELECT statement is evaluated only once, which could improve performance without causing errors.

For example, assume that a program contains many ELSE IF constructs like the following program.

```
if (e1) then ...
else if (e2) then ...
...
else if (en) then ...
else ...
```

Use this SELECT statement instead.

```
select;
when (e1): ...
when (e2): ...
...
when (en): ...
otherwise: ...
```

IF Expression Data Type

The data type of an IF expression is determined by examining the type of the first result expression, *expression-2*.

```
IF expression-1 THEN expression-2 ELSE expression-3
```

If *expression-2* is not a numeric data type, then the IF expression is assigned the type of *expression-2*.

If *expression-2* is a numeric data type, then the IF expression is assigned the wider numeric data type of *expression-2* and *expression-3*. For example, if *expression-2* is an SMALLINT and *expression-3* is a DOUBLE, then the IF expression is assigned type DOUBLE. If *expression-2* is a numeric data type and *expression-3* is not a numeric data type, then the IF expression is assigned the type of *expression-2*.

If the first result expression in a nested IF expression chain is a numeric data type, then all the result expressions are examined to find the widest numeric data type to assign as the type of the nested IF expression chain. In the following example, *t* is a TINYINT, *b* is a BIGINT, and *d* is a DECIMAL(10,5). The 0 in the ELSE is assigned type BIGINT. Therefore, the nested IF expression chain is assigned the type decimal(10,5), the widest numeric type of TINYINT, BIGINT, and DECIMAL(10,5).

```
r = if n < 0 then t
    else if n = 0 then b
    else if n > 0 then d
    else 0;
```

Note: If 0.0 had been used for the ELSE value instead of 0, then the ELSE result would have been assigned type DOUBLE instead of type BIGINT. With the type DOUBLE ELSE expression, the widest numeric type of the result expressions would be type DOUBLE. Therefore, the nested IF expression chain would be assigned type DOUBLE instead of type decimal(10,5).

Lazy Evaluation of IF Expressions

The IF expression uses lazy evaluation for the result expressions.

```
IF expression-1 THEN expression-2 ELSE expression-3
```

For example, you could use this code to check for division by zero.

```
a = if c ne 0 then b/c else null;
```

Expression *expression-1* is always evaluated, but only one of *expression-2* or *expression-3* is evaluated. The expression that is not selected as the result of the IF expression is not evaluated. Thus, if *expression-1* is a nonzero value (true), then only *expression-2* is evaluated. If *expression-1* is zero (false), then only *expression-3* is evaluated.

Lazy evaluation also applies to the result expressions of nested IF expression chains.

```
IF condition-expression-1 THEN result-expression-1
  ELSE IF condition-expression-2 THEN result-expression-2
  ...
  ELSE IF condition-expression-n THEN result-expression-2
  ELSE result-expression-default
```

The selected result expression is the only result expression evaluated. Lazy evaluation also applies to the condition expressions. Condition expressions are evaluated in order until a condition evaluates to true (nonzero) or all conditions are evaluated. If the IF expression has n condition expressions and the i th condition is the first nonzero condition, then only the first 1 to i conditions are evaluated. The $i + 1$ to n conditions are not evaluated.

SELECT Expression

Overview of the SELECT Expression

A SELECT expression is used to select between multiple expressions based on the values of other expressions.

To invoke a SELECT expression, use this syntax:

```
SELECT <(select-expression)>
  WHEN (when-expression) <...WHEN (when-expression)> result-expression
  <...WHEN (when-expression) <...WHEN (when-expression)> result-expression>
  OTHERWISE <default-result-expression>
END
```

The SELECT expression evaluates each WHEN expression in order until a matching expression is found. Then the associated *result-expression* is evaluated as the result of the SELECT expression.

The SELECT expression can be used wherever any other expression can be used. Here is an example.

```
r = 25.5 + select (t) when (1) -a when (3) b*2 end;
```

SELECT Expression with a Selection Expression

If a selection expression is present, then it is evaluated. Then the WHEN expressions are evaluated in order. The result of the SELECT expression is the result expression of the first WHEN expression that evaluates to the same value as the selection expression. If all the WHEN expressions evaluate to different values than the selection expression, the result of the SELECT expression is the default

result expression if present. Otherwise, it is a missing or null value. Here is an example.

```
s = select (t)
  when (1) x*10
  when (3) x
  when (5) x*100
  when (0) 0
  otherwise .
end;
```

If `t` is 5, then the SELECT expression evaluates to 'x*100'.

SELECT Expression without a Selection Expression

If a selection expression is not present, then the WHEN expressions are evaluated in order. The result of the SELECT expression is the result expression of the first WHEN expression that evaluates to true (a nonzero value). If all the WHEN expressions evaluate to false (zero), the result of the select expression is the default result expression if present. Otherwise, it is a missing or null value. Here is an example.

```
grade = select
  when (score >= 90) 'A'
  when (score >= 80) 'B'
  when (score >= 70) 'C'
  when (score >= 60) 'D'
  when (score >= 0 ) 'F'
end;
```

If `score` is 76, then the first *when-expression* to evaluate to true is `score>=70`. The *select-expression* evaluates to 'C'.

Optional Otherwise Expression

If an otherwise default result expression is not supplied, then DS2 provides a default result value to select when none of the WHEN expressions are selected. If the SELECT expression has type DOUBLE or CHAR in SAS mode, the default result value is a missing value (.). For all other data types in either mode, the default result value is NULL.

Result Expression with Multiple When Expressions

Multiple WHEN expressions can be associated with a single result expression. The WHEN expressions are listed consecutively followed by the single result expression. If any of the WHEN expressions associated with a result expression is the first matching WHEN expression, then the result of the SELECT expression is the result expression.

For example, the following SELECT expression evaluates to 'airplane' if the value of variable *t* is either 'A', 'a', 'P', or 'p'.

```
s = select (t)
  when ('A')
  when ('a')
  when ('P')
  when ('p') 'airplane'
  when ('C')
  when ('c') 'car'
  when ('T')
  when ('t') 'train'
  otherwise 'walk'
end;
```

SELECT Expression Data Type

The type of a SELECT expression is determined by examining the type of the first result expression. If the first result expression is not a numeric data type, then the SELECT expression is assigned the type of the first result expression.

If the first result expression is a numeric data type, then all the result expressions are examined to find the widest numeric data type to assign as the type of the SELECT expression.

In the following example, *t* is a TINYINT, *b* is a BIGINT, *d* is a DECIMAL(10,5), and *s* is a CHAR(10). The select expression is assigned the type DECIMAL(10,5), the widest numeric type of TINYINT, BIGINT, DECIMAL(10,5), and CHAR(10).

```
r = select (t)
  when (1) t
  when (2) b
  when (3) d
  otherwise s
end;
```

Note: If *s* had been assigned to the first result expression, then the type of the SELECT expression would have been CHAR(10). If the first result expression has a non-numeric type, then the non-numeric type is assigned as the type of the SELECT expression.

Lazy Evaluation of the SELECT Expression

The SELECT expression uses lazy evaluation for the WHEN expressions. The WHEN expressions are evaluated in order until a matching WHEN expression is found or all when expressions are evaluated. If the SELECT expression has *n* WHEN expressions and the *i*th WHEN expression is selected, then only the first 1 to *i* WHEN expressions are evaluated. The *i*+1 to *n* when expressions are not evaluated.

Lazy evaluation also applies to the result expressions of the SELECT expression. The selected result expression is the only result expression evaluated.

In the following example, if `n[i]` equals 0, then only the first two WHEN expressions (`n[i] < 0` and `n[i] = 0`) are evaluated and only the second result expression (`y*10-r2`) is evaluated.

```
m(select
  when (n[i] < 0) y*100-r1
  when (n[i] = 0) y*10-r2
  when (n[i] > 0) y*10
end);
```

Operators in Expressions

Operator Precedence

An operator symbolizes a type of operation that is to be performed on an operand, such as addition, comparison, and logical negation. When an expression contains multiple operators and operands, DS2 resolves the expression by using operator precedence. Operations are performed from the highest order of precedence to the lowest order of precedence.

The highest order of precedence is 1 and the lowest order of precedence is 9. Within a precedence level, with the exception of exponentiation, minimum, and maximum operators, operators associate from left to right. The exponentiation, minimum, and maximum operators associate from right to left.

By using the precedence order in [Table 9.5 on page 95](#), in the expression `5+a**b*3`, `a**b` is calculated first and then multiplied by 3, and that result is added to 5.

TIP In DS2, `x < y < z` is evaluated like `x < y` and `y < z`.

The following table lists the operators and their order of precedence:

Table 9.5 Operators and Their Order of Precedence in DS2 Expressions

Order of Precedence	Symbol	Associativity
1	()	left to right
1	SELECT expression	left to right
2	+, -	right to left

Order of Precedence	Symbol	Associativity
2	[^] or ~	left to right
2	^{**} , <>, ><	left to right
3	[*] , /	left to right
4	⁺ , -	left to right
5	or !!	left to right
5	..	left to right
6	IN, LIKE	left to right
7	=, [^] = or ~=	right to left
7	>=, <=, >, < ¹	left to right
8	&	left to right
9	or !	left to right
10	IF expression	right to left
none	:=	none
none	_NEW_	none
none	Method expression	none

¹ In DS2, $x < y < z$ is evaluated like $x < y$ and $y < z$.

Expression Values by Operator

The following table shows the resolved value for expressions that are based on an operator:

Table 9.6 Expression Values by Operator

Operator	Value
Unary expressions	

Operator	Value
Unary plus	Is the same as the expression operand
Unary minus	Is the arithmetic negation of the operand
NOT or ^ or ~	If the operand is nonzero, result is 0. If the operand is zero or missing, result is 1. If the operand is null, result is null.
Logical expressions	
OR or or !	■ the logical OR of the two operands ■ null when one operand is zero or missing and the other operand is null ■ null when both operands are null ■ 1 when either operand is nonzero (even if the other operand is null or missing) ■ 0 when both operands are zero or missing
AND or &	■ the logical AND of the two operands ■ null when one operand is nonzero and the other operand is null ■ null when both operands are null ■ 1 when both operands are nonzero ■ 0 when either operand is zero or missing (even if the other operand is null)
Arithmetic expressions	
+	the arithmetic sum of the operands
-	the arithmetic difference of the operands
*	the arithmetic product of the operands
/	the arithmetic quotient of the operands
**	the left operand raised to the power of the right operand
><	the minimum of the left and right operands
<>	the maximum of the left and right operands
any arithmetic operator	null when either or both operands are null
Relational expressions	

Operator	Value
<	1 when the left operand is less than the right operand; otherwise, 0
>	1 when the left operand is greater than the right operand; otherwise, 0
<=	1 when the left operand is less than or equal to the right operand; otherwise, 0
>=	1 when the left operand is greater than or equal to the right operand; otherwise, 0
=	1 when the left operand is equal to the right operand; otherwise, 0
^=	1 when the left operand is not equal to the right operand; otherwise, 0
any logical operator	null when either or both operators are null
Concatenation expression	
or !!	the left operand merged with the right operand to form a new value
..	strips each argument before concatenating
IF expression	
IF	1 when the comparison expression is contained in the constant list
IN expression	
IN	1 when the comparison expression is contained in the constant list
LIKE expression	
LIKE	1 when the comparison expression is contained in the constant list
SELECT expression	
SELECT	1 when the comparison expression is contained in the constant list

¹ The || concatenation operator does not remove any spaces. You can use the TRIM function to remove trailing spaces. However, the .. operator performs the same function as TRIM followed by concatenation. It is faster to use the .. operator (a .. b) than to use the TRIM function (TRIM(a) || TRIM(b)).

Short-Circuit Evaluation in SAS

Minimal evaluation, or short-circuit evaluation, is a technique that some programming languages use to evaluate Boolean operators. In short-circuit evaluation, the second argument in a Boolean expression is executed only if the first argument does not determine the value of the overall expression. For example, in the expression (X AND Y), if the first argument, (X) evaluates to FALSE, then the overall expression (X AND Y) must evaluate to FALSE. So there is no need to calculate the second argument (Y).

SAS does not guarantee short-circuit evaluation. When using Boolean operators to join expressions, you might get undesired results if your intention is to short circuit, or avoid the evaluation of, the second expression. To guarantee the order in which SAS evaluates an expression, you can rewrite the expression using nested IF statements.

In the first example below, the logical expression does not short-circuit at the first expression, and the program fails because a division by zero is detected in the second expression.

```
data _null_;
  declare double a;
  method init();
    a=0;
    if (a>1 AND 1/a) then put 'hello';
    else put 'goodbye';
  end;
enddata;
run;
quit;
```

The following lines are written to the SAS log:

```
ERROR: Float divide by zero
ERROR: General error
```

In the next example, the same logical expression is written using nested IF statements to guarantee the order of evaluation and to ensure that the program runs successfully.

```
data _null_;
  declare double a;
  method init();
    a=0;
    if a>1 then
      if 1/a then put 'hello';
      else put 'goodbye';
    else put 'goodbye again';
  end;
enddata;
run;
quit;
```

The following lines are written to the SAS log:

```
goodbye again
```


Dates and Times in DS2

<i>DS2 Dates, Times, and Timestamps</i>	101
Overview of DS2 Dates, Times, and Timestamps	101
Declaring Date, Time, and Timestamp Variables	102
DS2 Date, Time, and Timestamp Values	102
Operations on DS2 Dates and Times	103
<i>SAS Date, Time, and Datetime Values</i>	103
<i>Converting SAS Date, Time, and Datetime Values to a DS2 Date, Time, or Timestamp Value</i>	104
<i>Converting DS2 Date, Time, and Timestamp Values to SAS Date, Time, or Datetime Values</i>	105
<i>Date, Time, and Datetime Functions</i>	106
<i>Date, Time, and Datetime Formats</i>	107

DS2 Dates, Times, and Timestamps

Overview of DS2 Dates, Times, and Timestamps

DS2 supports the SQL style date and time conventions that are used in other data sources. When your data source is not a SAS data set, DS2 can process dates and times that have a data type of DATE, TIME, and TIMESTAMP.

Date and time values with a data type of DATE, TIME, and TIMESTAMP can be converted to a SAS date, time, or datetime value. When a numeric column is read from a SAS data set and the numeric column has a SAS date, time, or datetime format associated with it, the column is converted to a DS2 type DATE, TIME, or TIMESTAMP data type.

DS2 provides date and time functions that convert any date or time value to SAS date, time, and datetime values, and back again to a recognizable date or time value. For more information, see [“Date, Time, and Datetime Functions” on page 106](#).

The date and time intervals that are supported in ANSI SQL are not supported in DS2.

Declaring Date, Time, and Timestamp Variables

You declare a date, time, or timestamp variable by using the DATE, TIME, or TIMESTAMP data types in the DECLARE statement, as in this example:

```
dcl date dt;
dcl time tm;
dcl timestamp tmstp;
```

Note: If you use a precision when you declare a time or timestamp variable, the time or timestamp values are not rounded to the specified precision until they are generated by the DATA statement. Internally, the time or timestamp constant values are simply copied to the time or timestamp variable.

Note: If you are working with TIME and TIMESTAMP values in a data source other than SAS and you do not specify a precision, the default precision is the DS2 default precision of 6 for TIME and 6 for TIMESTAMP.

For additional information about the DS2 date and time data types, see [Chapter 5, “DS2 Data Types,”](#) on page 47.

DS2 Date, Time, and Timestamp Values

Once you declare a date, time, or timestamp variable, the value of the variable can be only a DS2 date, time, or timestamp constant that has the following syntax:

DATE'yyyy-mm-dd'

TIME'hh:nn:ss[.fraction]'

TIMESTAMP'yyyy-mm-dd hh:nn:ss[.fraction]'

where

- *yyyy* is a four-digit year
- *mm* is a two-digit month, 01–12
- *dd* is a two-digit day, 01–31
- *hh* is a two-digit military hour
- *nn* is a two-digit minute
- *ss* is a two-digit second
- *fraction* can be one to nine digits, 0–9, is optional, and represent a fraction of a second

The string portion of the value after the DATE, TIME, or TIMESTAMP keyword must be enclosed in single quotation marks.

In the date constant, the hyphens are required and the length of the date string must be 10.

In the time constant, the colons are required. If the fraction of a second is not present, the time string must be 8 characters long and exclude the period. DS2 issues an error if the period is present without a fraction. If the fraction of second is present, the fraction can be up to 9 digits long and the time string can be up to 18 characters long (including the period).

In the timestamp constant, the hyphens in the date are required as well as the colons in the time. If the fraction of a second is not present, the timestamp string must be 19 characters long and exclude the period. If the fraction of a second is present, the fraction can be up to 9 digits long and the timestamp string can be up to 29 characters long.

Here are some examples of DS2 date, time, and timestamp constants:

```
date '2017-01-31'
time '20:44:59'
timestamp '2017-02-07 07:00:00.7569'
```

Operations on DS2 Dates and Times

The only operations that can be performed on DATE, TIME, and TIMESTAMP values are operations that use the relational operators <, >, <=, >=, =, ^=, and IN, such as in the following statement:

```
if tm in(time'10:22:31', time'12:55:01') then
  if tm < time'13:30:00' then put 'Early afternoon';
  else put 'Time not available';
```

DS2 does not calculate date and time intervals on values that have the data types of DATE, TIME, and TIMESTAMP.

SAS Date, Time, and Datetime Values

A SAS date value is the number of days between January 1, 1960 and a specified date. Dates before January 1, 1960 are negative numbers; dates after are positive numbers. For example, the SAS date value for January 1, 1960 is 0, -365 for January 1, 1959, and 17532 for January 1, 2008.

A SAS time value is the number of seconds since midnight of the current day. SAS time values are between 0 and 86400.

A SAS datetime value is the number of seconds between January 1, 1960 and a specific hour, minute, and second of a specific date.

When a numeric column is read from a SAS data set and the numeric column has a SAS date, time, or datetime format associated with it, the column is converted to a DS2 type DATE, TIME, or TIMESTAMP. If the numeric column in a SAS data set does not have a format or has a format that is not a SAS date, time, or datetime format, the column is processed as type DOUBLE.

Once a DOUBLE has been converted to a DS2 type DATE, TIME, or TIMESTAMP, all calculations on dates and times are done as a SAS date value, a SAS time value, or a SAS datetime value. For more information, see [“Date, Time, and Datetime Functions” on page 106](#).

After calculations are complete, there are other functions that can then format the SAS date, time, and datetime values to recognizable date and time formats.

Converting SAS Date, Time, and Datetime Values to a DS2 Date, Time, or Timestamp Value

SAS date, time, and datetime values can be converted to DS2 dates, time, and timestamp values by using the TO_DATE, TO_TIME, and TO_TIMESTAMP functions. The argument of these functions is any value or expression that represents a SAS date, time, or datetime value and has a type DOUBLE. You can then use either the PUT statement or a format in the DECLARE statement to format the date, time, or timestamp value.

Here is an example.

```
data _null_;
  dcl date ds2d having format YYMMDD10.;
  dcl time ds2t having format TIME18.9;
  dcl timestamp ds2dt having format DATETIME28.9;
  dcl double d t ts;
  method init();
    d = 20854;
    ds2d = to_date(d);
    ds2t = to_time(d);
    ds2dt = to_timestamp(d);
    put ds2d ds2t ds2dt;
  end;
enddata;
run;
```

The following lines are written to the SAS log.

```
2017-02-04
5:47:34.000000000
01JAN1960:05:47:34.000000000
```

For more information, see the “[TO_DATE Function](#)” in *SAS DS2 Language Reference*, the “[TO_TIME Function](#)” in *SAS DS2 Language Reference*, and the “[TO_TIMESTAMP Function](#)” in *SAS DS2 Language Reference*.

Converting DS2 Date, Time, and Timestamp Values to SAS Date, Time, or Datetime Values

DS2 date, time, and timestamp values can be converted to a SAS datetime value by using the `TO_DOUBLE` function. This function converts the date, time, or timestamp CHAR or NCHAR string to a SAS datetime value with a data type of DOUBLE. You can then use any DS2 format to display the value in a date, time, or datetime format.

The following DS2 program illustrates how you can convert a DS2 timestamp to a SAS date, time, and datetime values:

```
data _null_;
  method run();
    dcl timestamp DS2ts;
    dcl double sasdtval sasd sastm;
    dcl char(28) fmtdate fmttime fmtdt;
    DS2ts = timestamp '2017-06-13 10:54:34.012';
    put DS2ts;
    sasdtval = to_double(DS2ts);
    sasd = datepart(sasdtval);
    sastm = timepart(sasdtval);
    put sasdtval:best16.7;
    put sasd:best.;
    put sastm:best.;
    fmtdate = put(sasd, yymmdd10.);
    fmttime = put(sastm, time.);
    fmtdt = put(sasdtval, datetime21.7);
    put fmtdate=;
    put fmttime=;
    put fmtdt=;
  end;
enddata;
run;
```

The following output is written to the SAS log:

```
2017-06-13 10:54:34.012000000
1812970474.012
20983
39274.012
fmtdate=2017-06-13
fmttime=10:54:34
fmtdt=13JUN17:10:54:34.0120
```

In this example, a SAS date value is formatted to look like DS2 date value, but it has a data type of DOUBLE and not DATE. SAS date and time values cannot be assigned to DS2 date or time variables. Their data types are different. If you attempt to assign a SAS date or time value to a DS2 date or time variable, DS2 issues a data type invalid conversion error.

For more information, see the “[PUT Function](#)” in *SAS DS2 Language Reference*. For a complete list of formats, see “[Date, Time, and Datetime Formats](#)” on page 107.

Date, Time, and Datetime Functions

In order to perform date and time calculations, DS2 date and time functions do the following:

- retrieve or convert a date and time as a SAS date, time, or datetime value
- format a SAS date, time, or datetime value into a recognizable date or time
- extract a date or a time from a SAS datetime value

The following tables list the date and time functions and what they do. For specific information about any of these functions, see “[DS2 Functions](#)” in *SAS DS2 Language Reference*.

Table 10.1 *Functions That Retrieve or Convert Dates and Times into a SAS Date, Time, or Datetime Value*

Function	Description
DATE	Returns the current date as a SAS date value
TODAY	Returns the current date as a SAS date value
DATEJUL	Converts a Julian date to a SAS date value
DHMS	Returns a SAS datetime value from date, hour, minute, and second values
HMS	Returns a SAS time value from hour, minute, and second values
MDY	Returns a SAS date value from month, day, and year values
TIME	Returns the current time of day as a SAS time value.
YYQ	Returns a SAS date value from a year and quarter year values

Table 10.2 *Functions That Format a SAS Date or Datetime Value as a Recognizable Date or Time*

Function	Description
DATE	Returns the day of the month from a SAS date value
HOUR	Returns the hour from a SAS time or datetime value
JULDATE	Returns the Julian date from a SAS date value
JULDATE7	Returns a seven-digit Julian date from a SAS date value
MINUTE	Returns the minute from a SAS time or datetime value
MONTH	Returns a number that represents the month from a SAS date value
QTR	Returns the quarter of the year from a SAS date value
SECOND	Returns the second from a SAS time or datetime value
WEEKDAY	Returns an integer that corresponds to the day of the week, from a SAS date value
YEAR	Returns the year from a SAS date value

Table 10.3 *Functions That Extract Date and Times from SAS Datetime Values*

Function	Description
DATEPART	Extracts the date from a SAS datetime value and returns the date as a SAS date value
TIMEPART	Extracts the time from a SAS datetime value and returns the time as a SAS datetime value

Date, Time, and Datetime Formats

DS2 formats write SAS date, time, and datetime values as recognizable dates and times. You use the PUT function to format a SAS date, time, or datetime value:

```
PUT(SAS-value, format.);
```

The first argument to the PUT function is the SAS date value, a SAS datetime value, or a SAS time value. The second argument is the format.

Here is a simple DS2 program that shows how the PUT function is executed:

```
proc ds2;
data _null_;
  declare double d;
  declare varchar(15) x;
  method run();
    d=SAS-date-value;
    x=put (d,format-name.);
    put x=;
  end;
enddata;
run;
quit;
```

The input variable is a DOUBLE. The output variable is a character variable. The PUT function operates on character variables.

Table 10.4 on page 108 illustrates the results of formatting SAS date, datetime, and time values that represent June 13, 2017 (2017-06-13 10:54:34.012) using the various DS2 format types. For information about how the SAS date, time, and datetime values were obtained, refer back to “Converting DS2 Date, Time, and Timestamp Values to SAS Date, Time, or Datetime Values” on page 105.

Note: When working with SAS date, time, and datetime formats, the SAS input value must be appropriate for the format type. That is, specify a SAS date value with date formats. Specify a SAS datetime value with datetime formats. Specify a SAS time value with time formats. A SAS date value is used with quarter and year formats. If you do not specify an appropriate value, DS2 returns an inappropriate response.

Table 10.4 Examples of DS2 Date and Time Formats

Format Type	Format Name	SAS Input Value	Format Specification	Result
Date	DATEw.	20983	Default	13JUN17
		20983	DATE9.	13JUN2017
	DAYw.	20983	Default	13
	DDMMYYw.	20983	Default	13/06/17
		20983	DDMMYY10.	13/06/2017
	DDMMYYxw.	20983	DDMMYYB.	13 06 17
		20983	DDMMYYB10.	13 06 2017
		20983	DDMMYYC.	13:06:17

Format Type	Format Name	SAS Input Value	Format Specification	Result
		20983	DDMMYYC10.	13:06:2017
		20983	DDMMYYD.	13-06-17
		20983	DDMMYYD10.	13-06-2017
		20983	DDMMYYN6.	130617
		20983	DDMMYYN8.	13062017
		20983	DDMMYYP.	13.06.17
		20983	DDMMYYP10.	13.06.2017
		20983	DDMMYYYS.	13/06/17
		20983	DDMMYYYS10.	13/06/2017
	DOWNAMEw.	20983	Default	Tuesday
	JULIANw.	20983	Default	17164
	MMDDYYw.	20983	Default	06/13/17
		20983	MMDDYY10.	06/13/2017
	MMDDYYxw.	20983	MMDDYYB.	06 13 17
		20983	MMDDYYB10.	06 13 2017
		20983	MMDDYYC.	06:13:17
		20983	MMDDYYC10.	06:13:2017
		20983	MMDDYYD.	06-13-17
		20983	MMDDYYD10.	06-13-2017
		20983	MMDDYYN6.	061317
		20983	MMDDYYN8.	06132017
		20983	MMDDYYP.	06.13.17
		20983	MMDDYYP10.	06.13.2017
		20983	MMDDYYYS.	06/13/17
		20983	MMDDYYYS10.	06/13017
	MMYYw.	20983	Default	06M2017

Format Type	Format Name	SAS Input Value	Format Specification	Result
	MMYYxw.	20983	MMYYC.	06:2017
		20983	MMYYD.	06-2017
		20983	MMYYN.	062017
		20983	MMYYP.	06.2017
		20983	MMYYs.	06/2017
	MONNAMEw.	20983	Default	June
	MONTHw.	20983	Default	6
	MONYYw.	20983	Default	JUN17
	WEEKDATEw.	20983	Default	Tuesday, June 13, 2017
	WEEKDATXw. .	20983	Default	Tuesday, 13 June 2017
	WEEKDAYw.	20983	Default	3
Quarter	QTRw.	20983	Default	2
	QTRRw.	20983	Default	II
Datetime	DATEAMPMw. .d	181297047 4.012	Default	13JUN17:10:54:34 AM
	DATETIMEw. d	181297047 4.012	Default	13JUN17:10:54:34
	DTDATEw.	181297047 4.012	Default	13JUN17
	DTMONYYw.	181297047 4.012	Default	JUN17
	DTWKDATXw. .	181297047 4.012	Default	Tuesday, 13 June 2017
	DTYEARw.	181297047 4.012	Default	2017
	DTYYQCw.	181297047 4.012	Default	17:2
Time	HOURw.d	39274.012	Default	11
	TIMEw.d	39274.012	Default	10:54:34

Format Type	Format Name	SAS Input Value	Format Specification	Result
	TIMEAMPWw.d	39274.012	Default	10:54:34 AM
	TODw.d	39274.012	Default	10:54:34
Year	YEARw.	20983	Default	2017
	YYMMw.	20983	Default	2017M06
	YYMMxw.	20983	YYMMC.	2017:06
		20983	YYMMD.	2017-06
		20983	YYMMN.	201706
		20983	YYMMP.	2017.06
		20983	YYMMS.	2017/06
	YYMMDDw.	20983	Default	17-06-13
	YYMMDDxw.	20983	YYMMDDB.	17 06 13
		20983	YYMMDDC.	17:06:13
		20983	YYMMDDD.	17-06-13
		20983	YYMMDDN.	20170613
		20983	YYMMDDP.	17.06.13
		20983	YYMMDDS.	17/06/13
	YYMONw..	20983	Default	2017JUN
Year/ Quarter	YYQw.	20983	Default	2017Q2
	YYQxw.	20983	YYQC.	2017:2
		20983	YYQD.	2017-2
		20983	YYQP.	2017.2
		20983	YYQS.	2017/2
		20983	YYQN.	20172
	YYQRw.	20983	Default	2017QII
	YYQRxw.	20983	YYQRC.	2017:II

Format Type	Format Name	SAS Input Value	Format Specification	Result
		20983	YYQRD.	2017-II
		20983	YYQRP.	2017.II
		20983	YYQRS.	2017/II
		20983	YYQRN.	2017II
	YYQZw.	20983	Default	1702

DS2 Arrays

Overview of DS2 Arrays	113
Temporary Arrays	114
Overview of Temporary Arrays	114
Temporary Array Declaration	115
Variable Arrays	115
Overview of Variable Arrays	115
Variable Array Declaration	116
Definition of Variables in a VARARRAY Statement	117
Delayed Variable Definition with DIM Variable Array Bounds	118
Using the OF Operator with Arrays	119
Declaring Arrays with a HAVING Clause	120
Array Assignment	122
Overview of Array Assignment	122
Array Assignment from Another Array	122
Special Case for Double Missing Values	123
Array Assignment with Variable Arrays	123
Array Assignment from a Constant List	124
Array Arguments	125
Overview of Array Arguments	125
Defining Array Parameters	126
Bounded Array Parameters	127
Unbounded Array Parameters	127
How to Query Array Dimensions	128
How to Write Array Content	129

Overview of DS2 Arrays

In DS2, an array is a named aggregate collection of homogeneous data. DS2 has two types of arrays: temporary and variable. These arrays have the following characteristics.

- homogeneous by type
- multidimensional (the number of bounds can be ≥ 1)
- indexed by signed integer values
- exists only for the duration of the DS2 program or DS2 procedure
- not a DS2 variable in the PDV, though the elements of a variable array can refer to variables in the PDV
- array elements do not appear in a result table, though variables referenced by elements of a variable array can appear in the results table

The following table shows some of the differences between temporary and variable arrays.

Temporary Array	Variable Array
set of temporary elements	set of references to variables in the PDV
created with a DECLARE statement	created with a VARARRAY statement
can be declared in local or global scope	must be declared in global scope
similar to arrays seen in other languages	similar to arrays of pointers or references seen in other languages
in DATA step, created with an ARRAY statement with <code>_TEMPORARY_</code> argument	in DATA step, created with an ARRAY statement

Temporary Arrays

Overview of Temporary Arrays

The elements of a temporary array are temporary in that they are not located in the PDV and therefore do not appear in any result table. Temporary data element values are automatically retained across iterations rather than being reset to missing at the beginning of the next iteration. Temporary arrays exist only for the duration of the DS2 program.

You use the DECLARE statement to specify the name, data type, and number and size of the array bounds. You can also use a HAVING clause in the DECLARE statement to associate label, format, and informat attributes with a temporary array. For example, the following DECLARE statement specifies a three-element temporary array that stores three temporary double values outside the PDV.

```
declare double a[3]
```

Temporary Array Declaration

Temporary array declarations are similar to scalar declarations. In addition to the data type and name, you can also specify the number and size of the array bounds. Multiple bounds (or dimensions) are specified using comma separators.

The form of signed integer pairs specifies the lower and upper bounds for each dimension of the array $[l:h]$, where l represents the lowest index for the given bound and h represents the highest index for the given bound. The lower bound specification, l , is optional. If the lower bound of a dimension is not specified, then the lower bound defaults to 1.

An error is returned if the upper bound, h , is less than the lower bound. If you specify an array bound with only one integer, then that integer is interpreted as the upper bound. The default lowest bound is 1.

The upper bound of an array can also be sized based on the number of elements in a dimension of a previously declared array. You use a DIM function call for the upper bound. The DIM function is the only function that can be used to specify an upper array bounds. The DIM function cannot be used to specify the lower bound of a dimension.

The part of the DECLARE statement for temporary array declaration is as follows.

DECLARE *data-type* <variable-list> [*having-clause*];

<variable-list>::=<variable> [...<variable>]

<variable>::=*identifier* <array-declaration>

<array-declaration>::=\[<array-bound>[, ...<array-bound>]\]

<array-bound>::= {[*dim-lower*:]*dim-upper*} | {[*dim-lower*:] {DIM(*a*[, *n*)]} | *}

For more information, see [“DECLARE Statement” in SAS DS2 Language Reference](#).

Variable Arrays

Overview of Variable Arrays

Variable arrays are a way to simplify processing of a series of variables that have a similar name or purpose in the input data. The elements of a variable array refer to variables in the PDV. Variable arrays exist only for the duration of the DS2 program. However, the content of the referenced variables might be preserved in one or more result sets.

You use the VARARRAY statement to specify the name, data type, and number and size of the array bounds. For example, the following VARARRAY statement specifies a three-element variable array that refers to three double variables (a1, a2, a3) in the PDV.

```
vararray double a[3];
```

The VARARRAY statement in the previous example creates any of the double variables (a1, a2, a3) that have not previously been created. Variable array element a[1] references variable a1, a[2] references variable a2, and a[3] references variable a3.

After a variable array is created, the variable array elements act as a second set of identifiers that can be used to read or modify the data that is stored in the referenced variables in the PDV.

You can also use a HAVING clause in the VARARRAY statement to associate label, format, and informat attributes with a variable array.

Note: All character variables in a character variable array must have the same length and encoding.

For more information, see [Chapter 11, “DS2 Arrays,” on page 113](#) and the [“VARARRAY Statement” in SAS DS2 Language Reference](#).

Variable Array Declaration

Variable array declarations are similar to scalar declarations. In addition to the data type and name, you can also specify the number and size of the array bounds. Multiple bounds (or dimensions) are specified using comma separators.

Array bounds have two forms.

signed integer pairs

The form of signed integer pairs specifies the lower and upper bounds for each dimension of the array. $[l:h]$, where l represents the lowest index for the given bound and h represents the highest index for the given bound. The lower bound specification, l , is optional. If the lower bound of a dimension is not specified, then the lower bound defaults to 1.

An error is returned if the upper bound, h , is less than 1. If you specify an array bound with only one integer, then that integer is interpreted as the upper bound. The default lowest bound is 1.

`vararray double a[5];` declares an array a of type double, with five elements that are indexed from 1 to 5. `vararray char b[5,10];` declares a two-dimensional character array b with 5 elements in the first dimension and 10 elements in the second dimension for a total of 50 elements in the array. `vararray int c[3] x y z;` declares an array c with three elements. The array is indexed with a lower bound of 1 and an upper bound of 3.

The upper bound of an array can also be sized based on the number of elements in a dimension of a previously declared array. You use a DIM function call for the

upper bound. The DIM function is the only function that can be used to specify an upper array bound. The DIM function cannot be used to specify the lower bound of a dimension.

* (asterisk)

The * form specifies a one-dimensional array in which the lower bound is 1 and the upper bound is the number of variables in the variable list.

For more information, see [“Variable Lists” on page 28](#). For more information about how to declare variable arrays and how to specify multiple bounds, see the [“VARARRAY Statement” in SAS DS2 Language Reference](#).

Definition of Variables in a VARARRAY Statement

A VARARRAY statement defines any variable in its variable list that has not previously been defined. Some variable list types reference only existing variables and therefore do not result in the definition of new variables. The following table shows which variable list types can define new variables and which variable list types reference only existing variables.

Variable List	Example	Variable Expansion	Description
name	x y z	immediate	can define new variables
numbered range	x1-x5	immediate	can define new variables
name range	sales_jan--sales_mar	delayed	reference only existing variables
name prefix	sales:	delayed	reference only existing variables
type	smallint	delayed	reference only existing variables
special name	_all_	delayed	references only existing variables

The name and numbered range variable lists can be expanded without examining all the variables in the DS2 program. Therefore, these types of variable lists are expanded immediately when the VARARRAY statement is encountered in the program.

The other types of variable lists must examine all variables that are defined in the DS2 program. Expansion of these variable lists is delayed until after all statements in the DS2 program have been examined and all variables have been defined.

This delay can lead to some unexpected error conditions. For example, consider the following program.

```
data;
  1 vararray int x[1] x;;
  method run();
```

```

2 x1 = 5.0;
end;
enddata;
run;

```

- 1 The VARARRAY statement does not create any variables because the prefix variable list `x:` references only existing variables. The expansion of prefix variable list `x:` is delayed until all statements in the program have been examined.
- 2 In the assignment statement variable `x1` is undefined. Therefore, the assignment statement assigns the type of the right-hand side value (DOUBLE) to variable `x1`.

After all the program statements are examined, the prefix variable list `x:` is expanded to `x1`, the only existing variable with prefix `x`. The DS2 compiler then issues a compilation error because variable `x1` of type DOUBLE is incompatible with variable array `x` of type INTEGER.

One way to remove the error condition is to change the VARARRAY statement, `vararray int x[1] x:;`, to `vararray int x[1] x1;` or `vararray int x[1];`. The revised statement defines the variable `x1` as type INTEGER.

Delayed Variable Definition with DIM Variable Array Bounds

If a variable array has an upper dimension bound based on the dimension of another array, then the definition of variables for the array can be delayed until all statements in the program have been examined. Here is an example.

```

data;
1 vararray double x[*] x;;
2 vararray int out[dim(x)];

method init();
3 out1 = 0.0;
end;

method run();
set in;
end;
enddata;
run;

```

- 1 The expansion of prefix variable list `x:` is delayed until all statements in the program have been examined. Therefore, the size of variable array `x` is not known until all statements have been examined.
- 2 The `out` variable array has the default variable list `out1-out $n$` , where n is the number of elements specified for the variable array. The determination of the number of elements in `out` is delayed until the size of array `x` is known (this occurs when all statements have been examined).

- 3 In the assignment statement, variable out1 is undefined. Therefore, the assignment statement assigns the type of the right-hand side value, DOUBLE, to variable out1.

After all statements in the above DS2 program are processed, the following occurs. Assume the table in has 3 double variables, x1 x2 x3.

- The prefix variable list x: is expanded to x1 x2 x3.
- The size of variable array x is determined to be 3.
- The size of variable array out is determined to be 3 (dim(x)).
- The default variable list out1-out3 is expanded to out1 out2 out3.
- The variables out2 and out3 are defined as type INTEGER because they were not previously defined. Note that out1 was defined as type DOUBLE by the assignment statement `out1 = 0.0;`.

The DS2 compiler issues a compilation error because variable out1 of type DOUBLE is incompatible with variable array out of type INTEGER.

One way to remove the error condition is to change the array assignment, `out1 = 0.0;`, to `out[1]=0.0;`. This change updates the out1 data value by means of an out array reference to prevent the assignment statement from assigning type DOUBLE to the variable out1.

Using the OF Operator with Arrays

You can use the OF operator with DS2 variable arrays. This capability enables the passing of variable arrays to most functions whose arguments contain a varying number of parameters.

There are some rules and limitations when using variable arrays. These rules and limitations are listed after the example.

The following example shows how you can use a variable array in a SUM function:

```
data _null_;
  vararray double a[4];
  method init();
    a:=(1,2,3,4);
  end;

  method run();
    dcl double x y z;
    y=99.0;
    z=100.0;
    x=sum(z, of a[*], y);
    put x=;

    x= sum(of a:, z);
    put x=;
  end;
enddata;
run;
```

Example Code 11.1 Log Output for the Example of Using Variable Arrays

```
x=209
x=110
```

The following rules and limitations apply to variable array OF lists:

- can be used in functions where the number of parameters matches the number of elements in the OF list

Note: A function can contain no OF lists, one OF lists, some OF lists, some OF lists with other expressions, and so on. The total number of arguments after the all of the OF lists are expanded must match the number of arguments that the function takes if the function has a fixed number of arguments.

- can be used in functions that take a varying number of parameters
- cannot be used as array indices
- cannot be used with the DIF and LAG functions, nor with any of the variable information functions such as VLENGTH
- cannot be used with functions that are specified in a WHERE clause. Here is an example:

```
where range(of x1-x3);
```

Declaring Arrays with a HAVING Clause

The declaration statement for a temporary or variable array can contain a HAVING clause. The HAVING clause associates a label, format, and informat attribute with the array. If the array is a variable array, then the HAVING clause is also associated with the variables referenced by the variable array.

The decision about when to apply the HAVING clause to a variable that is referenced by the variable array depends on when the variable list that contains the variable reference is expanded. Name and numbered range variable lists are normally expanded when the VARARRAY statement is processed. Therefore, the HAVING clause is applied to all variables referenced by these lists at that time. Name range, name prefix, type, and special name variable lists are expanded after all statements in the program have been examined and all variables in the program have been defined. Consequently, the HAVING clause for all variables referenced by these variable lists is applied after all statements in the DS2 program have been examined.

Consider the following program.

```
data;
1 declare double x1 x2 having format 5.0;
2 vararray double x[3] having format 5.2;
3 declare double x3 having format ROMAN5.;
```

```
enddata;
```

- 1 The DECLARE statement is processed. Variables x1 and x2 are defined. The HAVING clause `format 5.0` is associated with variables x1 and x2.

Variable	x1	x2
Format	5.0	5.0

- 2 The VARARRAY statement is processed. Default variable list x1-x3 is expanded to x1 x2 x3. Variable x3 is defined. The HAVING clause `format 5.2` is associated with variables x1 x2 x3.

Variable	x1	x2	x3
Format	5.2	5.2	5.2

- 3 The DECLARE statement is processed. The HAVING clause `format ROMAN5.` is associated with variable x3.

Variable	x1	x2	x3
Format	5.2	5.2	ROMAN5.

Now consider what happens if the variable list type is modified to a type that results in delayed processing of the variable list.

```
data;
1 declare double x1 x2 having format 5.0;
2 4 vararray double x[*] x: having format 5.2;
3 declare double x3 having format ROMAN5.;
enddata;
```

- 1 The DECLARE statement is processed. Variables x1 and x2 are defined. The HAVING clause `format 5.0` is associated with variables x1 and x2.

Variable	x1	x2
Format	5.0	5.0

- 2 The VARARRAY statement processing is delayed or begins processing. Prefix variable list x: cannot be expanded until all statements in the program have been examined and all variables are defined.

Variable	x1	x2
Format	5.0	5.0

- 3 The DECLARE statement is processed. Variable x3 is defined. The HAVING clause `format ROMAN5.` is associated with variable x3.

Variable	x1	x2	x3
Format	5.0	5.0	ROMAN5.

- 4 The VARARRAY statement completes processing. Prefix variable list x: is expanded to x1 x2 x3. The HAVING clause `format 5.2` is associated with variables x1 x2 x3.

Variable	x1	x2	x3
Format	5.2	5.2	5.2

Array Assignment

Overview of Array Assignment

DS2 supports array assignment with the `:=` operator. The syntax for assigning an array or constant list is as follows:

```
array:=array;
```

```
array:=(constant list);
```

In an array assignment, *array* can be either a temporary or variable array.

Array Assignment from Another Array

When you assign one array to another array, the data types of the two arrays must be compatible (either the same or convertible). The number of dimensions and the total number of elements in each dimension do not have to be the same.

Consider the assignment from array *y* to array *x* as shown in this statement.

```
x:=y;
```

During the assignment, each element of array *y* is assigned to each element of array *x*, for example, `x[1]=y[1]; . . . x[n]=y[n];`

The basic algorithm for evaluating `x[i] = y[i]` follows. First `y[i]` is examined to see whether it is missing (SAS mode) or null (ANSI mode).

- If `y[i]` is missing or null, then missing or null is assigned to `x[i]`.¹
- If `y[i]` is not missing or null, then the types of `x[i]` and `y[i]` are examined.
 - If the type of `y[i]` is different from the type of `x[i]`, then `y[i]` is converted to the type of `x[i]`.
 - If the conversion of `y[i]` succeeds, then the result of the conversion is assigned to `x[i]`.
 - If the conversion of `y[i]` fails, then missing or null is assigned to `x[i]`.
 - If the type of `y[i]` is the same as the type of `x[i]`, then `y[i]` is assigned to `x[i]`.

If array *x* and array *y* do not have the same number of elements, then as many elements as possible are assigned from array *y* to array *x* and null or missing is

1. The decision to assign missing or null to an array element depends on data type of the array and whether the program is running in SAS or ANSI mode. For more information, see [Chapter 7, “How DS2 Processes Nulls and SAS Missing Values,” on page 61](#).

assigned to any remaining elements in array *x*. The length of array *x* is not modified by the assignment.

In the following example, array *x* has ten elements, array *y* has seven elements, and array *y* is assigned to array *x*. Therefore, the seven elements from array *y* are assigned to the first seven elements of array *x*, and missing is assigned to the last three elements of array *x*.

```
data _null_;
  method init();
    declare double x[10];
    declare double y[7];

    x := (0 0 0 0 0 0 0 0 0 0);
    y := (1 2 3 4 5 6 7);
    x := y;
    put x[*]=;
  end;
enddata;
run;
```

The following lines are written to the SAS log.

```
x[1]=1 x[2]=2 x[3]=3 x[4]=4 x[5]=5 x[6]=6 x[7]=7 x[8]=. x[9]=. x[10]=.
```

Special Case for Double Missing Values

If *y*[*i*] is a DOUBLE with a SAS missing value (for example, .Z), then DS2 tries to preserve the SAS missing value during the assignment according to these rules:

- If *x*[*i*] is a DOUBLE, then the SAS missing value from *y*[*i*] is assigned to *x*[*i*].
- If *x*[*i*] is a character string, then the missing character representation of *y*[*i*] (for example, Z for .Z, is assigned to *x*[*i*]).

This special case processing occurs only when DS2 is in SAS mode and the type of *y*[*i*] is a DOUBLE.

For more information about null and missing values, see [Chapter 7, “How DS2 Processes Nulls and SAS Missing Values,”](#) on page 61. For an example of an array assignment, see [“Example: Arrays” in SAS DS2 Language Reference](#).

Array Assignment with Variable Arrays

The elements of a variable array reference variables in the PDV. In the array assignment statement *x* := *y*, the value of the elements of *y* are assigned elementwise to the elements of *x*. If either *x* or *y* is a variable array (vararray), then the assignment is always *x*[*i*]=*y*[*i*].

Array assignment to a variable array does not modify the elements (this is, the references) in the variable array. Instead, the data in the variables referenced by the elements of the array are modified. Similarly, in an array assignment from a variable array, the data in the variables referenced by the elements of the variable array are used for the assignment.

Array Assignment from a Constant List

To assign from a constant list to an array, the constants in the constant list must be compatible (either the same or convertible) with the data type of the array. The constant list and the array do not have to have the same number of dimensions or the same number of elements in each dimension.

Assume this array assignment statement.

```
x := (c1 c2 c3 ... cn);
```

During array assignment from a constant list to array *x*, each element of the constant is assigned to each element of array *x* as shown in the following expanded form.

```
x[1] = c1;
x[2] = c2;
...
x[i] = ci;
...
x[n] = cn;
```

The basic algorithm for evaluating $x[i] = ci$ follows. First *ci* is examined to see whether it is missing or null.¹

- If *ci* is missing or null, then missing (SAS mode) or null (ANSI mode) is assigned to $x[i]$.
- If *ci* is not missing or null, then the types of $x[i]$ and *ci* are examined.
 - If the type of *ci* is different from the type of $x[i]$, then *ci* is converted to the type of $x[i]$.
 - If the conversion of *ci* succeeds, then the result of the conversion is assigned to $x[i]$.
 - If the conversion of *ci* fails, then missing or null is assigned to $x[i]$.
 - If the type of *ci* is the same as the type of $x[i]$, then *ci* is assigned to $x[i]$.

If the constant list and the array *x* do not have the same number of elements, then as many constants as possible are assigned from the constant list to array *x* and a null or missing value is assigned to any remaining elements in array *x*. The length of array *x* is not modified by the assignment.

In the following example, a constant list having five constants is assigned to array *x* having seven double elements. The five constants in the constant list are assigned

1. The decision to assign missing or null to an array element depends on data type of the array and whether the program is running in SAS or ANSI mode. For more information, see [Chapter 7, "How DS2 Processes Nulls and SAS Missing Values,"](#) on page 61.

to the first five elements of array **x**, and missing is assigned to the last two elements of array **x**.

```
data _null_;
  method init();
    declare double x[7];
    x := (1 '2' 3.3 '' .Z);
    put x[*]=;
  end;
enddata;
run;
```

The following lines are written to the SAS log.

```
x[1]=1 x[2]=2 x[3]=3.3 x[4]=. x[5]=Z x[6]=. x[7]=.
```

Note: The types of the elements in the constant list can be heterogeneous as long as all the types of the elements are convertible to the type of the assigned to array.

Here is another example of an array assignment from a constant list.

```
declare char(2) a[2, 3];
...
a := (('aa' 'bb' 'cc') ('dd' 'ee' ''));
```

The elements in array **a** after the above assignment statement would look like this.

```
'aa'  'bb'  'cc'
'dd'  'ee'  ''
```

Array Arguments

Overview of Array Arguments

DS2 arrays can be passed as arguments to DS2 methods. DS2 arrays are always passed by reference to methods. DS2 arrays cannot be passed by value, that is, a copy of the array cannot be supplied as an argument to a method. DS2 array arguments must have the same type as specified by the array parameter in order to match the array parameter. Array arguments do not support implicit type conversion to a different type. DS2 arrays are passed as either a bounded array parameter (for example, `a[8]`) or an unbounded array parameter (for example, `a[*]`).

Defining Array Parameters

A DS2 method can be defined with array parameters. The type of array (temporary or variable) is specified in the parameter definition. The following table illustrates the syntax for defining different types of parameters.

Type of Parameter	Parameter Syntax	Example
scalar parameter	<i>data-type parameter-name</i>	double x
temporary array parameter	<i>data-type parameter-name [bounds]</i>	double x[5]
variable array parameter	VARARRAY <i>data-type parameter-name [bounds]</i>	vararray double x[2,4]

The data type and type of an array argument must exactly match the type and kind specified in the array parameter definition. DS2 does not convert array arguments to a different kind or data type. For example, if a parameter is defined as a temporary array of doubles, then the argument must be a temporary array of doubles. If a variable array of doubles or a temporary array of integers is passed as an argument for the temporary array of doubles parameter, then an error occurs.

The following DS2 program illustrates the definition of a method that has array parameters and illustrates calls to the method using array arguments.

```
data _null_;
  declare double x;
  declare double y[4];
  vararray double z[4];

  method m(double u, double v[4], vararray double w[4]);
    do i = 1 to 4;
      v[i] = u;
      w[i] = u;
    end;
  end;

  method init();
    m(x, y, z);      /* call method m */
    put y[*]='
    put z[*]=;
  end;
enddata;
run;
```

Bounded Array Parameters

A bounded array parameter supplies explicit bounds information for accessing elements of the array argument. These are examples.

```
method m(double a[4]);

method m(vararray double a[5:10,3:6]);
```

A bounded array parameter matches any DS2 array argument with the same number of elements regardless of the dimensionality of the array argument. For example, bounded array parameter `a[2,4]` would match array arguments `a1[8]`, `a2[11:12,11:14]`, and `a3[2,2,2]` because arrays `a1`, `a2`, and `a3` each have 8 elements. If the dimensionality of the array argument differs from the array parameter, then an element of the array parameter is mapped to the corresponding element in the array argument. This mapping is based on the position of the element in the array, using row-major order. For example, array parameter `a1[2,1]` accesses the fifth element of an 8-element array and thus would map to `a1[5]`, `a2[12,11]`, and `a3[2,2,1]`.

Unbounded Array Parameters

An unbounded array parameter does not supply any explicit bounds information for the corresponding array argument. Here is an example.

```
method m(double a[*]);
```

The asterisk (*) for the array bounds specifies that parameter `a` is an unbounded array parameter.

An unbounded array parameter matches any DS2 array argument regardless of the number of elements or dimensionality of the array argument. In a DS2 method, the array parameter is treated as a one-dimensional array even if the corresponding array argument is a multi-dimensional array. The unbound array parameter is mapped to the multi-dimensional array using row-major order. Consider the 2x3 array `a[2, 3]`.

```
1  2  3
4  5  6
```

If array `a` is passed to a method as an unbounded array parameter `b`, then `b` is accessed as a one-dimensional array of six elements.

```
1  2  3  4  5  6
```

Note that accessing an element of array parameter `b` results in the access of an element of array `a`, because array `a` is passed by reference to the DS2 method. Here is an example.

```
method m(double b[*]);
    b[1] = 10;           /* assigns 10 to a[1, 1] */
```

```

b[2] = 20;          /* assigns 20 to a[1, 2] */
b[3] = 30;          /* assigns 30 to a[1, 3] */
b[4] = 40;          /* assigns 40 to a[2, 1] */
b[5] = 50;          /* assigns 50 to a[2, 2] */
b[6] = 60;          /* assigns 60 to a[2, 3] */
end;

method init();
  declare double a[2, 3];
  m(a);
end;

```

In an array expression of the form `a[i]`, where `a` is an unbound array parameter, bounds checking of the index, `i`, is performed at run time. If an index of an array parameter is beyond the boundaries of the array argument, DS2 issues an error and the array expression evaluates to NULL or missing.

Note: An unbounded array parameter cannot be used as an argument for a bounded array parameter.

How to Query Array Dimensions

The following functions can be used to obtain dimension information about an array. For more information about each function, see [“DS2 Functions” in SAS DS2 Language Reference](#).

`DIM(a)`

Returns the number of elements in the first dimension of array *a*

`DIM(a, n)`

Returns the number of elements in dimension *n* of array *a*

`LBOUND(a)`

Returns the lower bound of the first dimension of array *a*

`LBOUND(a, n)`

Returns the lower bound of dimension *n* of array *a*

`HBOUND(a)`

Returns the upper bound of the first dimension of array *a*

`HBOUND(a, n)`

Returns the upper bound of dimension *n* of array *a*

`NDIMS(a)`

Returns the number of dimensions of array *a*

For any of the query functions, the array argument *a* can be a temporary array or a variable array, and the dimension argument *n* should be an expression that evaluates to an integral value. The following example illustrates these query functions.

```

do i = 1 to dim(a1);
    put a1[i];
end;

numelems = 0;
do i = 1 to ndims(a2);
    numelems = numelems + dim(a2, i);
end;

do i = lbound(a2, 1) to hbound(a2, 1);
    do j = lbound(a2, 2) to hbound(a2, 2);
        do k = lbound(a2, 3) to hbound(a2, 3);
            sum = sum + a2[i,j,k];
        end;
    end;
end;

```

If an array function is called with a dimension value outside the dimensions of the array, then a run-time error occurs and the function returns a NULL integer value.

How to Write Array Content

The DS2 PUT statement can be used to write individual elements of an array or all elements of an array.

The syntax to write an individual array element is as follows.

PUT *array-name*[*element*]<=>;

The syntax to write all elements of an array is as follows.

PUT *array-name*[*]<=>;

The PUT statement can write elements of temporary arrays and variable arrays. When all elements of an array are written with the *array-name*[*] syntax, all of the elements of the array are written with the same format. In other words, different formats cannot be specified for different array elements with *array-name*[*].

The following example illustrates the put statement output of the contents of an array:

```

data _null_;
    vararray varchar(10) x[10];
    declare double      y[2,2,2];
    method init();
        x[1] = 'a';
        do i = 2 to dim(x);
            x[i] = x[i-1] || x[1];
        end;
        put 'X:' x[*];

        y := (10 20 30 40 50 60 70 80);
        put 'Y:' y[*]=;
    end;

```

```

    end;
enddata;
run;

```

The following lines are written to the SAS log.

```

X: a aa aaa aaaa aaaaa aaaaaa aaaaaaa aaaaaaaaa aaaaaaaaaa aaaaaaaaaa
Y: y[1,1,1]=10 y[1,1,2]=20 y[1,2,1]=30 y[1,2,2]=40 y[2,1,1]=50 y[2,1,2]=60
   y[2,2,1]=70 y[2,2,2]=80

```

For more information, see the [“PUT Statement”](#) in *SAS DS2 Language Reference*.

DS2 Packages

<i>Introduction to DS2 Packages</i>	131
<i>Packages, Scope, and Lifetime</i>	134
Package Variable Scope and Package Instance Lifetime	134
Example: Package Instance Lifetime	134
Returning Package Instances from Methods	136
Passing Package Arguments	137
Attributes and Methods	138
<i>Dot Operator in Packages</i>	139
<i>Package Constructors and Destructors</i>	140
<i>User-Defined Packages</i>	140
Overview of User-Defined Packages	140
Implementing a Simple List Package	141
<i>Predefined DS2 Packages</i>	143
Overview of Predefined DS2 Packages	143
Using the FCMP Package	144
Using the Hash Package	148
Using the Hash Iterator Package	162
Using the HTTP Package	162
Using the JSON Package	167
Using the Logger Package	171
Using the MATRIX Package	174
Using the PCRXFIND Package	180
Using the PCRXREPLACE Package	184
Using the SQLSTMT Package	185
Using the TZ Package	190

Introduction to DS2 Packages

A DS2 package is a collection of methods and variables that can be used in DS2 programs. A DS2 package supports a set of related tasks and is designed for reuse.

There are two types of packages:

User-defined packages

These are packages that you can use to store methods for any purpose.

For more information, see [“User-Defined Packages” on page 140](#).

Predefined packages

These packages are predefined in DS2.

For more information, see [“Predefined DS2 Packages” on page 143](#).

FCMP

Supports calls to FCMP functions and subroutines from within the DS2 language.

Note: The FCMP package is not supported on the CAS server.

Hash and hash iterator

Enables you to quickly and efficiently store, search, and retrieve data based on unique lookup keys.

HTTP

Constructs an HTTP client to access HTTP web services.

JSON

Enables you to create and parse JSON text.

Logger

Provides a basic interface (open, write, and level query) to the SAS logging facility.

Matrix

Provides a powerful and flexible matrix programming capability.

Note: The MATRIX package is not supported on the CAS server.

PCRXFIND and PCRXREPLACE

Provides a way to find a substring within a given string and replace a substring.

SQLSTMT

Provides a way to pass FedSQL statements to a DBMS for execution and to access the result set returned by the DBMS.

Note: The SQLSTMT package is not supported on the CAS server.

TZ

Provides a way to process local and international time and date values.

To use a package, a DS2 program, another package, or a thread instantiates the package and accesses its methods. For a comparison between packages, DS2 programs, and threads, see [“Block Statements” in SAS DS2 Language Reference](#).

TIP A DS2 thread can instantiate a local instance of a package within the thread program. Package types are not supported as parameters in the THREAD statement. The package instance is not accessible from any other DS2 thread, and data is not shared across threads. If the DS2 program spawns 10 DS2 threads, then there are 10 package instances in memory.

Note: You can invoke a DS2 package method as a function in a FedSQL SELECT or FedSQL CALL statement. For more information, see [“Using DS2 Packages in Expressions” in SAS FedSQL Language Reference](#) and [“Package Method Expression” on page 87](#). This functionality is not available on the CAS server.

A package is used as a template to construct an instance of the package. A package variable is used to reference a particular instance of the package. Here is an example.

```
/* Create package animal */
package animal;
  declare varchar(100) s;
  method animal(varchar(100) s);
    this.s = s;
  end;

  method speak();
    put s;
  end;
endpackage;

data _null_;
  method init();
    /* Create variable a1 of type animal. */
    declare package animal a1;

    /* Create variable a2 of type animal.
       Construct an instance of type animal.
       Set variable a2 to reference the
       newly constructed animal instance. */
    declare package animal a2('meow');

    /* Set variable a1 to reference the same animal
       instance referenced by variable a2. */
    a1 = a2;

    a1.speak();
    a2.speak();

    /* Construct an instance of type animal.
       Set variable a1 to reference the
       newly constructed animal instance. */
    a1 = _new_ animal('woof');
    a1.speak();
    a2.speak();
  end;
enddata;
```

```
run;
```

Packages, Scope, and Lifetime

Package Variable Scope and Package Instance Lifetime

The lifetime of a package variable is governed by scope, but the lifetime of a package instance is governed by references to the instance.

The lifetime of a package variable depends on the scope in which the variable is created. A package variable is deleted automatically when execution exits the scope in which the variable was created.

A package variable that is created in a method is created in the local scope of the method. As a result, these package variables are local to a method and are deleted when on return from the method.

Package variables cannot be returned from a method. Only package instances can be returned from a method.

The lifetime of a package instance depends on whether a DS2 package variable references it. The lifetime of a package instance can extend beyond the scope in which the instance itself was created.

Multiple package variables might reference a single package instance. A package instance continues to exist as long as at least one package variable references it. When no more package variables reference a package instance, the package instance is automatically destroyed.

Example: Package Instance Lifetime

This example illustrates how package instances are created and destroyed.

```
proc ds2;
  package mypkg/overwrite=yes;
    dcl varchar(50) name;

    /* constructor - automatically invoked when a package instance is
     * created. */
    method mypkg(varchar(50) name);
      this.name = name;
      put 'creating' this.name;
    end;
```

```

/* destructor - automatically invoked when a package instance is
 * destroyed. */
method delete();
    put 'destroying' this.name;
end;
endpackage;

data _null_;
    dcl package mypkg var1('pkga'); 1

    method init();
        dcl package mypkg var2('pkgb');
        dcl package mypkg var3('pkgc'); 2

        var1 = var2; 3
    end; 4

    method term();
        var1 = _new_ mypkg('pkgd'); 5
    end;

    enddata; 6
run;
quit;

```

- 1 After the global variables are created, here is the result:
 - package instance **pkga** is referenced by package variable **var1**
- 2 After the local variables in the INIT method are created, here are the results:
 - package instance **pkga** is referenced by package variable **var1**.
 - package instance **pkgb** is referenced by package variable **var2**.
 - package instance **pkgc** is referenced by package variable **var3**.
- 3 When package variable **var2** is assigned to package variable **var1**, here are the results:
 - package instance **pkga** is not referenced by a package variable and therefore is destroyed.
 - package instance **pkgb** is referenced by package variable **var1** and package variable **var2**.
 - package instance **pkgc** is referenced by package variable **var3**.
- 4 When the INIT method returns, the method's local package variables (**var2** and **var3**) are deleted:
 - package instance **pkgb** is referenced by package variable **var1**.
 - package instance **pkgc** is not referenced by a package variable and is therefore destroyed.
- 5 When the new package instance **pkgd** is assigned to package variable **var1**, here are the results:
 - package instance **pkgb** is not referenced by a package variable and therefore is destroyed.

- package instance `pkgd` is referenced by package variable `var1`.
- 6 When the program finishes, the program's global variables (`var1`) are deleted:
- package instance `pkgd` is not referenced by a package variable and is therefore destroyed.

The following lines are written to the SAS log:

```
creating pkga
creating pkgb
creating pkgc
destroying pkga
destroying pkgc
creating pkgd
destroying pkgb
destroying pkgd
```

Returning Package Instances from Methods

You can use the RETURN statement to return package instances from methods. Here is an example.

```
package mypkg/overwrite=yes;
  method m(double x) returns double;
    return x+99;
  end;
endpackage;

data _null_;
  dcl package mypkg p;
  dcl double x;

  method r() returns package mypkg;
    return _new_ mypkg();
  end;

  method init();
    p = r();
    x = p.m(100);
    put x=;
  end;

  method term();
    x = p.m(200);
    put x=;
  end;

enddata;
```

In this case, the method `r` returns an instance of the package, `mypkg`, which is then used in the INIT method. Because `p` is declared outside the method call, it can be used again in other methods as shown here in the TERM method.

The variable `p` must be declared outside the method call in order to do this. If it had been declared local to the `INIT` method, it would not, of course, have been available in the `TERM` method.

The `DELETE` method destroys the package instance that is referenced by a package variable and assigns `NULL` to the package variable and all other package variables that referenced the destroyed package instance. Here is an example:

```
method init();
  p = r();
  x = p.m(100);
  put x;
  p.delete();
end;
```

When `p.delete()` is executed, the package instance that is referenced by `p` is destroyed, and `p` is assigned a `NULL` value.

This effect could be more easily achieved by declaring `p` to be local to the `INIT` method.

Passing Package Arguments

In addition to returning packages from methods, DS2 allows package instances to be passed to methods.

In this example, the instance that is referenced by global variable `p2` is passed to the method `tp` where it is referenced by local variable `p2`. During execution of method `tp`, both the global variable `p2` and the local variable `p2` reference the same package instance. In method `tp`, a new package instance is created and assigned to package variable `p` in the package instance that is referenced by local variable `p2`.

```
package mypkg/overwrite=yes;
  method m(double x) returns double;
    return x+99;
  end;
endpackage;

package mypkg2/overwrite=yes;
  dcl package mypkg p;
endpackage;

data _null_;
  dcl package mypkg2 p2();
  dcl double x;

  method tp(package mypkg2 p2);
    p2.p=_new_ mypkg();
  end;

  method init();
    dcl package mypkg p;
    tp(p2);
    p=p2.p;
```

```

        x = p.m(100);
        put x=;
    end;
enddata;
run;

```

Attributes and Methods

The **PRIVATE** access modifier can be used for attributes or methods that are intended for internal use within the package. This enables you to manage the complexity of your program by exposing only the attributes that you intended for the user to directly get or set. Private attributes are useful for saving state information that results from calling a method which, if touched directly by the user, could invalidate further results. Here is an example of using private attributes where you do not want the attributes **min**, **max**, or **sum** modified directly in the **nextNumber** method.

```

proc ds2;
package stats / overwrite=yes;
    /*-- put scratch space in private attributes --*/
    dcl private double min max sum ini;
    /*-- put shared logic in a private method --*/
    private method p_update( double v );
        if missing(v) then return;
        if missing(ini) then do;
            min=v;
            max=v;
            sum=v;
            ini = 1;
        end;
        else do;
            if v < min then min = v;
            if v > max then max = v;
            sum = sum + v;
        end;
        return;
    end;

    method nextNumber( double v );
        put 'in the double method';
        p_update( v );
    end;

    method nextNumber( char(20) c );
        dcl double v;
        v = c;
        put 'in the char method';
        p_update( v );
    end;

    method nextNumber( int i );
        dcl double v;

```

```

        v = i;
        put 'in the int method';
        p_update( v );
    end;

    method getStats( in_out double min, in_out double max, in_out double sum );
        min = this.min;
        max = this.max;
        sum = this.sum;
    end;
endpackage;

data;
dcl package stats st();
dcl double min max sum;
method run();
    st.getStats( min, max, sum );
    output;
    st.nextNumber( 5 );
    st.nextNumber( '4.0' );
    st.nextNumber( 2.0 );
    st.getStats( min, max, sum );
    output;
end;
enddata;
run;
quit;

```

Dot Operator in Packages

In the DS2 language, standard dot notation is restricted to three-level names. If you have nested packages, the standard dot notation requires a series of operations to make the method call. Here is an example.

```

dcl package TOP top();
...
t1=top.middle;
t2=t1.bottom;
result=t2.calledmethod();

```

However, when referencing DS2 packages, you can use the dot(.) as a standard binary operator. This enables you to access methods of nested packages by using a four-level name.

The previous example can be simplified as follows.

```

dcl package TOP top();
...
result=top.middle.bottom.calledmethod();

```

Package Constructors and Destructors

Constructors and destructors are special package methods that are used during construction and destruction of package instances. The constructor method initializes a newly constructed package instance. The destructor method performs cleanup, releases resources held by the package instance before the package instance is destroyed, or both. A package's constructor method has the same name as the class, and a package's destructor is the DELETE method. Constructors and destructors do not have return types and do not return values.

DS2 automatically calls a package's constructor when an instance of the package is constructed. DS2 automatically calls a package's destructor when a package instance goes out of scope and is destroyed. Note that creating a package variable with a DECLARE PACKAGE statement does not result in DS2 calling the package's constructor. A package's constructor is called only when a package instance is constructed with either a DECLARE PACKAGE statement with constructor arguments or with a _NEW_ operator.

For more information, see the applicable DECLARE PACKAGE statements and _NEW_ operators in the language reference section of this document.

User-Defined Packages

Overview of User-Defined Packages

You can store methods that you create in user-defined packages. These packages can be thought of as libraries of your methods. Any type of method can be saved in a package. Once you have stored methods in a package (using the PACKAGE statement), you can access them by creating an instance of the package with only a DECLARE statement or with the _NEW_ operator.

```
declare package package-name instance-name;  
instance-name = _new_ package-name();
```

Alternatively, you can use the condensed constructor syntax:

```
declare package package-name instance-name();
```

For more information, see the [“PACKAGE Statement” in SAS DS2 Language Reference](#), [“DECLARE PACKAGE Statement” in SAS DS2 Language Reference](#), and the [“_NEW_ Operator” in SAS DS2 Language Reference](#).

Note: You cannot use a user-defined package to hide a predefined DS2 package by overloading the package name. A package reference to a built-in package name resolves to the built-in package rather than to the user-defined package.

Here is an example of a very simple user-defined package called MATH. It contains a method that adds two numbers.

```
package math;
  method add(double x, double y) returns double;
    return x+y;
  end;
endpackage;
```

In the next example, two numbers are added by using the ADD method in the MATH package that was created in the previous example. First, the MATH package is declared and instantiated. Then the ADD method is called and the result is assigned to SUM.

```
data _null_;
  dcl double sum;
  method init();
    dcl package math f();
    sum = f.add(2,3);
    put 'sum= ' sum;
  end;
enddata;
```

Implementing a Simple List Package

The DS2 package syntax enables you to create compound structures and assign actions (methods) to these structures. A DS2 package cannot inherit attributes or methods from another package, but it can contain references to other packages. Therefore, it can support composition (has-a) relationships.

Here is an example of a simple but specific list. It starts with a list package called MYLIST that holds a specific element called MYELEMENT. A small data program exercises the MYLIST and MYELEMENT packages.

```
proc ds2;
  /*-- Define a simple list element --*/
  package myElement/overwrite = yes;
    dcl package myElement next;
    dcl int d;

    /*-- Custom constructor with parameter --*/
    method myElement( int d );
      this.d = d; /*-- Use "this" to differentiate d's --*/
    end;

    method print();
      put d=;
    end;
```

```

    /*-- A default delete method is implicitly defined --*/
endpackage;

/*-- Define a package to hold the elements --*/
package myList/overwrite=yes;
  dcl package myElement front;
  dcl package myElement back;

  method add( package myElement element );
    if null( front ) then do; /*-- first element --*/
      front = element;
      back = element;
    end;
    else do; /*-- link in new element --*/
      back.next = element;
      back = element;
    end;
  end;

  method printFront();
    if ^null( front ) then front.print();
    else put 'Front is NULL';
  end;

  method printBack();
    if ^null( back ) then back.print();
    else put 'Back is NULL';
  end;

/*-- A custom destructor - called when references go to zero --*/
method delete();
  dcl package myElement cur next;
  cur = front;

  /*-- Explicitly empty the list --*/
  do while ( ^null( cur ) );
    next = cur.get_next();
    cur.print();
    cur.delete();
    cur = next;
  end;
end;
endpackage;
run;

data _null_;
  method run();
    dcl int i;
    dcl package myList ml(); /*-- Instantiated --*/
    dcl package myElement me; /*-- Reference only --*/
    do i = 1 to 10;
      me = _new_ myElement(i); /*-- Constructor with parms --*/
      ml.add(me);
    end;
    ml.printFront();
  end;
end;

```

```

        ml.printBack();
    end;
enddata;
run;
quit;

```

Notice how the MYELEMENT package declares a reference to itself, next, as the first element of its internal structure. This is the foundation of building linked structures in memory: a package's ability to reference itself. Next, it declares the content that the list holds, in this case a single integer called `d`. Two methods are defined. The first `myElement(int d)` has the same name as the package, which means that it is the constructor for the package. It is not necessary to define a constructor, but, if you do define it, it can take initializing parameters such as the value of `d` in this case. The second method, `PRINT`, simply calls `PUT` on `d` to display the value of the element in the SAS log. A default destructor is supplied by the compiler if a user-written `DELETE` method is not included.

Next we have the MYLIST package that includes two elements of type MYELEMENT, `front` and `back`. With those and the `ADD` method, you have a simple list structure. For example, the MYLIST package contains a `PRINTFRONT` and `PRINTBACK` method, which in turn calls the `PRINT` method of the MYELEMENT package. Finally, there is a custom `DELETE` method, which is recommended whenever one package holds references to other packages. References to other packages could lead to a circle of references. When the execution leaves the `RUN` method, `DELETE` is implicitly called on the instance of MYLIST that is held by `m1`, which in turn explicitly calls `DELETE` on all the instances of MYELEMENT in the list.

Predefined DS2 Packages

Overview of Predefined DS2 Packages

SAS provides the following predefined packages for use in the DS2 language:

FCMP

Supports calls to FCMP functions and subroutines from within the DS2 language.

For more information, see [“Using the FCMP Package” on page 144](#).

Note: The FCMP package is not supported on the CAS server.

Hash and hash iterator

Enables you to quickly and efficiently store, search, and retrieve data based on unique lookup keys. The hash package keys and data are variables. Key and data values can be directly assigned constant values, values from a table, or values can be computed in an expression.

For more information, see [“Using the Hash Package” on page 148](#) and [“Using the Hash Iterator Package” on page 162](#).

HTTP

Constructs an HTTP client to access HTTP web services.

For more information, see [“Using the HTTP Package” on page 162](#).

JSON

Enables you to create and parse JSON text.

Logger

Provides a basic interface (open, write, and level query) to the SAS logging facility.

For more information, see [“Using the Logger Package” on page 171](#).

Matrix

Provides a powerful and flexible matrix programming capability. It provides a DS2-level implementation of SAS/IML functionality.

For more information, see [“Using the MATRIX Package” on page 174](#).

Note: The MATRIX package is not supported on the CAS server.

PCRXFIND and PCRXREPLACE

Provides a way to find a substring within a given string or replace a substring.

SQLSTMT

Provides a way to pass FedSQL statements to a DBMS for execution and to access the result set returned by the DBMS.

For more information, see [“Using the SQLSTMT Package” on page 185](#).

Note: The SQLSTMT package is not supported on the CAS server.

TZ

Provides a way to process local and international time and date values.

For more information, see [“Using the TZ Package” on page 190](#).

Using the FCMP Package

Overview of FCMP Packages

Note: The FCMP package is not supported on the CAS server.

The DS2 language supports calls to functions and subroutines that are available or are created with the FCMP procedure through an FCMP package.

You create an FCMP package by using the LANGUAGE=FCMP and TABLE= options in a PACKAGE statement. After the package is created, you declare an instance of the FCMP package. There are two ways to construct an instance of an FCMP package.

- Use the DECLARE PACKAGE statement along with the _NEW_ operator:

```
declare package fcmp banking;
banking = _new_ fcmp();
```

- Use the DECLARE PACKAGE statement along with its constructor syntax:

```
declare package fcmp banking();
```

For more information, see [“DECLARE PACKAGE Statement: FCMP Package” in SAS DS2 Language Reference](#) and [“PACKAGE Statement” in SAS DS2 Language Reference](#).

FCMP Package Capabilities

These are the capabilities of using the FCMP package in the DS2 language:

- Call an FCMP function or subroutine with scalar DOUBLE, CHAR, and NCHAR parameters, return parameters of both.

The DS2 language does automatic type conversion so that almost any type is supported. An example is a conversion from TINYINT to DOUBLE. For more information, see [“Overview of Type Conversions” on page 70](#).

- Call an FCMP function or subroutine with scalar DOUBLE OUTARGS parameters. The FCMP procedure’s DOUBLE OUTARGS parameter is treated as an IN_OUT parameter in the METHOD statement. For more information about the IN_OUT parameter, see the [“METHOD Statement” in SAS DS2 Language Reference](#).

Note: DS2 must pass a DOUBLE variable, not an expression, through an OUTARGS parameter.

Considerations and Limitations When Using the FCMP Package

- The FCMP package does not support VARARGS functions calls and therefore cannot use the FCMP procedure’s VARARGS interface.
- Errors caused when information is passed between the FCMP procedure and the FCMP package are not always reported correctly. For example, if you supply an incorrect table name in the PACKAGE statement, an error is written in the log file. However, there is no indication given that the operation fails.

- The FCMP package assumes the session encoding and currently has no mechanism that allows different encodings for different parameters within the same function call or for the same parameter across multiple function calls.
- You can access any FCMP library as long as the connection string defines the catalog in which the FCMP library is located.
- When called from a DS2 program, the following FCMP functions do not recognize seed values that are set in the DS2 program.

COMPCOST

COMPGED

RAND

STREAMINIT

In a DATA step, all random numbers from RAND are drawn from the same stream by default, but you can create multiple independent streams by using the CALL STREAM routine.

In PROC DS2, as a result of an unexpected feature, random numbers from RAND that are called inside an FCMP package come from a separate stream. To get reproducible random numbers, use the CALL STREAMINIT routine inside each FCMP function that calls RAND. To get independent streams, also use the CALL STREAM routine inside each FCMP function that calls RAND. If you do so, the results from DS2 will be consistent with a DATA step and with future releases.

Here is an example:

```

title1 'Generate 3 Uniform and 3 Normal Pseudorandom Variables' ;
title2 'With and Without FCMP' ;

%let seed = 1234; %put seed=&seed;
%let nobs = 10;    %put nobs=&nobs;

* To get reproducible streams in FCMP, call STREAMINIT in each FCMP function;
* To get independent streams in FCMP, call STREAM in each FCMP function;
proc fcmp outlib=work.funcs.rand;

    function rand_uniform( stream );
        call streaminit( &seed );
        call stream (stream );
        u = rand('uniform' );
        return (u);
    endsub;

    function rand_normal( stream, mean, std );
        call streaminit( &seed );
        call stream (stream );
        z = rand('normal', mean, std );
        return (z);
    endsub;

run;
quit;

title2 'PROC DS2 Step With and Without FCMP';

```

```

proc ds2;
  package pkg / overwrite=yes language='fcmp' table='work.funcs';
  run;
  data ds2(overwrite=yes);
    dcl package pkg randpkg();
    declare double obs u1 u2 u3 z1 z2 z3;
    method run();
      declare double junk;
      junk = streaminit( &seed );
      do obs = 1 to &nobs;
        junk = stream( 1 );
        u1 = rand( 'uniform' );
        u2 = randpkg.rand_uniform( 2 );
        u3 = randpkg.rand_uniform( 3 );
        junk = stream( 4 );
        z1 = rand( 'normal', 100, 20 );
        z2 = randpkg.rand_normal( 5, 100, 20 );
        z3 = randpkg.rand_normal( 6, 100, 20 );
        output;
      end;
    end;
  enddata;
run;
quit;

proc print data=ds2;
run;

```

The following table is generated.

Output 12.1 Random Numbers

**Generate 3 Uniform and 3 Normal Pseudorandom Variables
PROC DS2 Step With and Without FCMP**

OBS	obs	u1	u2	u3	z1	z2	z3
1	1	0.28215	0.91417	0.70279	120.205	81.990	85.187
2	2	0.51896	0.62759	0.82619	52.955	83.050	96.879
3	3	0.34428	0.75150	0.10642	139.520	128.227	105.486
4	4	0.99869	0.26466	0.55477	123.377	112.573	92.993
5	5	0.29200	0.79337	0.34503	62.937	114.600	86.801
6	6	0.50000	0.72109	0.07985	116.885	118.569	71.318
7	7	0.02342	0.93284	0.24532	87.582	151.621	83.920
8	8	0.29646	0.15810	0.51957	71.924	90.825	101.612
9	9	0.30978	0.61928	0.24458	115.001	116.015	75.750
10	10	0.75230	0.45879	0.71742	135.418	74.332	115.295

Using the Hash Package

Overview of Hash Packages

The hash package provides an efficient, convenient mechanism for quick data storage and retrieval. The hash package stores and retrieves data based on unique lookup keys. Depending on the number of unique lookup keys and the size of the table, the hash package lookup can be significantly faster than a standard format lookup or an array.

Before you use a DS2 hash package, you must define and construct an instance of (**instantiate**) the hash package.

After you define and create a hash package instance, you can perform many tasks, including the following:

- Store and retrieve data.
- Replace and remove data.
- Generate a table that contains the data in the hash package.

For example, suppose that you have a large table that contains numeric lab results that correspond to a unique patient number and weight and a small table that contains patient numbers (a subset of those in the large table). You can load the large table into a hash package using the unique patient number as the key and the weight values as the data. You can then iterate over the small table using the patient number to look up the current patient in the hash package whose weight is over a certain value and output that data to a different table.

Defining and Creating a Hash Package Instance

To create an instance of a hash package, you provide keys, data, and optional initialization data about the hash instance to construct. A hash package instance can be defined either fully at construction or at construction and through a subsequent series of method calls.

In the following example, the hash instances, **h1** and **h2**, have the same instance definition. The hash instance **h1** is fully defined at construction while **h2** is defined at construction and through a series of method calls.

```
declare package hash h1([key], [data1 data2 data3],
    0, 'testdata', '', '', '', 'multidata');

declare package hash h2();
    method init();
        h2.keys([key]);
        h2.data([data1 data2 data3]);
        h2.dataset('testdata');
```



```

h2.multidata();
h2.defineDone();
end;

```

For more information, see [“Defining a Hash Instance By Using Constructors” on page 149](#) and [“Defining a Hash Instance By Using Method Calls” on page 150](#).

Defining a Hash Instance By Using Constructors

A **constructor** is a method that you can use to instantiate a hash package and initialize the hash package data.

There are three different methods for creating a hash package instance with constructors.

- Create a partially defined hash instance.

DECLARE PACKAGE HASH *instance*(*hashexp*, {'datasource' | '\{sql-text\}'}, 'ordered', 'duplicate', 'suminc', 'multidata');

The key and data variables are defined by method calls. The optional parameters that provide the initialization data can be specified either in the DECLARE PACKAGE statement as shown above, in the `_NEW_` operator, by method calls, or a combination of any of these. A single `DEFINEDONE` method call completes the definition.

- Create a completely defined hash instance with the specified key and data variables.

DECLARE PACKAGE HASH *instance*(\[*keys*\], \[*data*\]
[, *hashexp*, {'datasource' |
'\{sql-text\}'}, 'ordered', 'duplicate', 'suminc', 'multidata']);

The key and data variables are defined in the DECLARE PACKAGE statement, which indicates that the instance should be created as completely defined. No additional initialization data can be specified with subsequent method calls.

- Create a completely defined hash instance with only the specified key variables (a keys-only hash instance). There are no data variables.

DECLARE PACKAGE HASH *instance*(\[*keys*\], *hashexp*, {'datasource' | '\{sql-text\}'}, 'ordered', 'duplicate', 'suminc', 'multidata');

The key and data variables are defined in the DECLARE PACKAGE statement, which indicates that the instance should be created as completely defined. No additional initialization data can be specified with subsequent method calls.

For more information about the optional parameters, see [“Providing Initialization Data for a Hash Package” on page 151](#). For more information about defining the optional parameters using method calls, see [“Defining a Hash Instance By Using Method Calls” on page 150](#).

Note: All variables that are passed to a hash instance must be global variables.

Defining a Hash Instance By Using Method Calls

If a hash instance is partially defined during construction of the instance, then the instance can be further defined through calls to the following methods.

KEYS
 DEFINEKEY
 DATA
 DEFINEDATA
 DATASET
 DUPLICATE
 HASHEXP
 ORDERED
 MULTIDATA
 SUMINC
 DEFINEDONE

For more information about these methods, see [“DS2 Hash and Hash Iterator Package Attributes, Methods, Operators, and Statements” in SAS DS2 Language Reference](#).

Note: After a hash instance specification is completed by a call to the DEFINEDONE method, a subsequent call to any of the above methods results in an error.

Here is an example of a hash instance, `h`, defined by using the method calls.

```
data _null_;
  declare package hash h(0, 'testdata');
  method init();
    h.keys([key]);
    h.data([data1 data2 data3]);
    h.ordered('descending');
    h.duplicate('error');
    h.defineDone();
  end;
enddata;
```

Defining Key and Data Variables

The hash package uses unique lookup keys to store and retrieve data. The keys and the data are variables that you use to initialize the hash package by using dot notation method calls.

You can define the key and data variables in one of three ways.

- Use the variable methods, `DEFINEDATA` and `DEFINEKEY`.
- Use the variable list methods, `DATA` and `KEYS`.
- Use key and data variable lists specified in the `DECLARE PACKAGE` statement.

If an instance of the hash package is not completely defined at construction, that is keys and data variables are not specified at construction, you must call the `DEFINEDONE` method to complete initialization of the hash instance.

Here are examples.

```
/* Keys and data defined using the implicit variable method */
declare package hash h();
h.definekey('k');
h.definedata('d');
h.definedone();
```

```
/* Keys and data defined using the variable list methods */
declare package hash h();
h.keys([k]);
h.data([d]);
h.definedone();
```

```
/* Keys and data defined using the variable list constructors */
declare package hash h([k],[d]);
```

Key variables must be a DS2 built-in type (character, numeric, or date-time). Data variables can be either a DS2 built-in type or a built-in or user-defined package type.

For more information, see [“Implicit Variable and Variable List Methods” on page 153](#).

Providing Initialization Data for a Hash Package

In addition to the keys and data, you can provide the following optional parameters when you initialize a hash package:

- the internal table size (*hashexp*) where the size of the hash table is 2^n
- the name of the table to load (*datasource*) or a FedSQL query to select the data to load

Note: Using a FedSQL query is not supported on the CAS server.

- whether or how the data is returned in key-value order (*ordered*)
- whether to ignore duplicate keys when loading a table (*duplicate*)
- the name of a variable that maintains a summary count of hash package keys (*suminc*)
- whether multiple data items are allowed for each key (*multidata*)

You can specify the initialization data in the `DECLARE PACKAGE` statement, the `_NEW_` operator, by method calls, or a combination of these ways.

Note: When you initialize hash package data using a constructor in the DECLARE PACKAGE statement or the _NEW_ operator, you must provide the optional parameters in this order: *hashexp*, *datasource*, *ordered*, *duplicate*, *suminc*, and *multidata*. These positional constructor parameters must all be enclosed in a single set of parentheses, separated by commas, and, except for the *hashexp* parameter, wrapped by single quotation marks. Because the optional parameters are positional, you must provide a place holder for each parameter to the last parameter that you specify. The placeholder that must be used depends on the parameter. For more information about the placeholders, see the “[DECLARE PACKAGE Statement: Hash Package](#)” in *SAS DS2 Language Reference* or the “[_NEW_ Operator: Hash Package](#)” in *SAS DS2 Language Reference*. In the following example, to specify an ascending order and to replace duplicates, you must use -1 as a place holder for the *hashexp* parameter, empty single quotations marks (' ') as the place holder for the *datasource* parameter, 'a' for *ordered* parameter, 'replace' for *duplicate*. Because the *duplicate* parameter is the last one specified, no place holder is required for the *suminc* and *multidata* parameters.

```
declare package hash variable-name(8, ' ', 'a', 'replace');
```

For more information, see “[Defining a Hash Instance By Using Constructors](#)” on page 149, “[Defining a Hash Instance By Using Method Calls](#)” on page 150, and “[Using the _NEW_ Operator to Create a Hash Instance](#)” on page 152.

Using the _NEW_ Operator to Create a Hash Instance

As an alternative to using the DECLARE PACKAGE statement to create a hash variable and a hash instance, you can use the DECLARE PACKAGE statement create the hash variable and the _NEW_ operator to create the hash instance. You declare a hash package variable using the DECLARE PACKAGE statement. Then you use the _NEW_ operator to instantiate an instance of the hash package and set the hash variable to reference the newly instantiated hash instance. With this scenario, initialization data cannot be provided using the DECLARE PACKAGE statement. You can provide initialization data for the hash instance with the _NEW_ operator and subsequent method calls if the hash instance was not constructed fully defined.

In the following example the DECLARE PACKAGE statement tells the compiler that the variable MYHASH is of type hash package. At this point, you have declared only the variable MYHASH. It has the potential to reference a hash instance, but it currently references nothing and therefore is a null package reference. You should declare the hash variable package only once. The _NEW_ operator creates an instance of the hash package and assigns it to the variable MYHASH.

```
declare package hash myhash();
myhash = _new_ hash(8, 'mytable', 'yes', 'replace', 'sumnum', 'y');
```

The above statement is equivalent to the following code:

```
declare package hash myhash(8, 'mytable', 'yes', 'replace', 'sumnum', 'y');
```

For more information, see the “[_NEW_ Operator: Hash Package](#)” in *SAS DS2 Language Reference*.

Implicit Variable and Variable List Methods

When you define a hash instance, you specify a series of key and data variables. After the hash instance is completely defined, the key and data variables can be implicitly or explicitly read and written during subsequent operations.

Note: All variables that are passed to a hash instance must be global variables.

There are two ways to pass keys and data variables to the hash instance: implicit variables methods and variable list methods.

The implicit variable method is similar to the DATA step hash object interface. Using the implicit variable methods, the key and data variables are defined through a series of DEFINEKEY and DEFINEDATA method calls and a single DEFINEDONE method call. Then the set of key and data variable definitions is used during execution of the other hash package methods as implicit arguments.

The following example defines a hash table with two key variables, **k1** and **k2**, and two data variables, **d1** and **d2**. The FIND method reads the values of the implicit key variables, looks up the key values in the hash table. If the key values are found, DS2 writes the corresponding data values to the implicit data variables defined for the hash instance.

```
declare package hash h();
  h.definekey('k1');
  h.definekey('k2');
  h.definedata('d1');
  h.definedata('d2');
  h.definedone();

  /* No explicit arguments specify what key values to find */
  /* or what to do with the data values if keys are found. */
  /* Implicitly uses key variables k1 and k2 and                */
  /* data variables d1 and d2.                                  */
  h.find();
```

The variable list method involves specifying variables of interest in variable lists as explicit arguments when the hash methods are called.

This example uses the variable list methods and is the same as the one above that uses implicit variable methods.

```
declare package hash h();
rc=h.keys([k1 k2]);
rc=h.data([d1 d2]);
rc=h.definedone();
h.find([k1 k2], [d1 d2]);
```

The variable list method provides flexibility to use variables other than the implicit key and data variables. The following FIND method looks for the values that are stored in **x** and **y**. If the values are found, they are written to **u** and **v**.

```
h.find([x y], [u v]);
```

For more information about variable lists, see [“Variable Lists” on page 28](#).

All of the hash implicit variable methods work with hash instances that have both keys and data and with hash instances that are keys-only.

Some variable list methods work only with keys-only hash instances while others work only with hash instances that have both keys and data. A run-time error occurs if a keys-only hash instance invokes a variable list method that works only for a hash instance that has keys and data, and vice versa. For more information about which methods work with keys-only hash instances, see each method in [“DS2 Hash and Hash Iterator Package Attributes, Methods, Operators, and Statements” in SAS DS2 Language Reference](#).

Non-Unique Key and Data Pairs

By default, all of the keys in a hash package are unique. This means one set of data variables exists for each key. In some situations, you might want to have duplicate keys in the hash package, that is, associate more than one set of data variables with a key.

For example, assume that the key is a patient ID and the data is a visit date. If the patient were to visit multiple times, multiple visit dates would be associated with the patient ID. When you create a hash package with the MULTIDATA parameter or method set to YES, multiple sets of the data variables are associated with the key.

If the table contains duplicate keys, by default, the first instance is stored in the hash package and subsequent instances are ignored. To store the last instance in the hash package, use the DUPLICATE parameter or method. The DUPLICATE parameter or method also writes an error to the SAS log if there is a duplicate key.

However, the hash package allows storage of multiple values for each key if you use the MULTIDATA parameter or method. The hash package keeps the multiple values in a list that is associated with the key. This list can be traversed and manipulated by using several methods such as HAS_NEXT or FIND_NEXT.

To traverse a multiple data item list, you must know the current list item. Start by calling the FIND method for a given key. The FIND method sets the current list item. Then to determine whether the key has multiple data values, call the HAS_NEXT method. After you have determined that the key has another data value, you can retrieve that value with the FIND_NEXT method. The FIND_NEXT method sets the current list item to the next item in the list and sets the corresponding data variable or variables for that item.

In addition to moving forward through the list for a given key, you can loop backward through the list by using the HAS_PREV and FIND_PREV methods in a similar manner.

Note: The items in a multiple data item list are maintained in the order in which you insert them.

For more information about the MULTIDATA and DUPLICATE parameters, see the [“DECLARE PACKAGE Statement: Hash Package” in SAS DS2 Language Reference](#) or the [“_NEW_ Operator: Hash Package” in SAS DS2 Language Reference](#). For more information about the MULTIDATA and DUPLICATE methods, see the [“MULTIDATA Method” in SAS DS2 Language Reference](#) and the [“DUPLICATE Method” in SAS DS2 Language Reference](#).

Maintaining Key Summaries

You can maintain a summary count for a hash package key by using the SUMINC parameter or method. SUMINC instructs the hash package to allocate internal storage in each record to store a summary value in the record each time that the record is used by a FIND, CHECK, or REF method. The SUMINC value is also used to maintain a summary count of hash parameter keys after a FIND, CHECK, or REF method.

SUMINC is given a variable, which holds the sum increment, that is, how much to add to the key summary for each reference to the key. The SUMINC value can be greater than, less than, or equal to 0. The SUMINC value is also used to initialize the summary on an ADD method. Each time the ADD method occurs, the key to the SUMINC value is initialized.

The SUM method retrieves the summary value for a given key when only one data item exists per key.

If multiple items exist, the SUMDUP method retrieves the current value of the key summary.

You can use key summaries in conjunction with the DATASOURCE parameter or DATA method. As a table is read into the hash package using the DEFINEDONE method or a DECLARE PACKAGE statement, all key summaries are set to the SUMINC value and all subsequent FIND, CHECK, or ADD methods change the corresponding key summaries.

For more information about the SUMINC parameter, see the [“DECLARE PACKAGE Statement: Hash Package” in SAS DS2 Language Reference](#). For more information about the SUMINC and SUMDUP methods, see the [“SUMINC Method” in SAS DS2 Language Reference](#) and the [“SUMDUP Method” in SAS DS2 Language Reference](#).

Storing and Retrieving Data

After you initialize the hash package's key and data variables, you can store data in the hash package using the ADD method, or you can use the DATASOURCE parameter or DATASET method to load a table into the hash package. If you use the DATASOURCE parameter or DATASET method, and if the table contains more than one row with the same value of the key, by default, SAS keeps the first row in the hash table and ignores subsequent rows. To store the last instance in the hash package or to send an error to the log if there is a duplicate key, use the DUPLICATE parameter or method. To allow duplicate values for each key, use the MULTIDATA parameter or method.

You can then use the FIND method to search and retrieve data from the hash package. Use the FIND_NEXT and FIND_PREV methods to search and retrieve data if multiple data items exist for each key.

For more information, see the [“ADD Method: Hash Package”](#) in *SAS DS2 Language Reference*, the [“FIND Method”](#) in *SAS DS2 Language Reference*, the [“FIND_NEXT Method”](#) in *SAS DS2 Language Reference*, and the [“FIND_PREV Method”](#) in *SAS DS2 Language Reference*.

You can consolidate a FIND method and ADD method using the REF method. In the following example, you can reduce the amount of code from this:

```
rc = h.find();
    if (rc != 0) then
        rc = h.add();
```

to a single method call:

```
rc = h.ref();
```

For more information, see the [“REF Method”](#) in *SAS DS2 Language Reference*.

Note: You can also use the hash iterator package to retrieve the hash package data, one data item at a time, in forward and reverse order. For more information about the hash iterator package, see [“Using the Hash Iterator Package”](#) on page 162.

Replacing and Removing Data

You can remove or replace data in the hash package using one of the following methods:

- Use the REMOVE method to remove the data items from the specified key.
- Use the REMOVEALL method to remove all the data items.
- Use the REMOVEDUP method to remove data items for keys that have multiple data items.
- Use the REPLACE method to replace all data items.
- Use the REPLACEDUP method to replace only the current data item.

Note: If an associated hash iterator is pointing to the key, the REMOVE method does not remove the key or data from the hash package. An error message is issued to the log.

For more information, see the [“REMOVE Method”](#) in *SAS DS2 Language Reference*, the [“REMOVEALL Method”](#) in *SAS DS2 Language Reference*, the [“REMOVEDUP Method”](#) in *SAS DS2 Language Reference*, the [“REPLACE Method”](#) in *SAS DS2 Language Reference*, and the [“REPLACEDUP Method”](#) in *SAS DS2 Language Reference*.

Saving Hash Package Data in a Table

You can create a table that contains the data in a specified hash package by using the OUTPUT method.

In the following example, the first table program generates the **data1** table. The second table program creates a hash package **h** with one key, **k**, and two data, **d1** and **d2**. Then each row from the **data1** table is read and the keys and data are added to hash package **h**. Finally, the data values stored in hash package **h** are written to the **out1** table. The third table program writes the contents of the **out1** table.

```
/* Generate and output 5 rows for table data1. */
data data1(overwrite=yes);
  declare double k d1 d2;
  method init();
    declare int i;
    do i = 1 to 5;
      k = i; d1 = i*10; d2 = i*2; output;
    end;
  end;
enddata;
run;

data _null_;
  declare double k d1 d2;
  declare package hash h(0, '', 'descending');

  /* Define key and data variables for hash h. */
  method init();
    h.defineKey('k');
    h.defineData('d1');
    h.defineData('d2');
    h.defineDone();
  end;

  /* Read rows from table data1.
   * Add key and data values from rows to hash h. */
  method run();
    set data1;
    h.add();
  end;

  /* Add additional key and data values to hash h.
   * Output hash h to table out1. */
  method term();
    k = 11; d1 = 110; d2 = 22; h.add();
    k = 12; d1 = 120; d2 = 24; h.add();
    k = 13; d1 = 130; d2 = 26; h.add();
    k = 14; d1 = 140; d2 = 28; h.add();
    h.output('out1');
  end;

enddata;
```

```

run;

/* Outputs rows from table out1. */
data;
    method run();
        set out1;
    end;
enddata;
run;

```

Output 12.2 *Output Rows from Table OUT1*

d1	d2
140	28
130	26
120	24
110	22
50	10
40	8
30	6
20	4
10	2

Note that the hash package keys are not stored as part of the output table. If you want to include the key in the output table, you must define the key as data in the DEFINEDATA method. In the previous example, the DEFINEDATA method would be written this way:

```

h.defineKey('k');
h.defineData('k');
h.defineData('d1');
h.defineData('d2');

```

With the above modification, the following lines output.

Output 12.3 Output Rows from Table OUT1 with Keys and Data

k	d1	d2
14	140	28
13	130	26
12	120	24
11	110	22
5	50	10
4	40	8
3	30	6
2	20	4
1	10	2

Using a FedSQL Query with a Hash Instance to Get Rows Dynamically at Run Time

You can delay the decision of which rows to get from a table until run time by using a hash instance. At run time, the hash instance is created and loaded with the selected rows. The rows are selected based on a FedSQL query specified for the data source. You can use a hash iterator to loop over the rows and access the row data.

In the following example, the rows are selected by the `SELECT * FROM test WHERE a=1` FedSQL query. This query is passed to the `execute` method that loads them into the hash package. A hash iterator loops over the rows and writes the selected rows to the SAS log and the `result` table.

```
data test;
  a=1; b=2;output;
  a=11; b=22;output;
  a=1; b=3;output;
  a=22; b=44;output;
run;

proc ds2;
package pkg /overwrite=yes;
  dcl double a b;

  method execute(char(200) sql);
    dcl package hash h();
    dcl package hiter hi(h);
    h.keys([a]);
    h.data([a b]);
    h.multidata('yes');
```

```

        h.dataset(sql);
        h.definedone();

        put 'SELECTED ROWS: ';
        rc = hi.first();
        do while(rc = 0);
            put a= b=;
            rc = hi.next();
        end;

        h.output('result');
    end;
endpackage;
run; quit;

proc ds2;
data _null_;
    method init();
        declare package pkg p1();
        p1.execute('{SELECT * FROM test WHERE a=1}');
    end;

enddata;
run; quit;

proc print data=test;
    title2 'TEST TABLE';
run; quit;

proc print data=result;
    title2 'RESULT TABLE';
run; quit;

```

The following lines are written to the SAS log.

```

SELECTED ROWS:
a=1 b=2
a=1 b=3

```

The input and output tables are as follows.

Output 12.4 Input Table

TEST TABLE		
Obs	a	b
1	1	2
2	11	22
3	1	3
4	22	44

Output 12.5 Output Table

RESULT TABLE		
Obs	a	b
1	1	2
2	1	3

Using Hash Package Attributes

There are two attributes available to use with hash packages. NUM_ITEMS returns the number of items in a hash package and ITEM_SIZE returns the size (in bytes) of an item.

The following example retrieves the number of items in a hash package:

```
num_items = myhash.num_items;
```

The following example retrieves the size of an item in a hash package:

```
item_size = myhash.item_size;
```

You can obtain an idea of how much memory the hash package is using with the ITEM_SIZE and NUM_ITEMS attributes. The ITEM_SIZE attribute does not reflect the initial overhead that the hash package requires, nor does it take into account any necessary internal alignments. Therefore, the use of ITEM_SIZE does not provide exact memory usage, but it gives a good approximation. For more information, see the [“ITEM_SIZE Attribute” in SAS DS2 Language Reference](#) and [“NUM_ITEMS Attribute” in SAS DS2 Language Reference](#).

Using the Hash Iterator Package

You use a hash iterator package to store and search data based on unique lookup keys. The hash iterator package enables you to retrieve the hash package data in forward or reverse key order.

You declare a hash iterator package by using the `DECLARE PACKAGE` statement. After you declare the new hash iterator package, use the `_NEW_` operator to instantiate the package, using the hash package name as a parameter. Here is an example:

```
declare package hiter myiter;  
myiter = _new_ hiter('h');
```

The `DECLARE PACKAGE` statement tells the compiler that the variable `MYITER` is of type hash iterator. At this point, you have declared only the variable `MYITER`. It has the potential to reference a hash iterator instance, but it currently references nothing and thus is a null package reference. You should declare the hash iterator package variable only once. The `_NEW_` operator constructs an instance of the hash iterator package and assigns it to the variable `MYITER`. The hash package, `H`, is passed as a constructor parameter.

As an alternative to the two-step process of using the `DECLARE PACKAGE` and the `_NEW_` operators to declare and instantiate a hash iterator package, you can declare and instantiate a package in one step by using the `DECLARE PACKAGE` statement as a constructor method. Here is the same example using only the `DECLARE PACKAGE` statement.

```
declare package hiter myiter('h');
```

For more information, see the [“DECLARE PACKAGE Statement: Hash Iterator Package”](#) in *SAS DS2 Language Reference*, and the [“_NEW_ Operator: Hash Iterator Package”](#) in *SAS DS2 Language Reference*.

Note: You must declare and instantiate a hash package before you create a hash iterator package.

Using the HTTP Package

Overview of the HTTP Package

Use the HTTP package to construct an HTTP client in order to access HTTP web servers.

Here are the general tasks:

- 1 Declare and instantiate an HTTP package.
- 2 Create an HTTP GET, HEAD, or POST method.
You can use additional HTTP package methods to add header information and to send request data.
- 3 Execute the HTTP GET, HEAD, or POST method.
- 4 Retrieve the response information from the web server:
 - The response body as a complete entity or by streaming
 - The response content type
 - The response header

The HTTP package also enables you to perform these tasks:

- Retrieve status codes from HTTP responses.
- Set a socket time-out value.
- In SAS 9.4M6 and SAS Viya 3.4, specify a URL or a proxy URL to which the client HTTP instance should connect.
- In SAS 9.4M6 and SAS Viya 3.4, specify a user name and password if the URL or proxy URL requires them.
- In SAS 9.4M6 and SAS Viya 3.4, specify a character set to use when encoding the request body or decoding the response body.
- In SAS 9.4M6 and SAS Viya 3.4, either specify the OpenAuthorization (OAuth) access token for the request or search the SAS environment for the token.
- Log the HTTP traffic between the HTTP client and server using the SAS logging facility.

Declaring and Instantiating an HTTP Package

You must first declare and instantiate an HTTP package. There are two ways to construct an instance of an HTTP package.

- Use the DECLARE PACKAGE statement along with the `_NEW_` operator:

```
declare package http httpclt;
httpclt = _new_ http();
```

- Use the DECLARE PACKAGE statement along with its constructor syntax:

```
declare package http httpclt();
```

For more information, see [“DECLARE PACKAGE Statement: HTTP Package”](#) in *SAS DS2 Language Reference* and [“_NEW_ Operator: HTTP Package”](#) in *SAS DS2 Language Reference*.

TIP Web service applications might require only a single HTTP client to synchronously handle HTTP traffic. Or, if your application requires it, you can instantiate multiple HTTP clients to asynchronously request and process data.

Create an HTTP GET, HEAD, or POST Method

- 1 Use the `CREATEGETMETHOD`, `CREATEHEADMETHOD`, or `CREATEPOSTMETHOD` method to create the GET, HEAD, and POST method.
For more information, see [“CREATEGETMETHOD Method” in SAS DS2 Language Reference](#), [“CREATEHEADMETHOD Method” in SAS DS2 Language Reference](#), and [“CREATEPOSTMETHOD Method” in SAS DS2 Language Reference](#).
- 2 (Optional) To add a header to the HTTP GET method, use the `ADDREQUESTHEADER` method.
For more information, see [“ADDREQUESTHEADER Method” in SAS DS2 Language Reference](#).
- 3 (Optional) To add a request body to the HTTP method, use the `SETREQUESTBODYASBINARY` or `SETREQUESTBODYASSTRING` method.
For more information, see [“SETREQUESTBODYASBINARY Method” in SAS DS2 Language Reference](#) and [“SETREQUESTBODYASSTRING Method” in SAS DS2 Language Reference](#).
- 4 (Optional) To specify the request content type in the HTTP method, use the `SETREQUESTCONTENTTYPE` method.
For more information, see [“SETREQUESTCONTENTTYPE Method” in SAS DS2 Language Reference](#).

Executing an HTTP GET, HEAD, and POST Method

When you execute the HTTP GET, HEAD, and POST methods, you send the request to an HTTP web server.

Note: Before you execute the HTTP GET, HEAD, and POST methods, you must create the HTTP GET, HEAD, or POST method. For more information, see [“Create an HTTP GET, HEAD, or POST Method” on page 164](#).

For most HTTP methods, use the `EXECUTEMETHOD` method to send the request to the web server. For more information, see [“EXECUTEMETHOD Method” in SAS DS2 Language Reference](#).

The EXECUTEMETHOD method does not support streaming of the response body. If you want to stream the response body, a different execute method, EXECUTEMETHODSTREAM, is required. For more information, see [“Retrieving an HTTP Resource” on page 166](#).

To send another request, repeat the process starting with creating the GET, HEAD, or POST request and ending with the EXECUTEMETHOD or EXECUTEMETHODSTREAM method.

This example program instantiates an HTTP package (the client), creates an HTTP GET method, executes the GET method to send a request to an HTTP web service, and retrieves the body information from the response from the HTTP web service as a string.

```
data _null_;
  method run();
    /* instantiate the package */
    declare package http h();
    declare varchar(1024) character set utf8 body;
    declare int rc;

    /* create a GET */
    h.createGetMethod('http://api.worldbank.org/countries/fr/');
    /* execute the GET */
    h.executeMethod();
    /* retrieve the response body as a string */
    h.getResponseBodyAsString(body, rc);
    put body;
end;
```

The following lines are written to the SAS log.

```
<?xml version="1.0" encoding="utf-8"?>
<wb:countries page="1" pages="1" per_page="50" total="1"
xmlns:wb="http://www.worldbank.org">
  <wb:country id="FRA">

    <wb:iso2Code>FR</wb:iso2Code>
    <wb:name>France</wb:name>
    <wb:region id="ECS">Europe
&amp; Central Asia (all income levels)</wb:region>
    <wb:adminregion id="" />

    <wb:incomeLevel id="OEC">High income: OECD</wb:incomeLevel>
    <wb:lendingType id="LNX">Not
classified</wb:lendingType>
    <wb:capitalCity>Paris</wb:capitalCity>

    <wb:longitude>2.35097</wb:longitude>
    <wb:latitude>48.8566</wb:latitude>

  </wb:country>
</wb:countries>
```

Retrieving an HTTP Resource

You can use the following HTTP package methods to retrieve headers, the response body, and the response body type from an HTTP resource. If you are retrieving the response body, you can retrieve it as one entity or stream the response body.

Task	HTTP package method
Retrieve header	“GETRESPONSEHEADERSASSTRING Method” in SAS DS2 Language Reference
Get response body as one entity ¹	“GETRESPONSEBODYASBINARY Method” in SAS DS2 Language Reference “GETRESPONSEBODYASSTRING Method” in SAS DS2 Language Reference
Stream the response body ¹	“STREAMRESPONSEBODYASBINARY Method” in SAS DS2 Language Reference “STREAMRESPONSEBODYASSTRING Method” in SAS DS2 Language Reference
Retrieve the response body content type	“GETRESPONSECONTENTTYPE Method” in SAS DS2 Language Reference

¹ If you get the response body as one entity, you must create the GET method first and execute that method with the EXECUTEMETHOD method. If you stream the response body, you must create the GET method first and execute that method with the EXECUTESTREAMMETHOD method.

Considerations When Using the HTTP Package

- The HTTP package supports only GET, HEAD, and POST HTTP methods.
- Each client sends requests and processes responses synchronously. The application can create multiple clients to asynchronously perform actions on HTTP resources.
- The HTTP package stores data for requests and from responses in memory as DS2 string values or binary values. If you want to store the data on disk as a file, consider using the HTTP procedure. For more information, see [“HTTP Procedure” in Base SAS Procedures Guide](#).
- The HTTP package can send requests to a secure HTTP endpoint that requires authentication. When an HTTP end-point requires client authentication, it responds to the client with its list of supported authentication mechanisms. The HTTP package currently supports two of the three most common authentication mechanisms: Basic and Negotiate. Since Basic authentication in

itself does not provide any credential confidentiality, it should be used only when the data is being encrypted via Transport Layer Security (TLS). For complete information about how SAS validates TLS, see *Encryption in SAS*. Negotiate authentication supports Kerberos, and, when on Windows, supports NT LAN Manager (NTLM).

Open Authentication

Open Authorization (OAuth) is an open standard for token-based authentication and authorization on the internet. OAuth is designed specifically to work with HTTP and essentially allows access tokens to be issued to third-party clients by an authorization server, with the approval of the resource owner. The third-party client then uses the access token to access the protected resources that are hosted by the resource server.

In SAS 9.4M6 and SAS Viya 3.4, for the HTTP package, you can either provide a token by using the SETAUTHTOKEN method or you can allow SAS to search the environment for an OAuth access token by using the ADDSASOAUTHTOKEN method. In both cases, the OAuth access token is added to the request header as a Bearer value of an Authorization header field.

For more information, see the [“SETAUTHTOKEN Method” in SAS DS2 Language Reference](#) and the [“ADDSASOAUTHTOKEN Method” in SAS DS2 Language Reference](#).

Logging HTTP Traffic

The HTTP package supports logging through the SAS logging facility.

The App.TableServices.d2pkg.HTTP logger logs errors, headers, and data that are sent back and forth from the HTTP client and the web server.

For more information, see [“HTTP Package Logger” on page 310](#).

Using the JSON Package

Overview of the JSON Package

Java Script Object Notation (JSON) is a text-based, open standard data format that is designed for human-readable data interchange. JSON is based on a subset of the JavaScript programming language and uses JavaScript syntax for describing data objects.

The JSON package provides an interface to create and parse JSON text. The JSON package Write methods accumulate the Write requests in memory, and the text can be retrieved. The JSON package parser enables you to read and parse text.

Declaring and Instantiating the JSON Package

There are two ways to construct an instance of a JSON package.

- Use the DECLARE PACKAGE statement along with the `_NEW_` operator:

```
declare package json myjsonpkg;
myjsonpkg = _new_ json();
```

- Use the DECLARE PACKAGE statement along with its constructor syntax:

```
declare package json myjsonpkg();
```

For more information, see the “[DECLARE PACKAGE Statement: JSON Package](#)” in *SAS DS2 Language Reference*, and the “[_NEW_ Operator: JSON Package](#)” in *SAS DS2 Language Reference*.

Writing JSON Text

To create JSON text, you create a JSON writer instance by using the `CRETEWRITER` method.

JSON output consists of two types of data structure containers:

JSON object container (`{ }`)

begins with a left brace (`{`) and ends with a right brace (`}`). An object container collects name-value pairs that are written as pairs of names and values. A value can be any of the supported JSON data types, an object, or an array. Each name is followed a colon and then the value. The name-value pairs are separated by a comma.

Use the `WRITEOBJOPEN` and `WRITECLOSE` methods to create the object container.

JSON array container (`[ ]`)

begins with a left bracket (`[`) and ends with a right bracket (`]`). An array container collects a list of values that are written as a list of values without names. A value can be any of the supported JSON data types, an object, or an array. Values are separated by a comma.

Use the `WRITEARRAYOPEN` and `WRITECLOSE` methods to create the array container.

The top-level container can include any number of containers. Containers, likewise, can nest containers to an arbitrary depth. When nesting containers, be careful to observe the data structure requirements of the current container.

- Objects require a list of name-value pairs, where the value can itself be an object or array.

- Arrays have no such structural requirement of name-value pairs and are merely a list of values, objects, or arrays.

With JSON package methods, you can write character and numeric values, null values, and Boolean true and false values.

Here is an example of JSON text output:

```
{ "SASJSONExport": "1.0", "SASTableData+CLASS": [{ "Name": "Joyce", "Sex": "F", "Age": 11, "Height": 51.3, "Weight": 50.5 }, { "Name": "Thomas", "Sex": "M", "Age": 11, "Height": 57.5, "Weight": 85 } ] }
```

You can get the JSON text that is produced by the writer by using the WRITERGETTEXT method.

When you finish writing the text, call the DESTROYWRITER method to remove the writer instance.

The following example creates some JSON text and writes it to the SAS log.

```
data _null_;
  method init();
    dcl package json j();
    dcl double dblVal;
    dcl int rc;
    dcl nvarchar(30) jsontxt;

    rc = j.createWriter();
    if rc=0 then rc = j.writeArrayOpen();
    if rc=0 then rc = j.writeString( '      Hello World!      ' );
    if rc=0 then rc = j.writeClose();
    j.writerGetText( rc, jsontxt );
    put rc= jsontxt=;
  end;
enddata;
run;
```

The following line is written to the SAS log.

```
rc=0 jsontxt=["      Hello World!      "]
```

For more information about the methods that enable you to write JSON text, see [“DS2 JSON Package Methods, Operators, and Statements” in SAS DS2 Language Reference](#).

Parsing JSON Text

To parse, or read, JSON text, you create a JSON writer instance by using the CREATEPARSER method. You can provide the input JSON text to the parser with either the CREATEPARSER method, the SETPARSERINPUT method, or both.

The GETNEXTTOKEN method is used to return the next validate JSON language element form the JSON text. The GETNEXTOKEN method can also return the token type, parse flags, the line number, and the column number. The JSON package IS* methods (for example, ISLEFTBRACE) enable you to query the following token types:

Boolean true
 Boolean false
 float
 integer
 label
 left brace ({)
 left bracket ([)
 null
 numeric
 partial
 right brace (})
 right bracket (])
 string

When you finish parsing the text, call the DESTROYPARSER method to remove the parser instance.

The following example creates a parser and uses the CREATEPARSER method to provide the input JSON text.

```

data _null_;
  method init();
    dcl package json j();
    dcl int rc tokenType parseFlags;
    dcl bigint lineNum colNum;
    dcl nvarchar(128) token abc t1;
    abc = 'xyz';
    t1 = '{"abc" : 1 }';
    rc = j.createParser( t1 );
    if (rc ne 0) then goto TestError;

    * obj open;
    j.getNextToken( rc, token, tokenType, parseFlags, lineNum, colNum );
    if ( rc ne 0 ) then goto TestError;

    * obj label;
    j.getNextToken( rc, token, tokenType, parseFlags, lineNum, colNum );
    if ( rc ne 0 ) then goto TestError;

    * obj value;
    j.getNextToken( rc, token, tokenType, parseFlags, lineNum, colNum );
    if ( rc ne 0 ) then goto TestError;

    * obj close;
    j.getNextToken( rc, token, tokenType, parseFlags, lineNum, colNum );
    if ( rc ne 0 ) then goto TestError;

  Exit:
    rc = j.destroyParser();
    return;

  TestError:
    put 'Test ended abnormally.';
  
```

```

        goto Exit;

    end;
enddata;
run;

```

For more information about the methods that enable you to parse JSON text, see [“DS2 JSON Package Methods, Operators, and Statements” in SAS DS2 Language Reference](#).

JSON Package Logger

The JSON package supports logging through the SAS logging facility.

The DS2 JSON package provides a logger, App.tk.DS2PKG.JSON, to collect the internal DS2 JSON package DEBUG and TRACE information. If you cannot easily add TRACE statements to a DS2 program that uses the JSON package, set the App.tk.DS2PKG.JSON logger level to DEBUG or TRACE.

For more information, see [“JSON Package Logger” on page 310](#).

Using the Logger Package

Overview of the Logger Package

In the SAS logging facility, a logger is a named entity that identifies a message category. A logger's attributes consist of a level and one or more appenders that process the log events for the message category. The level indicates the threshold, or lowest event level, that is processed for this message category.

You use a logger package to interface with the SAS logging facility. After you declare the new logger package, you can send messages to the logger at a specified logging level.

Note: You cannot use the logger package from a SAS Enterprise Guide session.

For more information about DS2 loggers and the SAS logging facility, see [Appendix 2, “DS2 Loggers,” on page 307](#) and the *SAS Logging: Configuration and Programming Reference*.

Declaring and Instantiating a Logger Package

There are two ways to construct an instance of a logger package.

- Use the DECLARE PACKAGE statement along with the _NEW_ operator:

```
declare package logger logpkg;
logpkg = _new_ logger();
```

- Use the DECLARE PACKAGE statement along with its constructor syntax:

```
declare package logger logpkg();
```

For more information, see the “[DECLARE PACKAGE Statement: Logger Package](#)” in *SAS DS2 Language Reference*, and the “[_NEW_ Operator: Logger Package](#)” in *SAS DS2 Language Reference*.

Unformatted and Formatted Messages

You can specify a raw, or unformatted, message that is written directly to the SAS logging facility.

You can also use \$s format markers to create a formatted message. Each \$s format marker in the message format is replaced by the content of the corresponding argument that you specify.

For more information, see “[LOG Method: Logger Package](#)” in *SAS DS2 Language Reference*.

Log Messages to a Table

- 1 Create an empty table to store the messages.

Here is an example:

```
libname logs 'c:\temp';
data logs.edmlog;
  length seqno 8;
  length date 8; format date DATETIME19.;
  length msg $ 256;
  stop;
run;
```

- 2 Create a logger and appender with an XML logging configuration file.

Here is an example that creates an App.Program logger named **App.Program.EDM**. The appender that the logger writes to is **EDMAppender**. The appender is configured to write to a SAS data set name **edmlog** in **c:\temp**. Save this XML logging configuration file as **edm-14s.xml**.

```
<?xml version="1.0"?>
<logging:configuration xmlns:logging="http://www.sas.com/xml/logging/1.0/">

  <appender name="EDMAppender" class="DBAppender">
    <param name="ConnectionString" value="
      DRIVER=base; CATALOG=base;
      schema=(name=base; primarypath='C:\temp')"/>
    <param name="MaxBufferedEvents" value="1000"/>
    <param name="TableName" value="edmlog"/>
    <param name="Column" value="sn"/>
```



```

        <param name="Column" value="d"/>
        <param name="Column" value="m"/>
    </appender>

    <appender name="null" class="ConsoleAppender">
        <filter class="DenyAllFilter"/>
    </appender>

    <logger name="App.Program.EDM">
        <appender-ref ref="EDMAppender"/>
    </logger>

    <root>
        <appender-ref ref="null"/>
    </root>
</logging:configuration>

```

- 3 Start SAS with the LOGCONFIGLOC system option set to the name of the XML configuration file that you created in Step 2.

Here is an example.

```
options logconfigloc edm-l4s.xml
```

- 4 Create a DS2 logger package instance that is associated with the logger that you created in the XML configuration file from Step 2.

Here is an example.

```

data _null_;
  dcl package logger l('App.Program.EDM');
  method init();
    dcl double i;
    i = l.islevelactive(4); put i=;
    l.log('T', 'Hello World! Trace');
    l.log('D', 'Hello World! Debug');
    l.log('I', 'Hello World! Info');
    l.log('W', 'Hello World! Warning');
    l.log('E', 'Hello World! Error');
  end;
enddata;
run;

```

You can print the contents of the **edmlog** data set.

```

libname logs 'c:\temp';
proc print data=logs.edmlog; run;

```

The data set contents looks like this.

Obs	seqno	date	msg
1	285	25APR2013:17:11:42	Hello World! Trace
2	286	25APR2013:17:11:42	Hello World! Debug
3	287	25APR2013:17:11:42	Hello World! Info
4	288	25APR2013:17:11:42	Hello World! Warning
5	289	25APR2013:17:11:42	Hello World! Error

For more information about the SAS logging facility, see [SAS Logging: Configuration and Programming Reference](#).

Using the MATRIX Package

Overview of the MATRIX Package

Note: The MATRIX package is not supported on the CAS server.

A matrix is a two-dimensional array of numeric or character values. The dimensions of a matrix are defined by the number of rows and columns. The elements of an $n \times p$ matrix are arranged in n rows and p columns.

The matrix package provides a DS2-level implementation of SAS/IML functionality. You can use matrix package methods to perform complex tasks such as matrix inversion. You can perform arithmetic, relational, and logical operations. You can perform some operations on an elementwise basis and other operations on the entire data matrix.

You can load data in a matrix package using an array or external data. You can generate data by writing the entire matrix to an array at one time or row-by-row. The array can then be written to a result table.

For more information about these methods, see [“DS2 Matrix Package Methods, Operators, and Statements” in SAS DS2 Language Reference](#).

Declaring and Instantiating a MATRIX Package

A matrix package is created by declaring and instantiating the matrix package using the DS2 DECLARE PACKAGE statement. Here is an example:

```
declare package matrix m(2, 2);
```

This statement creates a 2 x 2 matrix and stores its instance in the variable `m`.

Note: The matrix is filled with zeros, not null or missing values.

For more information, see [“DECLARE PACKAGE Statement: Matrix Package” in SAS DS2 Language Reference](#).

Matrix Data Input

There are two ways to initialize or load data into a matrix package.

- initialize with zero values

- `declare package matrix instance-name(rows, columns);`

```
□ instance-name=_new_ matrix(rows,columns);
```

■ initialize with array values

```
□ /* loads an array using the array name */
```

```
instance-name=_new_ matrix(array-name, rows, columns);
```

```
□ /* loads an array using the IN method */
```

```
instance-name=_new_ matrix(rows,columns);
instance-name.in(array-name);
```

```
□ /* loads row-by-row using an input table*/
/* a variable array, and the SET statement */
```

```
instance-name=_new_ matrix(rows,columns);
. . .
set table-name;
instance-name.in(variable-array-name, i);
```

These examples create a 2x2 matrix that is initialized with zero values.

```
declare package matrix m(2,2);
```

```
m=_new_ matrix(2, 2);
```

These examples load array **a** into the matrix **m**.

```
/* simple array */
method init();
  dcl double a[3, 3];
  dcl package matrix m;

  a :=(1, 2, -1, 2, 1, 0, -1, 1, 2);
  m=_new_ matrix(a, 3, 3);
end;
```

```
/* variable array */
vararray double a[3,3];
dcl package matrix m;

method init();
  a := (1, 2, -1, 2, 1, 0, -1, 1, 2);
  m = _new_ matrix(a, 3, 3);
end;
```

These examples load array **va** into the matrix **m** using the IN method.

```
/* simple array */
dcl double va[3,3];
dcl package matrix m;
m = _new matrix(3,3);
m.in(va);
```

```
/* variable array */
vararray double va[3,3];
dcl package matrix m;

m.in(va); */
```

This example reads table **x** using the SET statement into variable array **a**. That variable array is then used to load the matrix **m**.

```
vararray double a[4];
dcl package matrix m;

/* Create an empty matrix to hold the input values */
method init();
    m = _new_ [this] matrix(4, 4);
    i = 1;
end;

/* Read and initialize each row of matrix from vararray a */
method run();
    set x;
    m.in(a, i);
    i + 1;
end;
```

For more information, see the “[_NEW_ Operator: Matrix Package](#)” in *SAS DS2 Language Reference* and the “[IN Method](#)” in *SAS DS2 Language Reference*.

Note: The `_NEW_` operator is part of the code stream. It is not used in the declarations.

Matrix Data Output

You can generate matrix data by writing the entire matrix to an array at one time or row-by-row.

Use the `TOARRAY` or `TOVARARRAY` methods to write the matrix to an array at one time. In this example, array **a** is loading into matrix **m**. The transpose of matrix **m** is then calculated and is written to array **c**.

```
dcl double a[3,3];
dcl double b[3,3];

method run();
    dcl package matrix m r;
    dcl double i j;

    a := (1,2,3,4,5,6,7,8,9);

    m = _new_ matrix(a, 3, 3);
    r = m.trans();
    r.toarray(b);

    do i = 1 to 3;
        do j = 1 to 3;
            put b[i,j];
        end;
    end;
end;
```

For more information, see the “[TOARRAY Method](#)” in *SAS DS2 Language Reference* and the “[TOVARARRAY Method](#)” in *SAS DS2 Language Reference*.

Variable arrays can be used to write data by using the OUT method and the OUTPUT statement. The OUT method writes the matrix row data to a variable array. The OUTPUT statement writes the data to a result table. The matrices are written one row at a time to the result table a row at a time by means of the variable array. For more information, see the “[OUT Method](#)” in *SAS DS2 Language Reference* and the “[Loading and Writing Data](#)” in *SAS DS2 Language Reference*.

Matrix Operations

The following table summarizes the operations that can be performed on matrices using typical DS2 dot syntax method calls.

Table 12.1 Matrix Operations

Type of Operation	Method Name	Operation performed	Notes
Binary Arithmetic	ADD	Addition	- The array dimensions for the matrices used in addition or subtraction operations do not have to be compatible. But the number of columns in the first matrix has to equal the number of rows in the second matrix or the second matrix has to be a 1x1 matrix. - The array dimensions for the matrices used in a multiplication operation have to be compatible. The number of columns in the first matrix must equal the number of rows in the second matrix.
	MULT	Multiplication	
	SUB	Subtraction	
Unary	ABS	absolute value	- Unary operations are performed on a single matrix. - Matrices must be square or singular. For inverse and determinant operations, the matrix must be square.
	COPY	copy matrix	
	DET	determinant of a square matrix	
	EXP	exponential value	
	FLOOR	integer part of each matrix value	
	INVERSE	inverse	
	LOG	natural logarithm	

Type of Operation	Method Name	Operation performed	Notes
	SQRT	square root	
	TRANS	transposition	
Binary relational	EQ	equal to	■ The result of any binary relational operation is a matrix whose entries tell how the $[i, j]^{\text{th}}$ element of the first matrix compares to the $[i, j]^{\text{th}}$ element of the second matrix. The result values are either 0 if the comparison is false or 1 if the comparison is true. ■ The matrix sizes must match or you can use a scalar comparison.
	GT	greater than	
	GE	greater than or equal to	
	LT	less than	
	LE	less than or equal to	
	NE	not equal to	
Binary logical	AND	and comparison	■ The result of any binary logical operation is a matrix whose entries tell how the $[i, j]^{\text{th}}$ element of the first matrix compares to the $[i, j]^{\text{th}}$ element of the second matrix. The result values are either 0 if the comparison is false or 1 if the comparison is true.
	OR	or comparison	
ALL relational	ALL_EQ	ALL equal	■ The ALL relational operations produce a scalar result that indicates whether the $[i, j]^{\text{th}}$ element of the first matrix satisfies the comparison with the $[i, j]^{\text{th}}$ element of the second matrix. The scalar result is 0 or 1. All of the $[i, j]$ element comparisons must be true in order for the result to be 1. Otherwise, the result is 0. ■ The matrix sizes must match or you can use a scalar comparison.
	ALL_GT	ALL greater than	
	ALL_GE	ALL greater than or equal to	
	ALL_LT	ALL less than	
	ALL_LE	ALL less than or equal to	
	ALL_NE	ALL not equal to	
ANY relational	ANY_EQ	ANY equal	■ The ANY relational operations produce a scalar result that indicates whether the $[i, j]^{\text{th}}$ element of the first matrix satisfies the comparison with the $[i, j]^{\text{th}}$ element of the second matrix. The scalar result is 0 or 1. If any of the $[i, j]$ element comparisons is true, the result is 1. Otherwise, the result is 0. ■ The matrix sizes must match or you can use a scalar comparison.
	ANY_GT	ANY greater than	
	ANY_GE	ANY greater than or equal to	
	ANY_LT	ANY less than	
	ANY_LE	ANY less than or equal to	

Type of Operation	Method Name	Operation performed	Notes
	ANY_NE	ANY not equal to	
ALL logical	ALL_AND	ALL AND	■ The ALL logical operations produce a scalar result that indicates whether the $[i, j]^{\text{th}}$ element of the first matrix satisfies the comparison with the $[i, j]^{\text{th}}$ element of the second matrix. The scalar result is 0 or 1. All of the $[i, j]$ element comparisons must be true in order for the result to be 1. Otherwise, the result is 0. ■ The matrix sizes must match or you can use a scalar comparison.
	ALL_OR	ALL OR	
ANY logical	ANY_AND	ANY AND	■ The ANY relational operations produce a scalar result that indicates whether the $[i, j]^{\text{th}}$ element of the first matrix satisfies the comparison with the $[i, j]^{\text{th}}$ element of the second matrix. The scalar result is 0 or 1. If any of the $[i, j]$ element comparisons is true, the result is 1. Otherwise, the result is 0. ■ The matrix sizes must match or you can use a scalar comparison.
	ANY_OR	ANY OR	
Elementwise	EDIV	elementwise division	■ Elementwise operations enable you to apply an operation to a general matrix using a matrix with the same dimensions, a vector whose row dimension matches the row dimension of the general matrix, a vector whose column dimension matches the column dimension of the general matrix, or a 1x1 matrix effectively allowing a scalar operation on each $[i, j]$ element. ■ Elementwise operations produce a result matrix from the element-by-element operations on two argument matrices.
	EMAX	elementwise maximum	
	EMIN	elementwise minimum	
	EMOD	elementwise remainder of the division of elements	
	EMULT	elementwise multiplication	
	EPOW	elementwise raise to a power	

Considerations When Using a Matrix Package

- When a matrix is created using a constructor (for example, `m= _new_matrix(2,2);` or declare `package matrix m(2, 2);`), the matrix is filled with zeros, not missing values.

- A matrix that contains null or missing values is not very useful. If a matrix does contain null or missing values, some operations on it might be considered anomalous. For example, if you multiply matrices with null or missing values, you receive a run-time error. But if you add or subtract matrices with null or missing values, you do not receive an error. If you divide a matrix element by zero, you do not get a floating-point exception. The result is a missing value.
- If you use a loop to input or output matrix data, you want to avoid operations on the matrix while the loop is being processed. The reason is that the matrix is only partially filled until the loop is complete.
- It is not always easy to keep track of what size matrix you have. Make sure that the dimensions for your matrix operations are consistent.
- When a matrix is declared in a thread program, each thread program has its own, individual instance of a matrix. The DS2 matrix package does not support data partitioning between nodes or threads to perform parallel matrix operations. Instead, each thread performs the matrix operations on its own instance of the matrix.

Moving Values from a Matrix into a DS2 Array

The following example shows how to use the TOARRAY method. Using this method is the only way that you can move values from a matrix into a DS2 array for use in a DS2 program.

```

dcl double a[3, 3];
dcl double c[3, 3];
declare package matrix m;

a := (1, 2, 3, 4, 5, 6, 7, 8, 9);
m=_new_matrix(a, 3, 3);
m.toarray(c);

do i=1 to 3;
  do j=1 to 3;
    put c[i, j];
  end;
end;
```

Using the PCRXFIND Package

Overview of the PCRXFIND Package

The PCRXFIND package enables you to see whether a particular string of text is located within another string of text. The package is used for light-weight text analytics and includes ease-of-use functions for getting details from a text match.

These details include finding where a substring begins or ends within a block of text, or retrieving a particular portion of a match from a string.

Declaring and Instantiating a PCRXFIND Package

You declare a PCRXFIND package by using the DECLARE PACKAGE statement. After you declare the new PCRXFIND package, you can parse a Perl-compatible regular expression. This expression can be passed in when the package is declared. Passing the expression during package declaration is optional. The loaded expression can then be used to perform match operations by providing the indexes of the beginning and ending of the match, as well as indexes related to each parenthetical capture group within the previously parsed regular expression.

There are three ways to construct an instance of a PCRXFIND package.

- Use the DECLARE PACKAGE statement along with its constructor syntax:

```
declare package pcrxfind variable(<regular-expression>);
```

- Use the DECLARE PACKAGE statement along with the `_NEW_` operator:

```
declare package pcrxfind variable;  
variable = _new_ pcrxfind(<regular-expression>);
```

- Use the DECLARE PACKAGE statement without an expression:

```
declare package pcrxfind variable;  
variable = _new_ pcrxfind();
```

For more information, see the [“DECLARE PACKAGE Statement: PCRXFIND Package”](#) in *SAS DS2 Language Reference*, and the [“_NEW_ Operator: PCRXFIND Package”](#) in *SAS DS2 Language Reference*.

Example

Here is an example that looks at addresses and extracts a couple pieces of information using the PCRXFIND package. Addresses are reported as invalid if they do not match the provided regular expression. This example showcases case insensitive matching, as well as the GETGROUP, GETGROUPLength and the GETMATCH methods.

```
/* Generate a set of valid and invalid addresses */  
data sampleaddr/overwrite=yes;  
  dcl varchar(256) address;  
  method init();  
    address = '123 Pineforest ln Cary NC 27513'; output;  
    address = 'Extra text before the match 5123 Oakwood rd Cary NC  
27513';  
    output;  
    address = '24 Durham NC 27519'; output;  
    address = '0 INVALID BLVD NC 27605'; output;  
    address = 'APT. 731 1008 TRADE AVE RALEIGH NC 27605'; output;  
    address = 'HWY 55 123 Thistlecrest ln Cary NC 27511'; output;  
    address = 'This one is invalid due to length of the street...';
```

```

        423 XYYYSDDDDDDTHHHHHHZZJJJXQTYTRRRRRRRRASDAOIIKKKK CT JASDGTJ
        UW 5290889';
    output;
end;
enddata; run;

data _null_;
    /*
    * Instantiate the address validator with a regular expression that
    does
    * a case insensitive match on addresses. This will be used to
    extract
    * each component of the address or reject the address as invalid.
    */
    dcl package pcrxfind addrValidator
        ('/(apt\.[ ]?[#]?\d+ |hwy \d+ )?(\d{1,5}) (\w+) (
        rd|ln|cir|st|blvd|ave|ct) (\w+) ([A-Z]{2}) (\d+)/i' );

    method run();
        dcl int valid havePrefix streetLength cityLength;
        dcl char(2) state;
        dcl varchar(256) city prefix zipText street streetType streetNum
    outaddr
        streetAddress;
        dcl double zip;
        set sampleaddr;

        /* Perform the match */
        valid = addrValidator.match( address );

        /* Confirm that a match was found before proceeding */
        if valid > 0 then do;
            /*-- Grab each component of the match --*/
            addrValidator.getGroup( state, 6 );
            addrValidator.getGroup( city, 5 );
            cityLength = addrValidator.getGroupLength( 5 );
            addrValidator.getGroup( streetType, 4 );
            addrValidator.getGroup( street, 3 );
            streetLength = addrValidator.getGroupLength( 3 );
            addrValidator.getGroup( streetNum, 2 );

            /* Assemble the street address */
            streetAddress = cat( streetNum, ' ', street, ' ',
    streetType );
            havePrefix = addrValidator.getGroup( prefix, 1 );
            /* Convert text zip to numeric */
            addrValidator.getGroup( zipText, 7 );
            zip = inputn( zipText, 5 );

            /*
            * Make sure that the city and street didn't match over 25
    chars.
            * Filters potentially false results.
            */
            if (cityLength > 25) || (streetLength > 25) then do;
                addrValidator.getMatch( outaddr );
            end;
        end;
    end;
enddata; run;

```

```

        put 'City or street likely invalid due to length: '
    outaddr;
    end;
    else do;
        /* output the details for this address */
        put;
        put 'Valid address: ';
        if havePrefix = 0 then put 'prefix: ' prefix;
        put streetAddress=;
        put city=;
        put state=;
        put zip=;
        put;
    end;
    end;
    /* If valid is -1, the expression was run and no match was found
    */
    else if valid = -1 then do;
        outaddr = trim( address );
        put 'Invalid address: ' outaddr;
    end;
    end;
enddata;

run;

```

The following lines are printed to the log:

```

Valid address:
streetAddress=123 Pineforest ln
city=Cary
state=NC
zip=27513

Invalid address:  Extra text before the match 5123 Oakwood rd Cary NC 27513
Invalid address:  24 Durham NC 27519
Invalid address:  0 INVALID BLVD NC 27605

```

```

Valid address:
prefix:  APT. 731
streetAddress=1008 TRADE AVE
city=RALEIGH
state=NC
zip=27605

```

```

Valid address:
prefix:  HWY 55
streetAddress=123 Thistlecrest ln
city=Cary
state=NC
zip=27511

```

```

Invalid address:  This one is invalid due to the length of the street...
YYYYSDDDDTHHHHHZZZJJJXQTYTRRRRRRASDAOIIKKKK CT JASDGTJ      UW 5290889

```

Using the PCRXREPLACE Package

Overview of the PCRXREPLACE Package

The PCRXREPLACE package enables you to perform a replace or substitute operation on text. For example, sensitive names or phone numbers could be replaced with the word REDACTED, or updates could be made programmatically to text.

The REPLACE package can handle UTF-8 and Latin1 text natively, so transcoding these formats is not required.

Declaring and Instantiating a PCRXREPLACE Package

You declare a PCRXREPLACE package by using the DECLARE PACKAGE statement. You can include the regular expression to associate a PCRXREPLACE package with an expression and substitution text. Including the regular expression during package declaration is optional. After you declare the new PCRXREPLACE package, you can use the package to perform substitution operations on strings.

There are three ways to construct an instance of a PCRXREPLACE.

- Use the DECLARE PACKAGE statement along with its constructor syntax:

```
declare package pcrxreplace variable('/textToReplace/replacementText');
```

- Use the DECLARE PACKAGE statement along with the _NEW_ operator:

```
declare package pcrxreplace variable;  
variable = _new_ pcrxreplace('/textToReplace/replacementText');
```

- Use the DECLARE PACKAGE statement without an expression:

```
declare package pcrxreplace variable;  
variable = _new_ pcrxreplace();
```

For more information, see the [“DECLARE PACKAGE Statement: PCRXREPLACE Package”](#) in *SAS DS2 Language Reference*, and the [“_NEW_ Operator: PCRXREPLACE Package”](#) in *SAS DS2 Language Reference*.

Example

Here is an example that switches the first and last names in a data set.

```
data names(overwrite=yes);  
  dcl varchar(32) name;
```

```

        method init();
            /* create a dataset to work on */
            name = 'Jones, Fred'; output;
            name = 'Kavich, Kate'; output;
            name = 'Turley, Ron'; output;
            name = 'Dulix, Yolanda'; output;
        end;
    enddata;
run;

data _null_;
    /* Variable declaration */
    dcl package pcrxReplace nameSwap;

    method init();
        nameSwap = _new_ pcrxReplace( 's/(\w+), (\w+)/$2 $1/g' );
    end;

    /* Method that switches the first and last names in
       a dataset formatted as follows: 'lastname, firstname' */
    method run();
        dcl varchar(32) fname lname replacementText outputText;
        dcl int rc;
        set names;
        outputText = nameSwap.apply(name);
        put name=;
    end;

enddata;
run;

```

The following lines are printed to the log:

```

name=Fred Jones
name=Kate Kavich
name=Ron Turley
name=Yolanda Dulix

```

Using the SQLSTMT Package

Overview of the SQLSTMT Package

Note: The SQLSTMT package is not supported on the CAS server.

The SQLSTMT package provides a way to pass FedSQL statements to a DBMS for execution and to access the result set returned by the DBMS. The FedSQL statements can create, select, modify, insert, or delete rows from a table. If the FedSQL statements select rows from a table, the SQLSTMT package provides methods for interrogating the rows returned in a result set.

When a SQLSTMT instance is created, the FedSQL statement is sent to the FedSQL language processor which, in turn, sends the statement to the DBMS to be prepared and stored in the instance. The instance can then be used to efficiently execute the FedSQL statement multiple times. With the delay of the statement prepare until run time, the FedSQL statement can be built and customized dynamically during execution of the DS2 program.

Hadoop Distribution: If you are using a Hadoop distribution, the use of the SQLSTMT package requires Hive 0.13 or later.

Here is a simple example of using the SQLSTMT to insert values into a Teradata table.

```
dcl package sqlstmt s('insert into td.testdata (x, y, z) values (?, ?, ?)',
  [x y z]);

do i=1 to 5;
  x=i;
  y=i*1.1;
  z=i*10.01;
  s.execute();
end;
end;
```

The following rows are inserted into the table:

```
1 1.1 10.01
2 2.2 20.02
3 3.3 30.03
4 4.4 40.04
5 5.5 50.05
```

For more examples, see “DS2 Example Programs” in *SAS DS2 Language Reference*.

Declaring and Instantiating a SQLSTMT Package

You use the DECLARE PACKAGE statement to declare the SQLSTMT package. When a package is declared, a variable is created that can reference an instance of the package. If constructor arguments are provided with the package variable declaration, then a package instance is constructed and the package variable is set to reference the constructed package instance.

There are three ways to construct an instance of a SQLSTMT package.

- Use the DECLARE PACKAGE statement along with its constructor syntax. There are two syntax forms:

DECLARE PACKAGE SQLSTMT *variable*[(*'sql-txt'* [, \[*parameter-variable-list*\])];

DECLARE PACKAGE SQLSTMT *variable* [(*'sql-txt'* [, *connection-string*]);

- Use the DECLARE PACKAGE statement along with the _NEW_ operator. There are two syntax forms:

DECLARE PACKAGE SQLSTMT *variable*;

variable = **_NEW_ SQLSTMT**(*'sql-txt'* [, \[*parameter-variable-list*\]);

DECLARE PACKAGE SQLSTMT *variable*;

variable = **_NEW_ SQLSTMT** ('*sql-txt*' [, *connection-string*]);

Note: The DECLARE PACKAGE statement does not construct the SQLSTMT package instance until the **_NEW_** operator is executed. The SQL statement prepare does not occur until the **_NEW_** operator is executed.

- Use the DECLARE PACKAGE statement without SQL text.

DECLARE PACKAGE SQLSTMT *variable*();

variable = **_NEW_ SQLSTMT** ();

With the **_NEW_** operator, the *sql-text* can be a string value that is generated from an expression or a string value that is stored in a variable.

If the DECLARE statement includes arguments for construction within its parentheses (and omitting arguments is valid for the SQLSTMT package), then the package instance is allocated. If no parentheses are included, then a variable is created but the package instance is not allocated.

Multiple package variables can be created and multiple package instances can be constructed with a single DECLARE PACKAGE statement, and each package instance represents a completely separate copy of the package.

Specifying FedSQL Statement Parameter Values

If the FedSQL statement contains parameters, values to substitute for the parameters must be obtained to execute the FedSQL statement. The substitution values are one of the following:

- the current values of the variables specified in the constructor DECLARE PACKAGE statement or the **_NEW_** operator

For more information, see [“DECLARE PACKAGE Statement: SQLSTMT Package” in SAS DS2 Language Reference](#) and the [“_NEW_ Operator: SQLSTMT Package” in SAS DS2 Language Reference](#).

- the current values of the variables specified in the BINDPARAMETERS method

If you use the BINDPARAMETERS method and execute a FedSQL statement, the values of bound variables are read when the statement is executed and used as the values of the statement's parameters. If the type of a bound variable differs from the corresponding parameter's type, the bound variable's value is converted to the parameter's type. For more information, see the [“BINDPARAMETERS Method” in SAS DS2 Language Reference](#).

Note: If you use the SQLSTMT package to execute a DS2 package METHOD statement, use the REGISTEROUTPARAMETER method to map output parameters in the FedSQL statement to IN_OUT parameters in the DS2 package METHOD statement. The parameter data must be specified exclusively with bound variables. A run-time error results if a SETtype method and the

REGISTEROUTPARAMETER method are invoked. For more information, see [“REGISTEROUTPARAMETER Method” in SAS DS2 Language Reference](#).

- the values specified in the SETtype methods

For more information about these methods, see [“DS2 SQLSTMT Package Methods, Operators, and Statements” in SAS DS2 Language Reference](#).

Parameter values must be specified exclusively with bound variables or exclusively with the SETtype methods.

Note: The rules for identifiers for the FedSQL language apply to variables used in the SQLSTMT package, rather than the DS2 rules for identifiers. This occurs because FedSQL parses the string containing the SQL statement rather than DS2.

Specifying a Connection String

A connection string defines how to connect to the data. A connection string identifies the query language to be submitted as well as the information required to connect to the data source or sources.

You can specify a connection string when you declare and instantiate a SQLSTMT package. The connection string parameter is primarily designed for use with the SAS Federation Server. For more information about creating a fully specified connection string, see *SAS Federation Server: Administrator's Guide*.

If a connection string is not provided, the SQLSTMT package instance uses the connection string that is generated by the HPDS2 or DS2 procedure by using the attributes of the currently assigned libref.

Executing the FedSQL Statement

The EXECUTE method executes the FedSQL statement and returns a status indicator. Zero is returned for successful execution; 1 is returned if there is an error; 2 is returned if there is no data (NODATA). The NODATA condition exists when an SQL UPDATE or DELETE statement does not affect any rows.

When the FedSQL statement is executed, the values of bound variables are read and used as the values of the statement's parameters.

A SQLSTMT instance maintains only one result set. The result set from the previous execution, if any, is released before the FedSQL statement is executed.

The FedSQL statement executes dynamically at run time. Because the statement is prepared at run time, it can be built and customized dynamically during the execution of the DS2 program.

Accessing Result Set Data

The FETCH method returns the next row of data from the result set. A status indicator is returned. Zero is returned for successful execution; 1 is returned if there is an error; 2 is returned if there is no data (NODATA). The NODATA condition exists if the next row to be fetched is located after the end of the result set.

If variables are bound to the result set columns with the BINDRESULTS method or by the FETCH method, then the fetched data for each result set column is placed in the variable bound to that column. If the variables are not bound to the result set columns, the fetched data can be returned by the GET*type* methods.

Note: For character data, you can call the GET*type* method repeatedly until all of the result set column data is retrieved. For numeric data, you can call the GET*type* method only once to return the result set column data. Subsequent method calls result in a value of 2 (NODATA) for the *rc* status indicator. For more information, see the GET *type* methods in “DS2 SQLSTMT Package Methods, Operators, and Statements” in *SAS DS2 Language Reference*.

A SQLSTMT instance maintains only one result set. The CLOSERESULTS method automatically releases the result set when the FedSQL statement is executed or deleted.

A run-time error occurs if the FETCH method is called before the FedSQL statement is executed.

Comparing the SQLSTMT Package and the SQLEXEC Function

The following table compares the SQLSTMT package and the SQLEXEC function.

SQLSTMT Package	SQLEXEC Function
applicable when FedSQL statements are executed multiple times	applicable when a FedSQL statement is executed only once
allocates, prepares, executes, and frees a FedSQL statement dynamically at run time	allocates, prepares, executes, and frees a FedSQL statement dynamically at run time
supports the passing of parameters	does not support the passing of parameters
produces a result set	does not produce a result set

SQLSTMT Package	SQLEXEC Function
supports run-time SELECT query generation	cannot be used with a SELECT statement
similar to the Java Database Connectivity (JDBC) PreparedStatement class	similar to the SQL EXECUTE IMMEDIATE statement or the JDBC Statement.executeUpdate(String) method

Using the TZ Package

Overview of the TZ Package

DS2 supports the SQL style date and time conventions that are used in other data sources. When your data source is not a SAS data set, DS2 can process dates and times that have a data type of DATE, TIME, and TIMESTAMP. SAS date, time, and datetime values can be converted to DS2 dates, time, and timestamp values by using the TO_DATE, TO_TIME, and TO_TIMESTAMP functions. However, these functions do not incorporate a time zone.

The TZ package enables you to process local and international time and date values.

Declaring and Instantiating a TZ Package

There are two ways to construct an instance of a TZ package.

- Use the DECLARE PACKAGE statement along with the _NEW_ operator:

```
declare package tz tzpkg;
tzpkg = _new_ tz();
```

- Use the DECLARE PACKAGE statement along with its constructor syntax:

```
declare package tz tzpkg();
```

For more information, see the [“DECLARE PACKAGE Statement: TZ Package”](#) in *SAS DS2 Language Reference*, and the [“_NEW_ Operator: TZ Package”](#) in *SAS DS2 Language Reference*.

Returning Time and Time Zone Information

You can use the TZ package to return the following values:

- current local time
- current Coordinated Universal Time (UTC) time
- current time zone ID
- current time zone name
- the time zone offset of the time zone from UTC at the specified local time
- the time zone offset of the time zone from UTC at the specified UTC time

Here is an example of how to use the TZ package to get the world clock.

```
data _null_ ;
  method init();

  declare package tz tzone() ;
  dcl double tokyo_time london_time new_york_time utc_time ;

  tokyo_time = tzone.getLocalTime('Asia/Tokyo') ;
  london_time = tzone.getLocalTime('Europe/London') ;
  new_york_time = tzone.getLocalTime('America/New_York') ;

  utc_time = tzone.getUTCtime();

  put utc_time = datetime. ;
  put tokyo_time = datetime. ;
  put london_time = datetime. ;
  put new_york_time = datetime. ;
end ;
enddata ;
run;
```

The following lines are written to the SAS log:

```
utc_time=18NOV14:13:53:11
tokyo_time=18NOV14:22:53:11
london_time=18NOV14:13:53:11
new_york_time=18NOV14:08:53:11
```

For more information about the methods to perform these actions, see [“DS2 TZ Package Methods, Operators, and Statements” in SAS DS2 Language Reference](#). For more information about time zone ID and names, see [“Time Zone IDs and Time Zone Names” in SAS National Language Support \(NLS\): Reference Guide](#).

Returning Time Zone Offset

You can use the TZ package to return the time zone offset from UTC at either the specified local time or at the specified UTC time.

The time zone offset specifies the number of hours and minutes that a time zone is off from the UTC in the form `+|-hhmm` or `+|-hh:mm`.

Here is an example that returns the time zone offset from 'asia/tokyo' and from 'America/New_York'.

```
data _null_ ;
```

```

method init();

declare package tz tzone('asia/tokyo') ;
dcl double new_york local_time;
dcl char(40) cstr ;

local_time = tzone.getOffset() ;
put local_time time.;

new_york = tzone.getOffset('America/New_York') ;
put new_york time.;

end;
enddata ;
run;

```

The following lines are written to the SAS log:

```

9:00:00
-4:00:00

```

Converting Local or UTC Time

You can use the TZ package to convert local to one of the following time formats:

- ISO8601 with or without a time zone offset
- a TIMESTAMP string with a time zone ID
- UTC time

In addition, you can convert UTC time to local time.

Here is an example.

```

data _null_ ;
method init();

declare package tz tzone('asia/tokyo') ;
dcl double local_time ;
dcl char(35) local_time_iso local_time_utc local_time_tz;

local_time = tzone.tolocaltime(15550) ;
put local_time time.;

local_time_iso = tzone.toiso8601(15500) ;
put local_time_iso ;

local_time_utc = tzone.toutctime(15500) ;
put local_time_utc time.;

local_time_tz = tzone.totimestampz(15500);
put local_time_tz;

```

```
end;  
enddata ;  
run;
```

The following lines are written to the SAS log.

```
13:19:10  
1960-01-01T04:18:20.00+09:00  
-4:41:40  
1960-01-01 04:18:20.00 asia/tokyo
```


Threaded Processing

<i>Overview of Threaded Processing</i>	195
<i>Threading and DS2 Programs</i>	197
Overview of Serial and Parallel Programs	197
Data Manipulation Operations	197
DS2 Program Summary	198
<i>Automatic Variables That Are Useful in DS2 Threading</i>	199
<i>Threaded Processing and the SAS In-Database Code Accelerator</i>	199

Overview of Threaded Processing

Typically, DS2 code runs sequentially. That is, one process runs to completion before the next process begins. It is possible to run more than one process concurrently, using **threaded processing**. In threaded processing, each concurrently executing section of code is said to be running in a **thread**. DS2 threading works well both on a machine with multiple cores and within a massively parallel processing (MPP) database.

A DS2 program processes input data and produces output data. A DS2 program can run in two different ways: as a program and as a thread. When a DS2 program runs as a program, here are the results:

- Input data can include both rows from database tables and rows from DS2 program threads.
- Output data can be either database tables or rows that are returned to the client application.

When a DS2 program runs as a thread, here are the results:

- Input data can include only rows from database tables, not other threads.
- Output data includes the rows that are returned to the DS2 program that started the thread.

To enable DS2 code to run in threads:

- 1 Create the thread by enclosing your DS2 code between `THREAD...ENDTHREAD` statements.
- 2 Create one or more instances of the thread in a DS2 program by using a `DECLARE THREAD` statement.
- 3 Execute the thread or threads by using a `SET FROM` statement.

In this example, a very simple thread, `T`, is created by using the `THREAD` statement.

```
thread t;
  dcl int x;
  method init();
    dcl int i;
    do i = 1 to 3;
      x = i;
      output;
    end;
  end;
endthread;
```

In this DS2 program, an instance of `T` is declared, and two threads are executed, using the `SET FROM` statement in the `RUN` method. Each of the two threads generates three rows for `x` for a total of six rows in the output table.

```
data;
  dcl thread t t_instance;
  method run();
    set from t_instance threads=2;
    put 'x= ' x ;
  end;
enddata;
```

When you run the DS2 program, the SAS log might display the following output. Because of how threads are processed, the order of the output could be different.

```
x= 1
x= 2
x= 3
x= 1
x= 2
x= 3
```

Note: If one computation thread can keep up with the I/O thread, then that single thread is used for all computation.

Note: A single reader feeds all threads. A `SET` statement in a thread program shares a single reader for that `SET` statement. Each row in the input table is sent to exactly one thread.

Note: Types must be an exact match between a data program and a thread program. For the decimal data type, both the precision and scale must match. For character strings, the column size, the character set, and whether the string is of varying length or fixed length, must all match.

For more information, see the [“THREAD Statement” in SAS DS2 Language Reference](#).

Threading and DS2 Programs

Overview of Serial and Parallel Programs

A DS2 program can perform manipulations on multiple data rows concurrently, thus reducing the time required to process big data sets. Based on the structure of the DS2 program, the DS2 compiler determines which operations can be performed on multiple rows concurrently and which operations must be applied to each row sequentially. A DS2 program is classified as either a serial program, parallel program, or parallel-serial program.

serial program

contains operations with data dependencies across rows. Thus, rows must be processed in serial. A data program processes the data set and generates the result set.

parallel program

contains no operations with data dependencies across rows. Thus, multiple data rows can be processed in parallel. Each thread processes a subset of the data set and generates a subset of the result set.

parallel-serial program

contains some operations with data dependencies across rows and some operations without data dependencies. The processing of the operations is divided into two stages, a parallel stage and a serial stage. During the parallel stage, each thread processes a subset of the input data set and generates a subset of an intermediate data set. During the serial stage, one thread processes the complete intermediate data set and generates the complete result set.

Note: For more information, see [“How DS2 Runs in CAS” on page 273](#).

Data Manipulation Operations

A DS2 data manipulation operation is classified as either a serial operation or a parallel operation.

serial operation

operation having data dependencies across rows.

- Serial operations must be applied sequentially to each data row.

- Serial operations are implemented by statements in a DS2 data program.

parallel operation

operation having no data dependencies across rows.

- Parallel operations can be applied to multiple data rows in parallel.
- Parallel operations are implemented by statements in a DS2 thread program.

The DS2 compiler categorizes statements based on the placement of the statements within the DS2 program. The DS2 compiler assumes that all statements in a thread program implement parallel operations and all statements in a data program implement serial operations. The only exceptions are data input and output statements. The DS2 compiler does not consider SET, SET FROM, or OUTPUT as data manipulation statements.

DS2 Program Summary

A DS2 program is classified as either a serial program, parallel program, or a parallel-serial program based on the type of programs and type of data manipulation operations that it contains.

DS2 serial program

program that contains only serial operations.

- Has only a data program (no thread program).
- Data program contains serial data manipulation operations.

DS2 parallel program

program that contains only parallel operations.

- Has a thread program and a data program.
- Thread program contains parallel data manipulation operations.
- Data program contains no data manipulation operations. That is, the data program does not contain any statements besides SET FROM and OUTPUT.

DS2 parallel-serial program

program contains both parallel and serial operations.

- Has a thread program (parallel stage) and a data program (serial stage).
- Thread program contains parallel data manipulation operations.
- Data program contains serial data manipulation operations. That is, the data program contains at least one statement besides SET FROM and OUTPUT.

Automatic Variables That Are Useful in DS2 Threading

There are several automatic variables that are used for subsetting a problem across DS2 threads. These automatic variables are also useful for providing context when you are debugging with PUT statements.

Variable	Description
<code>_HOSTNAME_</code>	Returns the name of the worker node or host on which the DS2 program is running.
<code>_NTHREADS_</code>	Total number of DS2 threads running in the program. In a parallel environment, <code>_NTHREADS_</code> is the total number of DS2 threads across all nodes on which the DS2 program is running.
<code>_THREADID_</code>	During the execution of a serial program, that is, a program that does not contain a thread component, the executing data program is assigned <code>_THREADID_ = 0</code> . During the execution of a parallel program, the executing data program is assigned <code>_THREADID_ = 0</code> and each executing thread program is assigned a unique <code>_THREADID_</code> from 1 to number of threads.

Threaded Processing and the SAS In-Database Code Accelerator

A DS2 threaded program enables concurrent transformation of data. A DS2 threaded program consists of a thread program of transformations that support concurrency and a data program of transformations that require serialization.

When executed, a DS2 threaded program uses multiple threads. Each thread executes the thread program to concurrently read and transform a portion of the data. If the data program specifies serial transformations, an additional thread is used to combine the data from the multiple threads and perform serialized transformations on the data. The transformed data is then written to output.

The SAS In-Database Code Accelerator publishes the DS2 thread program to the database and executes the thread program in parallel inside the database. If the data program does not contain any transformations requiring serialization, the SAS In-Database Code Accelerator also publishes and executes the data program in parallel inside the database.

Note: The SAS In-Database Code Accelerator is not supported in SAS Viya or the CAS Server.

Examples of thread programs include large transpositions, computationally complex programs, scoring models, and BY-group processing.

The SAS In-Database Code Accelerator is available for Greenplum, Hadoop, and Teradata and must be licensed at your site.

For more information about using the SAS In-Database Code Accelerator, see *SAS In-Database Products: User's Guide*.

SAS In-Database Code Accelerator

<i>Overview of the SAS In-Database Code Accelerator</i>	202
<i>Requirements for Using the SAS In-Database Code Accelerator</i>	202
<i>SAS In-Database Code Accelerator for Greenplum</i>	203
<i>SAS In-Database Code Accelerator for Hadoop</i>	204
Overview	204
Supported File Types	205
How to Run the SAS In-Database Code Accelerator for Hadoop as a Spark Application	205
Considerations When Using SPD Engine Files	205
BY-Group Processing with Hadoop	205
Using the DBCREATE_TABLE_OPTS Table Option	206
Using the SCRATCH_DB Data Set Option or LIBNAME Option	206
Deprecated: Using HCatalog within the SAS Environment	206
Deprecated: Additional Prerequisites When Accessing Files That Are Processed Using HCatalog	208
<i>SAS In-Database Code Accelerator for Teradata</i>	209
<i>Using the DS2ACCEL Option to Control In-Database Processing</i>	209
<i>BY-Group Processing When Running Thread Programs inside the Database</i>	210
<i>Considerations and Limitations</i>	210
Greenplum, Hadoop, and Teradata	210
Greenplum	212
Hadoop	212
Teradata	214
<i>SAS In-Database Code Accelerator Examples</i>	215
Example 1: Running a Thread inside the Database	215
Example 2: Using User-Defined Formats	216
Example 3: Using User-Defined Formats and Packages	218
Example 4: BY-Group Processing	220

Overview of the SAS In-Database Code Accelerator

The SAS In-Database Code Accelerator enables you to publish a DS2 thread program to the database and execute that thread program in parallel inside the database. Examples of thread programs include large transpositions, computationally complex programs, scoring models, and BY-group processing.

The SAS In-Database Code Accelerator for Hadoop and the SAS In-Database Code Accelerator for Teradata also enable you to publish and execute the DS2 data program inside the database. You cannot publish and execute the DS2 data program inside the database with the SAS In-Database Code Accelerator for Greenplum.

With in-database processing, data is distributed on different data partitions. Each DS2 thread that is running inside the database has access to its own data partition. When doing BY-group processing, each DS2 thread with a BY statement can group and order only the rows in the same data partition. The data partition might have only part of the entire group of data. You need to do a final aggregation in the main data program. However, if you use the PROC DS2 statement's BYPARTITION=YES option, the entire group of data resides on the same data partition. For more information, see [“BY-Group Processing When Running Thread Programs inside the Database” on page 210](#).

Note: The SAS In-Database Code Accelerator is available only for Greenplum, Hadoop, and Teradata.

Requirements for Using the SAS In-Database Code Accelerator

To use the SAS In-Database Code Accelerator, the following requirements must be met. Otherwise, the thread program is run in multiple threads on the client machine.

- See [SAS System Requirements](#).
- The SAS Embedded Process must be installed and configured on your database.

For information about installing and configuring the SAS Embedded Process, see [SAS 9.4 and SAS Viya 3.5 Embedded Process: Deployment Guide](#).

Note: The SAS In-Database Code Accelerator is always in synchronization with the latest version of the SAS Embedded Process. For example, to use the SAS In-Database Code Accelerator that is associated with SAS 9.4M9, you must use the version of the SAS Embedded Process that is shipped with the SAS In-Database Code Accelerator. For the SAS releases that support each version of the SAS Embedded Process, see your data source in *SAS 9.4 and SAS Viya 3.5 Embedded Process: Deployment Guide*.

- In **SAS 9.4M9**, the SAS In-Database Code Accelerator for Hadoop has new requirements in configuration and usage. For more information, see [“Overview” on page 204](#).
 - Your DS2 code must include a thread program and a data program.
 - The tables used as input to the thread program must reside in the database.
 - Either the PROC DS2 DS2ACCEL= option must be set to YES or the DS2ACCEL= system option must be set to ANY.
-

Note: The SAS In-Database Code Accelerator is not supported in SAS Viya or on the CAS server.

SAS In-Database Code Accelerator for Greenplum

When you use the SAS In-Database Code Accelerator for Greenplum, the thread program and its associated files (format files, packages, and so on) are published to the database. The thread program is executed inside the database, and its result is brought to the data program running on client machine for final processing or aggregation if needed.

SAS In-Database Code Accelerator for Hadoop

Overview

IMPORTANT In SAS 9.4M9 for Hadoop, security policies have been introduced by a newer version of the Java Runtime Environment. Here is a summary of changes:

- You must dispatch Spark jobs by using the Apache Livy REST service. Previously, dispatching with Livy REST was required on Windows only. It is now required on UNIX and Windows. MapReduce is not supported. Here are the details:
- The [SAS_EP_JOB_MANAGEMENT_URL=](#) environment variable is required. Specify it before you submit the LIBNAME statement. Previously this environment variable was required on Windows only. It is now required on UNIX and Windows.
- The [SAS_EP_ENABLE_EPCS=](#) environment variable is no longer required, because TRUE is now the default.
- The [HADOOPPLATFORM=](#) system option is no longer required, because SPARK is now the default.
- Configuration changes are required. See [SAS 9.4](#) and [SAS Viya 3.5 Embedded Process: Deployment Guide](#).

Note: The SAS In-Database Code Accelerator for Hadoop is also available by licensing SAS Data Loader. SAS In-Database Code Accelerator functionality is available for use with SAS Data Loader directives. If you install the SAS In-Database Technologies for Hadoop software, you can also submit DS2 code directly from a SAS session without using SAS Data Loader for Hadoop directives. For more information about installing the SAS Data Loader for Hadoop software, see *SAS Data Loader for Hadoop: Installation and Configuration Guide*.

Note: The SAS In-Database Code Accelerator for Hadoop supports only specific versions of the Hadoop distributions. For more information, see [SAS 9.4 Supported Hadoop Distributions](#).

Supported File Types

The SAS In-Database Code Accelerator for Hadoop supports Hive tables. Beginning in SAS 9.4M9, HDMD files are not supported.:

How to Run the SAS In-Database Code Accelerator for Hadoop as a Spark Application

Beginning in SAS 9.4M9, When you use the SAS In-Database Code Accelerator for Hadoop, the data and thread programs run in a Spark application. You must dispatch Spark jobs by using the Apache Livy REST service. MapReduce is not supported. Changes are required. For details, see [“Overview” on page 204](#).

In earlier releases, by default the programs run in the MapReduce framework (YARN or MapReduce 2). In SAS 9.4M6, you can specify that the data and thread programs run as a Spark application by setting the HADOOPPLATFORM system option to SPARK. For more information, see [“HADOOPPLATFORM= System Option” in SAS In-Database Products: User's Guide](#).

Considerations When Using SPD Engine Files

If you are using SPD Engine files with the SAS In-Database Code Accelerator, be aware of the following considerations. For more information, see *SAS SPD Engine: Storing Data in the Hadoop Distributed File System*.

- In SAS 9.4M6, CEDA processing of SPD Engine input files is supported by the SAS In-Database Code Accelerator for Hadoop. Previously, only SPD Engine data sets whose architectures matched the architecture of the Hadoop cluster (that is, 64-bit Solaris or Linux) ran inside the database. (Solaris is not supported in SAS 9.4M9.)
- In SAS 9.4M4, Hive tables that use the SPD Engine SerDe are supported as input to the SAS In-Database Code Accelerator for Hadoop.

BY-Group Processing with Hadoop

For information about BY-group processing and other details, see the paper [Execution of User-Written DS2 programs inside Apache Spark using SAS In-Database Code Accelerator](#).

Using the DBCREATE_TABLE_OPTS Table Option

The DBCREATE_TABLE_OPTS table option is used to provide a free form string in the DATA statement. For the SAS In-Database Code Accelerator for Hadoop, you can use the DBCREATE_TABLE_OPTS table option to specify the output SerDe, the output delimiter of the Hive table, the output escaped by, and any other CREATE TABLE syntax allowed by Hive.

For more information, see [“DBCREATE_TABLE_OPTS= Table Option” in SAS DS2 Language Reference](#).

Using the SCRATCH_DB Data Set Option or LIBNAME Option

SAS/ACCESS Interface to Hadoop supports a LIBNAME and data set option, SCRATCH_DB, for a Hive database that is used when a temporary table is created. The SAS In-Database Code Accelerator for Hadoop supports this option.

There are two instances where the code accelerator uses a Hive temporary table:

- When the input source is a Hive view, the view is materialized out to a temporary table within the SCRATCH_DB database.
- When the code accelerator pulls the output back to Work or another database, the external table that is created for the output from the SAS Embedded Process is temporarily located in the table within the SCRATCH_DB database.

For more information, see the [“SCRATCH_DB= Data Set Option” in SAS/ACCESS for Relational Databases: Reference](#) and the [“SCRATCH_DB= LIBNAME Statement Option” in SAS/ACCESS for Relational Databases: Reference](#).

Deprecated: Using HCatalog within the SAS Environment

IMPORTANT When you update to SAS 9.4M9, MapReduce is no longer supported, and Spark is used instead. As a result, features that use HCatalog are not supported. For more information, see *SAS In-Database Products: User's Guide*.

HCatalog is a table management layer that presents a relational view of data in the HDFS to applications within the Hadoop ecosystem. With HCatalog, data structures that are registered in the Hive metastore, including SAS data, can be

accessed through standard MapReduce code and Pig. HCatalog is part of Apache Hive.

The SAS In-Database Code Accelerator for Hadoop uses HCatalog to process complex, non-delimited files.

Table 14.1 Summary of HCatalog File I/O

File Type	Input	Output
delimited	HDFS direct-read HCatalog if partitioned, skewed, or escaped	HDFS direct-read HCatalog if partitioned, skewed, or escaped
RCFile	HCatalog	HCatalog
ORC	HCatalog	HCatalog
Parquet	HCatalog ¹	CREATE TABLE AS SELECT ²
sequence	HDMD HCatalog if partitioned or skewed	HCatalog
Avro	HCatalog ¹	CREATE TABLE AS SELECT ³

¹ In SAS 9.4M4, partitioned Avro or Parquet data is supported as input to the SAS In-Database Code Accelerator for Hadoop.

² Unable to write output directly to Parquet files due to these issues: <https://issues.apache.org/jira/browse/HIVE-8838>.

³ Unable to write output directly to Avro files due to these issues: <https://issues.apache.org/jira/browse/HIVE-8687>.

Consider these requirements when using HCatalog:

- Data that you want to access with HCatalog must first be registered in the Hive metastore.
- The recommended Hive version for the SAS In-Database Code Accelerator for Hadoop is 0.14 or later.
- Older Hadoop distributions prior to Hive version 1.0 might not include the Avro JAR files by default. Additional JAR files are required and must be defined in the SAS_HADOOP_JAR_PATH environment variable.
- Support for HCatalog varies by vendor. For more information, see the documentation for your Hadoop vendor.

Deprecated: Additional Prerequisites When Accessing Files That Are Processed Using HCatalog

IMPORTANT When you update to SAS 9.4M9, MapReduce is no longer supported, and Spark is used instead. As a result, features that use HCatalog are not supported. For more information, see *SAS In-Database Products: User's Guide*.

If you plan to access complex, non-delimited file types such as Avro or Parquet, through HCatalog, there are additional prerequisites:

- To access Avro file types, the `avro-1.7.4.jar` file must be added to the `SAS_HADOOP_JAR_PATH` environment variable. To access Parquet file types, the `parquet-hadoop-bundle.jar` file must be added to the `SAS_HADOOP_JAR_PATH` environment variable. In addition, you need to add the following HCatalog JAR files to the `SAS_HADOOP_JAR_PATH` environment variable:

```
webhcat-java-client*.jar
hbase-storage-handler*.jar
hcatalog-server-extensions*.jar
hcatalog-core*.jar
hcatalog-pig-adapter*.jar
```

- On the Hadoop cluster, the SAS Embedded Process for Hadoop install script automatically adds HCatalog JAR files to its configuration file. The HCatalog JAR files are added to the Embedded Process Map Reduce job class path during job submission.

For information about installing and configuring the SAS Embedded Process for Hadoop, see *SAS In-Database Products: Administrator's Guide*.

- You must include the HCatalog JAR files in your `SAS_HADOOP_JAR_PATH` environment variable.

For more information about the `SAS_HADOOP_JAR_PATH` environment variable, see *SAS Hadoop Configuration Guide for Base SAS and SAS/ACCESS*.

- The `hive-site.xml` file must be in your `SAS_HADOOP_CONFIG_PATH`.

For more information about the `SAS_HADOOP_CONFIG_PATH` environment variable, see *SAS Hadoop Configuration Guide for Base SAS and SAS/ACCESS*.

- To access Hive SPD Engine tables from the SAS In-Database Code Accelerator, you must include the path to the SerDe JAR files in the `SAS_HADOOP_JAR_PATH`.

For more information about the SAS_HADOOP_JAR_PATH environment variable, see *SAS Hadoop Configuration Guide for Base SAS and SAS/ACCESS*.

SAS In-Database Code Accelerator for Teradata

When you use the SAS In-Database Code Accelerator for Teradata, the data program, the thread program, and their associated files (format files, packages, and so on) are published to the database. Both the data program and the thread program are executed inside the database.

Using the DS2ACCEL Option to Control In-Database Processing

The DS2ACCEL system option controls whether DS2 code is executed inside the database.

In the December 2013 release, the default behavior is to run the data and thread programs on the client machine (DS2ACCEL=NONE). You must set either the DS2ACCEL= system option to ANY or the DS2ACCEL= option in the PROC DS2 statement to YES for in-database processing to occur. The DS2ACCEL= option in the PROC DS2 statement overrides the DS2ACCEL system option.

Note: This is a change in behavior from the previous release in which the default value for the PROC DS2 INDB option (now named DS2ACCEL) caused the SAS In-Database Code Accelerator to automatically trigger in-database processing.

For more information, see [“DS2ACCEL= System Option” in SAS DS2 Language Reference](#) and [“PROC DS2 Statement” in Base SAS Procedures Guide](#).

BY-Group Processing When Running Thread Programs inside the Database

DS2 BY-group processing groups the rows from input tables and orders the rows by values of one or more columns in the BY statement.

With in-database processing, data is distributed on different data partitions. Each DS2 thread running inside the database has access to one data partition. Each DS2 thread can group and order only the rows in the same data partition. Consequently, the data partition might have only part of the entire group of data. You must do a final aggregation in the main data program.

But, in some instances, it is necessary for each thread to process the entire group of data. The SAS In-Database Code Accelerator provides a way to redistribute the input table to the thread program with a BY statement so that the entire group of data resides on the same data partition.

The PROC DS2 statement BYPARTITION argument controls whether the input data is re-partitioned. By default, the input data for the DS2 program is automatically re-partitioned by the first BY variable. All of the BY groups are in the same data partition and processed by the same thread. Each thread does the BY processing for the entire group of data. You might not need to do the final aggregation in the main data program.

Note: Using BYPARTITION=NO in the PROC DS2 statement is not supported by the SAS In-Database Code Accelerator for Hadoop.

For more information, see [“Interleaving” on page 240](#), and the DS2 procedure in [Base SAS Procedures Guide](#).

Considerations and Limitations

Greenplum, Hadoop, and Teradata

- If the thread program is run inside the database, the number of threads is set by the SAS In-Database Code Accelerator. When this occurs, the THREADS= argument in the SET FROM statement in the data program has no effect.

- When a matrix is declared in a thread program, each thread program has its own, individual instance of a matrix. The DS2 matrix package does not support data partitioning between nodes or threads to perform parallel matrix operations. Instead, each thread performs the matrix operations on its own instance of the matrix.
- The DS2 program fails if you try to use an empty format that you defined with PROC FORMAT.
- In-database processing does not occur when the following methods are used to load data. Instead, the data and thread programs are run on the client.
 - using an SQLSTMT package
 - using an initialized hash package
 - using an HTTP package
- Using an unrecognized catalog in the SET statement causes the thread program to run on the client.
- Only one SET FROM statement is allowed in the data program. Otherwise, an error occurs.
- Some data sources choose their own preferred order for columns in the output table from DS2. For example, on Hive, the BYPARTITION columns are always moved to the end of the table. This is common as various data sources try to optimize their performance.

The order of declaration in a DS2 program might not be used as the order of columns in the data source. For example, if you use `keep K1- - K4 ;`, you might not get the columns as you expect or you might get an error because `K1` appears after `K4` in the CREATE TABLE statement.
- Custom null values in delimited tables are not supported.
- Null values are converted to blank values.
- In SAS mode, the IN and NOT IN expressions return a Boolean result based on whether the result of an expression is contained in a list. In ANSI mode, the IN and NOT IN expressions return a null value.
- Tables that are merged or joined inside the database using the SAS In-Database Code Accelerator must be from the same schema.
- If you put the BY statement in the data program, all the rows leaving the thread program are forced into a single data program task. Using a BY statement in the data program works best when it is preceded by data reduction logic in the thread program.
- When using the SAS In-Database Code Accelerator, remember that column names that are enclosed in quotation marks are case sensitive. However, each data source has its own naming conventions and case sensitivity. For example, a column that is named "ID" becomes "id" in Hive because all names in Hive are normalized and case insensitive.

Greenplum

- Only the thread program runs inside the database.

Hadoop

- User-defined column names that contain a period (.) or colon (:) cause errors and are not supported.
- In SAS 9.4M6, the SAS In-Database Code Accelerator for Hadoop supports SQL queries containing a WHERE IN clause.
- Both the data and thread program can run inside the database if the output table from the data program resides in Hadoop.

You can use a different LIBNAME statement for the input and output table if the input and output librefs meet the following conditions:

- The librefs are on the same Hadoop cluster.
- Both files must be accessible by Hive, or both files must be accessible in HDFS by means of an HDMD file.
- When the connection strings are compared, they must be identical in value and case except for these values:
 - DBMAX_TEXT
 - SCHEMA
 - HDFS_METADIR
 - HDFS_PERMDIR
 - HDFS_TEMPDIR

Note: Beginning in SAS 9.4M9, the HDFS_METADIR=, HDFS_PERMDIR=, and HDFS_TEMPDIR= options are no longer supported.

If the output table from the data program does not reside in Hadoop, only the thread program is run inside the cluster.

- Follow these guidelines if you use multi-table SET statements:
 - Multi-table SET statements and a SET statement with embedded SQL code are allowed. An example is `set dblib.invoice dblib.paysched;`
 - The librefs in the SET statement must be the same (for example, they must have the same schema or permissions, or they must use the same catalog) for in-database processing to occur. Otherwise, the data and thread programs are run on the client.

- ❑ Only one SET statement is allowed. If more than one SET statement is used in the thread program, the thread program is not run inside the database. Instead, the thread program runs on the client.
- ❑ Using multi-table SET statements, embedded SQL, or the MERGE statement requires Hive.
- MERGE statements are allowed and available for in-database processing when using the SAS In-Database Code Accelerator for Hadoop.

Note: Tables with the SPD Engine format do not support the MERGE statement.

- A HAVING *format* clause is not supported.
- A Hive STRING data type is always converted to a VARCHAR data type using the following rules:
 - ❑ STRING -> VARCHAR(65355)
 - ❑ STRING + SASFMT:CHAR(*n*) -> VARCHAR(*n*)
 - ❑ STRING + SASFMT:VARCHAR(*n*) -> VARCHAR(*n*)
 - ❑ STRING + DBMAX_TEXT -> VARCHAR(DBMAX_TEXT)
- The Hive user needs Read and Write access to the TempDir and the Destination Warehouse directories. In addition, the MapReduce user needs Read and Write permission.

Note: Beginning in SAS 9.4M9, MapReduce is not supported.

- When working with delimited files, data is textualized using the closest fitting format. The data is stored in a textualized manner. Therefore, some discrepancies might occur, and the transformation causes alteration of precision. For example, Hadoop would create an HDFS text representation of a floating point DOUBLE value. After retrieving the value, the resulting DOUBLE value could be slightly different from the starting value.
- The BYPARTITION=NO option in the PROC DS2 statement specifies that the input data is not re-partitioned even if there is a BY statement and enables two-stage aggregation. When using the SAS In-Database Code Accelerator for Hadoop, this option setting is ignored and the BYPARTITION=YES is used. Alternate thread stage aggregation techniques such as a hash object should be used instead of BYPARTITION=NO.
- Hadoop reserved keywords cannot be used for table names. Quoting table names that are Hadoop reserved keywords does not work.
- If you are performing multi-table joins with either the SET or MERGE statement, how you structure your thread and data program can affect performance. Multi-table joins with SET or MERGE statements require a Hadoop SQL pre-step that must complete before the DS2 steps can start. The performance of this pre-step varies depending on the Hadoop SQL processor being used.
- If you are using HCatalog sources, you must add the HADOOP_HOME environment variable in the Windows environment. An example of the value for

this option is `HADOOP_HOME=c:\hadoop`. The directory must contain a subdirectory named `bin` that must contain the `winutils.exe` file for your distribution. Contact your distribution vendor for a copy of the `winutils.exe` file. For more information, see [Problems running Hadoop on Windows](#).

Note: Beginning in SAS 9.4M9, HCatalog is not supported.

Teradata

- Both the data and thread program run inside the database if the output table from the data program resides in Teradata.

Note: If the data program contains any data transformations beyond creating output table data, the data program is not run inside the database.

Note: It is recommended that you not use an `OUTPUT` statement in the `INIT` or `TERM` methods of your data program. Teradata generates duplicate results (one row for each AMP) rather than just one row.

You can use a different `LIBNAME` statement for the input and output table if the input and output librefs meet the following conditions:

- ☐ The librefs are in the same Teradata database.
- ☐ When the connection strings are compared, they must be identical in value and case except for these values:
 - `CATALOG`
 - `SCHEMA`

If the output table from the data program does not reside in Teradata, only the thread program is run inside the database.

- Multi-table `SET` statements and a `SET` statement with embedded SQL code are allowed. Here is an example.

```
set dblib.invoice dblib.paysched;
```

Note: The librefs in the `SET` statement must be the same (for example, they must have the same schema or permissions, or they must use the same catalog). Otherwise, the data and thread programs are run on the client.

Note: Only one `SET` statement is allowed. If more than one `SET` statement is used in the thread program, the thread program is not run inside the database. Instead, the thread program runs on the client.

- MERGE statements are allowed when using the SAS In-Database Code Accelerator for Teradata.
- The data program runs on all AMPs, but only one of those AMPs sees the input data. This behavior often goes unnoticed. If the data program has only a RUN method that iterates over the input and produces output rows, the RUN method logic fires only on one AMP because that AMP is the only one receiving input. However, if the data program has other programming blocks (such as an INIT or TERM method) that produce output rows, output of rows from INIT or TERM produce one output row for each AMP.

SAS In-Database Code Accelerator Examples

Example 1: Running a Thread inside the Database

Here is an example of a DS2 program whose data and thread programs are published and executed in database through the SAS In-Database Code Accelerator. The results from the thread program are processed by the data program inside the database.

```
options ds2accel=any;

libname teralib teradata server=terapin database=xxxxxx
        user=xxxxxx password=xxxxxx;

data teralib.indata;
    do i = 1 to 10;
        output;
    end;
run;

proc ds2;

thread th_pgm / overwrite=yes;
    retain isum 0;
    keep isum;
    dcl double x isum;

    method run();
        set teralib.indata;
        x=i+1;
        isum=isum+i;
    end;

    method term();
```

```

        output;
    end;

endthread;
run;

data out(overwrite=yes);
    retain fsum 0;
    retain nrows 0;
    keep fsum nrows;

    dcl thread th_pgm m;
    method run();
        /* The THREADS= argument in the SET FROM statement has no effect */
        /* if the SAS In-Database Code Accelerator is used to access a */
        /* database table. */
        set from m threads=1;
        fsum =fsum + isum;
        nrows = nrows + 1;
    end;

    method term();
        output;
    end;
enddata;
run;
quit;

```

Example 2: Using User-Defined Formats

The following example uses formats that are defined in PROC FORMAT. Those formats that are referred to in the thread program are used to create an XML file. In addition to the data and programs, the format XML file is published to the database. The format XML file is used when running DS2 inside the database.

```

options ds2accel=any;

libname teralib teradata server=terapin database=xxxxxx
    user=xxxxxx password=xxxxxx;

%let libname=teralib;

data &libname..indata_fmt;
    do i = 1 to 10;
        output;
    end;
run;

proc format;
    value yesno 1='YES' 0='NO';
run;

proc format;

```

```

        value $x '1'='YES' '0'='NO';
run;

proc ds2;
drop thread th_pgm; run;

thread th_pgm;
  dcl double x;
  dcl char z w;
  method run();
    set &libname..indata_fmt;
    x=i+1;
    z=put(1, yesno.);
    w=put('0', $x.);
  end;
endthread;
run;

data out (overwrite=yes);
  dcl thread th_pgm m;
  method run();
    dcl double y;
    /* The THREADS= argument in the SET FROM statement has no effect */
    /* if the SAS In-Database Code Accelerator is used to access a */
    /* database table. It could have been omitted from the SET FROM*/
    /* statement. */
    set from m threads=10;
    y=x+1;
  end;
enddata;
run;
quit;

```

The following output table is produced.

Output 14.1 Result Table for Example 2

x	z	w	i
2	YES	NO	1
3	YES	NO	2
4	YES	NO	3
5	YES	NO	4
6	YES	NO	5
7	YES	NO	6
8	YES	NO	7
9	YES	NO	8
10	YES	NO	9
11	YES	NO	10

Example 3: Using User-Defined Formats and Packages

The following example uses user-defined formats and user-defined DS2 packages. In addition to the data and thread programs, the user-defined formats and the user-defined DS2 packages are published to the database.

```
options ds2accel=any;

libname db teradata user=XXXX password=XXXX
        server=terapin database=XXXX;

proc ds2;
data db.ipassdata / overwrite=yes;
  declare double  score;
  method init();
    declare int i;
    do i = 1 to 20;
      score = i * 5;
      output;
    end;
  end;
enddata;
run;
quit;

proc format;
  value lettergrade
```

```

        90-high = 'A'
        80-89  = 'B'
        70-79  = 'C'
        60-69  = 'D'
        low-59 = 'F';
run;

proc format;
    value passfail
        70-high = 'PASS'
        low-69  = 'FAIL';
run;

proc ds2;
package pkgGrade;
    method compute(double s) returns char(1);
        declare char(1) g;
        g = put(s, lettergrade.);
        return g;
    end;
endpackage;

package pkgPassFail;
    method compute(double s) returns char(4);
        declare char(4) g;
        g = put(s, passfail.);
        return g;
    end;
endpackage;

thread th_pgm;
    declare char(1) grade;
    declare char(4) pass;
    declare package pkgGrade g();
    declare package pkgPassFail pf();

    method run();
        set db.ipassdata;
        grade = g.compute(score);
        pass  = pf.compute(score);
    end;
endthread;

data outdata;
    dcl thread th_pgm m;
    method run();
        /* The THREADS= argument in the SET FROM statement has no effect */
        /* if the SAS In-Database Code Accelerator is used to access a */
        /* database table. */
        set from m threads=1;
    end;
enddata;
run;
quit;

proc print data=outdata; quit;

```

The following output table is produced.

Output 14.2 Result Table for Example 3 (Partial Output)

The SAS System		
grade	pass	score
F	FAIL	5
F	FAIL	10
F	FAIL	15
F	FAIL	20
F	FAIL	25
F	FAIL	30
F	FAIL	35
F	FAIL	40
F	FAIL	45
F	FAIL	50
F	FAIL	55
D	FAIL	60
D	FAIL	65
C	PASS	70
C	PASS	75

Example 4: BY-Group Processing

The following example transposes customer data that has multiple records for each customer into one wide record for each customer. The SAS In-Database Code Accelerator for Teradata redistributes the input data `pivot_1m` by the first BY variable `Cust_Name`. All the rows with the same `Cust_Name` are on the same data partition and transposed by one thread.

```
options ds2accel=any;

%let nobs=1000000;
libname td teradata server=terapin user=xxxxxx password=xxxxxx database=xxxxxx;

proc delete data=td.pivot_1m; run;
data td.pivot_1m (tpt=no fastload=yes dbcommit=100000);
  drop i;
  length Cust_Name $20;
  do i = 1 to &nobs;
```



```

        month_id = floor(rand('Uniform')*12)+1;
        month_visits = floor(rand('Uniform')*1000)+1;
        month_amount = (floor(rand('Uniform')*1000000)+1)/100;
        Cust_Name = "Name"||strip(mod(i,1000));
        output;
    end;
run;

%let inputdata=td.pivot_1m;

proc ds2;
    thread work.p_thread / overwrite=yes;
        dcl double i;
        vararray double amount[12];
        vararray double num_visits[12];
        keep Cust_Name amount1-amount12 num_visits1-num_visits12;
        retain amount1-amount12 num_visits1-num_visits12;
        method clear_array();
            do i=1 to 12 ;
                amount[i] = 0;
                num_visits[i] = 0;
            end;
        end;
        method run();
            set &inputdata;
            by Cust_Name;
            if first.Cust_Name then
                clear_array();
                amount[month_id] = month_amount + amount[month_id];
                num_visits[month_id] = month_visits + num_visits[month_id];
            if last.Cust_Name then
                output;
            end;
        endthread;
run;

data td.pivot_results (overwrite=yes);
    dcl thread p_thread p;
    method run();
        set from p;
        output;
    end;
enddata;
run;
quit;

```

The following output table is produced (partial output).

Output 14.3 Result Table for Example 4 (Partial Output)

amount1	amount2	amount3	amount4	amount5	amount6
0.00	0.00	0.00	3942.11	0.00	0.00
0.00	7112.52	0.00	0.00	0.00	0.00
0.00	0.00	0.00	8531.73	0.00	0.00
0.00	0.00	0.00	0.00	0.00	4189.85
628.50	0.00	0.00	0.00	0.00	0.00
0.00	5436.21	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00	0.00

Using DS2 and FedSQL

Dynamically Executing FedSQL Statements from DS2 223

Dynamically Executing FedSQL Statements from DS2

Note: With the exception of the SET statement in a DS2 action or PROC DS2, using a FedSQL query within your DS2 program is not currently supported on the CAS server.

You can embed and execute FedSQL statements from within your DS2 programs. You can use FedSQL with DS2 in the following instances:

- You can invoke a DS2 package method expression as a function in a FedSQL SELECT statement.

For more information, see [“Using DS2 Packages in Expressions” in SAS FedSQL Language Reference](#).

- You can use the SQLSTMT package to generate, prepare, and execute FedSQL statements to create, select, modify, insert, or delete rows from a table at run time.

The SQLSTMT package is intended for use with FedSQL statements that are executed multiple times, statements with parameters, or statements that generate a result set. For more information, see [“Using the SQLSTMT Package” on page 185](#).

- You can also use the SQLEXEC function to generate, prepare, and execute FedSQL statements to create, select, modify, insert, or delete rows from a table at run time.

The SQLEXEC function is intended for use with FedSQL statements that are executed only one time, do not have parameters, and do not produce a result set.

For more information, see the [“SQLEXEC Function” in SAS DS2 Language Reference](#).

- You can load data into a hash instance at run time by using a FedSQL SELECT statement in the DECLARE PACKAGE statement or the DATASET method.

For more information, see [“Using a FedSQL Query with a Hash Instance to Get Rows Dynamically at Run Time” on page 159](#) the [“DATASET Method” in SAS DS2 Language Reference](#) and the [“DECLARE PACKAGE Statement: Hash Package” in SAS DS2 Language Reference](#).

- You can use the SET statement to read in data by using a FedSQL SELECT statement.

For more information, see [“SET Statement with Embedded FedSQL” on page 229](#) and the [“SET Statement” in SAS DS2 Language Reference](#).

DS2 Input and Output

<i>Overview of DS2 Input and Output</i>	225
<i>Reading Data Using the SET Statement</i>	226
Overview of the SET Statement	226
SET Statement Compilation	227
SET Statement Execution	227
SET Statement with Embedded FedSQL	229
<i>Reading Data Using the Hash Package</i>	230
<i>Reading and Writing Data Using the SQLSTMT Package and the SQLEXEC Function</i> ..	231
<i>Writing Data Using the OUTPUT Statement</i>	231
<i>Column Order in Output Tables When Using Data Sources Outside SAS</i>	232
<i>NLS Transcoding Failures</i>	232

Overview of DS2 Input and Output

You can use these methods to read in DS2 data:

- You can generate data using variable or array assignment within packages and methods in your DS2 program.
- You can read an existing table by using the SET statement.
For more information, see [“Reading Data Using the SET Statement”](#) on page 226 and the [“SET Statement”](#) in *SAS DS2 Language Reference*.
- You can read data using a hash package.
For more information, see [“Reading Data Using the Hash Package”](#) on page 230.
- You can retrieve data using an SQLSTMT package by executing a SELECT statement and accessing the result set.

Note: The SQLSTMT package is not supported on the CAS server.

For more information, see [“Reading and Writing Data Using the SQLSTMT Package and the SQLEXEC Function” on page 231](#).

You can use these methods to write DS2 data:

- You write DS2 data by using the OUTPUT statement. The OUTPUT statement writes rows to a result table.

For more information, see [“Writing Data Using the OUTPUT Statement” on page 231](#).

- You can use the hash package OUTPUT method.

For more information, see [“OUTPUT Method” in SAS DS2 Language Reference](#) and [“Saving Hash Package Data in a Table” on page 157](#).

- You can use FedSQL INSERT and UPDATE statements in the SQLEXEC function or the SQLSTMT package.

Note: The SQLEXEC function and the SQLSTMT package are not supported on the CAS server.

[“Reading and Writing Data Using the SQLSTMT Package and the SQLEXEC Function” on page 231](#), [“SQLEXEC Function” in SAS DS2 Language Reference](#), and [“Accessing Result Set Data” on page 189](#).

Note: The MongoDB data source has special requirements for creating tables. Depending on your data, you might want to create these tables: a root table, a parent table, and a child table. You can create only root tables with DS2. To create parent and child tables, you must use FedSQL. See the information about MongoDB in [SAS/ACCESS for Relational Databases: Reference](#).

Reading Data Using the SET Statement

Overview of the SET Statement

The SET statement is flexible and has a variety of uses in DS2 programming. These uses are determined by the options and statements that you use with the SET statement:

- reading rows and columns from existing tables for further processing in a DS2 program
- concatenating and interleaving tables, and performing one-to-one reading of tables

For more information, see [Chapter 17, “Combining Tables,” on page 235](#).

Each time the SET statement executes, one row is read into the program data vector. SET reads all columns and all rows from the input tables unless you specify otherwise. A SET statement can contain multiple tables; a DS2 program can contain multiple SET statements.

For more information, see [“SET Statement” in SAS DS2 Language Reference](#).

Note: A SET statement in a thread program shares a single reader for that SET statement. Each time a SET statement is executed, one row is read from the named input table. However, all threads share a single reader. Each row in the input table is sent to exactly one thread. If you use a SET statement in the INIT method, the first thread reads all the rows. The other threads reach end-of file, and processing is terminated without their advancing to their associated RUN methods. Therefore, it is recommended that you not use the SET statement in the INIT or TERM method of a thread program.

Note: The SET statement is best used in the RUN method to take advantage of the RUN method's implicit looping capability.

SET Statement Compilation

When the DS2 compiler evaluates a SET statement, it reads the column information for each *table-reference*. For each column in each table, it creates a global variable in the DS2 program with the same type attributes as those of the column. In this way, the SET statement creates a set of associated column variables. An error occurs if a global variable already exists and the type does not match the type of the table column. This means that if two tables have a column with the same name, those columns' types must be compatible.

SET Statement Execution

In a data program, when a SET statement executes, it reads the first row from the first table. Successive executions continue to read rows until the current table has been completely read. Then the first row from the next table is read. Each time a row is read, the row values are assigned to the corresponding column variables. Column variables that are created by the SET statement are retained. SET statement execution ends when all rows from all tables have been completely read.

For a thread program, the order of rows entering any given thread is undefined unless you use a BY statement. Even then, there is no way in a thread program for a thread to assume that all the rows are received before any rows from the other table or tables.

Any column variable that does not appear in the table being read is set to a SAS missing value or a null value depending on whether you are in SAS or ANSI mode. Variables that are declared by a SET statement are initialized as follows:

- In SAS mode:
 - DOUBLE data types are initialized to the SAS numeric missing value (.).
 - Fixed-width CHAR and NCHAR data types are initialized to the SAS character missing value (a blank filled string).
 - All other data types, including VARCHAR and NVARCHAR, are initialized to ANSI null.
- In ANSI mode all types are initialized to ANSI null.

Note: For more information, see [Chapter 7, “How DS2 Processes Nulls and SAS Missing Values,”](#) on page 61.

Here is an example. In this program, the column variable *modifiers* is not specified in the second and fourth rows. The OUTPUT method uses the values of all defined variables whether they have been assigned a value after the last OUTPUT statement.

```
proc ds2;
data test (overwrite=yes);
  dcl char string1 string2 modifiers having informat $char8. format $char8.;
  method init();
    string1='aBc'; string2='AbC'; modifiers='i'; output;
    string1='  abc'; string2='abc'; output;
    string1='  abc'; string2='abc'; modifiers='l'; output;
    string1=' abc'; string2='  abx'; output;
    string1=' abc'; string2='  abx'; modifiers='l'; output;
  end;
enddata;
run;

data test_out (overwrite=yes);
  dcl double result;
  method run();
    set test;
    result=compare(string1, string2, modifiers);
    put 'String 1= ' string1 'String 2= ' string2 'Modifier= ' modifiers
        'Result= ' result;
  end;
enddata;
run;
quit;
```

The following lines are written to the SAS log.

String 1=	aBc	String 2=	AbC	Modifier=	i	Result=	0
String 1=	abc	String 2=	abc	Modifier=	i	Result=	-1
String 1=	abc	String 2=	abc	Modifier=	l	Result=	0
String 1=	abc	String 2=	abx	Modifier=	l	Result=	-3
String 1=	abc	String 2=	abx	Modifier=	l	Result=	-3

To ensure that the second and fourth rows do not have a value that is specified for the *modifiers* column variable, you must set the variable to a missing or null value.

```
proc ds2;
data test (overwrite=yes);
  dcl char string1 string2 modifiers having informat $char8. format $char8.;
  method init();
    string1='aBc'; string2='AbC'; modifiers='i'; output;
    string1='  abc'; string2='abc'; modifiers=' '; output;
    string1='  abc'; string2='abc'; modifiers='l'; output;
    string1=' abc'; string2='  abx'; modifiers=' '; output;
    string1=' abc'; string2='  abx'; modifiers='l'; output;
  end;
enddata;
run;

data test_out (overwrite=yes);
  method run();
    set test;
    result=compare(string1, string2, modifiers);
    put 'String 1= ' string1 'String 2= ' string2 'Modifier= ' modifiers
        'Result= ' result;
  end;
enddata;
run;
quit;
```

The following lines are written to the SAS log.

String 1=	aBc	String 2=	AbC	Modifier=	i	Result=	0
String 1=	abc	String 2=	abc	Modifier=		Result=	-1
String 1=	abc	String 2=	abc	Modifier=	l	Result=	0
String 1=	abc	String 2=	abx	Modifier=		Result=	2
String 1=	abc	String 2=	abx	Modifier=	l	Result=	-3

SET Statement with Embedded FedSQL

A SET statement can use FedSQL code to read a table, as in this example:

```
set {select * from catalog_base.investment};
```

It is possible to interleave table names and embedded FedSQL in a SET statement. In this example, the SET statement reads the same table twice:

```
set {select * from catalog_base.investment} catalog_base.investment;
```

Embedded FedSQL used in a SET statement must be valid FedSQL code, and it must resolve to a set of table rows. Otherwise, an error occurs.

Note: There is no guarantee on the order or rows that is surfaced from embedded SQL, regardless of what that embedded SQL contains:

```
set {select * from sql13a order by "X", "Y"};
```

Some environments might preserve the order imposed by the ORDER BY clause, but others do not. DS2 wraps the embedded SQL with additional code. Also, the ORDER BY clause is not honored because DS2 cannot detect and produce FIRST or LAST information. A better way is to write the program with the BY outside of the embedded SQL text:

```
set {select * from sql13a}; by "X" "Y";
```

In the SAS In-Database Code Accelerator and in the CAS server, the ORDER BY ordering is not preserved in either the data program or the thread program. This happens because the results of the embedded SQL are saved to a temporary table by the FedSQL action. The ORDER BY ordering is applied by the FedSQL action before the temporary table is created. No ordering is implied when the temporary table is then read in by the DS2 action.

Reading Data Using the Hash Package

You can use a hash package to read data from a table.

The DECLARE PACKAGE statement, the _NEW_ operator, and the DATASET method accept either of these methods to identify a data source:

- a name that identifies a table
- a valid FedSQL SELECT statement that resolves to a set of table rows

Note: Using a FedSQL query to select the data is not supported on the CAS server.

Note: You cannot load data from a DS2 hash table into an FCMP package.

For more information, see [“Using the Hash Package” on page 148](#).

Reading and Writing Data Using the SQLSTMT Package and the SQLEXEC Function

Note: The SQLSTMT package and the SQLEXEC function are not supported on the CAS server.

The SQLSTMT package and the SQLEXEC function enable DS2 programs to dynamically generate, prepare, and execute FedSQL statements to update, insert, or delete rows from a table. With an instance of the SQLSTMT package or the SQLEXEC function, the FedSQL statement allocate, prepare, execute, and free occurs at run time.

In addition, the SQLSTMT package can interrogate the result set that is produced by the executed FedSQL statement.

CAUTION

Only FedSQL statements can be used with the SQLEXEC function and the SQLSTMT package. DBMS-specific SQL cannot be used. For more information, see [SAS FedSQL Language Reference](#).

For more information, see “Using the SQLSTMT Package” on page 185 and the “SQLEXEC Function” in [SAS DS2 Language Reference](#).

TIP By using a parameterized FedSQL query in the SQLSTMT package, you can achieve what the DATA step SET statement with KEY= does.

Writing Data Using the OUTPUT Statement

The OUTPUT statement creates an output row, using values for the row that are contained in the global variables when the output statement executes. The OUTPUT statement writes the current row to a table immediately, not at the end of

the DS2 program. If no table name is specified in the OUTPUT statement, the row is written to the table or tables that are listed in the DATA statement.

DS2 keeps track of the values in the order in which the compiler encounters them within a DS2 program, whether they are read from existing tables or created in the program.

If you do not supply an OUTPUT statement, DS2 adds one implicitly at the end of the RUN method that writes rows to the table or tables that are being created.

Placing an explicit OUTPUT statement in a DS2 program overrides the automatic output, and adds a row to a table only when an explicit OUTPUT statement is executed. Once you use an OUTPUT statement to write a row to any one table, however, there is no implicit OUTPUT statement at the end of the RUN method. In this situation, a DS2 program writes a row to a table only when an explicit OUTPUT executes. You can use the OUTPUT statement alone or as part of an IF-THEN/ELSE or SELECT statement or in DO loop processing.

Note: OUTPUT statements in thread programs cannot contain any table names. Each output row is returned to the data program that started the thread.

Column Order in Output Tables When Using Data Sources Outside SAS

Some data sources choose their one preferred order for columns in the output table from DS2. For example, on Hive, the BYPARTITION columns are always moved to the end of the table. This is common as various data sources try to optimize their performance.

The order of declaration in a DS2 program might not be used as the order of columns in the data source. For example, if you use `keep K1- - K4;`, you might not get the columns as you expect or you might get an error because `K1` appears after `K4` in the CREATE TABLE statement.

NLS Transcoding Failures

Transcoding is the process of converting character data from one encoding to another encoding. An NLS transcoding failure can occur during row input or output operations, or during string assignment. By default, this run-time error causes row processing to halt. You can change the default behavior by using one of the following options:

- SAS Federation Server: specify the DEFAULT_ATTR= connection option with the XCODE_WARN=n statement handle option.
- PROC FEDSQL and PROC DS2: set the XCODE= option.

Using the options, you can choose to ignore the errors and continue processing of the row.

For more information, see the SAS Federation Server and SAS procedure documentation.

Combining Tables

<i>Definitions for Combining Data</i>	235
<i>Combining Tables: Basic Concepts</i>	236
What You Need to Know Before Combining Information Stored in Multiple Tables	236
The Four Ways That Data Can Be Related	236
Overview of Methods for Combining Tables	239
How to Prepare Your Tables	243
<i>Combining DS2 Tables: Methods</i>	246
Concatenating	246
Interleaving	249
One-to-One Reading	256
Match-Merging	260

Definitions for Combining Data

In the context of DS2 processing, combining data has these meanings:

- concatenating
- interleaving
- one-to-one reading
- match-merging

The two statements that are used for combining tables are MERGE and SET.

Combining Tables: Basic Concepts

What You Need to Know Before Combining Information Stored in Multiple Tables

Many applications require input data to be in a specific format before the data can be processed to produce meaningful results. The data typically comes from multiple sources and might be in different formats. Therefore, you often, if not always, have to take intermediate steps to logically relate and process data before you can analyze it or create reports from it.

Application requirements vary, but there are common factors for all applications that access, combine, and process data. Once you have determined what you want the output to look like, you must perform the following tasks:

- Determine how the input data is related.
- Ensure that the data is properly sorted, if necessary.
- Select the appropriate access method to process the input data.
- Select the appropriate tools to complete the task.

The Four Ways That Data Can Be Related

Data Relationship Categories

Relationships among multiple sources of input data exist when each of the sources contains common data, either at the physical or logical level. For example, employee data and department data could be related through an employee ID column that shares common values. Another table could contain numeric sequence numbers whose partial values logically relate it to a separate table by row number.

You must be able to identify the existing relationships in your data. This knowledge is crucial for understanding how to process input data in order to produce desired results. All related data falls into one of these four categories, characterized by how rows relate among the tables:

- one-to-one
- one-to-many

- many-to-one
- many-to-many

To obtain the results that you want, you should understand how each of these methods combines rows, how each method treats duplicate values of common columns, and how each method treats missing values or nonmatched values of common columns. Some of the methods also require that you preprocess your tables by sorting them. See the description of each method in [“Methods for Combining Tables” on page 239](#).

One-to-One Relationship

In a one-to-one relationship, typically a single row in one table is related to a single row from another based on the values of one or more selected columns. A one-to-one relationship implies that each value of the selected column occurs no more than once in each table. When you work with multiple selected columns, this relationship implies that each combination of values occurs no more than once in each table.

In the following example, rows in tables Salary and Taxes are related by common values for EmployeeNumber.

Figure 17.1 One-to-One Relationship

SALARY		TAXES	
EmployeeNumber	Salary	EmployeeNumber	TaxBracket
1234	55000	1111	0.18
3333	72000	1234	0.28
4876	32000	3333	0.32
5489	17000	4222	0.18
		4876	0.24

One-to-Many and Many-to-One Relationships

A one-to-many or many-to-one relationship between input tables implies that one table has at most one row with a specific value of the selected column, but the other input table can have more than one occurrence of each value. When you work with multiple selected columns, this relationship implies that each combination of values occurs no more than once in one table. However, the combination can occur more than once in the other table. The order in which the input tables are processed determines whether the relationship is one-to-many or many-to-one.

In the following example, rows in tables One and Two are related by common values for column A. Values of A are unique in table One but not in table Two.

Figure 17.2 One-to-Many Relationship

ONE			TWO		
A	B	C	A	E	F
1	5	6	1	2	0
3	3	4	1	3	99
			1	4	88
			1	5	77
			2	1	66
			2	2	55
			3	4	44

In the following example, rows in tables One, Two, and Three are related by common values for column ID. Values of ID are unique in tables One and Three but not in Two. For values 2 and 3 of ID, a one-to-many relationship exists between rows in tables One and Two, and a many-to-one relationship exists between rows in tables Two and Three.

Figure 17.3 One-to-Many and Many-to-One Relationships

ONE		TWO		THREE	
ID	Name	ID	Sales	ID	Quota
1	Joe Smith	1	28000	1	15000
2	Sally Smith	2	30000	2	7000
3	Cindy Long	2	40000	3	15000
4	Sue Brown	3	15000	4	5000
5	Mike Jones	3	20000	5	8000
		3	25000		
		4	35000		
		5	40000		

Many-to-Many Relationships

The many-to-many category implies that multiple rows from each input table can be related based on values of one or more common columns.

In the following example, rows in tables BreakDown and Maintenance are related by common values for column Vehicle. Values of Vehicle are not unique in either table. A many-to-many relationship exists between rows in these tables for values AAA and CCC of Vehicle.

Figure 17.4 Many-to-Many Relationship

BREAKDOWN		MAINTENANCE	
Vehicle	BreakDownDate	Vehicle	MaintenanceDate
AAA	02MAR99	AAA	03JAN99
AAA	20MAY99	AAA	05APR99
AAA	19JUN99	AAA	10AUG99
AAA	29NOV99	CCC	28JAN99
BBB	04JUL99	CCC	16MAY99
CCC	31MAY99	CCC	07OCT99
CCC	24DEC99	DDD	24FEB99
		DDD	22JUN99
		DDD	19SEP99

Overview of Methods for Combining Tables

Methods for Combining Tables

You can use these methods to combine tables:

- concatenating
- interleaving
- one-to-one reading
- match-merging

Concatenating

The following figure shows the results of concatenating two tables. Concatenating the tables appends the rows from one table to another table. The data program reads Data1 sequentially until all rows have been processed, and then reads Data2. Table Combined contains the results of the concatenation. Note that the tables are processed in the order in which they are listed in the SET statement.

```
data concatenate;
  method run();
    set data1 data2;
  end;
enddata;
run;
```

Note: For a thread program, the order of rows entering any given thread is undefined unless you use a BY statement. Even then, there is no way in a thread program for a thread to assume that all the rows are received before any rows from the other table or tables.

Figure 17.5 Concatenating Two Tables

Data 1		Data 2		Combined																																		
<table><tr><th>year</th></tr><tr><td>1991</td></tr><tr><td>1992</td></tr><tr><td>1993</td></tr><tr><td>1994</td></tr><tr><td>1995</td></tr></table>	year	1991	1992	1993	1994	1995	+	<table><tr><th>year</th></tr><tr><td>1991</td></tr><tr><td>1992</td></tr><tr><td>1993</td></tr><tr><td>1994</td></tr><tr><td>1995</td></tr></table>	year	1991	1992	1993	1994	1995	=	<table><tr><th>data1</th><th>data2</th></tr><tr><td>1991</td><td></td></tr><tr><td>1992</td><td></td></tr><tr><td>1993</td><td></td></tr><tr><td>1994</td><td></td></tr><tr><td>1995</td><td></td></tr><tr><td></td><td>1991</td></tr><tr><td></td><td>1992</td></tr><tr><td></td><td>1993</td></tr><tr><td></td><td>1994</td></tr><tr><td></td><td>1995</td></tr></table>	data1	data2	1991		1992		1993		1994		1995			1991		1992		1993		1994		1995
year																																						
1991																																						
1992																																						
1993																																						
1994																																						
1995																																						
year																																						
1991																																						
1992																																						
1993																																						
1994																																						
1995																																						
data1	data2																																					
1991																																						
1992																																						
1993																																						
1994																																						
1995																																						
	1991																																					
	1992																																					
	1993																																					
	1994																																					
	1995																																					

For more information, see [“Combining DS2 Tables: Methods”](#) on page 246.

Interleaving

The following data program interleaves two tables. Interleaving intersperses rows from two or more tables, based on one or more common columns. Table Combined shows the results.

```
data interleave;
  method run();
    set data1 data2;
    by year;
  end;
enddata;
run;
```

Figure 17.6 Interleaving Two Tables

Data 1		Data 2		Combined																							
<table><tr><th>year</th></tr><tr><td>1991</td></tr><tr><td>1992</td></tr><tr><td>1993</td></tr><tr><td>1994</td></tr><tr><td>1995</td></tr></table>	year	1991	1992	1993	1994	1995	+	<table><tr><th>year</th></tr><tr><td>1992</td></tr><tr><td>1993</td></tr><tr><td>1994</td></tr><tr><td>1995</td></tr><tr><td>1996</td></tr></table>	year	1992	1993	1994	1995	1996	=	<table><tr><th>year</th></tr><tr><td>1991</td></tr><tr><td>1992</td></tr><tr><td>1992</td></tr><tr><td>1993</td></tr><tr><td>1993</td></tr><tr><td>1994</td></tr><tr><td>1994</td></tr><tr><td>1995</td></tr><tr><td>1995</td></tr><tr><td>1996</td></tr></table>	year	1991	1992	1992	1993	1993	1994	1994	1995	1995	1996
year																											
1991																											
1992																											
1993																											
1994																											
1995																											
year																											
1992																											
1993																											
1994																											
1995																											
1996																											
year																											
1991																											
1992																											
1992																											
1993																											
1993																											
1994																											
1994																											
1995																											
1995																											
1996																											

For more information, see [“Combining DS2 Tables: Methods”](#) on page 246.

One-to-One Reading

The following figure shows the results of one-to-one reading. One-to-one reading combines rows from two or more tables by creating rows that contain all of the columns from each contributing table. Rows are combined based on their relative position in each table, that is, the first row in one table with the first in the other, and so on. The data program stops after it has read the last row from the smallest table. Table Combined shows the results.

```
data one2one;
  method run();
    set data1;
    set data2;
  end;
enddata;
run;
```

The following data program results in one-to-one reading of two tables.

Figure 17.7 One-to-One Reading

Data 1		Data 2		Combined																								
<table><tr><th>VarX</th></tr><tr><td>X1</td></tr><tr><td>X2</td></tr><tr><td>X3</td></tr><tr><td>X4</td></tr><tr><td>X5</td></tr></table>	VarX	X1	X2	X3	X4	X5	+	<table><tr><th>VarY</th></tr><tr><td>Y1</td></tr><tr><td>Y2</td></tr><tr><td>Y3</td></tr><tr><td>Y4</td></tr><tr><td>Y5</td></tr></table>	VarY	Y1	Y2	Y3	Y4	Y5	=	<table><tr><th>VarX</th><th>VarY</th></tr><tr><td>X1</td><td>Y1</td></tr><tr><td>X2</td><td>Y2</td></tr><tr><td>X3</td><td>Y3</td></tr><tr><td>X4</td><td>Y4</td></tr><tr><td>X5</td><td>Y5</td></tr></table>	VarX	VarY	X1	Y1	X2	Y2	X3	Y3	X4	Y4	X5	Y5
VarX																												
X1																												
X2																												
X3																												
X4																												
X5																												
VarY																												
Y1																												
Y2																												
Y3																												
Y4																												
Y5																												
VarX	VarY																											
X1	Y1																											
X2	Y2																											
X3	Y3																											
X4	Y4																											
X5	Y5																											

For more information, see [“Combining DS2 Tables: Methods”](#) on page 246.

Match-Merging

The following figure shows the results of match-merging. Match-merging combines rows from two or more tables into a single row in a new table based on the values of one or more common columns. The following data program results in one-to-one reading of two tables. Table Combined shows the results.

```
data one2one;
  method run();
    merge data1 data2;
    by year;
  end;
enddata;
run;
```

Figure 17.8 Match-Merging Two Tables

Data 1		Data 2		Combined		
Year	VarX	Year	VarY	Year	VarX	VarY
1991	X1	1991	Y1	1991	X1	Y2
1992	X2	1991	Y2	1991		Y1
1993	X3	1993	Y3	1992	X2	
1994	X4	1994	Y4	1993	X3	Y3
1995	X5	1995	Y5	1994	X4	Y4
				1995	X5	Y5

For more information, see [“Combining DS2 Tables: Methods”](#) on page 246.

How to Prepare Your Tables

Guidelines to Prepare Your Tables

Before combining tables, follow these guidelines to produce the results that you want:

- Know the structure and the contents of the tables.
- Look at sources of common problems.
- Ensure that rows are in the correct order, or that they can be retrieved in the correct order (for example, by presorting them).
- Test your program.

Knowing the Structure and Contents of the Tables

To help determine how your data is related, look at the structure of the tables. To see the table structure, execute the DATASETS procedure, the CONTENTS procedure, or access the SAS Explorer window in your windowing environment to display the descriptor information. Descriptor information includes the number of rows in each table, the name and attributes of each column, and an alphabetic list of extended attributes (including table and column extended attributes). To print a sample of the rows, use the PRINT procedure or the REPORT procedure.

You can also use functions such as VTYPE and VLENGTH to show specific descriptor information. For more information, see [“DS2 Functions” in SAS DS2 Language Reference](#).

Looking at Sources of Common Problems

If your program does not execute correctly, review your input data for the following errors:

- columns that have the same name but that represent different data

DS2 includes only one column of a given name in the new table. If you are merging two tables that have columns with the same names but different data, the values from the last table that was read are written over the values from other tables.

To correct the error, you can rename columns before you combine the tables by using the RENAME= table option in the SET or MERGE statement. Or you can use the DATASETS procedure.

- common columns with the same data but different attributes

The way DS2 handles these differences depends on which attributes are different:

- type attribute

If the type attributes are incompatible, DS2 stops processing the data program and issues an error message stating that the columns are incompatible.

To correct this error, you must use a separate DS2 data program to change the types as necessary. The DS2 statements that you use depend on the nature of the column. The following table contains the result attribute when combining columns.

Table 17.1 *Type Promotion*

Left-hand side	Right-hand side	Result
BIGINT	BIGINT	BIGINT
BIGINT	INTEGER	BIGINT
BIGINT	SMALLINT	BIGINT
BIGINT	TINYINT	BIGINT
INTEGER	BIGINT	BIGINT
INTEGER	INTEGER	INTEGER
INTEGER	SMALLINT	INTEGER
INTEGER	TINYINT	INTEGER
SMALLINT	BIGINT	BIGINT
SMALLINT	INTEGER	INTEGER
SMALLINT	SMALLINT	SMALLINT
SMALLINT	TINYINT	INTEGER
TINYINT	BIGINT	BIGINT
TINYINT	INTEGER	INTEGER
TINYINT	SMALLINT	INTEGER
TINYINT	TINYINT	TINYINT
DOUBLE	DOUBLE	DOUBLE
DOUBLE	FLOAT	DOUBLE
INTEGER	DECIMAL	DOUBLE
REAL	REAL	DOUBLE
BIGINT	DOUBLE	DOUBLE
DECIMAL (X,Y)	DECIMAL (X,Z)	DOUBLE

Left-hand side	Right-hand side	Result
TIME ¹	TIME ¹	TIME ¹
TIMESTAMP	TIMESTAMP	TIMESTAMP
TIMESTAMP	DATE	ERROR
DATE	DATE	DATE
VARBINARY ¹	VARBINARY	VARBINARY
BINARY	BINARY	BINARY
BINARY	VARBINARY	VARBINARY
CHAR(N)	CHAR(N)	CHAR(N)
CHAR(N) ²	CHAR(N) ²	VARCHAR(N) ²
VARCHAR(N)	VARCHAR(N)	VARCHAR(N)
VARCHAR(N)	VARCHAR(M)	VARCHAR(MAX (N, M))
CHAR(N)	CHAR(M)	VARCHAR (MAX (N, M))
VARCHAR(N)	CHAR(M)	VARCHAR (MAX (N, M))
NCHAR(M)	VARCHAR(M)	VARCHAR(M)
NCHAR(M)	NCHAR(M)	NCHAR(M)
INTEGER	CHARACTER	ERROR
DATE	DOUBLE	ERROR

¹ No Hive support

² For DB2, Impala, ODBC, and Oracle

Note: A best practice is to declare every variable in a thread program. By doing so, you can avoid type mismatches among data sources.

□ length attribute

If the length attribute is different, DS2 takes the length from the table that contains the column with the maximum length. In the following example, all tables that are listed in the MERGE statement contain the column Mileage. In Quarter1, the length of the column Mileage is four bytes; in Quarter2, it is eight bytes and in Quarter3 and Quarter4, it is six bytes. In the output table Yearly, the length of the column Mileage is eight bytes, which is the length derived from Quarter2.

```
data yearly;
  dcl char(4) quarter1 char(8) quarter2 char(6) quarter3 quarter4;
  method run();
    merge quarter1 quarter2 quarter3 quarter4;
    by Account;
  end;
enddata;
```

```
run;
```

- label, format, and informat attributes

If any of these attributes are different, DS2 takes the attribute from the first table that contains the column with that attribute. However, any label, format, or informat that you explicitly specify overrides a default. If all tables contain explicitly specified attributes, the one specified in the first table overrides the others.

You can also use functions such as VLABEL to show specific descriptor information. For more information, see [“DS2 Functions” in SAS DS2 Language Reference](#).

Testing Your Program

As a final step in preparing your tables, you should test your program. Create small temporary tables that contain a sample of rows that test all of your program's logic. If your logic is faulty and you get unexpected output, you can debug your program.

Combining DS2 Tables: Methods

Concatenating

Definition

Concatenating tables is the combining of two or more tables, one after the other, into a single table. The number of rows in the new table is the sum of the number of rows in the original tables. The order of row is sequential. All rows from the first table are followed by all rows from the second table, and so on.

In the simplest case, all input tables contain the same columns. If the input tables contain different columns, rows from one table have missing values for columns that are defined only in other tables. In either case, the columns in the new table are the same as the columns in the old tables.

Syntax

Use this form of the SET statement to concatenate tables:

SET *table(s)*;

Arguments*table(s)*

specifies any valid table name.

For more information, see [“SET Statement” in SAS DS2 Language Reference](#).

DS2 Processing during Concatenation

Compilation phase

DS2 reads the descriptor information of each table that is named in the SET statement and then creates a program data vector that contains all the columns from all tables as well as columns created by the data program.

Execution — Step 1

DS2 reads the first row from the first table into the program data vector. It processes the first row and executes other statements in the data program. It then writes the contents of the program data vector to the new table.

The SET statement does not reset the values in the program data vector to missing, except for columns whose value is calculated or assigned during the data program. Columns that are created by the data program are set to missing at the beginning of each iteration of the data program unless they are retained. Variables that are read from a table are not.

Execution — Step 2

In the data program, DS2 continues to read one row at a time from the first table until it finds an end-of-file indicator. The values of the columns in the program data vector are then set to missing, and the data program begins reading rows from the second table, and so on, until it reads all rows from all tables. For a thread program, the order of rows entering any given thread is undefined unless you use a BY statement. Even then, there is no way in a thread program for a thread to assume that all the rows are received before any rows from the other table or tables.

Example 1: Concatenation Using the Data Program

In this example, each table contains the columns Common and Number, and the rows are arranged in the order of the values of Common. Generally, you concatenate tables that have the same columns. In this case, each table also contains a unique column to show the effects of combining tables more clearly. The following program uses a SET statement to concatenate the tables and then prints the results:

```
data animal(overwrite=yes);
  dcl varchar(10) common animal number;
  method init();
    common='a'; animal='Ant'; number='5'; output;
    common='b'; animal='Bird'; number=''; output;
    common='c'; animal='Cat'; number='17'; output;
    common='d'; animal='Dog'; number='9'; output;
```

```

        common='e'; animal='Eagle'; number=''; output;
        common='f'; animal='Frog'; number='76'; output;
    end;
enddata;
run;

data plant(overwrite=yes);
    dcl varchar(10) common plant number;
    method init();
        common='g'; plant='Grape'; number='69'; output;
        common='h'; plant='Hazelnut'; number='55'; output;
        common='i'; plant='Indigo'; number=''; output;
        common='j'; plant='Jicama'; number='14'; output;
        common='k'; plant='Kale'; number='5'; output;
        common='l'; plant='Lentil'; number='77'; output;
    end;
enddata;
run;

/* set concatenates */
data concatenate (overwrite=yes);
    method run();
        set animal plant;
    end;
enddata;
run;

proc print data=concatenate;
run;
quit;

```

Output 17.1 Concatenated Table (SET Statement)

Animal			Plant			Concatenate			
common	animal	number	common	plant	number	common	animal	number	plant
a	Ant	5	g	Grape	69	a	Ant	5	
b	Bird		h	Hazelnut	55	b	Bird		
c	Cat	17	i	Indigo		c	Cat	17	
d	Dog	9	j	Jicama	14	d	Dog	9	
e	Eagle		k	Kale	5	e	Eagle		
f	Frog	76	l	Lentil	77	f	Frog	76	
						g		69	Grape
						h		55	Hazelnut
						i			Indigo
						j		14	Jicama
						k		5	Kale
						l		77	Lentil

The resulting table, Concatenate, has 12 rows, which is the sum of the rows from the combined tables. The program data vector contains all columns from all tables. The values of columns found in one table but not in another are set to missing.

Example 2: Concatenation Using SQL

You can also use the SQL language to concatenate tables. In this example, SQL reads each row in both tables and creates a new table named Combined. The following shows the YEAR1 and YEAR2 input tables:

The following SQL code creates and prints the table Combined.

```
proc sql;
  create table combined as
    select * from animal
      union all
    select * from plant;
  select * from combined;
quit;

proc print data=combined;
run;
quit;
```

The output is exactly the same as the output when using the SET statement.

Interleaving

Definition

Interleaving uses a SET statement and a BY statement to combine multiple tables into one new table. This is also known as BY-group processing. BY-group processing is a method of combining rows from one or more tables that are grouped or ordered by values of one or more common columns. When a BY statement is specified immediately after a SET statement, the SET statement interleaves the rows of the input tables in sorted order. The sort order or sort key is specified by the column names in the BY statement. The number of rows in the new table is the sum of the number of rows from the original tables.

Note: Assume you have a table with a column that has a character data type. If you change the column to be a numeric data type on input with a DECLARE statement, the sort order of the resulting column is not numeric. For example, assume the following character column (CHAR) *s* exists in a table: 9, 10, 500. If you declare *s* as a numeric column (DOUBLE) when you read the table with a SET and BY statement, the data is generated as output in alphanumeric order, that is, 10, 500, 9.

The SET statement orders the rows in alphanumeric order before the string is converted to the numeric data type.

The keyword DESCENDING can be used before the name of the column in the BY statement in order to sort that column in descending instead of ascending order.

Syntax

Use this form of the SET statement to interleave tables when you use a BY variable:

SET *table(s)*;

BY <DESCENDING>*column* <...<DESCENDING> *column*>;

Arguments

table

specifies a table name.

DESCENDING

specifies that the tables are sorted in descending order by the column that is specified. DESCENDING means largest to smallest for numeric columns, or reverse alphabetical for character columns.

column

specifies each column by which the table is sorted. These columns are referred to as BY variables for the current data program.

Sort Requirements

When a BY statement is used, internally DS2 requests the rows in sorted order. If the rows are already sorted, "re-sorting" of the data might be necessary.

Note: When the SAS In-Database Code Accelerator executes the DS2 programs, BY groups might not necessarily be in sorted order. BY-group order depends on whether the host environment supports sorting as opposed to simply hashing. The SAS In-Database Code Accelerator chooses the most efficient technique for clustering like values together without causing local re-sorting of potentially large data volumes. Note that the SAS In-Database Code Accelerator is not supported in SAS Viya or on the CAS server.

DS2 Processing during Interleaving

Compilation phase

- DS2 reads the descriptor information of each table that is named in the SET statement and then creates a program data vector that contains all the columns from all tables as well as columns created by the data program.
- In the DS2 program, SAS identifies the beginning and end of each BY group by creating two temporary variables for each BY column: `FIRST.variable` and `LAST.variable`. Their values indicate whether a row has the following characteristics:
 - the first one in a BY group
 - the last one in a BY group
 - neither the first nor the last one in a BY group
 - both first and last, as is the case when there is only one row in a BY group.

When a row is the first in a BY group, SAS sets the value of `FIRST.variable` to 1 for the column whose value changed, as well as for all of the columns that follow in the BY statement. For all other rows in the BY group, the value of `FIRST.variable` is 0. Likewise, if the row is the last in a BY group, SAS sets the value of `LAST.variable` to 1 for the column whose value changes on the next row, as well as for all of the columns that follow in the BY statement. For all other rows in the BY group, the value of `LAST.variable` is 0. For the last row in a table, the values of all `LAST.variable` variables are set to 1. These temporary variables are available for DS2 programming but are not added to the output table.

You can take actions conditionally, based on whether you are processing the first or the last row in a BY group.

Note: See “[Interleaving Tables](#)” in *SAS DS2 Language Reference* for an example that illustrates BY-group processing.

Note: For an SPD Engine data set, utility files are used for certain operations that need extra space. The BY statement requires a utility file and the SAS `UTILLOC=` system option allocates space for that utility file. For more information, see the SAS `UTILLOC=` system option in *SAS System Options: Reference*.

Execution — Step 1

DS2 compares the first row from each table that is named in the SET statement to determine which BY group should appear first in the new table. It reads all rows from the first BY group from the selected table. If this BY group appears in more than one table, it reads from the tables in the order in which they appear in the SET statement. The values of the columns in the program data vector are set to missing each time DS2 starts to read a new table and when the BY group changes.

Execution — Step 2

DS2 compares the next rows from each table to determine the next BY group and then starts reading rows from the selected table in the SET statement that

contains rows for this BY group. DS2 continues until it has read all rows from all tables.

Example 1: Interleaving in the Simplest Case

In this example, each table contains the BY variable Common, and the rows are arranged in order of the values of the BY variable. The following example creates the Animal and the Plant input tables.

```
data animal(overwrite=yes);
  dcl varchar(10) common animal number;
  method init();
    common='a'; animal='Ant'; number='5'; output;
    common='b'; animal='Bird'; number=''; output;
    common='c'; animal='Cat'; number='17'; output;
    common='d'; animal='Dog'; number='9'; output;
    common='e'; animal='Eagle'; number=''; output;
    common='f'; animal='Frog'; number='76'; output;
  end;
enddata;
run;
```

```
data plant(overwrite=yes);
  dcl varchar(10) common plant number;
  method init();
    common='a'; plant=''; number='69'; output;
    common='b'; plant='Bamboo'; number='55'; output;
    common='c'; plant='Cabbage'; number=''; output;
    common='d'; plant='Daffodil'; number='14'; output;
    common='e'; plant='Eucalyptus'; number='5'; output;
    common='f'; plant='Fig'; number='77'; output;
  end;
enddata;
run;
```

The following program uses SET and BY statements to interleave the tables, and prints the results:

```
/* set with by interleaves */
data interleave (overwrite=yes);
  method run();
    set animal plant; by common;
  end;
enddata;
run;
```

```
proc print data=interleave;
run;
quit;
```


Output 17.2 Interleaved Table (SET Statement)

Animal			Plant			Interleave			
common	animal	number	common	plant	number	common	animal	number	plant
a	Ant	5	a	Azalea	69	a	Ant	5	
b	Bird		b	Bamboo	55	a		69	Azalea
c	Cat	17	c	Cabbage		b	Bird		
d	Dog	9	d	Daffodil	14	b		55	Bamboo
e	Eagle		e	Eucalyptus	5	c	Cat	17	
f	Frog	76	f	Fig	77	c			Cabbage
						d	Dog	9	
						d		14	Daffodil
						e	Eagle		
						e		5	Eucalyptus
						f	Frog	76	
						f		77	Fig

The resulting table Interleave has 12 rows, which is the sum of the rows from the combined tables. The new table contains all columns from both tables. The value of columns found in one table but not in the other are set to missing, and the rows are arranged by the values of the BY variable.

Example 2: Interleaving with Duplicate Values of the BY Variable

If the tables contain duplicate values of the BY variables, the rows are written to the new table in the order in which they occur in the original tables. This example contains duplicate values of the BY variable Common. The following program creates the Animal and Plant input tables:

```
data animal(overwrite=yes);
  dcl varchar(10) common animal number;
  method init();
    common='a'; animal='Ant'; number='5'; output;
    common='a'; animal='Bird'; number=''; output;
    common='b'; animal='Cat'; number='17'; output;
    common='c'; animal='Dog'; number='9'; output;
    common='d'; animal='Eagle'; number=''; output;
    common='e'; animal='Frog'; number='76'; output;
  end;
enddata;
run;

data plant(overwrite=yes);
  dcl varchar(10) common plant number;
  method init();
```

```

common='a'; plant='Grape'; number='69'; output;
common='b'; plant='Hazelnut'; number='55'; output;
common='c'; plant='Indigo'; number=''; output;
common='c'; plant='Jicama'; number='14'; output;
common='d'; plant='Kale'; number='5'; output;
common='e'; plant='Lentil'; number='77'; output;
end;
enddata;
run;

```

The following program uses SET and BY statements to interleave the tables, and prints the results:

```

/* set with by interleaves */
data interleave (overwrite=yes);
  method run();
    set animal plant; by common;
  end;
enddata;
run;

proc print data=interleave;
run;
quit;

```

Output 17.3 Interleaved Table with Multiple BY Variables (SET Statement)

Obs	common	animal	number	plant
1	a	Bird		
2	a	Ant	5	
3	a		69	Grape
4	b	Cat	17	
5	b		55	Hazelnut
6	c	Dog	9	
7	c		14	Jicama
8	c			Indigo
9	d	Eagle		
10	d		5	Kale
11	e	Frog	76	
12	e		77	Lentil

The number of rows in the new table is the sum of the rows in all the tables. The rows are written to the new table in the order in which they occur in the original tables.

Example 3: Interleaving with Different BY Values in Each Table

The tables `Animal` and `Plant` both contain values that are present in one table but not in the other. The following program creates the `Animal` and the `Plant` input tables:

```
data animal(overwrite=yes);
  dcl varchar(10) common animal number;
  method init();
    common='a'; animal='Ant'; number='5'; output;
    common='c'; animal='Bird'; number=''; output;
    common='d'; animal='Cat'; number='17'; output;
    common='e'; animal='Dog'; number='9'; output;
  end;
enddata;
run;
```

```
data plant(overwrite=yes);
  dcl varchar(10) common plant number;
  method init();
    common='a'; plant='Grape'; number='69'; output;
    common='b'; plant='Hazelnut'; number='55'; output;
    common='c'; plant='Indigo'; number=''; output;
    common='d'; plant='Jicama'; number='14'; output;
    common='e'; plant='Kale'; number='5'; output;
    common='f'; plant='Lentil'; number='77'; output;
  end;
enddata;
run;
```

This program uses `SET` and `BY` statements to interleave these tables, and prints the results:

```
data interleave (overwrite=yes);
  method run();
    set animal plant; by common;
  end;
enddata;
run;
```

```
proc print data=interleave;
run;
quit;
```

Output 17.4 Interleaved Table with Different BY Variables (SET Statement)

Obs	common	animal	number	plant
1	a	Ant	5	
2	a		69	Grape
3	b		55	Hazelnut
4	c	Bird		
5	c			Indigo
6	d	Cat	17	
7	d		14	Jicama
8	e	Dog	9	
9	e		5	Kale
10	f		77	Lentil

The resulting table has ten rows arranged by the values of the BY variable.

Comments and Comparisons

- In other languages, the term merge is often used to mean interleave. DS2 reserves the term merge for the operation in which rows from two or more tables are combined into one row. The rows in interleaved tables are not combined; they are copied from the original tables in the order of the values of the BY variable.
- If one table has multiple rows with the same BY value, the DATA step preserves the order of those rows in the result.
- To use the data program, the input tables must be appropriately sorted. SQL does not require the input tables to be in order.

One-to-One Reading

Definition

One-to-one reading combines rows from two or more tables into one row by using two or more SET statements to read rows independently from each table. This process is also called one-to-one matching. The new table contains all the columns from all the input tables. The number of rows in the new table is the number of rows in the smallest original table. If the tables contain common columns, the

values that are read in from the last table replace the values that were read in from earlier tables.

Syntax

Use this form of the SET statement for one-to-one reading:

SET *table-1*;

SET *table-2*;

Arguments

table-1

specifies a table name. *table-1* is the first table that the data program reads.

table-2

specifies a table name. *table-2* is the second table that the data program reads.

CAUTION

Use care when you combine tables with multiple SET statements. Using multiple SET statements to combine rows can produce undesirable results. Test your program on representative samples of the tables before using this method to combine them.

For more information, see [“SET Statement” in SAS DS2 Language Reference](#).

DS2 Processing during a One-to-One Reading

Compilation phase

DS2 reads the descriptor information of each table named in the SET statement and then creates a program data vector that contains all the columns from all tables as well as columns created by the data program.

Execution — Step 1

When DS2 executes the first SET statement, DS2 reads the first row from the first table into the program data vector. The second SET statement reads the first row from the second table into the program data vector. If both tables contain the same columns, the values from the second table replace the values from the first table, even if the value is missing. After reading the first row from the last table and executing any other statements in the data program, DS2 writes the contents of the program data vector to the new table. The SET statement does not reset the values in the program data vector to missing, except for those columns that were created or assigned values during the data program.

Execution — Step 2

DS2 continues reading from one table and then the other until it detects an end-of-file indicator in one of the tables. DS2 stops processing with the last row of the shortest table and does not read the remaining rows from the longer table.

Example 1: One-to-One Reading: Processing an Equal Number of Rows

The tables `Animal` and `Plant` both contain the column `Common`, and are arranged by the values of that column. The following program creates the `Animal` and the `Plant` input tables:

```
data animal(overwrite=yes);
  dcl varchar(10) common animal number;
  method init();
    common='a'; animal='Ant';  output;
    common='b'; animal='Bird'; output;
    common='c'; animal='Cat';  output;
    common='d'; animal='Dog';  output;
    common='e'; animal='Eagle'; output;
    common='f'; animal='Frog';  output;
  end;
enddata;
run;
```

```
data plant(overwrite=yes);
  dcl varchar(10) common plant number;
  method init();
    common='a'; plant='Grape'; output;
    common='b'; plant='Hazelnut'; output;
    common='c'; plant='Indigo';  output;
    common='d'; plant='Jicama';  output;
    common='e'; plant='Kale';    output;
    common='g'; plant='Lentil';  output;
  end;
enddata;
run;
```

The following program uses two `SET` statements to combine rows from `Animal` and `Plant`, and prints the results:

```
data one2one (overwrite=yes);
  method run();
    set animal;
    set plant;
  end;
enddata;
run;

proc print data=one2one;
run;
quit;
```

Output 17.5 One-to-One Reading with an Equal Number of Rows (SET Statement)

Obs	common	animal	plant
1	a	Ant	Grape
2	b	Bird	Hazelnut
3	c	Cat	Indigo
4	d	Dog	Jicama
5	e	Eagle	Kale
6	g	Frog	Lentil

Each row in the new table contains all the columns from all the tables. Note, however, that the Common column value in row 6 contains a “g.” The value of Common in row 6 of the Animal table was overwritten by the value in Plant. Plant was the table that DS2 read last.

One-to-One Reading with an Uneven Number of Rows

The tables Animal and Plant both contain the column Common, and are arranged by the values of that column. The tables have different number of rows. The following program creates the Animal and the Plant input tables:

```
data animal(overwrite=yes);
  dcl varchar(10) common animal;
  method init();
    common='a'; animal='Ant'; output;
    common='a'; animal='Bird'; output;
    common='b'; animal='Cat'; output;
    common='c'; animal='Dog'; output;
    common='d'; animal='Eagle'; output;
    common='e'; animal='Frog'; output;
  end;
enddata;
run;

data plant(overwrite=yes);
  dcl varchar(10) common plant;
  method init();
    common='a'; plant='Grape';output;
    common='b'; plant='Hazelnut'; output;
    common='c'; plant='Indigo'; output;
    common='d'; plant='Jicama'; output;
  end;
enddata;
run;
```

The following program uses two SET statements to combine rows from Animal and Plant, and prints the results:

```
data one2onerowsnotequal (overwrite=yes);
  method run();
    set animal;
    set plant;
  end;
enddata;
run;
quit;

proc print data=one2onerowsnotequal;
run;
```

Output 17.6 One-to-One Reading with an Uneven Number of Rows (SET Statement)

Obs	common	animal	plant
1	a	Ant	Grape
2	b	Bird	Hazelnut
3	c	Cat	Indigo
4	d	Dog	Jicama

The result table contains only four rows because DS2 stops processing with the last row from the shortest table.

Comments and Comparisons

- Using multiple SET statements with other DS2 statements makes the following applications possible:
 - ☐ merging one row with many
 - ☐ conditionally merging rows
 - ☐ reading from the same table twice

Match-Merging

Definition

Match-merging combines rows from two or more tables into a single row in a new table according to the values of a common column. The number of rows in the new table is the sum of the largest number of rows in each BY group in all tables. To

perform a match-merge, use the MERGE statement with the required BY statement. When you perform a match-merge, all tables are sorted by the columns that you specify in the BY statement.

Syntax

Use this form of the MERGE statement to match-merge tables:

MERGE *table(s)*;

BY *column(s)*;

Arguments

table

names at least two existing tables from which rows are read.

column

names each column by which the table is sorted. These columns are referred to as BY variables.

For more information, see the [“MERGE Statement” in SAS DS2 Language Reference](#) and the [“BY Statement” in SAS DS2 Language Reference](#).

DS2 Processing during Match-Merging

Compilation phase

DS2 reads the descriptor information of each table that is named in the MERGE statement and then creates a program data vector that contains all the rows from all tables as well as rows created by the data program.

Execution – Step 1

DS2 looks at the first BY group in each table that is named in the MERGE statement to determine which BY group should appear first in the new table. The data program reads into the program data vector the first row in that BY group from each table, reading the tables in the order in which they appear in the MERGE statement. If a table does not have rows in that BY group, the program data vector contains missing values for the rows that are unique to that table.

Execution – Step 2

Each row in any of the input tables is used exactly once in the output table. Columns that are unique to a table are filled with missing or null values if that table is exhausted while producing a BY group.

Execution – Step 3

DS2 repeats these steps until it reads all rows from all BY groups in all tables.

CAUTION

BY variables in a DS2 merge that have a DECIMAL or NUMERIC data type are converted to a DOUBLE data type. If matching DECIMAL columns are not BY variables, the DECIMAL columns remain as a DECIMAL data type.

CAUTION

If there is a type, scale, or precision mismatch between columns with a DECIMAL or NUMERIC data type between tables, the column is converted to a DOUBLE data type.

Using RETAIN to Create a DATA Step Merge

In [SAS V9.4M6](#) and [SAS Viya 3.4](#), you can use the RETAIN argument in the MERGE statement to produce a many-to-many match merge that is similar to a DATA step merge. However, MERGE with RETAIN can have many correct answers because the order of the rows within a BY-group is undefined for most data sources. Therefore, it is possible to get different results from the same program – both of which are correct.

When you specify the RETAIN argument, the final row of a data set in a particular BY group is used repeatedly until there are no more rows in any of the contributing data sets.

For more information, see the [“MERGE Statement” in SAS DS2 Language Reference](#).

Example 1: Merging Rows

The tables Animal and Plant each contain the BY variable Common, and the rows are arranged in order of the values of the BY variable. The following program creates the Animal and the Plant input tables:

```
data animal(overwrite=yes);
  dcl varchar(10) common animal;
  method init();
    common='a'; animal='Ant';   output;
    common='b'; animal='Bird';  output;
    common='c'; animal='Cat';   output;
    common='d'; animal='Dog';   output;
    common='e'; animal='Eagle'; output;
    common='f'; animal='Frog';  output;
  end;
enddata;
run;
```

```
data plant(overwrite=yes);
  dcl varchar(10) common plant;
  method init();
    common='a'; plant='Grape'; output;
    common='b'; plant='Hazelnut'; output;
```

```

        common='c'; plant='Indigo';  output;
        common='d'; plant='Jicama';  output;
        common='e'; plant='Kale';    output;
        common='f'; plant='Lentil';   output;
    end;
enddata;
run;

```

The following program merges the tables according to the values of the BY variable Common, and prints the results:

```

data mmerge (overwrite=yes);
    method run();
        merge animal plant;
        by common;
    end;
enddata;
run;
quit;

proc print data=mmerge;
run;
quit;

```

Output 17.7 Simple Match Merge (MERGE Statement)

Obs	common	animal	plant
1	a	Ant	Grape
2	b	Bird	Hazelnut
3	c	Cat	Indigo
4	d	Dog	Jicama
5	e	Eagle	Kale
6	f	Frog	Lentil

Each row in the new table contains all the columns from all the tables.

Example 2: Match-Merge with Duplicate Values of the BY Variable

In the following example, the tables Animal and Plant contain duplicate values of the BY variable Common. The following program creates the Animal and the Plant input tables:

```

data animal(overwrite=yes);
    dcl varchar(10) common animal;
    method init();
        common='a'; animal='Ant'; output;
        common='a'; animal='Ape'; output;
        common='b'; animal='Bird'; output;

```

```

        common='c'; animal='Cat';   output;
        common='d'; animal='Dog';   output;
        common='e'; animal='Eagle'; output;
    end;
enddata;
run;

data plant(overwrite=yes);
    dcl varchar(10) common plant;
    method init();
        common='a'; plant='Apple';output;
        common='b'; plant='Banana';  output;
        common='c'; plant='Coconut'; output;
        common='c'; plant='Celery';  output;
        common='d'; plant='Dewberry'; output;
        common='e'; plant='Eggplant'; output;
    end;
enddata;
run;

```

The following program produces the merged table MATCH1, and prints the results:

```

data mmdiffby (overwrite=yes);
    method run();
        merge animal plant;
        by common;
    end;
enddata;
run;
quit;

proc print data=mmdiffby;
run;
quit;

```

Output 17.8 Match-Merge with Duplicate BY Variables (MERGE Statement)

Obs	common	animal	plant
1	a	Ape	Apple
2	a	Ant	
3	b	Bird	Banana
4	c	Cat	Coconut
5	c		Celery
6	d	Dog	Dewberry
7	e	Eagle	Eggplant

In row 2 of the output, the value of the column Plant is not retained. Match-merging also did not duplicate values in Animal for row 5.

Note: The MERGE statement does not produce a Cartesian product on a many-to-many match-merge. Instead, it performs a one-to-one merge while there are rows in the BY group in at least one table. When all rows in the BY group have been read from one table and there are still more rows in another table, DS2 fills the columns with missing or null values.

Example 3: Match-Merge with Non-matched Rows

When DS2 performs a match-merge with nonmatched rows in the input tables, DS2 retains the values of all columns in the program data vector even if the value is missing. The tables Animal and Plant do not contain all values of the BY variable Common. The following program creates the Animal and the Plant input tables:

```
data animal(overwrite=yes);
  dcl varchar(10) common animal;
  method init();
    common='a'; animal='Ant'; output;
    common='c'; animal='Cat'; output;
    common='d'; animal='Dog'; output;
    common='e'; animal='Eagle'; output;
  end;
enddata;
run;
```

```
data plant(overwrite=yes);
  dcl varchar(10) common plant;
  method init();
    common='a'; plant='Apple';output;
    common='b'; plant='Banana'; output;
    common='c'; plant='Coconut'; output;
    common='e'; plant='Eggplant'; output;
    common='f'; plant='Fig'; output;end;
enddata;
run;
```

The following program produces the merged table Mmnomrow, and prints the results:

```
data mmnomrow (overwrite=yes);
  method run();
    merge animal plant;
    by common;
  end;
enddata;
run;
quit;
```

```
proc print data=mmnomrow;
run;
quit;
```

Output 17.9 Match-Merge with Non-Matched Rows (MERGE Statement)

Obs	common	animal	plant
1	a	Ant	Apple
2	b		Banana
3	c	Cat	Coconut
4	d	Dog	
5	e	Eagle	Eggplant
6	f		Fig

As the output shows, all values of the column Common are represented in the new table, including missing values for the columns that are in one table but not in the other.

Reserved Words

Reserved Words in the DS2 Language 267

Reserved Words in the DS2 Language

The following words are reserved as DS2 language keywords and cannot be used as variable names or in any other way that differs from their intended use.

Note: You can use a reserved word as a variable name if the word is enclosed in double quotation marks. For more information and an example, see [“Delimited Identifiers” on page 58](#).

Table 18.1 DS2 Reserved Words

Special Characters	A	B	C	D
___KPLIST	ABORT	BIGINT	CALL	DATA
ALL	AND	BINARY	CATALOG	DATE
HOSTNAME	AS	BY	CHAR	DCL
NEW	ASM		CHARACTER	DECIMAL
NTHREADS			COMMIT	DECLARE
NULL			CONTINUE	DELETE
RC				DESCENDING
ROWSET				DIM
TEMPORARY				DO
THREADID				DOUBLE
				DROP
				DS2_OPTIONS

E ELIF ELSE END ENDDATA ENDMODULE ENDPACKAGE ENDSTAGE ENDTABLE ENDTHREAD EQ ERROR ESCAPE	F FLOAT FORMAT FORTRAN FORWARD FROM FUNCTION	G GE GLOBAL GOTO GROUP GT	H HAVING	I IDENTITY IF IN INDSNAME INDSNUM INFORMAT INLINE INPUT INT INTEGER IN_OUT
K KEEP	L LABEL LE LEAVE LIKE LIST LT	M MERGE METHOD MISSING MODIFY MODULE	N NATIONAL NCHAR NE NG NL NOT NULL NUMERIC NVARCHAR	O ODS OF OR ORDER OTHER OTHERWISE OUTPUT OVERWRITE
P PACKAGE PARTITION PRECISION PRIVATE PROGRAM PUT	R REAL REMOVE RENAME REPLACE REQUIRE RETAIN RETURN RETURNS ROLLBACK	S SELECT SET SMALLINT SQLSUB STAGE STOP STORED SUBSTR SYSTEM	T TABLE THEN THIS THREAD THREADS TIME TIMESTAMP TINYINT TO TRANSACTION T_UDF TSPL_OPTIONS	U UNTIL UPDATE

V VARARRAY VARBINARY VARCHAR VARLIST VARYING	W WHEN WHERE WHILE			
--	------------------------------------	--	--	--

PART 3**DS2 and CAS**

Chapter 19	
<i>DS2 in CAS: Concepts</i>	273
Chapter 20	
<i>Running DS2 Programs in CAS</i>	285
Chapter 21	
<i>DS2 and DATA Step Model Publishing and Scoring in CAS</i>	291

DS2 in CAS: Concepts

<i>How DS2 Runs in CAS</i>	273
DS2 in CAS	273
Why Run a DS2 Program in CAS?	274
Processing Modes	274
Multi-Threaded Processing	274
Multi-Threaded Example	275
Single-Threaded Processing	276
Single-Threaded Example	276
Partially Multi-Threaded Processing	277
Why Run Partially Multi-Threaded?	278
Partially Multi-Threaded Example	279
DS2 Threaded Programs in CAS	280
Librefs and Caslibs	281
Comments	281
DS2 Language Elements Not Supported in CAS	281
<i>BY-Group Processing in CAS</i>	281
<i>Integer Type Conversion in the CAS Server</i>	282
<i>DS2 Logging in the CAS Server</i>	283
<i>DS2 Session Encoding in the CAS Server</i>	283

How DS2 Runs in CAS

DS2 in CAS

A DS2 program can run in CAS as well as in SAS 9.4. CAS is the cloud-based, run-time environment that enables distributed, parallel execution. A DS2 program running in CAS typically works on in-memory CAS tables using multiple threads that are distributed across multiple machines. How you run a DS2 program in CAS is almost identical to how you run it in SAS 9.4. DS2 in CAS supports most of the

same language elements and provides much of the same functionality as DS2 in SAS 9.4.

Why Run a DS2 Program in CAS?

Big data processing on a single machine is slow. When your DS2 program runs in CAS, parallel execution speeds the processing of big data by running the program on multiple machines and by dividing the processing workload among threads on these machines.

Processing Modes

- Single-threaded

The program runs in a single thread on a single machine. The single thread processes all the data. Running single-threaded is not recommended for very large data sets.

- Multi-threaded

The program runs in multiple threads distributed across multiple machines, in parallel. Each thread processes a portion of the data.

Only DS2 programs that use the SET FROM statement in a DS2 threaded program run multi-threaded. All other DS2 programs run in a single thread on a single machine. The single thread processes all the data sequentially.

A DS2 program that uses the SET FROM statement in a DS2 threaded program runs either partially or fully multi-threaded. The DS2 thread program always runs multi-threaded, but the DS2 data program runs multi-threaded only if the only statements it contains are a SET FROM statement and, as an option, an OUTPUT statement. If the DS2 data program contains any other statements, the DS2 data program runs single-threaded. A DS2 program whose DS2 thread program runs multi-threaded and whose DS2 data program runs single-threaded is partially multi-threaded.

Multi-Threaded Processing

A DS2 program runs multi-threaded if the data program sets from a DS2 thread and the data program contain only a SET FROM statement and, as an option, an OUTPUT statement. When a DS2 program runs multi-threaded, observations are processed in multiple threads that are distributed in parallel across all CAS workers. During a multi-threaded run, each CAS worker processes a subset of the input table and generates a subset of the output table. CAS data management supports each CAS worker, reading in parallel a subset of the input table and writing in parallel a subset of the output table.

Here is how a DS2 program is run multi-threaded:

- In each thread in each CAS worker, the DS2 thread program does the following:
 - reads its subset of the data from the input table.
 - processes its subset of the data.
 - sends its subset of the data to the DS2 data program.
- In each thread in each CAS worker, the DS2 data program does the following:
 - receives its subset of the data from the DS2 thread program.
 - writes its subset of the data to the output table.

Note: The DS2 data program does not perform any additional processing of the data beyond writing it to the output table.

Multi-Threaded Example

In the following example, a DS2 program filters `cars` to only those where the `msrp` is greater than \$100,000. The program is run in multiple threads, and each thread reads a subset of the records from the `cars` table. Each thread also writes a subset of the records to the `cars_luxury` table.

```
cas casauto; 1
proc casutil sessref=casauto; 2
  load data=sashelp.cars;
run;

proc ds2 sessref=casauto; 3
  thread cars_thd / overwrite=yes; 4
    method run();
      set cars;
      if (msrp > 100000) then do;
        output;
      end;
    end;
  endthread;

data cars_luxury / overwrite=yes; 5 \
  dcl thread cars_thd t;
  method run();
    set from t;
  end;
enddata
run;
quit;
```

- 1 Start a session named `casauto` with the CAS server.
- 2 Load `sashelp.cars` from the SAS client to the `cars` in-memory table in the `casauto` CAS session.

- 3 Run the DS2 program in the `casauto` CAS session. The `SESSREF=` option is necessary to run the DS2 program in CAS.
- 4 In each thread in each CAS worker, the thread program reads a subset of rows from the `cars` in-memory table. The thread program then sends to the data program the subset of rows where the `msrp` is greater than \$100,000.
- 5 In each thread in each CAS worker, the data program receives from the thread program a subset of the rows where `msrp` is greater than \$100,000. The data program then writes to the `cars_luxury` in-memory table the subset of rows where the `msrp` is greater than \$100,000.

When the DS2 program has finished running, the `cars_luxury` table contains the complete set of cars where the `msrp` is greater than \$100,000.

Single-Threaded Processing

A DS2 program that does not use the SET FROM statement in a DS2 threaded program runs single-threaded. When a DS2 program runs single-threaded, the program runs in one thread in a single CAS worker. The single thread processes the input table and generates the output table. All data observations are processed sequentially.

In a single thread in a single CAS worker, the DS2 program does the following:

- reads all the data from the input table.
- processes all the data.
- writes all the data to the output table.

Note: When you run programs single-threaded in CAS, processing is localized to a single worker regardless of the number of available workers in the CAS session.

Single-Threaded Example

In the following example, a DS2 program computes the mean `msrp` for all cars in the `sashelp.cars` table. The program is run single-threaded because it does not use the SET FROM statement in the data program. Regardless of the number of workers in the CAS session, all data processing is localized to a single thread in a single CAS worker.

```
cas casauto; 1
proc casutil sessref=casauto; 2
  load data=sashelp.cars;
run;

proc ds2 sessref=casauto; 3
data _null_; 4
```



```

dcl double sum_msrp n;
retain sum_msrp n;
method init();
    sum_msrp = 0;
    n = 0;
end;
method run();
    set cars;
    sum_msrp = sum_msrp + partial_sum_msrp;
    n = n + partial_n;
end;
method term();
    dcl double mean_msrp having format dollar8.;
    mean_msrp = sum_msrp / n;
    put 'Mean MSRP: ' mean_msrp;    end;
enddata;
run;
quit;

```

- 1 Start a session named **casauto** with the CAS server.
- 2 Load **sashelp.cars** from the SAS client to the cars in-memory table in the **casauto** CAS session.
- 3 Run the DS2 program in the **casauto** CAS session. The **SESSREF=** option is necessary to run the DS2 program in CAS.
- 4 In a single thread in a single CAS worker, the data program reads all the rows from the **cars** in-memory table, sums the **msrp** values across all the rows, computes the mean of the **msrp** values, and prints the mean.

The SAS log contains output similar to the following:

```

Mean MSRP:    $32,775
NOTE: Running DATA program on one node

```

Partially Multi-Threaded Processing

A DS2 program that uses the SET FROM statement in the DS2 threaded program runs partially multi-threaded if the DS2 thread program runs multi-threaded and the DS2 data program runs single-threaded. A data program containing any statements other than SET FROM or OUTPUT runs single-threaded. A partially multi-threaded run has two stages:

Parallel Stage

Runs the DS2 thread program in multiple threads distributed across all CAS workers.

Serial Stage

Runs the DS2 data program in a single thread in a single CAS worker.

During the parallel stage, each CAS worker processes a subset of the input table and generates a subset of an intermediate table. During the serial stage, a single

CAS worker processes the complete intermediate table and generates the complete output table.

Here is how a DS2 program is run partially multi-threaded:

■ Parallel Stage

In each thread in each CAS worker, the DS2 thread program does the following:

- Reads its subset of the data from the input table.
- Processes its subset of the data.
- Writes its subset of the data to a temporary table.

■ Serial Stage

In a single thread in a single CAS worker, the DS2 data program does the following:

- Reads all the data from the temporary table.
- Processes all the data.
- Writes all the data to the output table.

Why Run Partially Multi-Threaded?

Parallel, distributed execution speeds the processing of big data by running the program on multiple machines and by dividing processing workload among threads on these machines. Multi-threaded execution requires each division of data processing to be independent. Data processing algorithms that aggregate or combine data violate the data processing independence requirement for fully multi-threaded execution. If the data processing algorithm can be refactored into a split-apply-combine model of data processing, the processing can be partially parallelized. With a split-apply-combine strategy, the algorithm splits the data, applies data processing to each data split independent of other splits, and then combines the splits. MapReduce is a common specialization of the split-apply-combine strategy.

A DS2 program that runs partially multi-threaded implements a split-apply-combine strategy by the thread program applying the data processing to the data splits in parallel. The data program combines the results of the data processing in serial. For the best performance, as much data processing as possible should be performed by the thread program. Only data processing that requires synchronization or serialization should be performed by the data program.

Note: Only data processing in the thread program is distributed across the available workers in the CAS session. Data processing in the data program is always localized to a single worker regardless of the number of available workers in the CAS session.

Partially Multi-Threaded Example

In the following example, a DS2 program computes the mean **msrp** for all cars in the **sashelp.cars** table. The program is run partially multi-threaded because sets from a DS2 thread and the data program are not limited to a SET FROM and OUTPUT statement.

```
cas casauto; 1

proc casutil sessref=casauto; 2
  load data=sashelp.cars;
run;

proc ds2 sessref=casauto; 3
thread cars_thd / overwrite=yes; 4
  dcl double partial_sum_msrp partial_n;
  retain partial_sum_msrp partial_n;
  keep partial_sum_msrp partial_n;
  method init();
    partial_sum_msrp = 0;
    partial_n = 0;
  end;
  method run()
    set cars;
    partial_sum_msrp = partial_sum_msrp + msrp;
    partial_n = partial_n + 1;
  end;
  method term();
    output;
  end;
endthread;

data _null_; 5
  dcl thread cars_thd t;
  dcl double sum_msrp n;
  retain sum_msrp n;
  method init();
    sum_msrp = 0;
    n = 0;
  end;
  method run();
    set from t;
    sum_msrp = sum_msrp + partial_sum_msrp;
    n = n + partial_n;
  end;
  method term();
    dcl double mean_msrp having format dollar8.;
    mean_msrp = sum_msrp / n;
    put 'Mean MSRP: ' mean_msrp;
  end;
enddata;
run;
```

```
quit;
```

- 1 Start a session named **casauto** with the CAS server.
- 2 Load **sashelp.cars** from the SAS client to the cars in-memory table in the **casauto** CAS session.
- 3 Run the DS2 program in the **casauto** CAS session. The **SESSREF=** option is necessary to run the DS2 program in CAS.
- 4 In each thread in each CAS worker, the thread program reads a subset of rows from the **cars** in-memory table. The thread program sums the **msrp** of the subset of rows, computing a partial sum of the **msrp**. The thread program then writes the partial sum of the **msrp** to a temporary in-memory table.
- 5 In a single thread in a single CAS worker, the data program reads all the partial sums of the **msrp** from the temporary in-memory table, sums the partial sums to total the sum of all **msrp** values, computes the mean of the **msrp** values, and prints the mean.

The SAS log contains output similar to the following:

```
NOTE: Running THREAD program on all nodes
Mean MSRP:    $32,775
NOTE: Running DATA program on one node
```

Note that this partially multi-threaded example is a refactor of the single-threaded example. If a serial algorithm that requires single-threaded execution can be transformed into a split-apply-combine strategy, refactoring the algorithm often results in significant run-time performance benefits for big data that is processed in a distributed, parallel environment such as CAS.

DS2 Threaded Programs in CAS

The basic computational model for analytics in CAS combines distributed computing and multi-threaded computing. CAS actions that pass through the data typically do so by involving multiple threads: each thread receives a portion of the data, and each record in the input table is consumed by only one thread. These threads are called *worker threads* to emphasize the concurrent nature of their execution. The maximum number of concurrent worker threads that you are allowed to use is determined by your software license.

If your DS2 program specifies more threads than the maximum that you are allowed, DS2 reduces the number of threads to that maximum number. If your DS2 program either specifies 0 threads or does not specify the number of threads, DS2 uses the maximum number of threads allowed, as determined by your software license.

Librefs and Caslibs

When a DS2 program runs in CAS, the DS2 program does not have access to the librefs that are created in the SAS client. Only caslibs that are created in the CAS session are available to the DS2 program.

In a DS2 program running in CAS, the following form specifies a CAS table in a caslib:

```
caslib.table-name
```

If the caslib has a parenthesis or other special character, you must enclose the caslib in double quotation marks. Here is an example:

```
set "caslib(janedoe)".mytable;
```

Comments

You can use the `/* message */` construct to add comments to your DS2 program in order to document the purpose of the program, explain unusual segments of the program, or describe steps in a complex program or calculation.

IMPORTANT Although you can write comments within statements or anywhere that a single blank can appear in your DS2 program, comments using the `/* message */` construct cannot be nested.

DS2 Language Elements Not Supported in CAS

There are some DS2 language elements that are not supported in CAS. These language elements are documented with the restriction “This *language-element* is not supported on the CAS server.” in [SAS DS2 Language Reference](#).

BY-Group Processing in CAS

On the SAS client, DS2 BY-group processing groups the rows from input tables and orders the rows by values of one or more columns in the BY statement.

On the SAS client, pre-sorted data arrives to DS2 ordered, and the BY-groups are distributed to the threads as whole BY-group blocks. In CAS, the data is inherently unordered on the partitions. If there is no BY statement, each thread receives the data that is local to the node. If a BY statement is used in either the thread program or data program, the table rows are transmitted between workers such that each thread sees one or more whole BY-groups. Because data transport across the grid can be resource intensive, you should consider whether it would be best to aggregate the raw data that exists on each partition by using a hash table in the thread program before using BY processing to obtain the results in the data program. If there are comparatively few distinct data groups compared to input rows, it might be preferable to collapse data into those few distinct groups in the thread program before redistributing them using a BY statement in the data program.

Integer Type Conversion in the CAS Server

DS2 supports BIGINT, INTEGER, SMALLINT, and TINYINT integer data types when run on the SAS Platform. A CAS server table supports only BIGINT (INT64) and INTEGER (INT32) integer data types.

The CAS runDS2 action can execute a DS2 program in CAS that declares a variable with a SMALLINT or TINYINT data type. If the program writes the variable to a CAS table, then the SMALLINT or TINYINT values that are written are converted to an INTEGER (INT32) data type for the CAS table.

A value of -2147483647 represents a SAS missing value in an INTEGER (INT32) column in a CAS table. A value of -9223372036854775808 represents a SAS missing value in a BIGINT (INT64) column in a CAS table. Note that -2147483647 is a valid value in a BIGINT (INT64) column in a CAS table, so -2147483647 does not represent SAS missing values everywhere.

In DS2, -2147483647 is valid for a BIGINT variable, but it is out of range for INTEGER, SMALLINT, and TINYINT variables. In DS2, -9223372036854775808 is out of range for BIGINT, INTEGER, SMALLINT, and TINYINT variables. If an integral variable is assigned a value that is out of the range, but supported by the variable's type, then a SAS missing value or ANSI null is assigned to the variable. Therefore, if -2147483647 or -9876543210 is assigned to an INTEGER, SMALLINT, or TINYINT variable, the variable is assigned a SAS missing value or ANSI null value.

If an INTEGER (INT32) column in a CAS table is read into a DS2 program and an observation has the value -2147483647 for the INTEGER (INT32) column, the value is converted to DS2's internal representation of a SAS missing value during input. If a BIGINT (INT64) column in a CAS table is read into a DS2 program and an observation has the value -2147483647 for the BIGINT (INT64) column, the value is read as-is without conversion.

DS2 Logging in the CAS Server

DS2 and CAS support the SAS logging facility. For more information about logging in SAS Viya, see “Logging” in *SAS Viya Administration*.

DS2 Session Encoding in the CAS Server

The DS2 session encoding for DS2 programs that are compiled and executed in a CAS server is always UTF-8.

Running DS2 Programs in CAS

<i>How to Run DS2 Programs in CAS</i>	285
<i>DS2 Program Walk-Through</i>	287

How to Run DS2 Programs in CAS

There are two ways to run a DS2 program in CAS:

- PROC DS2

```
cas casauto;
caslib _all_ assign;

proc cas;
  session casauto;
  /* This sets the active caslib to casdata.*/
  table.addCaslib /
    caslib="casdata"
    dataSource={srcType="path"}
    path="path-to-your-data";

  /* Load source data (cars) into a table.*/
  table.loadTable /
    caslib="casdata"
    path="cars.sashdat"
    casOut={name="cars", replace=true};
run;
quit;

proc ds2 sessref=casauto;
thread cars_thd / overwrite=yes;
  method run();
    set casdata.cars;
    if (msrp > 100000) then do;
      put make= model= msrp=;
      output;
    end;
```

```

        end;
    endthread;

data cars_luxury / overwrite=yes;
    dcl thread cars_thd t;
    method run();
        set from t threads=4;
    end;
enddata;
run;
quit;

```

For an explanation of this program, see “DS2 Program Walk-Through” on page 287.

For more information about PROC DS2, see “DS2 Procedure” in *Base SAS Procedures Guide*.

■ DS2 runDS2 action

```

cas casauto;
caslib _all_ assign;

proc cas;
    session casauto;
    /* This sets the active caslib to casdata.*/
    table.addCaslib /
        caslib="casdata"
        dataSource={srcType="path"}
        path="path-to-your-data";

    /* Load source data (cars) into a table.*/
    table.loadTable /
        caslib="casdata"
        path="cars.sashdat"
        casOut={name="cars", replace=true};
run;

/*DS2 program to search for cars over $100K */
ds2.runDS2 program="thread cars_thd / overwrite=yes;
method run();
    set casdata.cars;
    if (msrp > 100000) then do;
        put make= model= msrp=;
        output;
    end;
end;
endthread;

data cars_luxury / overwrite=yes;
    dcl thread cars_thd t;
    method run();
        set from t threads=4;
    end;
enddata;";
run;
quit;

```

For more information, see the “DS2 Action Set” in *SAS Viya: System Programming Guide*.

TIP Unless you are using Lua or Python, it is recommended that you use PROC DS2 to run your DS2 program. There are advantages to using PROC DS2. For more information, see “DS2 Action Set Details” in *SAS Viya: System Programming Guide*.

DS2 Program Walk-Through

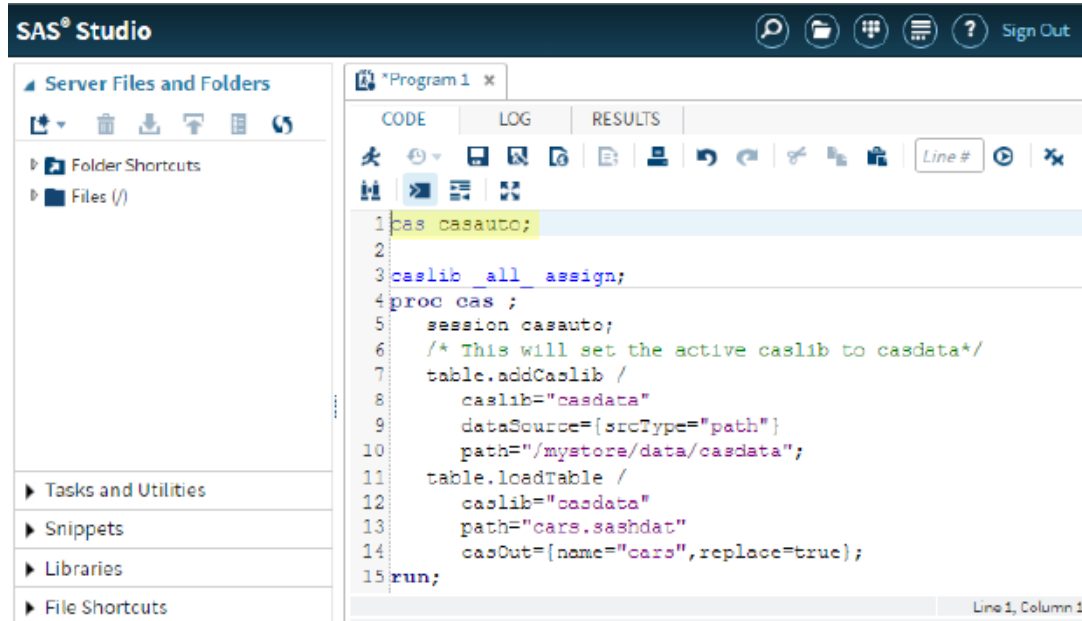
To run a DS2 program in CAS:

- 1 Start a CAS session.

To access CAS, start and activate the CAS session. Specify the name of a CAS session in the CAS statement.

```
cas casauto;
```

Here is a view of the **SAS Studio Code** tab:



The SAS log contains notes similar to the following:

```

NOTE: The session CASAUTO connected successfully to Cloud Analytic Services
cloud.example.com using port 5570. The UUID is
      session-UUID. The user is casdemo and the active caslib is
CASUSERHDFS(casdemo).
NOTE: The SAS option SESSREF was updated with the value CASAUTO.
NOTE: The SAS macro _SESSREF_ was updated with the value CASAUTO.
NOTE: The session is using nnn workers.
59      caslib _all_ assign;
NOTE: CASLIB CASUSER(casdemo) will be mapped to SAS Library CASUSER.
NOTE: CASLIB CASUSERHDFS(casdemo) will be mapped to SAS Library CASUSERH.

```

2 Associate a caslib.

You can use the CASLIB statement to associate the default CASUSER caslib with all SAS librefs that you create.

```
caslib _all_ assign;
```

Here is a partial listing of the notes that are displayed in the SAS log:

```

NOTE: CASLIB CASUSER(casdemo) will be mapped to SAS Library CASUSER.
NOTE: CASLIB CASUSERHDFS(casdemo) will be mapped to SAS Library CASUSERH.

```

3 Create a caslib and load data.

To create a caslib that provides access to files on your file system, use the `table.addCaslib` action along with the `dataSource` option set to **path**. In this example, the CASDATA caslib is the interface between the CASAUTO session and the source tables found on `/mystore/data/casdata`.

```

proc cas ;
  session casauto;
  /* This will set the active caslib to casdata*/
  table.addCaslib /
    caslib="casdata"
    dataSource={srcType="path"}
    path="/mystore/data/casdata";

```

For more information, see [“Caslibs” in SAS Cloud Analytic Services: Fundamentals](#).

To load your source data (cars) into an in-memory table on CAS, you can use the `table.loadTable` action.

```

table.loadTable /
  caslib="casdata"
  path="cars.sashdat"
  casOut={name="cars",replace=true};
run;

```

Results describing the CASDATA caslib are displayed on the **Results** tab in SAS Studio.

Results from table.addCaslib

CAS Library Information					
Library	Type	Path	Sub-directories included	Session local	Active
casdata	PATH	/mystore/data/casdata/	No	Yes	Yes

4 View your CAS table.

You can view the rows of data from the in-memory table (cars) using the `table.fetch` action.

```
table.fetch / table="cars" to=10;
run;
```

Here are the results of fetching the first 10 rows from the in-memory table.

Index	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	Engine Size (L)	Cylinders	Horsepower	MPG (City)	MPG (Highway)	Weight (LBS)	Wheelbase (IN)	Length (IN)
1	BMW	X3 3.0i	SUV	Europe	All	\$37,000	\$33,673	3	6	225	18	23	4023	110	180
2	Honda	S2000 convertible 2dr	Sports	Asia	Rear	\$33,280	\$29,955	2.2	4	240	20	25	2835	85	162
3	Nissan	Altima S 4dr	Sedan	Asia	Front	\$19,240	\$18,030	2.5	4	175	21	26	3039	110	192
4	Audi	A6 4.2 Quattro 4dr	Sedan	Europe	All	\$49,880	\$44,935	4.2	6	300	17	24	4024	109	190
5	Honda	Civic LX 4dr	Sedan	Asia	Front	\$15,880	\$14,531	1.7	4	115	32	38	2513	103	175
6	Mitsubishi	Eclipse GTS 2dr	Sports	Asia	Front	\$25,092	\$23,455	3	6	210	21	28	3241	101	177
7	Acura	3.5 RL w/Navigation 4dr	Sedan	Asia	Front	\$40,100	\$41,100	3.5	6	225	18	24	3893	115	197
8	GMC	Safari SLE	Sedan	USA	Rear	\$25,540	\$23,215	4.3	6	190	18	20	4300	111	190
9	Mercury	Merauder 4dr	Sedan	USA	Rear	\$34,495	\$31,558	4.6	8	302	17	23	4195	115	212
10	Volvo	S80 2.5T 4dr	Sedan	Europe	All	\$37,995	\$35,658	2.5	6	194	20	27	3681	110	190

5 Run a DS2 program.

When used with the `sessref=` option, PROC DS2 enables you to run your program in CAS using your in-memory CAS table.

```
proc ds2 sessref=casauto; 1

thread cars_thd / overwrite=yes; 2
  method run();
  set casdata.cars; 3
  if (msrp > 100000) then do;
    put make= model= msrp=;
    output;
  end;
end;
endthread;

data cars_luxury / overwrite=yes; 4
  dcl thread cars_thd t;
  method run();
  set from t threads=4;
end;
enddata;

run;
quit;
```

- 1 Add the `sessref=` option in the PROC DS2 statement to run your program in CAS.
- 2 Operations in the parallel stage of the program are applied to multiple data rows in parallel.
- 3 Read a row from the in-memory table.
- 4 The thread instances are created and executed.

Here are the results that are displayed in the SAS log:

```
NOTE: Running THREAD program on all nodes
Make=Porsche      Model= 911 GT2 2dr      MSRP=$192,465
Make=Mercedes-Benz Model= CL600 2dr      MSRP=$128,420
Make=Mercedes-Benz Model= SL55 AMG 2dr    MSRP=$121,770
Make=Mercedes-Benz Model= SL600 convertible 2dr MSRP=$126,670
NOTE: Created thread cars_thd in data set "casdata".cars_thd.
NOTE: Running THREAD program on all nodes
NOTE: Running DATA program on all nodes
NOTE: Execution succeeded. 4 rows affected.
104          quit;

NOTE: PROCEDURE DS2 used (Total process time):
      real time      4.89 seconds
      cpu time       0.02 seconds
```

DS2 and DATA Step Model Publishing and Scoring in CAS

<i>Introduction to DS2 and DATA Step Model Publishing and Scoring in CAS</i>	291
Overview of Model Publishing and Scoring	291
Model Publishing and Scoring Components	292
How to Publish and Score DS2 and DATA Step Models in CAS	293
<i>Using PROC SCOREACCEL to Publish, Run, and Delete Models in CAS</i>	294
<i>Using DS2 Actions to Publish, Run, and Delete Models in CAS</i>	295
Publishing a Model in CAS	295
Running a Model in CAS	296
Deleting a Model in CAS	297
<i>Using DS2 Actions to Copy to, Publish from, Run, and Delete Models in an External Data Source</i>	297
Publishing a Model to an External Data Source	297
Copying a Model from CAS to an External Data Source	299
Running a Model from CAS in an External Data Source	300
Delete a Model in an External Data Source	301

Introduction to DS2 and DATA Step Model Publishing and Scoring in CAS

Overview of Model Publishing and Scoring

Beginning in SAS 9.4M6 and SAS Viya 3.3, you can publish and score both DS2 and DATA step models in the CAS server. In addition, you can copy or publish the DS2 and DATA step models to an external data source. You can then score the model programs on the data source using the SAS Embedded Process.

Note: For SAS Viya 3.3 and later, only Hadoop and Teradata data sources are supported.

Here are the software requirements for model publishing and scoring in CAS:

- SAS 9.4 or SAS Viya 3.3 and later
 - SAS In-Database Technologies for Hadoop (or Teradata) on SAS Viya
 - SAS/ACCESS Interface to Hadoop (or Teradata) on SAS Viya
 - (Optional) SAS Model Manager on SAS Viya
-

Note: This documentation covers only the DS2 model publishing actions. For information about using SAS Model Manager to score models in CAS, see *SAS Model Manager User's Guide*.

- Linux operating environment

Model Publishing and Scoring Components

A model can consist of the following items:

- DS2 or DATA step model program

This SAS language file can be generated by SAS Enterprise Miner or you can write your own model program. The default SAS Enterprise Miner file name is `score.sas`. However, any file name can be specified.

- (Optional) Input and output variable metadata

This is information about the scoring variables and other properties that are used and created by the scoring code. This information is available in XML. The default SAS Enterprise Miner or SAS Factory Miner file name is `score.xml`.

- (Optional) Analytic stores

An *analytic store* is a binary entity that can be saved as a binary file, normally as a result of a `SAVESTATE` statement from an analytic procedure. The analytic store contains information about the state of an analytic object. It stores information that enables you to load and restore the state of the analytic object and set it in a score-ready mode. The analytic store is transportable. That is, it can be produced on one host and consumed on others without need for the traditional SAS export or import.

The analytic store files can be generated by SAS Factory Miner, SAS Enterprise Miner, and the `HPFOREST`, `HPSVM`, and `ASTORE` procedures. The default SAS Factory Miner file name is `score.sasast`. However, any file name can be specified.

- (Optional) User-defined formats

Information about these formats is available if the training data contains SAS user-defined formats. This information is available in either XML or an item store.

How to Publish and Score DS2 and DATA Step Models in CAS

There are two methods by which DS2 and DATA step models can be published and scored in CAS:

■ PROC SCOREACCEL

The SCOREACCEL procedure provides an interface from a SAS 9.4 client to CAS for DS2 and DATA step model publishing and scoring.

PROC SCOREACCEL provides an interface to the DS2 actions. When a model is run using PROC SCOREACCEL, the procedure invokes the DS2 runModel action, which runs the program in the CAS server. If the specified target is Teradata or Hadoop, the Model Publishing runModelExternal action is called to run the model inside the data source by using the SAS Embedded Process.

Here are the advantages of using PROC SCOREACCEL:

- It is easier to use. For example, you can specify the model component files instead of inserting the file content in your program.
- It uses the familiar SAS procedure interface.
- Your program is run from the SAS client.

For more information, see [“Using PROC SCOREACCEL to Publish, Run, and Delete Models in CAS” on page 294](#).

■ DS2 actions

The DS2 actions provide an interface from the CAS server for DS2 and DATA step model publishing and scoring.

PROC SCOREACCEL provides an interface to the DS2 actions. When a model is run using PROC SCOREACCEL, the procedure invokes the DS2 runModel action, which runs the program in the CAS server. If the specified target is Teradata or Hadoop, the Model publishing runModelExternal action is called to run the model inside the data source by using the SAS Embedded Process.

Here are the advantages of using the DS2 actions:

- Your program is run directly in the CAS server.
- You can program in Python, Lua, or R.

Note: Your model program must still be written in the DS2 or DATA step language.

- There are more options available when publishing and scoring models in a data source.

For more information, see [“Using DS2 Actions to Publish, Run, and Delete Models in CAS”](#) on page 295.

IMPORTANT This documentation refers to the actions generically as “DS2 actions”. However, the `deleteModel`, `publishModel`, `publishModelExternal`, `runModelLocal`, `runModelExternal`, and `copyModelExternal` actions are members of the Model Publishing and Scoring action set. The `runModel` action is a member of the DS2 action set.

Using PROC SCOREACCEL to Publish, Run, and Delete Models in CAS

PROC SCOREACCEL enables you to perform the following actions:

- Publish a DS2 or DATA step model to a CAS table
- Run a model in the CAS server
- Publish your model from CAS to an external data source and run the model using the SAS Embedded Process
- In [SAS Viya 3.4](#), delete a model previously published in CAS or an external data source

When you publish DATA step code, PROC SCOREACCEL translates the DATA step code to DS2 code on the SAS client before publishing the model to the CAS server. The DATA step code can contain macro references (code that includes “%” and “&”). The DS2 code that is produced is suitable for running in CAS or by the SAS Embedded Process on an external data source. The DS2 code can also contain references to the `sasep.in` and `sasep.out` variables. If the model is published to CAS, a DS2 thread program is produced. If the model is published to Hadoop or Teradata, the generated DS2 code is in the form of a standard data program that is the same as the one that is produced by the SAS Scoring Accelerator publishing macros.

PROC SCOREACCEL has three statements, `PUBLISHMODEL`, `RUNMODEL`, and `DELETEMODEL`. These statements use the DS2 model publishing actions on the CAS server.

PROC SCOREACCEL supports a file interface for passing the model components (model program, formal XML, and analytic stores). The procedure reads the specified files and passes their contents to the model publishing CAS actions. These files must be visible from the SAS client.

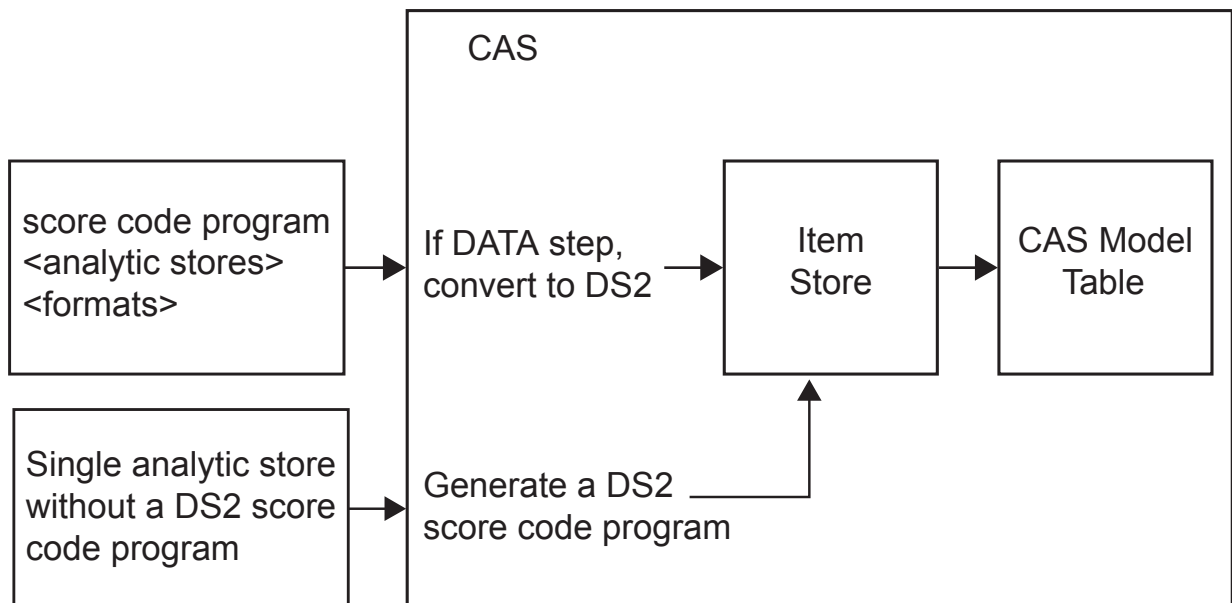
For more information, see [“SCOREACCEL Procedure”](#) in *Base SAS Procedures Guide*.

Using DS2 Actions to Publish, Run, and Delete Models in CAS

Publishing a Model in CAS

You use the DS2 `publishModel` action to publish a model in CAS.

Figure 21.1 DS2 `publishModel` Action



As shown in [Figure 21.1](#), there are two types of input to the `publishModel` action:

- You can specify the name of the DS2 or DATA step model program.

Optional components include formats and analytic stores. In CAS, if the model program is DATA step code, the code is converted to DS2. Then the model components are bundled into a consolidated item store. This item store is written as a blob to a VARBINARY column of the CAS model table.

- In [SAS Viya 3.4](#), you can specify the name of an analytic store without a DS2 model program.

The `publishModel` action generates the DS2 model program. Then the model components are bundled into a consolidated item store.

Note: Autogeneration of DS2 model programs is not supported for models that include multiple analytic stores.

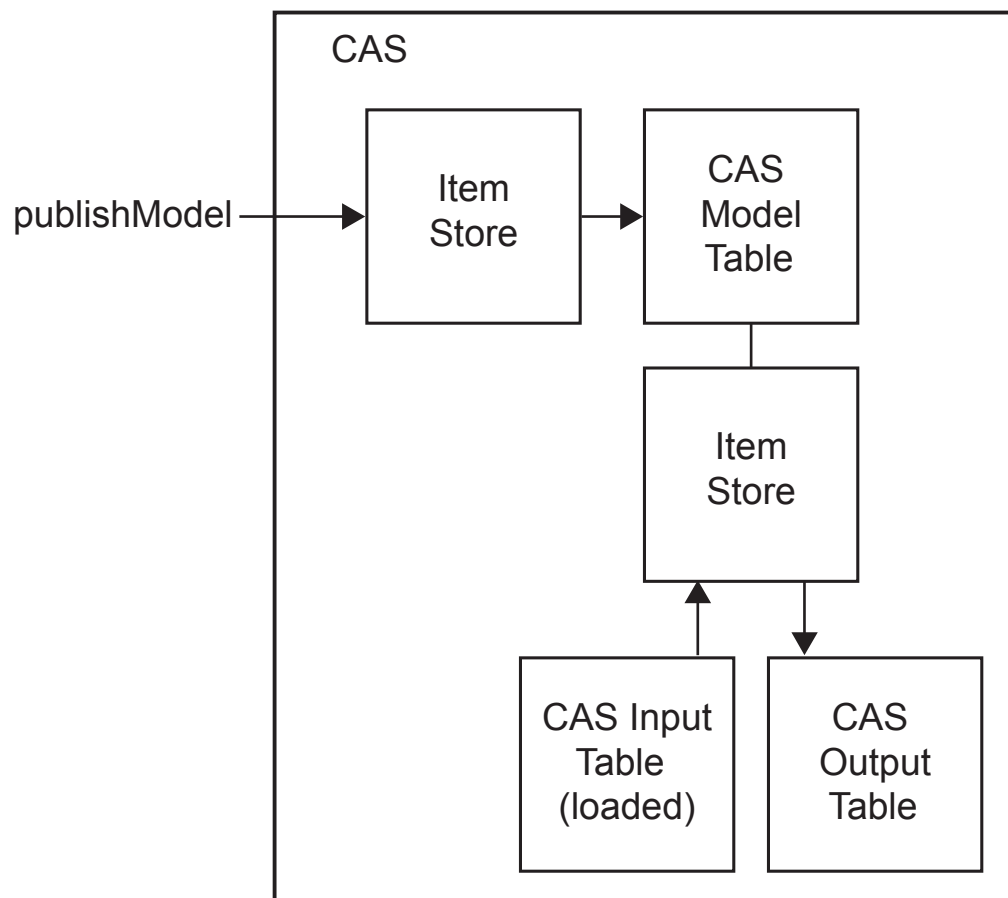
The DS2 `runModel` and `runModelExternal` actions can read the item store from the model table, unpack it, and run the model either on the CAS grid or on an external data source.

For more information, see [publishModel action](#) and “About the CAS Model Table” in *SAS Visual Analytics: Programming Guide*.

Running a Model in CAS

You can use either the DS2 `runModelLocal` action or the DS2 `runModel` action to run a model in CAS. The `runModelLocal` action is a member of the `modelPublishing` action set and the `runModel` action is a member of the DS2 action set. The `runModelLocal` action is simply an interface to the `runModel` action.

Figure 21.2 DS2 `runModel` or `runModelLocal` Action



As shown in [Figure 21.2](#), it is assumed that the model to be run is previously published to a CAS model table by the `publishModel` action. The `runModel` or

runModelLocal action unpacks the item store from the CAS model table for running the model.

Here are some considerations for running models in CAS:

- The input table must be loaded before the runModel or runModelLocal action is run.
- If the table and casOut action parameters are specified, the action replaces any occurrences of sasep.in and sasep.out in the DS2 model program with the specified caslib and table name. If the DS2 model program does not use **sasep.in** and **sasep.out**, then any values passed for table and casOut are ignored.
- It is recommended that the DS2 code that is run with the runModel or runModelLocal action in CAS include a thread program. This enables the DS2 model program to run in parallel on multiple worker nodes on the CAS server.

For more information, see the [runModel action](#) and the [runModelLocal action](#).

Deleting a Model in CAS

Use the deleteModel action to delete a model.

The deleteModel action deletes the row for the specified model from the model table. To delete the model, the model table must first be loaded into the CAS server.

For more information, see [deleteModel action](#).

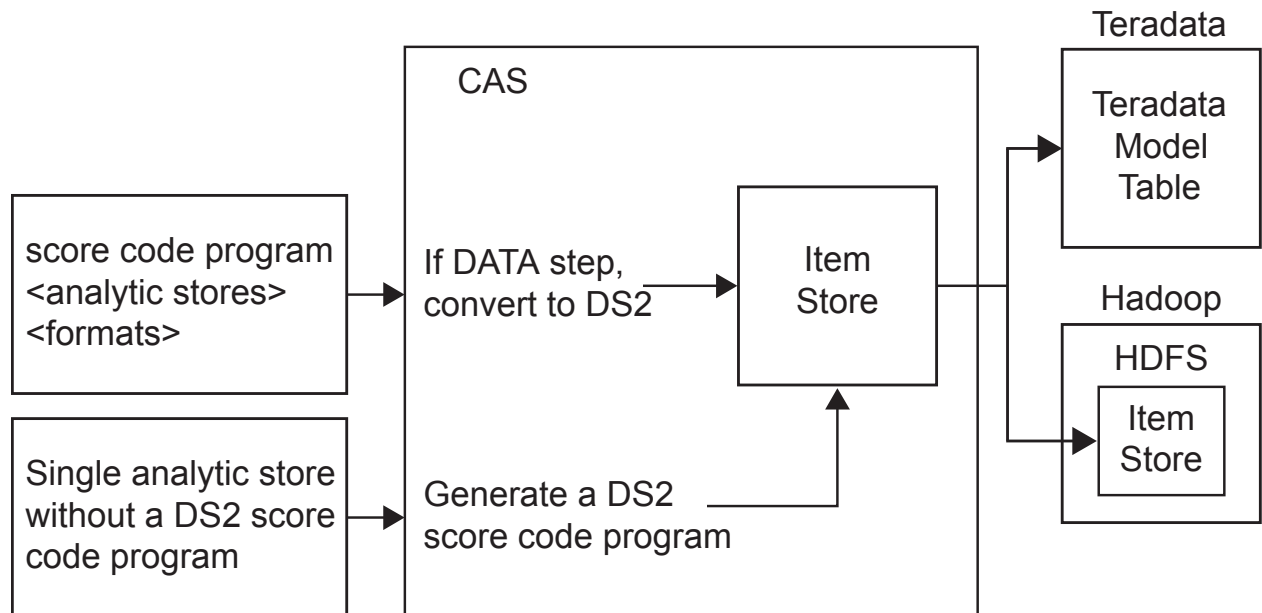
Using DS2 Actions to Copy to, Publish from, Run, and Delete Models in an External Data Source

Publishing a Model to an External Data Source

To run a model in an external data source (Hadoop or Teradata), you must first publish the model to the data source. You can use the DS2 publishModelExternal action to publish the model components to the data source.

Note: Alternatively, if the model already exists in a CAS model table, you can copy the model to the data source. For more information, see [“Copying a Model from CAS to an External Data Source”](#) on page 299.

Figure 21.3 DS2 publishModelExternal Action



As shown in [Figure 21.3](#), the sequence is similar to the publishModel action: there are two types of input to the publishModelExternal action:

- You can specify the name of the DS2 or DATA step model program.

Optional components include formats and analytic stores. In CAS, if the model program is DATA step code, the code is converted to DS2. Then the model components are bundled into a consolidated item store.

- In [SAS Viya 3.4](#), you can specify the name of an analytic store without a DS2 model program.

In CAS, the publishModelExternal action generates the DS2 model program. Then the model components are bundled into a consolidated item store.

Note: Autogeneration of DS2 model programs is not supported for models that include multiple analytic stores.

In Teradata, this item store is written as a blob to a VARBINARY column of the Teradata model table. In Hadoop, the item store is copied to the HDFS file system on the cluster. The DS2 runModelExternal action can read the item store, unpack it, and run the model on the external data source.

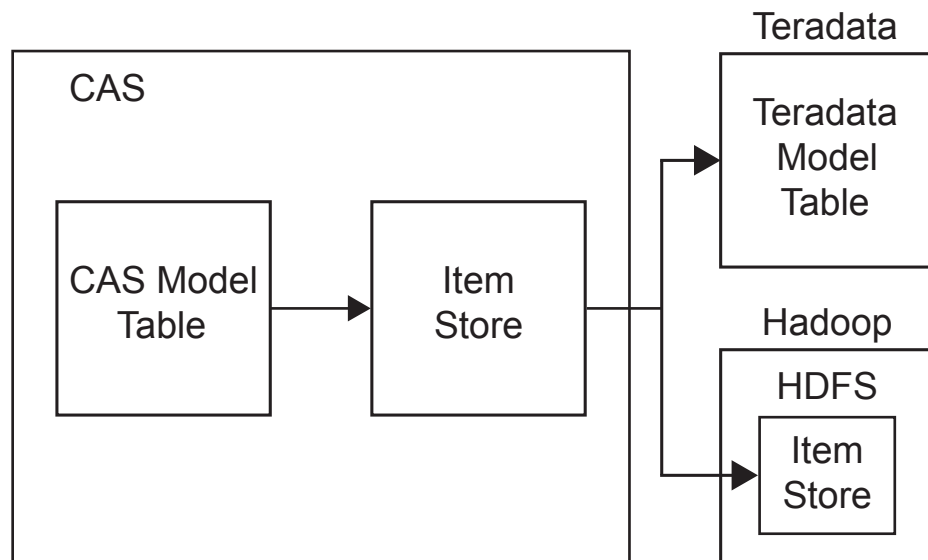
For more information, see [publishModelExternal action](#).

Copying a Model from CAS to an External Data Source

To run a model in an external data source (Hadoop or Teradata), you must first publish the model on the data source. If the model already exists in a CAS model table, you can use the DS2 `copyModelExternal` action to copy the model components to the data source.

Note: Alternatively, you can publish the model to the data source. For more information, see [“Publishing a Model to an External Data Source” on page 297](#).

Figure 21.4 DS2 `copyModelExternal` Action



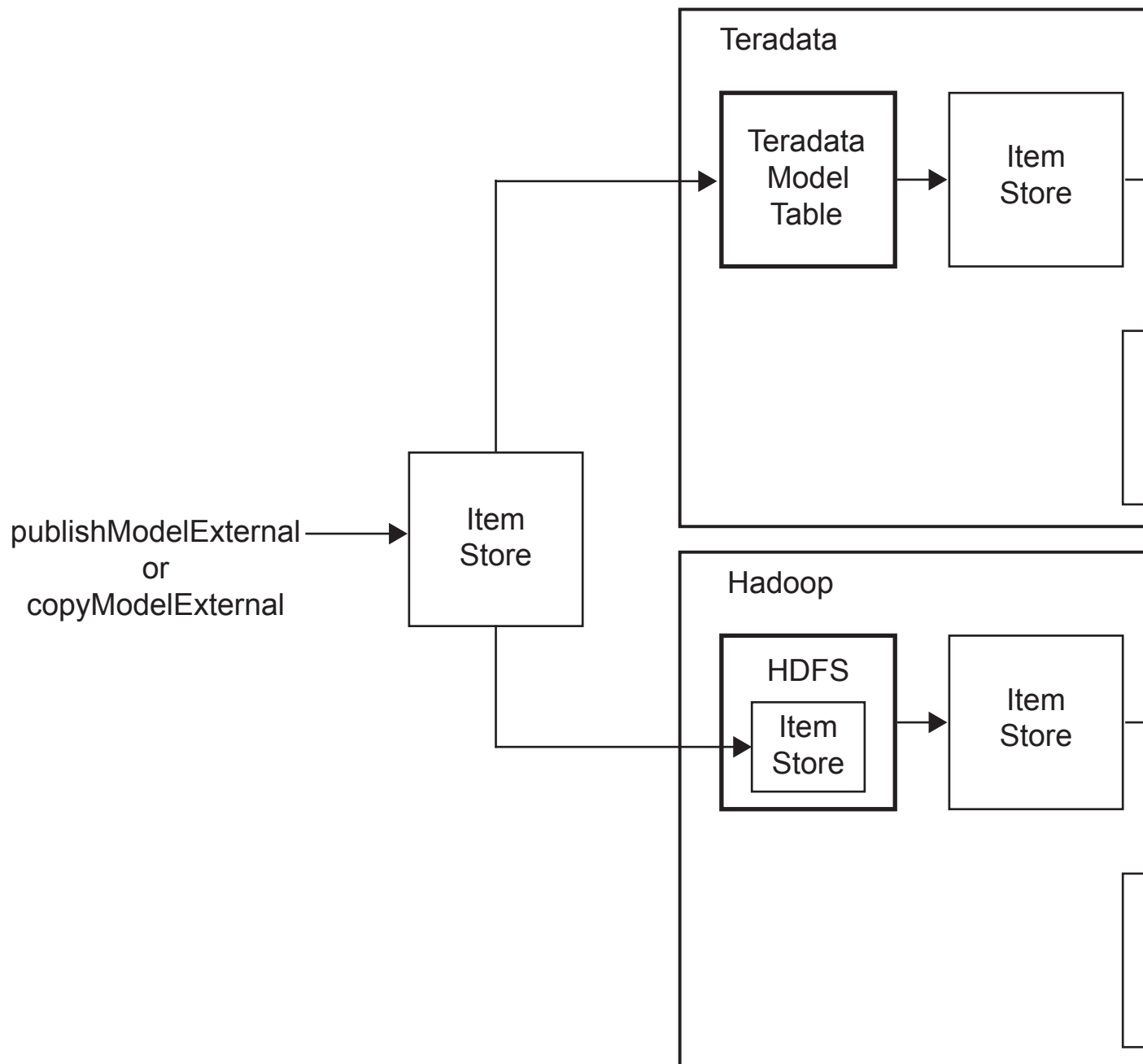
As shown in [Figure 21.4](#), the `copyModelExternal` action takes the item store from the CAS model table and copies it to the external data source. In Teradata, this item store is written as a blob to a VARBINARY column of the Teradata model table. In Hadoop, the item store is copied to the HDFS file system on the cluster. The DS2 `runModelExternal` action can read the item store, unpack it, and run the model on the external data source.

For more information, see [copyModelExternal action](#).

Running a Model from CAS in an External Data Source

You use the DS2 `runModelExternal` action to invoke the SAS Embedded Process to read a model and run it in an external database.

Figure 21.5 DS2 `runModelExternal` Action



As shown in [Figure 21.5](#), the model must be previously published or copied to the data source using either the `publishModelExternal` or `copyModelExternal` action, respectively. When the `runModelExternal` action is called, the item store is unpacked and read by the SAS Embedded Process. The input and output tables reside in the data source.

For more information, see [runModelExternal action](#).

Note: In SAS Viya 3.4, you can run your model in Hadoop as either a MapReduce job or as a Spark job using In-Database Technologies for Hadoop (on SAS Viya). In either case, you do this by setting the `PLATFORM=` parameter in the `runModelExternal` action or in the `RUNMODEL` statement of `PROC SCOREACCEL`. Running the model as a Spark job with this approach does not require SAS/ACCESS Interface to Spark.

Delete a Model in an External Data Source

In SAS Viya 3.4, use the `deleteModelExternal` action to delete a model from Hadoop or Teradata.

In Teradata, the table entry for the model is deleted from the specified model table. In Hadoop, the item store containing the model components is removed from HDFS on the Hadoop cluster.

PART 4

Appendixes

<i>Appendix 1</i>	
<i>DS2 Type Conversions for Expression Operands</i>	305
<i>Appendix 2</i>	
<i>DS2 Loggers</i>	307
<i>Appendix 3</i>	
<i>Resolving DS2 Errors</i>	317

Appendix 1

DS2 Type Conversions for Expression Operands

DS2 Automatic Type Conversions for Expression Operands 305

DS2 Automatic Type Conversions for Expression Operands

The following table lists the automatic type conversions for expression operands. The first column is the “from” type. The first row is the “to” type.

Table A1.1 DS2 Automatic Type Conversions for Assignment Statement

From/To	TinyInt	SmallInt	Integer	BigInt	Decimal/ Numeric	Real	Double	Date	Time	Timestamp	Char	Varchar	NChar	NVarchar	Binary	Varbinary
TinyInt	Y		Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
SmallInt	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
Integer	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
BigInt	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
Decimal/ Numeric	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
Real	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
Double	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
Date	N	N	N	N	N	N	N	Y	N	N	Y	Y	Y	Y	N	N
Time	N	N	N	N	N	N	N	N	Y	N	Y	Y	Y	Y	N	N
Timestamp	N	N	N	N	N	N	N	N	N	Y	Y	Y	Y	Y	N	N
Char	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
Varchar	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
NChar	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
NVarchar	Y	Y	Y	Y	Y	Y	Y	N	N	N	Y	Y	Y	Y	N	N
Binary	N	N	N	N	N	N	N	N	N	N	Y	Y	Y	Y	Y	Y
Varbinary	N	N	N	N	N	N	N	N	N	N	Y	Y	Y	Y	Y	Y

Appendix 2

DS2 Loggers

<i>Overview of DS2 Loggers</i>	307
<i>Configuration Loggers</i>	308
<i>Run-Time Loggers</i>	308
<i>HTTP Package Logger</i>	310
<i>JSON Package Logger</i>	310
<i>Example: Logging All SQL Operations</i>	312
<i>Example: Logging Trace Variables</i>	313
Invoking the TraceVariables Logger	313
Understanding Logger Messages	314

Overview of DS2 Loggers

The SAS logging facility is a framework that categorizes and filters log messages in SAS server and SAS programming environments, and writes log messages to various output devices. In the server environment, the logging facility logs messages based on predefined message categories, such as Admin for administrative messages, App for application messages, and Perf for performance messages. Messages for a category can be written to files, consoles, and other system destinations simultaneously. The logging facility also enables messages to be filtered based on the following thresholds: TRACE, DEBUG, INFO, WARN, ERROR, and FATAL.

DS2 provides several loggers to report both configuration and run-time information. In addition, DS2 provides a logger for the HTTP package and the JSON package. In general, INFO provides the minimum amount of information and DEBUG provides most (perhaps all) of the information that is needed to debug a field problem. The TRACE level provides anything and everything that might be of interest.

For more information about loggers and the SAS logging facility, see *SAS Logging: Configuration and Programming Reference*.

Configuration Loggers

Configuration loggers track information as the DS2 compiler starts up and provide context for the actual execution of the user's code.

The following configuration loggers are available.

`App.TableServices.DS2.Config.Options`
shows the options supplied to the DS2 compiler.

`App.TableServices.DS2.Config.Source`
shows the DS2 source code processed by the DS2 compiler.

`App.TableServices.DS2.Config.Version`
shows version information for all threaded kernel extensions loaded by the DS2 compiler.

When diagnosing DS2 problems, it is important to understand that the `App.TableServices.DS2.Config.*` loggers do not inherit configuration from their ancestors. The default level for the `App.TableServices.DS2.Config.*` loggers is OFF. If you want to capture `App.TableServices.DS2.Config.*` logger events to the server's log, you must explicitly set each applicable `App.TableServices.DS2.Config.*` logger. It is recommended that you configure them only when diagnosing a DS2 problem since the additional logging traffic affects performance.

Run-Time Loggers

Run-time loggers track actual execution. Some of the tracked information is generated for each row processed. Therefore, a large input table can produce very large amounts of data.

The following run-time loggers are available.

`App.TableServices.DS2.Runtime.Calls`
shows a trace of all method calls during execution.

`App.TableServices.DS2.Runtime.SQL`
shows all SQL statements either prepared by the DS2 compiler, executed by the DS2 compiler, or both.

`App.TableServices.DS2.Runtime.Timing`
shows the time that is spent during code compilation and execution. Depending on the level of information that is requested (INFO, DEBUG, TRACE), timing information is provided at various points throughout DS2 execution. Examples

are parse time, compilation time, various audit and transformation pass times, as well as the INIT, RUN, and TERM method execution times.

`App.TableServices.DS2.Runtime.Put`
records all PUT statement output.

`App.TableServices.DS2.Runtime.Log`
records all messages that are sent to the SAS log in a SAS session. On SAS Federation Server, these messages are also appended to the ODBC statement handle as diagnostic records by default.

`App.TableServices.DS2.Runtime.TraceVariables`

Note: This logger is available in SAS Viya 3.5 for use with the CAS server and the SAS Micro Analytic Service server only.

when set to DEBUG or TRACE, records changes to variable values during DS2 program execution. Tracing information is reported only for variables that are modified by these DS2 statements and functions:

- Assignment statement (=)
- Array assignment statement (:=)
- Index variable assignment by the DO statement
- SET or MERGE statement
- SUBSTR (left of =) function
- SUM statement (variable + expression; variable – expression;)

Tracing information for variables is not reported for variables modified in these ways:

- Variable data modified by a built-in DS2 package. For example, if the HASH.FIND method modifies a data variable, that variable modification is not reported in the trace.
- Implicit updates of DS2 automatic variables. For example, the implicit incrementing of the N automatic variable is not reported in the trace.
- Implicit initialization of variables to missing or null during variable creation.
- Implicit reinitialization of non-retained variables to missing or null before each iteration of the RUN method.

Note: The `App.TableServices.DS2.Runtime.TraceVariables` logger is used with the TRACEVARIABLES option. The TRACEVARIABLES option is set in the “DS2_OPTIONS Statement” in [SAS DS2 Language Reference](#). If the TRACEVARIABLES option is not enabled, no variable tracing information is reported. For an example of how the TRACEVARIABLES option is set, see [“Example: Logging Trace Variables” on page 313](#).

When diagnosing DS2 problems, it is important to understand that the `App.TableServices.DS2.Runtime.*` loggers do not inherit configuration from their ancestors. The default level for the `App.TableServices.DS2.Runtime.*` loggers is OFF. If you want to capture `App.TableServices.DS2.Runtime.*` logger events to the server’s log, you must explicitly set each applicable

App.TableServices.DS2.Runtime.* logger. It is recommended that you configure the loggers only when diagnosing a DS2 problem since the additional logging traffic affects performance.

Note: The logging facility on the SAS Viya 3.5 CAS server truncates any logger message that is longer than 8000 bytes to 8000 bytes.

HTTP Package Logger

The HTTP client supports logging of HTTP traffic through the SAS logging facility.

The name of the logger is App.TableServices.d2pkg.HTTP. The logger supports these logging levels:

INFO

shows general traffic information such as connections and disconnections from the web server, request information, and status information.

DEBUG

shows the headers from all requests and responses plus the first 64 bytes of body data. This enables you to see what the client and server are doing without having to see all of the data that is transmitted to the server.

TRACE

shows all of the data sent to and received from the web server.

JSON Package Logger

The JSON package supports logging through the SAS logging facility.

The name of the logger is App.tk.DS2PKG.JSON. The logger supports the DEBUG and TRACE logging levels. For example, if you want to set the logger level to TRACE at system start-up, add this line to your logging configuration file:

```
<logger name="App.tk.DS2PKG.JSON" <level value="trace"/> </logger>
```

To avoid affecting the logging level settings of others who use that logging configuration file, modify your own session by using the SAS logging facility:

```
%log4sas();
%log4sas_logger(App.tk.D2PKG.JSON, 'level=trace');
```

The JSON package provides little DEBUG or TRACE information. The GETNEXTTOKEN method does provide some TRACE information. The following log excerpt is an example:

```

INFO App.Program - 1  %log4sas();
INFO App.Program - 2  %log4sas_logger(App.tk.D2PKG.JSON, 'level=trace');
INFO App.Program - 3  proc ds2;
INFO App.Program - 4  ds2_options sas;
INFO App.Program - 5  data _null_;
INFO App.Program - 6      dcl package json j();
INFO App.Program - 7      dcl int i rc ttype parseFlags;
INFO App.Program - 8      dcl varchar(256) x;
INFO App.Program - 9      dcl varchar(256) token;
INFO App.Program - 10     method init();
INFO App.Program - 11         x = ' [{"labl 1": [{"labl 2": " a string"}]},
                        {"labl 3": ["", 3.14, null, true, false]}]';
INFO App.Program - 12         put x=;
INFO App.Program - 13         rc = j.createParser( x );
INFO App.Program - 14         i = 0;
INFO App.Program - 15         do while (rc < 101 );
INFO App.Program - 16             i = i + 1;
INFO App.Program - 17             token = '';
INFO App.Program - 2    The SAS System      16:33 Friday, October 21, 2016
INFO App.Program -
INFO App.Program - 18             j.getNextToken( rc, token, ttype, parseFlags );
INFO App.Program - 19         end;
INFO App.Program - 20     end;
INFO App.Program - 21     method term();
INFO App.Program - 22         rc = j.destroyParser();
INFO App.Program - 23     end;
INFO App.Program - 24 enddata;
INFO App.Program - 25 run;
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 1, type: 16,
                        flags: 0x0, token: [
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 2, type: 64,
                        flags: 0x0, token: {
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 10, type: 256,
                        flags: 0x1, token: labl 1
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 12, type: 16,
                        flags: 0x0, token: [
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 13, type: 64,
                        flags: 0x0, token: {
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 21, type: 256,
                        flags: 0x1, token: labl 2
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 33, type: 256,
                        flags: 0x0, token: a string
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 34, type: 128,
                        flags: 0x0, token: }
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 35, type: 32,
                        flags: 0x0, token: ]
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 36, type: 128,
                        flags: 0x0, token: }
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 38, type: 64,
                        flags: 0x0, token: {
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 46, type: 256,
                        flags: 0x1, token: labl 3
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 48, type: 16,
                        flags: 0x0, token: [
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 49, type: 16,
                        flags: 0x0, token: [

```

```

TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 51, type: 256,
                        flags: 0x0, token:
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 57, type: 512,
                        flags: 0x4, token: 3.14
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 63, type: 1024,
                        flags: 0x0, token: null
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 69, type: 4,
                        flags: 0x0, token: true
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 76, type: 8,
                        flags: 0x0, token: false
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 77, type: 32,
                        flags: 0x0, token: ]
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 78, type: 32,
                        flags: 0x0, token: ]
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 79, type: 128,
                        flags: 0x0, token: }
TRACE App.tk.D2PKG.JSON - d2jsnGetNextToken: line: 1, column: 80, type: 32,
                        flags: 0x0, token: ]
INFO App.Program - x=[{"labl 1":[{"labl 2":" a string"}]},{"labl 3":["", 3.14,
                        null, true, false]]}
INFO App.Program - NOTE: Execution succeeded. No rows affected.
INFO App.Program - 26          quit;

```

Example: Logging All SQL Operations

The following example creates a logging configuration file to log all SQL commands. After the configuration file is created, the LOGCONFIGLOC= system option is set to specify the name of the configuration file that is used to initialize the SAS logging facility.

The following code creates the logging configuration file.

```

<?xml version="1.0"?>
  <logging:configuration xmlns:logging="http://www.sas.com/xml/logging/1.0/">
    <logger name="App.TableServices.DS2.Runtime.SQL">
      <level value="trace"/>
    </logger>
    <logger name="App.TableServices.DS2" additivity="false">
      <appender-ref ref="DetailedOutput"/>
    </logger>
    <root>
      <appender-ref ref="RootLogger"/>
      <level value="info"/>
    </root>
    <appender name="DetailedOutput" class="FileAppender">
      <param name="append" value="false"/>
      <param name="FileNamePattern" value="sql.%S{App.Log}"/>
      <layout>
        <param name="ConversionPattern" value="%-5p:%sn:[%c{3}]:%m"/>
      </layout>
    </appender>

```

```

<appender name="RootLogger" class="FileAppender">
  <param name="Append" value="false"/>
  <param name="ImmediateFlush" value="true"/>
  <param name="FileNamePattern" value="%S{App.Log}"/>
  <layout>
    <param name="ConversionPattern" value="%m"/>
  </layout>
</appender>
</logging:configuration>

```

The LOGCONFIGLOC= system option is set to reference the configuration file as **config.14s**.

```
sas -log test.log -logconfigloc config.14s
```

For more information about this system option, see *SAS Logging: Configuration and Programming Reference*.

This produces a file, **sql.test.log**, which contains a series of messages similar to these.

```

DEBUG:00000084:[DS2.Runtime.SQL]:Found 0 NOCHANGE columns
INFO :00000085:[DS2.Runtime.SQL]:0x0afce4b0:exec-direct:CREATE TABLE WORK.outp
      ("i" DOUBLE, "j" DOUBLE )
DEBUG:00000086:[DS2.Runtime.SQL]:0x0afce4b0:exec-direct:passed:rc=0x00000000
DEBUG:00000087:[DS2.Runtime.SQL]:Found 0 NOCHANGE columns
INFO :00000088:[DS2.Runtime.SQL]:0x0afce4b0:exec-direct:SELECT * FROM WORK.outp
DEBUG:00000089:[DS2.Runtime.SQL]:0x0afce4b0:exec-direct:passed:rc=0x00000000
INFO :00000095:[DS2.Runtime.SQL]:stmt=0x0afae060:prepare:SELECT * FROM WORK.outp
DEBUG:00000096:[DS2.Runtime.SQL]:stmt=0x0afae060:prepare:passed:rc=0x00000000
INFO :00000097:[DS2.Runtime.SQL]:stmt=0x0afae060:execute
DEBUG:00000098:[DS2.Runtime.SQL]:stmt=0x0afae060:execute:passed:rc=0x00000000

```

The exact content of the output file is defined by the ConversionPattern parameter of the layout within the DetailedOutput appender. The DetailedOutput appender is associated with the definition of the App.TableServices.DS2 logger. This causes every logger in the hierarchy that is rooted at App.TableServices.DS2 to be logged to the same file. The additivity="false" modifier prevents the log messages from moving upward.

Example: Logging Trace Variables

Invoking the TraceVariables Logger

To enable logging of trace variables:

- 1 Configure the App.TableServices.DS2.Runtime.TraceVariables logger, specifying one of the DEBUG and TRACE logging levels.

- 2 Specify the TRACEVARIABLES option in your DS2 program. The TRACEVARIABLES option is set in the DS2_OPTIONS statement.

Note: You must complete both steps. If either one of these tasks is not done, no variable tracing information is reported to the SAS logging facility.

To configure the logger, add the following line to your logging configuration file. This example sets a logger level of DEBUG. A value of TRACE or DEBUG must be specified.

```
<logger name="App.TableServices.DS2.Runtime.TraceVariables"><level value="debug"/></logger>
```

Here is an example of how the TRACEVARIABLES option is set. Specify the DS2_OPTIONS statement directly before the program block whose variables you want to trace. The DS2_OPTIONS statement must be set for each program block that you want to examine.

```
proc ds2;
  ds2_options tracevariables;
  package swapper / overwrite=yes;
    method swap(in_out double x, in_out double y);
      dcl double t;
      t = x;
      x = y;
      y = t;
    end;
  endpackage;

  ds2_options tracevariables;
  data _null_;
    method init();
      dcl package swapper s();
      dcl double a[2];
      a := (10, 20);
      put 'before swap:' a[*]=;
      s.swap(a[1], a[2]);
      put 'after swap:' a[*]=;
    end;
  enddata;
run;
quit;
```

Understanding Logger Messages

Here is the SAS log from the preceding program:

```

1      proc ds2;
2      ds2_options tracevariables;
3      package swapper / overwrite=yes;
4      method swap(in_out double x, in_out double y);
5          dcl double t;
6          t = x;
7          x = y;
8          y = t;
9      end;
10     endpackage;
11
12     ds2_options tracevariables;
13     data _null_;
14         method init();
15             dcl package swapper s();
16             dcl double a[2];
17             a := (10, 20);
18             put 'before swap:' a[*]=;
19             s.swap(a[1], a[2]);
20             put 'after swap:' a[*]=;
21         end;
22     enddata;
23     run;
before swap: a[1]=10 a[2]=20
after swap: a[1]=20 a[2]=10
NOTE: Created package swapper in data set work.swapper.
NOTE: Execution succeeded. No rows affected.
24     quit;

```

Here are the trace messages reported to the SAS logging facility by the App.TableServices.DS2.Runtime.TraceVariables logger:

```

2021-01-12T13:33:35,063 DEBUG [000000007] - Line 17 (data block line 5): a[1]=10
2021-01-12T13:33:35,063 DEBUG [000000007] - Line 17 (data block line 5): a[2]=20
2021-01-12T13:33:35,063 DEBUG [000000007] - Line 6 (package swapper line 4): t=10
2021-01-12T13:33:35,064 DEBUG [000000007] - Line 7 (package swapper line 5): x=20
2021-01-12T13:33:35,064 DEBUG [000000007] - Line 8 (package swapper line 6): y=10

```

Trace messages have this format:

Line n1 (program_block_type program_block_name line n2): variable_name=variable_value

n1

Line number of the statement that experienced activity in the SAS log.

program_block_type

Type of the program block that contains the statement (“package”, “thread”, or “data”).

program_block_name

Name of the program block that contains the statement. For package and thread program blocks, the user-defined name of the package or thread is the name. For data program blocks, “block” is the name.

n2

Program block line number. Program block line numbers start with 1 at the PACKAGE, THREAD, or DATA statement that starts the program block and continue in sequence until reaching the ENDPACKAGE, ENDTHREAD, or ENDDATA statement that ends the program block. n2 is the line number of the statement that experienced activity, relative to the beginning of the package, thread, or data block.

This number is provided because the line number in the SAS log can change from run to run, depending on what else was run in the current SAS session. The line number of the statement offset from the beginning of the program block does not change. Knowing the program block line number is useful when a stored DS2 package or stored DS2 thread is not in the source listing of the SAS log.

Here are details about this example:

- The first two messages in the logger output reference an array assignment statement from the data program block in Line 17 in the SAS log.

```
2021-01-12T13:33:35,063 DEBUG [000000007] - Line 17 (data block line 5): a[1]=10
2021-01-12T13:33:35,063 DEBUG [000000007] - Line 17 (data block line 5): a[2]=20
```

The statement `a := (10, 20)` in Line 17 assigns a constant list containing the values 10 and 20 to the array `a`. The first message, `a[1]=10`, records the assignment of the value 10 to the first element of array `a`. The second message, `a[2]=20`, records the assignment of the value 20 to the second element of array `a`. The reference to Line 5 in each message restates the location of the array assignment statement as an offset from the DATA statement. The statement `a := (10, 20)` is located four lines below the DATA statement.

- The remaining messages describe variable assignments resulting from transformations that occurred in the swapper package.

```
2021-01-12T13:33:35,063 DEBUG [000000007] - Line 6 (package swapper line 4): t=10
2021-01-12T13:33:35,064 DEBUG [000000007] - Line 7 (package swapper line 5): x=20
2021-01-12T13:33:35,064 DEBUG [000000007] - Line 8 (package swapper line 6): y=10
```

Lines 6, 7, and 8 contain variable assignment statements that assign values to variables `t`, `x`, and `y`.

In the assignment statement `t=x`, `t` is assigned the value of `x`. Parameter `x` was passed the value 10 by the first argument, `a[1]`, of the `s.swap` method call on Line 19. Therefore, the value 10 is assigned to `t`, or `t=10`.

In the assignment statement `x=y`, `x` is assigned the value of `y`. Parameter `y` was passed the value 20 by the second argument, `a[2]`. Therefore, the value 20 is assigned to `x`, or `x=20`.

In the assignment statement `y=t`, `y` is assigned the value of `t`. Variable `t` still has the value 10 that was assigned by the earlier assignment statement `t=x`. Therefore, `y=10`.

Note: Variable tracing can result in a large number of SAS logging messages, especially when the DS2 program is large or the data set or data sets that are input by the DS2 program are large. Therefore, it is recommended that you configure logging only when diagnosing a DS2 problem because the additional logging traffic adversely affects performance.

Appendix 3

Resolving DS2 Errors

<i>Types of DS2 Errors</i>	317
<i>How to Resolve DS2 Errors</i>	317
<i>DS2 Loggers</i>	318

Types of DS2 Errors

DS2 writes errors to the SAS log when it is unable to complete a request. Most errors can be categorized as follows:

- Usage errors

Usage errors arise because of incorrect or invalid requests. DS2 usage errors are typically caused by syntax, semantic, or logic errors in a DS2 program. Syntax and semantic errors are detected at compile time and prevent compilation of the DS2 program. A DS2 program that successfully compiles might still contain logic errors that surface at execution time.

- Resource errors

Resource errors arise when resource service requests cannot be fulfilled. Various conditions lead to these failures. For example, the resource could be inaccessible, a resource threshold could have already been reached, or you could lack authorization to access the resource.

How to Resolve DS2 Errors

To resolve errors:

- 1 Check the SAS log for error messages. These messages might help you understand the source of the problem or provide ideas to help you resolve the error.

Here is an example of an error message that indicates what the problem is. In this instance, a usage error occurred. The error message indicates that the DS2 program contains a syntax error at or near line 7.

```
ERROR: Parse encountered END when expecting ';'.
ERROR: Line 7: Parse failed: >>> end <<<;
```

- 2 In the SAS log, focus on the earliest error messages first. Sometimes a single problem can lead to a cascade of failures and resolving the first error can also resolve subsequent errors.
- 3 If an error message indicates a resource error, check with your system administrator to monitor system resources. Examples of system resources include memory usage and disk space. If system resources are low, look for ways to increase resources or consider ways that your application could use them more efficiently.

Here is an example that shows a resource error. The message indicates a memory threshold was reached.

```
ERROR: Memory allocation failed.
```

- 4 If you have been unable to resolve the error using the previous three steps, contact SAS Technical Support. Here is a list of information you will need to supply: [Four tips to remember when you contact SAS Technical Support](#).

If the SAS log contains error messages with internal error codes, provide the codes to your SAS Technical Support representative. Internal error codes are hexadecimal numbers that can help SAS more quickly determine the source of the problem.

Here is an example of an error message that contains an internal error code (highlighted).

```
ERROR: DS2 internal error code 0xECCC8C8BF7C049FD. Refer to DS2 documentation
for resolving errors.
```

DS2 Loggers

DS2 loggers can also be useful in resolving errors. They provide another way to obtain information from a running DS2 program. Here are examples:

- The App.TableServices.DS2.Config.Options logger reports a number of configuration settings, one of which is the DS2 session encoding. If you are unsure of the session encoding that is being used to run a DS2 program, you can enable this logger to report the DS2 session encoding.
- The HTTP package logger provides useful information for debugging problems in HTTP requests that are generated with the DS2 HTTP package.

- The TraceVariables logger, which traces a variable value's changes during the execution of a DS2 program, is useful for problem remediation and can provide auditing of your DS2 code. This logger is available in SAS Viya 3.5 for use with the CAS server and the SAS Micro Analytic Service server only.

For more information about these and other DS2 loggers, see [Appendix 2, “DS2 Loggers,” on page 307](#).

